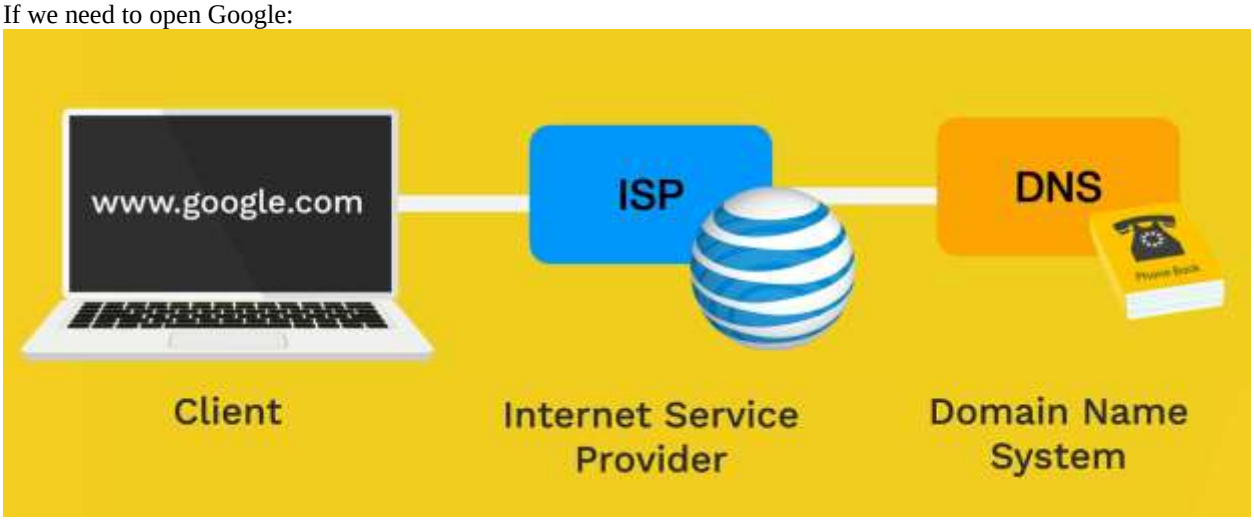
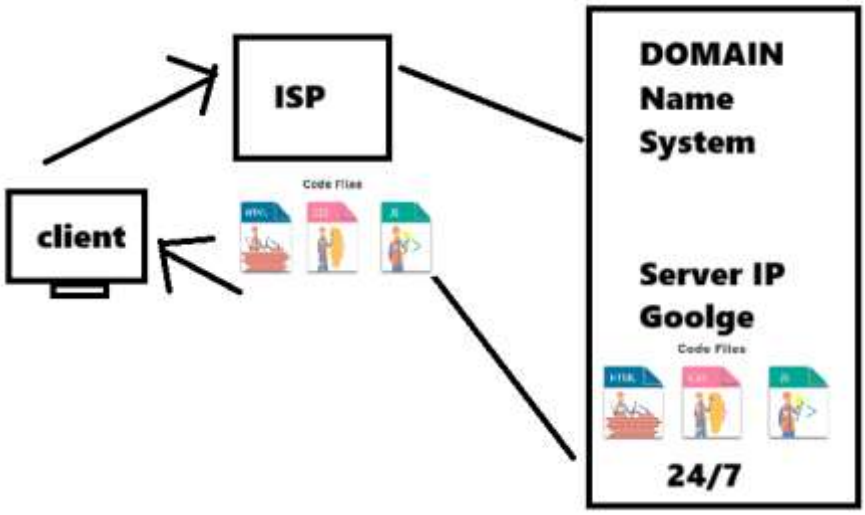
Internet

Is Long piece of wire that connects computers such client and servers? And server are on 24/7.
If we need to open Google:



Client send request – ISP – DNS (IP-Server Google)



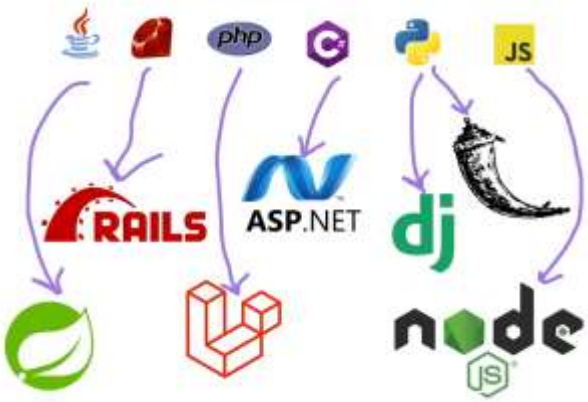
IP address: each computer has IP address same as we have post address.

Stuck?

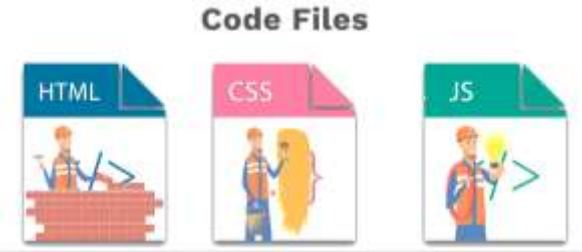
Check the code line by line
Review the code
Internet Google, Chatgpt, stackoverflow.com …

Front-end

Back-end

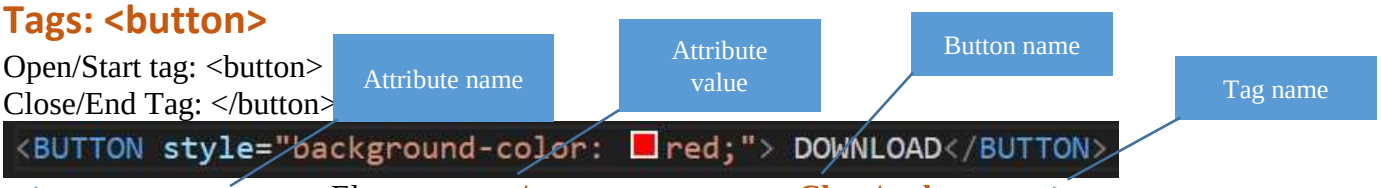


HTML: **Hyper Text Markup Language**[1]: to create the **structure of web site** and web app.
The HTML was written by TIM BERNERS_LEE in 1993.
Web (Network)
1 – **Web Application** (stores dynamic info) Facebook, google, YouTube, Instagram.
2 – **Web Sites** (stores static information) BBC, CV... (To make a portfolio for CV) which only Front End



Tags: <button>

Open/Start tag: <button>
Close/End Tag: </button>



Element: **start/open tag**+ content + **Close/end tag**.
Attribute: in the start tag after the tag name we can place attributes (خواص)
Attribute name: text before =
Attribute value: anything after =
Boolean Attributes: only name of attribute: those attributes which doesn't have value = disabled
Self-close tags ==
 which means break line also called empty element

[1] https://www.w3schools.com/html/default.asp
https://devdocs.io/

```
<!DOCTYPE html>
<html>
    <head>
        <title>Welcome to HTML</title>
    </head>
    <body>
        <h1>Lesson 1 of HTML</h1>

        <p title="Hyper text Markup Language">HTML</p>
        <button>Open</button>
        <br/>
        <hr/>
        <button>Delete</button>
        <div>
            <p title="a short paragraph">
                div is a parent tag and p is a child tag.
            </p>
        </div>
    </body>
</html>
```

- & Ampersand
- ; Semi-colon
- : Colon
- = equal sign
- < less than sign
- > Greater than sign
- ! Exclamation mark
- / forward slash
- \ back slash
- ‘ single quote
- “ double quote
- ( ) Parenthesis
- { } Curly Braces
- [ ] Square Brackets
- <> Angle Brackets
- * Asterisk
- , Comma

 means division which divides a web page
Div is parent element and p is child element and div is nested element. Cuz of paragraph inside it.

- **<Meta charset=”UTF-8”> supports all languages**
- => bold
- <hr> => horizontal line
-
 => break line
- => paragraph
- <h1></h1> => heading which we have h1 - h6

  my name is Khalid

Definition of DOCTYPE (Boilerplate)

---------- Day 2 the HTML Boilerplate

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Sample HTML5 Document</title>
</head>
<body>
    <h1>Hello, World!</h1>
    <p>This is a sample HTML5 document.</p>
</body>
</html>
```

<title> this tag is used for the title of the page </title>
** HTML title attribute is used for tips and abbreviations **
<link rel="icon" href="a.png"> this is self-close tag used to link icon to title area.
Entities: used to show our code same in output as html code
&lt : les than and greater than
&gt

```
&lth1&gtHeading 1&lt/h1&gt
```

= <h1>Heading1</h1>

HTML is not case sensitive.
SEO search engine optimization: is done by Meta tags

Meta tags

```
<meta charset="UTF-8">
```

Supports all languages in all old browsers.

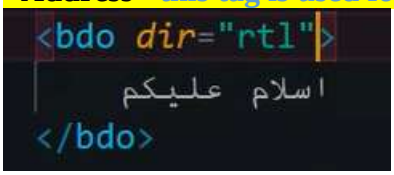
```
<head>
    <meta charset="UTF-8">
    <meta name="author" content="Popal">
    <meta name="keywords" content="health, food"
    <meta name="Instagram" content="create an account or login into your account">
```

Meta name is used to when someone type health direct him to our website or food..

Formatting tags:

** this tag is used to bold its contents **
<Strong> this tag is used to give more importance to its contents </Strong>
** this tag is used to italic its contents **
** this tag is used to give more emphasize to its contents **
<u> this tag is used to underline its contents </u>
<ins> this insert tag is used to give more importance to its contents </ins>

<Mark> this tag is used to highlight its contents **</mark>**
<Small> this tag is used to decrease the size of its contents **</small>**
<Del> this tag is used to draw a line through its contents **</Del>**
<Sup> this SuperScript tag is used to decrease the size of a letter on the top **</ Sup >**
<Sub> this **Subscript** tag is used to decrease the size of a letter on the bottom **</ sub >**
Quotations': defines a section which is quoted from another source
<blockquote cite = "https://www.google.com"> this tag is used for larger paragraphs when they are quoted from another source**</blockquote>**
<q> this tag is used for small quotes when they are quoted from another source**</q>**
<abbr title="Hypertext Markup Language"> HTML this tag is used for abbreviations **</abbr>**
<Address> this tag is used for address **</Address>**



is used for right to left and left to right

Link tags Anchor element

**<a *href*="https://www.bbc.com"> is used to link file, image, video or any file **
**<a *href*="a.jpeg"> is used to link file, image, video or any file **
**<a *href*="a.html"> Login to my profile **
**<a *href*="#"> for blank link **
**<a *href*="mailto: g.furqani@gmail.com"> Contact Us **
<a *href*="https://www.google.com"> <img src="./p.jpg"> link an image to google

```
<body>
    <h1>Lesson 2 of HTML</h1>
    <b>Bold</b>
    <br>
    <strong>Same as Bold</strong>
    <br>
    <i>Italic text</i>
    <br>
    <u>Underline Text</u>
    <br>
    <marked> Marked Text</marked>
    <br>
    <small>Small</small>
    <br>
    <del>Deleted</del>
    100<sup>2</sup>
    100<sub>2</sub>
    <blockquote cite="wwww.google.com/lorem">Lorem ipsum dolor sit amet consectetur adipisicing elit. Nesciu
    <p>Lorem ipsum dolor sit <q> amet consectetur adipisicing elit. </q> Vitae, quam.</p>
    <abbr title="Hyper Text Markup Language">HTML</abbr>
    <address>345 Hill Rock Blvd, Apt 101, Newport News VA, 23443</address>
    <bdo dir="rtl">عليكم السلام!</bdo>
    <a href="https://www.google.com">google</a>
    <a href="a.png">HTML Logo Photo</a>
    <a href="a.zip">Download & Install</a>
    <a href="Lesson1.html" target="_blank">Open Test file</a>
```

./ is used for relative path, / mean go inside ../ mean go outside of folder
Most of the time we use relative path not absolute cause if we use absolute and we take code to another computer so the code with new user name wont work. Qazif or …

Media:

The HTML **<img>** tag is used to embed an image in a web page.
<img>
Attributes
Src – specifies the path to the image
Alt – specifies an alternate text for the image
The HTML **<video>** is used to show a video on a web page.
The HTML **<audio>** is used to play audio on a web page.

Attributes

1. src
2. control
3. autoplay
4. loop
5. muted
6. poster

<img src=”a.jpg” width=”400” alt=”Website logo” >

Video: we can pick up video and audio
<video *src*=”a.mp4” controls muted autoplay loop > Login to my profile </video>
<video *src*=”a.mp3” controls muted autoplay loop > song </video>
Poster: ددی لپاره دی چې موږ لمړی عکس وښیه او بیا ویډیو
<video *src*=”a.mp4” controls muted autoplay loop poster =”a.jpg”> Login to my profile </video>
<audio *src*=”a.mp3” controls muted autoplay loop > song </audio>

File paths

Absolute: which is related to the root of computer hard drive.
Eg: C:\Users\qazif\OneDrive\Desktop\Dev School\JS-4\index.html
Relative: it is relative to a file. And is short and movable with folder.
(../) means go up level eg ../folder3/p.jpg
(./) current directory is a direct path eg ./p.jpg

```
Relative Path
<a href="./cv/p.jpg" target="_blank">image</a> go froword one folder
<a href="../data.xlsx" target="_blank">image1</a>   go back one folder
<br>
Absolute Path
<a href="C:\Users\qazif\OneDrive\Desktop\HTML Practice\cv\p.jpg" target="_blank">Photo</a> <br>

<img src="./cv/p.jpg" width="150" alt="Popal photo"/>
 <br>
<video src="./ab.mp4" width="400" controls >Watch full video of project</video>
<audio src="./a.mp3" controls>Play Music</audio>
```

Self-closing tags

<hr/> horizontal rule
**
 break tag**

We use diffchecker.com for to compare our code to the original code

```
<a href="a.png " target="_blank" >HTML Logo</a>
```

Target =”_blank” is used to open our link in a new blank page.

A web site that checks your code whether your code is correct

This is where we can learn **HTML** TAGS
https://developer.mozilla.org/en-US/docs/Web/HTML

HTML Space =

HomeWork

Mohammad Idress Furqani

I am 10 years old and I am in 3rd grade, and I live in Sherwood Place, I would like to learn Computer Enginering.
I love to learn HTML, CSS, JS, Python, and AI Machine learning.

Gulam Hazrat Furqani Web Developer

Gulam Furqani

this is **Gulam Furqani** a web dev and designer, i have skills in HTML, CSS, JS, Java, Python, Selenium, Postman, MySQL, JIRA, Frame works Git Hub

Relative Path

<img src=”**a.jpg**”/>
<img src=”**url**”/>
<img src=”**https://picsum.photos/200**” **alt=”image discription”**/>

When file is the same folder
./images/a.jpg
../a.jpg

Means go outside of the folder
../images/a.jpg
Means go outside of the folder then go to images folder then take a.jpg
Absolute path:

```
Absolute Path
<a href="C:\Users\qazif\OneDrive\Desktop\HTML Practice\cv\p.jpg" target="_blank">Photo</a> <br>
```

List

An unordered list starts with the <ul> tag. Each list item starts with the <li> tag.
An ordered list starts with the <ol> tag. Each list item starts with the <li> tag.
OL attribute
Type =”1” or 1,a,A,I, i

Ordered List

<ol type=”A” start=”4”> معنی ډول یی په ای بی سی سره او شروع یی د څلورم توری څخه راته شروع که
Question
Question
Question


```
<ol type="1">
    <li><a href="https://google.com">API</a></li>
    <li><a href="https://google.com">Html</a></li>
    <li><a href="https://google.com">Python</a></li>
    <li><a href="https://google.com">API</a></li>

</ol>
```

Unordered List

Question
Question
Question

Description list

<dl >
<dt>HTML</dt>
<dd>**lorem10**</dd>
</dl>

```
<!--Comment in HTML-->   short cut ctrl+/
1 deactivate the code. 2 we can add note to the code.
```

Table

- Use the HTML **<table>** element to define a table
- Use the HTML **<thead>** element to define a table head
- Use the HTML **<tbody>** element to define a table body
- Use the HTML **<tfoot>** element to define a table foot
- Use the HTML **<tr>** element to define a table row
- Use the HTML **<td>** element to define a table data

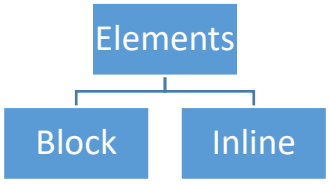
- Use the HTML **<th>** element to define a table heading
- Use the HTML **<caption>** element to define a table caption
- Use the colspan attribute to make a cell span many columns(merge columns)
- Use the rowspan attribute to make a cell span many rows (merge rows)

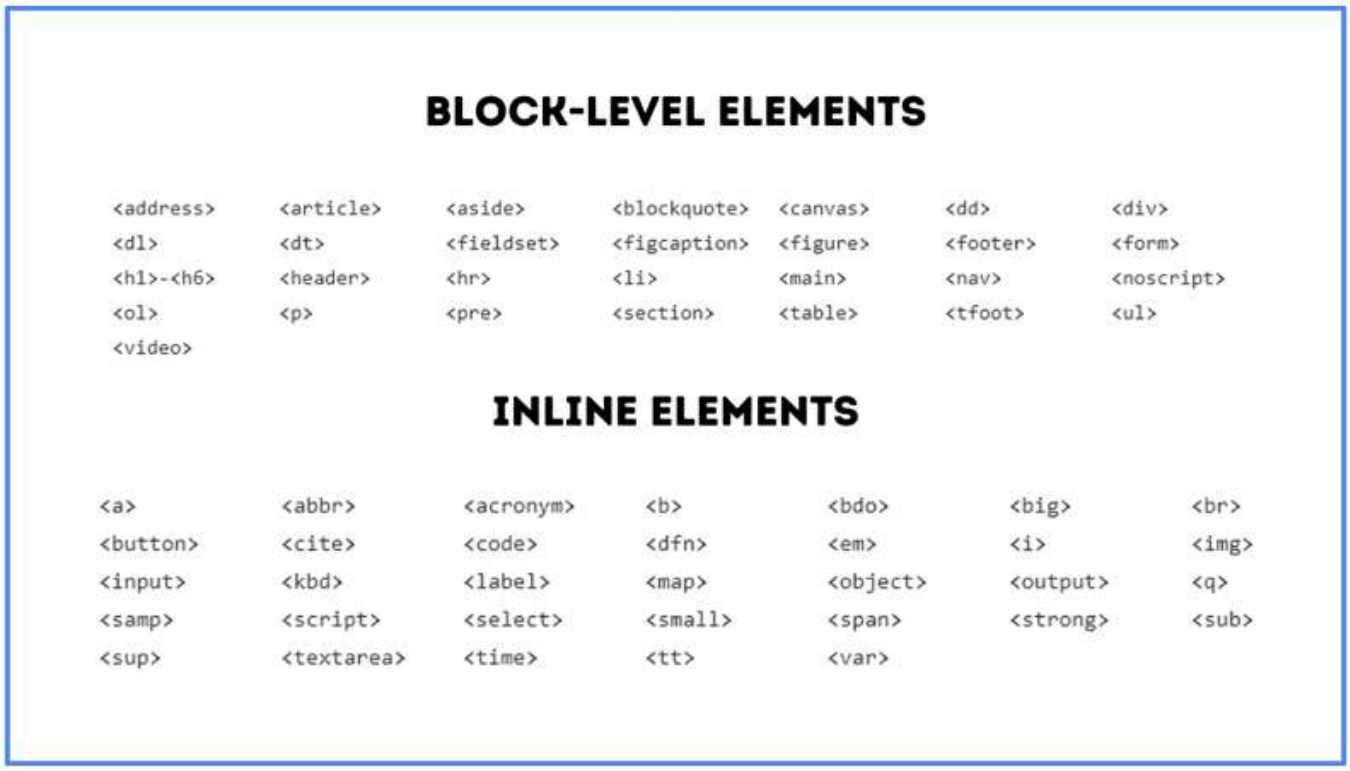
```
<table border="1">
    <caption>Salary Table</caption>
    <thead>
        <tr>
            <th>Name</th>
            <th>Position</th>
            <th>Salary</th>
        </tr>
    </thead>
    <tbody>
        <tr>
            <td>Ali</td>
            <td>QA Tester</td>
            <td>6000</td>
        </tr>
        <tr>
            <td>Ahmad</td>
            <td rowspan="2">Developer</td>
            <td>6300</td>
        </tr>
        <tr>
            <td>Khan</td>
            <td>3000</td>
        </tr>
    </tbody>
    <tfoot>
        <tr>
            <th colspan="2">Total</th>
            <th>12300</th>
        </tr>
    </tfoot>
</table>
```

Salary Table

Name	Position	Salary
Ali	QA Tester	6000
Ahmad	Developer	6300
Khan		3000
Total		12300

A block-level element always starts on a new line. AND an inline element does not start on a new line.





Institute

Form

An HTML from is used to collect user input. The user input is most often sent to a s
<form>

The HTML <input> element is the most used from element.

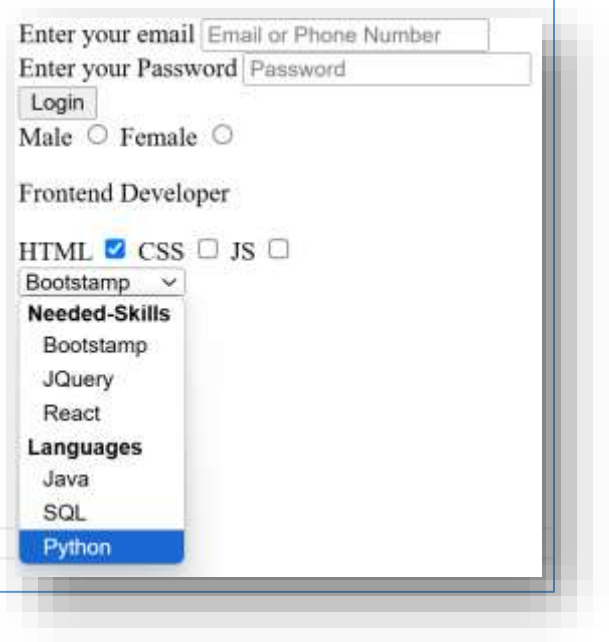
<input type=''text''> displays a single-line text input field
<input type=''radio''> displays a radio button (for selecting one of many choic
<input type=''check box''> displays a checkbox (for selecting zero for more of many
<input type=''submit''> displays a submit button (for submitting the form)
<Input type=''button''> displays a clickable button
<Input type=''color''> displays a clickable button
<Input type=''date''> displays a clickable button

Elements

<input>	
<label>	to name an input
<select>	to make a selection list and option should be child of it
<option>	
<textarea>	comment or note area where people can write more text
<button>	
<feildset>	seperates forms into groups if we have more than one
<legend>	gives title to feildset
<datalist>	define a list where we can use it for search as well always connect it with input

<output>
<optgroup>
<Input type = “radio”/> creates a radio button
Note: Type shows type of the input text box, for password submit that works as button and value defines default value for input tags and also it goes to url while we press submit and works as rename for submit ➔ Login button.
id=”” names element which will be known to call it. ep I chosen as ID for this element.
For=”” is used to for the focus to link two elements such as lable tag with input

```
<form >
    <label for="ep">Enter your email</label>
    <input type="text" placeholder="Email or Phone Number" name="username" id="ep"><br>
    <label>Enter your Password
        <input type="password" placeholder="Password" name="passcode99"><br>
    </label>
    <input type="submit" value="Login"><br>
    <label for="male">Male</label>
    <input type="radio" name="gender" value="male" id="male">
    <label for="female">Female</label>
    <input type="radio" name="gender" value="female" id="female">
    <p>Frontend Developer</p>
    <label for="">HTML
        <input type="checkbox" name="HTML" checked>
    </label>
    <label for="">CSS
        <input type="checkbox" name="CSS">
    </label>
    <label for="">JS
        <input type="checkbox" name="JS">      <br>
    </label>
    <select name="OtherSkills" id="">
        <optgroup label="Needed-Skills">
            <option value="Bs">Bootstamp</option>
            <option value="Jq">JQuery</option>
            <option value="Re">React</option>
        </optgroup>
        <optgroup label="Languages">
            <option value="Bs">Java</option>
            <option value="Jq">SQL</option>
            <option value="Re">Python</option>
        </optgroup>
    </select>
</form>
```



Drop-Down List or Search List <datalist>

```html
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Document</title>
</head>
<body>
    <label for="tech-input">Select a technology:</label>
    <input type="text" id="tech-input" list="f">
    <datalist id="f">
        <option value="Jira"></option>
        <option value="SQL"></option>
        <option value="Java"></option>
        <option value="Selenium"></option>
        <option value="TestNG"></option>
        <option value="Jenkins"></option>
        <option value="API"></option>
        <option value="Framework"></option>
        <option value="Manual"></option>
    </datalist>
</body>
</html>
```

Attributes

```html
<!-- input Attributes -->
<input type="button">
<input type="text">
<input type="submit">
<input type="checkbox">
<input type="radio">
<input type="datetime-local">
<input type="color">
<input type="date">
<input type="week">
<input type="time">
<input type="month">
<input type="email" placeholder="email" name="email">
<input type="file" multiple> <br>
<input type="number" max="100" min="0" value="50" step="10"> <br>
<input type="password">
<input type="range"> <br>
<input type="reset"> <br>
<input type="search"> <br>
<input type="tel" pattern="[0-9]{3}-[0-9]{3}" required readonly> <br>
<input type="url"> <br>
<!-- input Restrictions such as Required, disabled, value, size, readonly, max, min -->
```

Here is a list of some common input restrictions

- **disabled** – specifies that an input field should be disabled
- **max** – specifies the maximum value for an input field
- **maxlenghth** – specifies the maximum munber charictor for an input field 1
- **min** – specifies the minimum value of an input field
- **readonly** – specifies that an input field is read only (cannot be changed)
- **required** – specifies that an input field is required (must be filled out)
- **size** – specifies the width (in charactors) of an input field
- **<input – type=”tel” id=”phone” name=”phone” pattern= “[0-9]{3}-[0-9]{2}-[0-9]{3}”>**
- **step** – specifies the legel number intervals for an input field

- **value** – specifies the default value for an input field

<input type = “Button”> submit send data to DB and button won’t
<input type = “Color”> show color with color picker
<input type = “date”> show date with date picker

```
<!-- input Attributes -->
<input type="submit">
<input type="color">
<input type="date">
<input type="email" placeholder="email" name="email">
<input type="file" multiple> <br>
<input type="number" max="100" min="0" value="50" step="10"> <br>
<input type="range"> <br>
<input type="reset"> <br>
<input type="search"> <br>
<input type="tel" pattern="[0-9]{3}-[0-9]{3}" required readonly> <br>
<input type="url"> <br>
<!-- input Restrictions such as Required, disabled, value, size, readonly, max, min -->
```

Iframe:

Iframe is an HTML tag that embeds another HTML document or webpage into a current web page.

Attributes

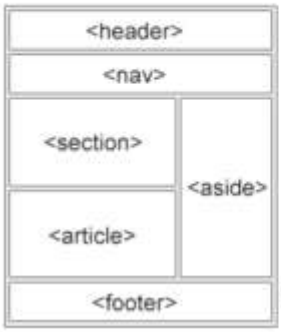
- Src
- Height
- Width
    - ❖ We can copy an embed YouTube video or google map and paste it our iframe
    - ❖ We open another html page in our iframe
    - ❖ We can link anchor tag with iframe

```
<body>

    <!-- iframe for a youtube video-->
    <iframe width="560" height="315" src="https://www.youtube.com/embed/C55zjPlqdYw?si=BQ5lFi9QAtk-MYXy" title="YouTube video player" frameborder="0"

    <!-- Link iframe with reglar iframe in our page with help of name and traget attributes-->
    <a href="https://www.google.com/maps/embed?pb=!1m18!1m1" target="map">
        My Location
    </a>

    <iframe src="./Lesson5Forms.html" width="300" height="250" name="map" ></iframe>

</body>
```

Layout:

HTML Layout Elements

HTML has several semantic elements that define the different parts of a web page:

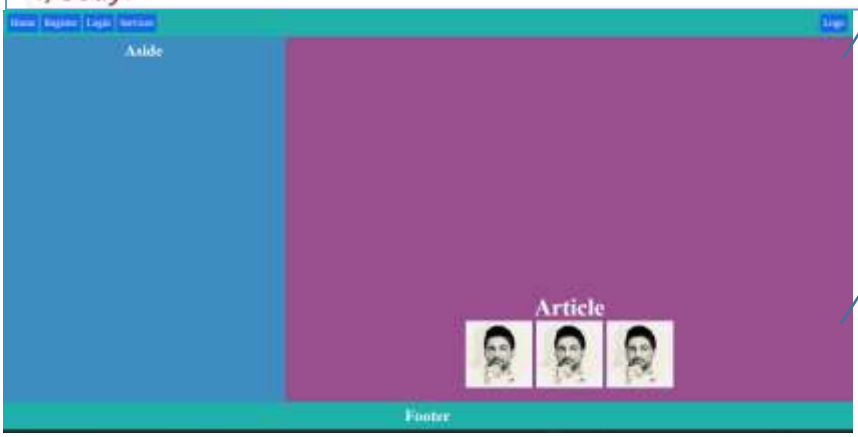


- `<header>` - Defines a header for a document or a section
- `<nav>` - Defines a set of navigation links
- `<section>` - Defines a section in a document
- `<article>` - Defines an independent, self-contained content
- `<aside>` - Defines content aside from the content (like a sidebar)
- `<footer>` - Defines a footer for a document or a section
- `<details>` - Defines additional details that the user can open and close on demand
- `<summary>` - Defines a heading for the `<details>` element

You can read more about semantic elements in our HTML Semantics chapter.

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Layout</title>
</head>
<body>
    <Header>
        Web Header
        <nav>
            Navigation Bar
        </nav>
    </Header>
    <main>
        <aside>
            Aside
        </aside>
        <Section>
            <H1>Layout of HTML</H1>
        </Section>
        <article>
            <h6>All contents of Webpage</h6>
            <p>Lorem ipsum dolor sit, amet consectetur adipisicing elit. Aliquam consequatur a</p>
        </article>
    </main>
    <footer>
        Footer
    </footer>
    <details>
        <summary>
            Information
        </summary>
        Lorem ipsum dolor sit amet.
    </details>
</body>
</html>
```

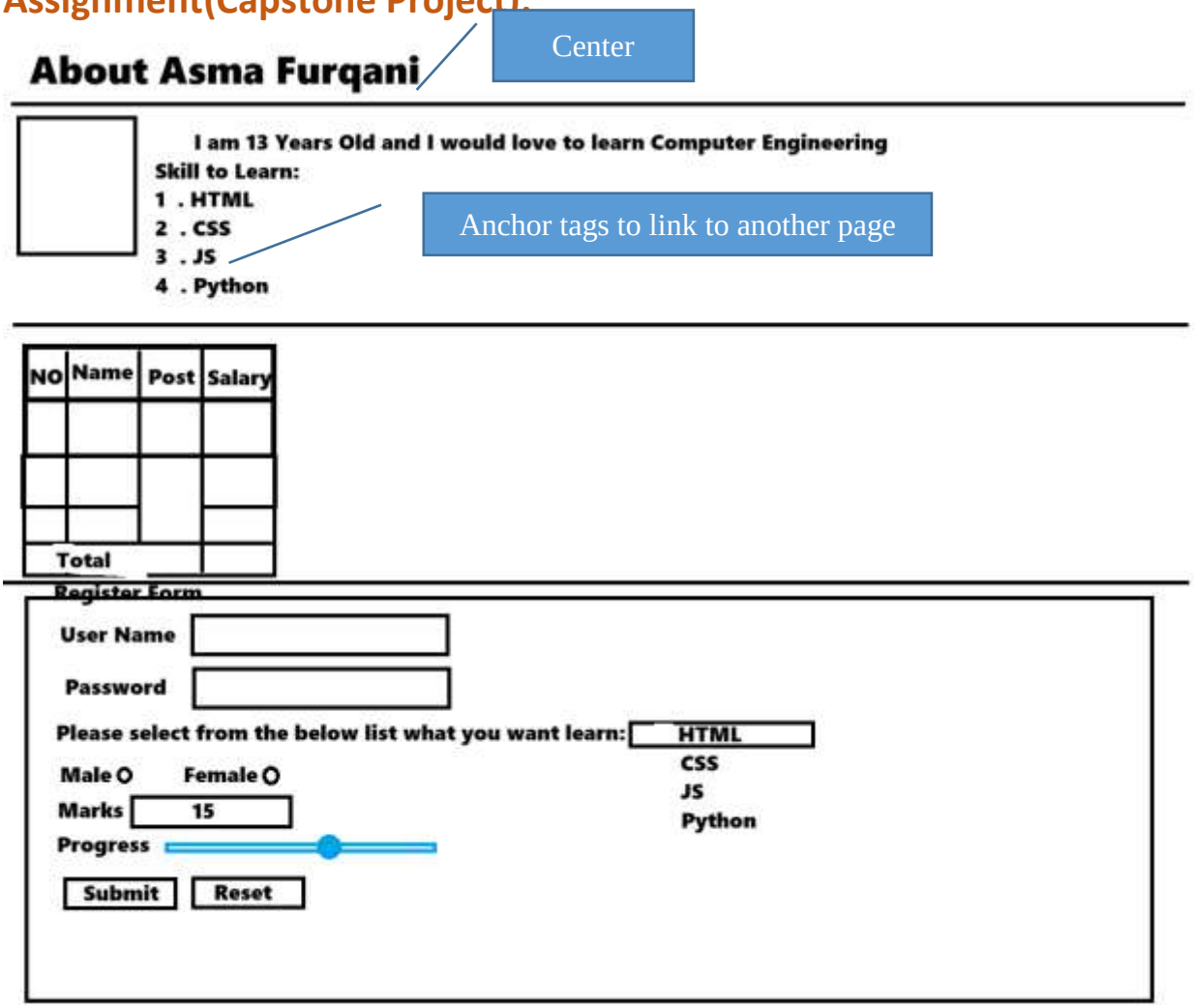
```
<body>

    <header> Header
        <nav>Nevegation Bar</nav>
    </header>
    <main>
        <aside>
            Aside
        </aside>
        <article>
            Article
        </article>
    </main>
    <footer>Footer</footer>

</body>
```



Header

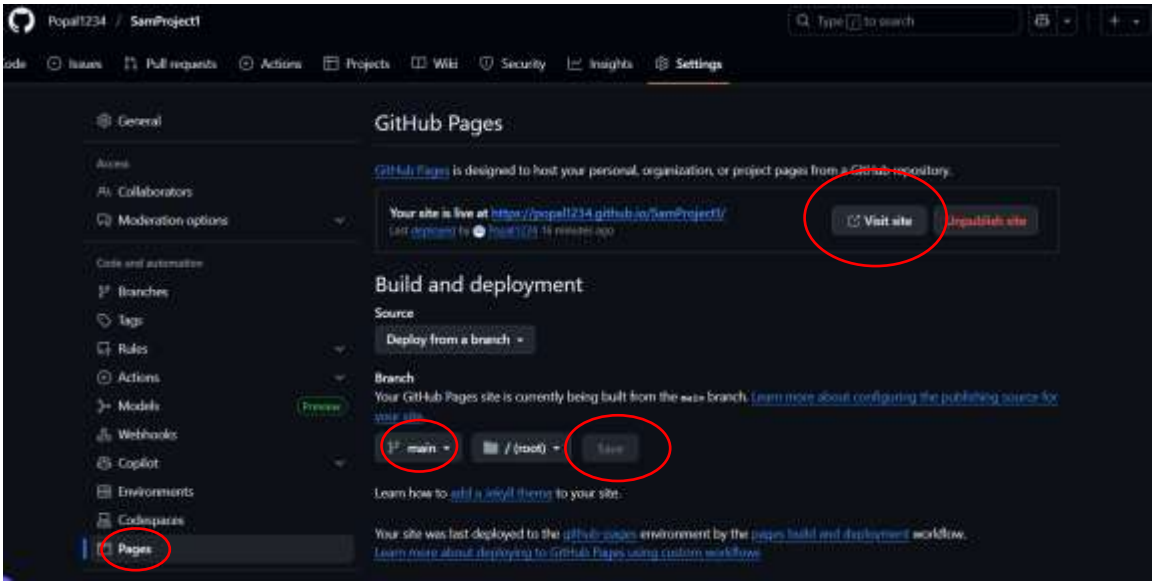
Main
Aside Article

Assignment(Capstone Project):



Hosting your Website

Hosting website so that everybody around the world can see that 24/7 with Github.com (Server)
1 **create new repository – make it public – select readme file as on.**
2 Add file - upload files your files
3 settings – pages – (Branch) make main then press save
4 copy link of visit


Popall1234 / SamProject1
Issues
Pull requests
Actions
Projects
Wiki
Security
Insights
Settings
General
Collaborators
Moderation options
Branches
Tags
Rules
Actions
Models
Webhooks
Copilot
Environments
Codespaces
Pages
GitHub Pages
Visit site
Build and deployment
Source
Deploy from a branch
Branch
main
/ (root)
Save

CSS: (Cascading Style Sheets)

It's a language used to describe the look and layout of a webpage written in HTML or XML.
1996 released. (dribble.com) where can find nice designs
CSS is used for the style and design of web page and website.

With CSS, you can control:

- The **appearance** of text (font size, color, style)
- The **layout** of elements (whether they appear side by side, stacked, etc.)
- **Spacing** between elements (margins, padding)
- **Backgrounds**, borders, and more

There are three ways of creating a style sheet: (**Inline, internal, External**)
Web Element: Anything present on the web page is a Web Element such as text box, button, table, list, form, image, etc. Web Element represents an HTML element.

1. Inline CSS:

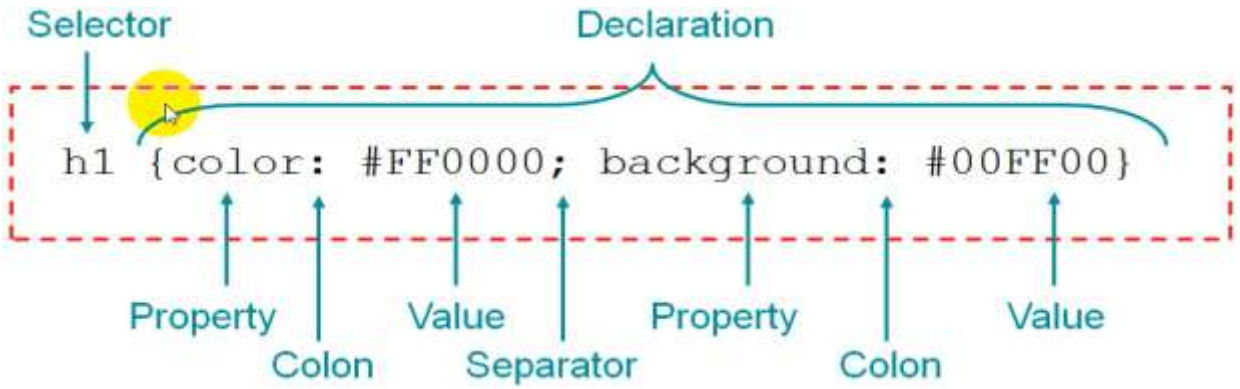
```
<!--Inline style-->
<h1 style="color: aqua; background-color: black;">Heading 1</h1>
```

Style =”color: red; background-color: blue;” is used to release style to a web element in same line, color is a **property** and red is a **value** and **semi colon**; is a separator

2. Internal CSS:

We put **style** tag inside the head of HTML which effects all elements with the same tags as below

```
<!--Internal Style-->
<style>
    p { color: red; background-color: azure;}
    h1 {text-align: center;}
</style>
```



3. External CSS:

External Style: we create a file with the name of (style.css) in same folder then we link it with html

```
<link rel="stylesheet" href="./style.CSS"/>
```

CSS Selectors Syntax:

- Element Selector: tagname
- Class Selector: .classAttributeValue
- Id Selector: #idAttributeValue
- Group Selector: ,
- Attribute Selector: [attribute name=”Att-value”]

```
[id]
{
  background-color: rgb(152, 143, 241);
}
```

- Attribute Value Selector:[title=”test”]

```
[id="m1"]
{
  background-color: rgb(177, 233, 136);
}
```

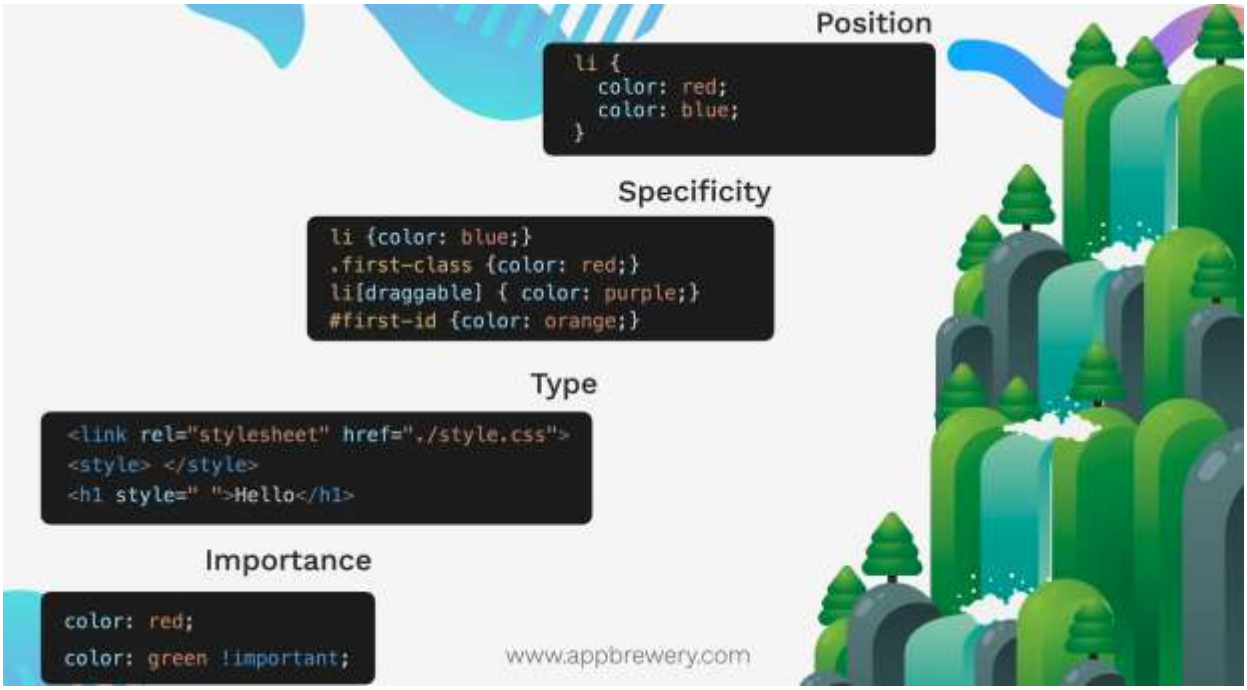
- CSS Declaration Block: {…};
- * mean all elements
- Child selector Body > div > div > p
  Chaining selectors div#lo.done{}
-

```
*
{
  background-color: rgb(177, 233, 136);
  color: chocolate;
  text-align: center;
}
```

!important means give priority to this style

`id`: Used to uniquely identify **one** specific HTML element on a page. No two elements should have the same `id`. (Identify one web element)

`class`: Used to identify **a group** of elements that share the same styling or behavior. Multiple elements can share the same `class`. (Identify group of web elements)


Position
li {
color: red;
color: blue;
}
Specificity
li {color: blue;}
.first-class {color: red;}
li[draggable] { color: purple;}
#first-id {color: orange;}
Type
<link rel="stylesheet" href="./style.css">
<style> </style>
<h1 style=" ">Hello</h1>
Importance
color: red;
color: green !important;
www.appbrewery.com

```html
<!DOCTYPE html>
<html >
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scal
    <title>Internal Style</title>
    <link rel="stylesheet" href="./style.css">
    <img src=".//p.jpg" width="300">
</head>
<body>
        <h1 id="A01">Gulam Hazrat Popal Furqani</h1>
        <h1>welcome to popal learning setion</h1>
        <h4 class="popal">Mohammad Idress Furqani</h4>
        <h4>Husna Furqani</h4>
        <h4>Asma Furqani</h4>
        <p class="popal">Lorem ipsum dolor sit amet consectetur adi
        <ol class="popal">
            <li>HTML</li>
            <li>CSS</li>
            <li>JS</li>
            <li>Python</li>
        </ol>
</body>
</html>
```

```css
h4
{
    color: crimson;
    background-color: black;
    text-align: center;
}
body
{
    background-color: antiquewhite;
}
.popal
{
    color: white;
    background-color: blue;
}
#A01
{color: brown;}
```

```html
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1
    <title>CSS Selectors for Style</title>
    <link rel="stylesheet" href="./style.CSS">
</head>
<body>
    <div>
        <h1 class="test">My name is Gulam HAZRAT POPAL FURQANI</h1>
    </div>
    <h6 class="test">I am a software engineer</h6>

    <b class="test">Bold</b>
    <button id="bld1">Button</button>
    <p title="some text">
        Lorem ipsum dolor sit amet consectetur adipisicing elit.
    </p>
</body>
</html>
```

```css
h1{  /* tagname selector*/
    background-color: blueviolet;
    color: aliceblue;
    padding: 20px;
    text-align: center;
}
.test
{ /* by classValue selector*/
    color: blueviolet;
    background-color: #eee;
}
#bld1
{ /* by idValue selector*/
    color: rgb(248, 246, 247);
    background-color: rgb(226, 43, 174) !important;
}

[title]
{ /* by Attribute selector*/
    background-color: orange;
    color: #eee;
}

p, h4, h6, b
{ /* group selector*/
    color: rgb(248, 246, 247);
    background-color: rgb(226, 43, 174) !important;
}
```

CSS Color properties:

- **Color by Name,**
- **Colors by RGB**
- **Colors By Hexadecimal**
- **Colors by HSL (**Hue Saturation Lightness**)**

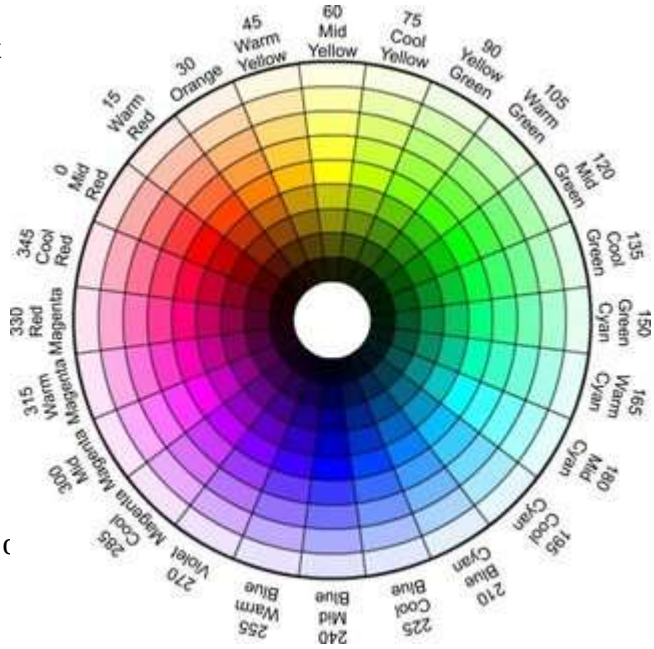
(https://colorhunt.co/palettes/popular)

Color by Name
Built-in Colors of browser
Ex: red, green, blue, yellow…

Colors by RGB

rgba (red, green, blue, alpha) (a is for to be brilliant
rgb(0-255, 0-255, 0-255)
If we chose all 255 in rgb the color will be white

- rgb(255,255,255) = white color
- rgb(0,0,0) = Black color
- rgb(255,0,0) = Red Color
- rgb(0,255,0) = green
- rgb(0,0,255) = blue
- Rgb(255,255,0) = Yellow color



Colors by Hexadecimal

Hexadecimal: is base of 16 alphanumeric value
Range of Hexadecimal: 0-9 and A-F
And color defined by hexadecimal are 6 characters (

- Ex: #FFFFFF or # 000000
- #RRGGBB, #RGB
- Lowest Value: 0
- Highest Value: F

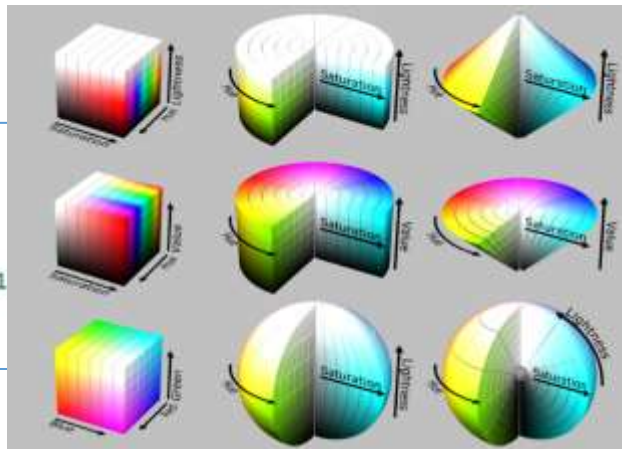
#FF0000 is red or we can #f00 is also the same red color and 00FF00 is green that is hexadecimal colors white #FFFFFF

Home Work

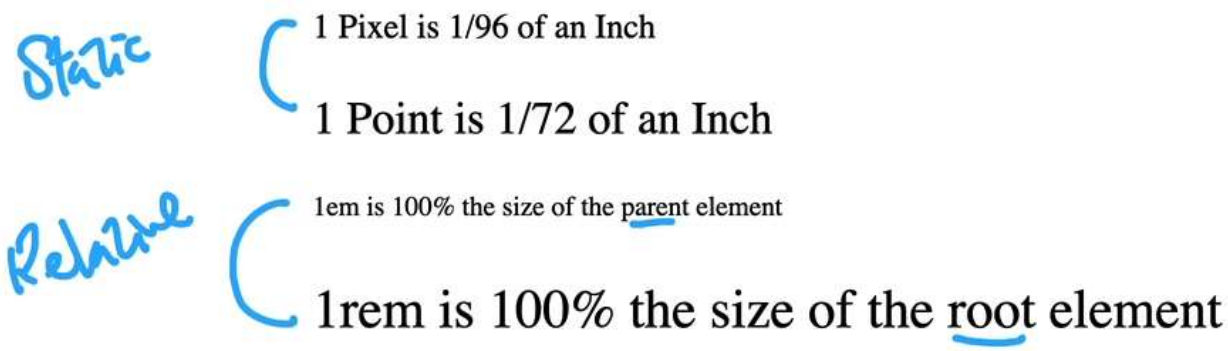
```css
.container
{
    display: grid;
    width: 900px;
    height: 450px;
    grid-template-columns:1fr 1fr 1fr;
}
.black
{
    width: 300px;
    height: 150px;
    background-color: #000;
}
.red
{
    width: 300px;
    height: 150px;
    background-color: #f00;
}
.green
{
    width: 300px;
    height: 150px;
    background-color: #0f0;
}
```

Colors by HSL (Hue Saturation Lightness**)**
hsl (0-360, 0-100%, 0-100%);
hsl(56, 80%, 50%);
Let's make cyan Color
180, 100%, 50%
Hue Saturation Lightness Alpha
hsla (0-360, 0-100%, 0-100%, 0.5);

```css
div p   /*means select child p*/
{
    width: 50%;
    height: 50%;
    background-color: rgba(155, 99, 201, 0.1
    border-radius: 10px;
}
```



Text-align: center, left, right, justify;
Sizing 1 Static Px, pt 2 Relative em , rem



Inspection or Developer Tools (Ctrl+Shift+I):
Here we add temporary style by clicking + sign and we bring changes in our CSS properties and values.

CSS Box Model

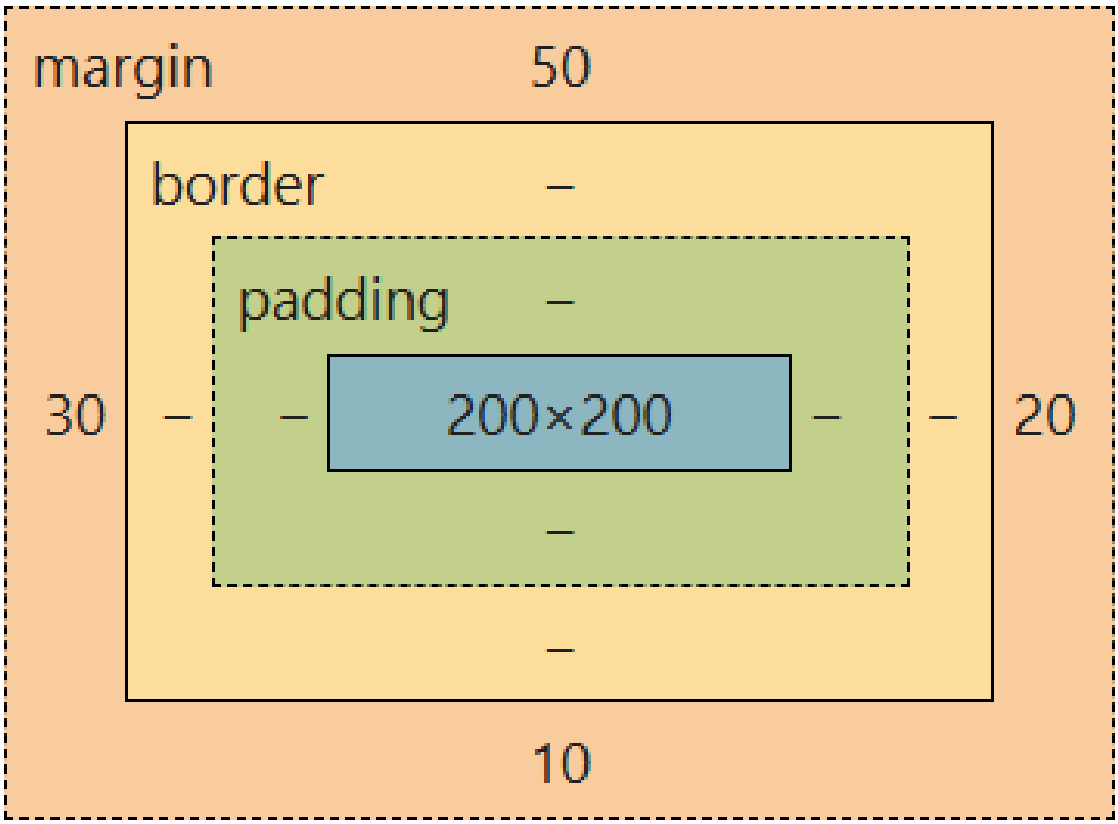
Background image

Margins:

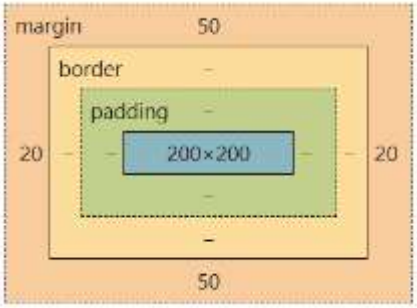
Margin Used for spacing between web elements create space outside the element

```
# style.CSS > ...
body
{
    /* used for cover or background image */
    background-image: url("it.png");
    /*background-size: 100%, 20%; */
    background-size: cover;
}

h1{
    background-color: rgba(138, 43, 226, 0.2);
    color: #EEEE;
    text-align: center;
    height: 200px;
    width: 200px;
    /* margin is used for spacing */
    margin-left: 30px;
    margin-top: 50px;
    margin-bottom: 10px;
    margin-right: 20px;
}
```

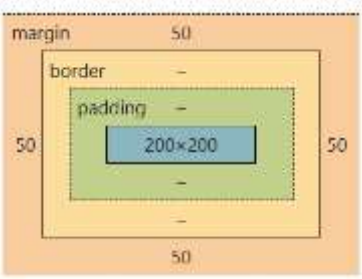
```
/* margin shortcut clock vise margin: top right bottom left ;*/
margin: 50px 20px 10Px 30px;
```




```
/* margin shortcut clock vise margin: top/bottom left/Right ;*/
margin: 50px 20px;
```




```
/* margin shortcut all sides same margin: valuepx ;*/
margin: 50px;
```



How to bring something to center equally

```
/* margin shortcut all sides same margin: valuepx ;*/
margin: 0px;
margin-left:auto ;
margin-right:auto;
```

Paddings:

Padding is the space between content and its border. It controls the inner spacing inside an element, pushing the content inward while maintaining the element's size or affecting its total dimensions depending on the `box-sizing` property

```
padding-top: 40px;
padding-left: 50px;
padding-right: 50px;
padding-bottom: 10px;

padding: 40px 50px 10px 50px; /*clock vise*/
```

Border:

A border that goes around the padding and content
The space between margin and padding is called border

```
border-width: 20px;
border-color: blueviolet;
border-style: ridge;
border-top-color: aqua;
border-left-style: dashed;
/* border: width style color*/
border: 20px solid black;
border-radius:50%;
```

Border-radius: 50%; make a complete circle

```
/* border-radius: topleft topRight BottomRight BottomLeft; */
border-radius: 20px 40px 60px 80px;
/* border-radius: topleft/BottomRight   BottomLeft/topRight; */
border-radius: 0 50%;
```

- border-width
- border-color
- **border-style**
  - ❖ **dotted** – defines a dotted border

- **dashed** – defines a dashed border
- **solid** – defines a solid border
- **double** – defines a double border
- **groove** – defines a 3D groove border. the effect depends on the border-color value
- **ridge** – defines a 3D ridge border. the effect depends on the border-color value
- **inset** – defines a 3D inset border. the effect depends on the border-color value
- **outset** – defines 3D outset border. the effect depends on the border-color value
- **none** – defines a no border
- **hidden** – defines a hidden border

Background:

The CSS background d properties are used to add background effects for elements.

- background-color
- background-image
- background-repeat (in case of small image to repeat or not)
- background-attachment (scroll image with text or not local/fixed)
- background-position (where to hold our background image)
- background (shorthand property) all in one
- **background: color image repeat attachment position/size;**

```
body
{
    /* background-color: hsl(326, 82%, 68%);
    background-image: url(./p.jpg);
    background-repeat:no-repeat;
    background-attachment:fixed;
    background-position: right;
    background-size: cover; */

    background: hsl(326, 82%, 68%) url(./p.jpg) no-repeat fixed right/cover ;
}
```

```css
body
{
    background-image: url(./it.png);
    background-size: cover;
    background-repeat: no-repeat;
    background-attachment: fixed;
    margin: 0;
}
h1{
    color: white;
    /* background-image: linear-gradient( black, red, green, yellow); */
    /* background-image: linear-gradient(to right, black, red, green, yellow); */
    background-image: linear-gradient(1deg, black, red, green, yellow);

    margin: 0;
    padding: 30px;
}
div
{
    color: white;
    padding: 20px;
    height: 500px;
}
div:nth-child(odd)
{
    background-color: rgba(255, 0, 0, 0.5);
}
div:nth-child(even)
{
    background-color: rgba(0, 255, 0, 0.5);
}
```

Linear – gradient:

Works in background image used to mix colors for background.

```css
h1{
    color: white;
    /* background-image: linear-gradient( black, red, green, yellow); */
    /* background-image: linear-gradient(to right, black, red, green, yellow); */
    /* background-image: linear-gradient(1deg, black, red, green, yellow); */
    background-image: linear-gradient(to right, black 30% 30%, red 30% 67%, green 10%);

    margin: 0;
    padding: 30px;
    line-height: 200px;
    text-align: center;
}
```

```
div {
    width: 800px;
    height: 600px;
    background: conic-gradient(
      red, orange, yellow, green, cyan, blue, indigo, violet,
      red, orange, yellow, green, cyan, blue, indigo, violet,
      red, orange, yellow, green, cyan, blue, indigo, violet,
      red, orange, yellow, green, cyan, blue, indigo, violet,
      red, orange, yellow, green, cyan, blue, indigo, violet
    );
    border-radius: 50%;
  }
```

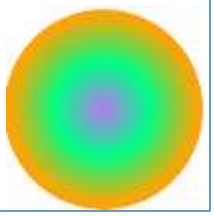
```
button
{
padding: 14px 30px;
margin: 30px;
border: 2px solid black;
background-color: transparent;
transition: 300ms;
border-radius: 10px;
background-image: linear-gradient(rgba(1, 146, 1, 0.5) , rgb(255, 255, 0, 0.4), rgb(0, 0, 255, 0.5));
}
button:hover
{
background-color: violet;
color: black;
border-radius: 5px;
}
```

With flag colors

Radial Gradient:

Used to mix color for background of elements. Which starts from center like a circle.

```
div:nth-child(odd)
{
    background-image: radial-gradient(circle,#b878f1, #00ff88, orange, hotpink);
    border-radius: 60%;
    width: 500px;
    margin: auto;
}
div:nth-child(even)
{
    background-color: rgba(0, 255, 0, 0.5);
}
```



Conic Gradient:

```
div:nth-child(even)
{                                   /* { 1 is all circle */
    background-image: conic-gradient(#64ede6 0turn 0.25turn, hotpink 0.25turn 0.5turn, #6ada4f 0.5turn 0.75turn, orange 0.75turn 1turn );
    width: 400px;
    height: 400px;
    margin: auto;
    border-radius: 50%;
}
```

```
{                                   /* { 1 is all circle */
    background-image: conic-gradient(#64ede6 0turn 0.25turn,  pink 0.25turn 0.5turn, #6ada4f 0.5turn 0.75turn,
    width: 400px;
    height: 400px;
    margin: auto;
    border-radius: 50%;
    background-size: 25%,25%;
}
```

```
div:nth-child(even)
{                                   /* { 1 is all circle */
    background-image: conic-gradient(#64ede6 0turn 0.25turn,  pink 0.25turn 0.5turn, #6ada4f 0.5turn 0.75turn, blue 0.75turn 1turn );
    width: 400px;
    height: 400px;
    margin: auto;
    border-radius: 50%;
    background-size: 25%,25%;

}
div:hover{
    background-image: conic-gradient(#64ede6 0turn 0.25turn,  pink 0.25turn 0.5turn, #6ada4f 0.5turn 0.75turn, blue 0.75turn 1turn );
    width: 400px;
    height: 400px;
    margin: auto;
    border-radius: 50%;
    background-size: 100%,100%;
    transition: 1000ms;
}
```

- background-color: colors definition;
- background-image: url("path");
- background-repeat: (default repeat) no-repeat, repeat-x, repeat-y;
- bachground-size: xpostionvalue ypostionvalue; px %
    - : same onevalue for both x and y;
    - 100px 100px
    - 100px;
    - 100%100%
    - 100%
- background-position:
    - (default top left)
    - top center,
    - top right,
    - center left,
    - canter right,
    - bottom left,
    - bottom center,
    - bottom right,
- background-attachment: (default scroll),fixed
- background-image: linear-gradient (position, color, color, color…);

- background-image: linear-gradiend(optional or it is default to bottom , color color_pos,color
- position = to top , to left, to right, to bottom;
- position = 90deg;
- background-image: radial-gradient (shap.color,color,…);
- shap = ellipse, circle;
- shap = y * 50% 50%;
- border-width
- border-color
- border-style
- dotted- defines a dotted border
- dashed – defines a dashed border
- solid – defines a solid border
- double – define a double border
- groove – a 3d grooved border. the effect depends on the border-color value
- ridge – a 3d ridge border. the effect depends on the border-color value

- inset- a 3d inset border. the effect depends on the border-color value
- outset- a 3d outset border. the effect depends on the border-color value
- none – defines no border
- hidden – defines a hidden border
- border-sides
- border-top-style: dotted;
- border-right-style: solid;
- Border-bottom-style: dotted;
- Border-left-style: solid;
- border: 5px solid red;
- border-radius: units;

Text:

- the direction properties can be used to change the text direction of an element
- **direction:** rtl, ltr
- the vertical-align property sets the vertical alignment of an element
- **vertical-align:** baseline, text-top, text-bottom, sub, and super
- Text-decoration property is used to set or remove decorations from the text.
- **text-decoration:** overline, underline, line-through
- The text-transform property is used to specify uppercase and lowercase letters in a text.
- **text-transform:** uppercase, lowercase, capitalize
- the text-indent property is used to specify the indention of the first line of a text:
- **text-ident:** give space to the first line of paragraph.
- **letter-spacing** : give space between each letter

- **line-height** : give space between each line
- **word-spacing** : give space between each word
- white-space:
- **text-shadow:** xp yp blur red; = x y blur color : give shadow the text
- **box-shadow:** xp yp blur red; = x y blur color : give shadow the box

Overflow: hidden, scroll, visible, auto;

```css
p
{
    direction: ltr;
    /* vertical-align: super; */
    vertical-align: bottom;
    text-align: center;
    text-decoration: dashed;
    text-transform: capitalize;
    text-transform: uppercase;
    text-indent: 20px; /*is like space for html we can use   for space */
    letter-spacing: 10px;  /* give space between each letter */
    line-height: 20 px; /* give space between each line */
    word-spacing: 20 px; /* give space between each word */
}
```

```css
div.box
{
    width: 400px;
    height: 400px;
    background-color: #eeee;
    letter-spacing: 3px;
    text-align: justify;
    padding: 15px;
    /* text-shadow: x y blur color; */
    text-shadow: 2px 2px 5px rgb(206, 30, 139);
    /* box-shadow: x y blur spreed color; */
    box-shadow: 2px 2px 20px rgb(206, 30, 139) ;
    border-radius: 10px;

}
```

Lorem, ipsum dolor sit amet consectetur adipisicing elit. Velit cupiditate nemo odio nam labore, ab repudiandae perferendis exercitationem repellendus suscipit quisquam quas saepe adipisci dolorem iusto tempora autem voluptate amet? Lorem, ipsum dolor sit amet consectetur adipisicing elit. Velit cupiditate nemo odio nam labore, ab repudiandae perferendis exercitationem repellendus suscipit quisquam quas saepe adipisci dolorem iusto tempora autem voluptate amet? Lorem, ipsum dolor sit amet consectetur adipisicing elit. Velit cupiditate nemo odio nam labore, ab repudiandae perferendis exercitationem repellendus suscipit quisquam quas saepe adipisci dolorem iusto tempora autem voluptate amet?.

```css
width: 400px;
height: 400px;
background-color: #eeee;
letter-spacing: 5px;
text-align: justify;
padding: 15px;

text-shadow: 2px 2px 5px rgb(206, 30, 139);
/* box-shadow: x y blur spreed color; */
box-shadow: 2px 2px 20px rgb(206, 30, 139) ;
border-radius: 10px;
/* overflow: hidden;  اضافه خط پنوی */
overflow: scroll;  /* give scoll bar to our text box*/
```

Fonts

- In CSS, we is the font-family property to specify the font.
- font-family: chose a font name
- font-style: we can italic style for font
- font-weight is used for bolding
- font variant: small-caps
- font-size: is for the size of font
- connect with google fonts
- `<link rel="stylesheet" href="https://fonts.googleapis.com/css?family=Sofia">`

```html
<head>
<link rel="stylesheet" href="https://fonts.googleapis.com/css?family=Sofia">
<style>
body {
  font-family: "Sofia", sans-serif;
}
</style>
</head>
```

```
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <link rel="stylesheet" href="Style.css">
    <link rel="preconnect" href="https://fonts.googleapis.com">
<link rel="preconnect" href="https://fonts.gstatic.com" crossorigin>
<link href="https://fonts.googleapis.com/css2?family=Big+Shoulders+Inline:ops
    <title>Document</title>
```

```
    font-family: "Big Shoulders Inline", sans-ser
    font-size: 25px;      /* font size*/
    font-style: italic;   /* italic, normal*/
    font-weight: 600;     /* bold*/


}
```

- display: none;
- visibility: hidden, visible;
- cursor: pointer;
- fontawsome

Pseudo selector for link:

- :visited
- :link
- :active
- **Display: none;** **we can hide an element with it**
- **: hover:** **when we hover the mouse changes can be applied.**
- **Cursor: pointer;** **we can use pointer hand sign when hover on an element**

button: hover:

- padding: 20px;
- border-radius: 10px
- background-color: brown:

- **Visited** **change color of anchor tag after visit**
- **link;** **change color before visits**
- **active:** **when we click on it, we can apply to button as well**
- **hover:** **when we hover mouse we can apply to everything**

Box-shadow: x y blur spread color;

- min-width
- max-width
- min-height
- max-height

Display: block; converts inline element to bock
Display: inline; coverts block element to inline
White-space: nowrap; continues text …
White-space: pre;

Table styling:

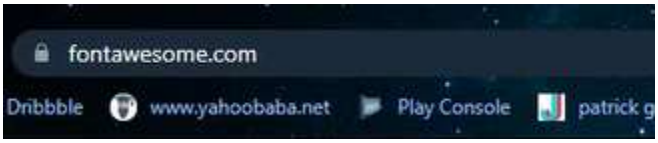
- border: 2px solid red; border: Size Style Color;
- border-collapse : collapse; Joins thead, tbody, tr and td borders
- border-spacing : 5px; give space between borders of td and ..
- caption-side : bottom; change the position of table caption.
- empty-cells : hide; hide empty cells

```
# style.css > ...
1   table, thead, tbody, th, td
2   {
3       border: 2px solid red;
4       border-spacing: 5px;
5       caption-side: bottom;
6       border-collapse: collapse;
7   }
8   td,th
9   {
10      padding: 10px;
11      text-align: center;
12      font-weight: 600;
13      empty-cells: hide;      /* merge empty  cells */
14      table-layout: fixed;  /* text will fixed with each cell */
15  }
```

List styling:

- **List-style-type** change bullets to other types as greek…
- list-style-position moves out of margin area
- list-style-image put image as bullet style
- list-style: Georgian inside ur1() list-style: type position image

```
ul, li
{
    /* list-style-type:georgian;
    list-style-position: inside;
    list-style-image: url(./a.jpg); */
    list-style: georgian inside url(./a.jpg);
    list-style: none;
}
```



Position:

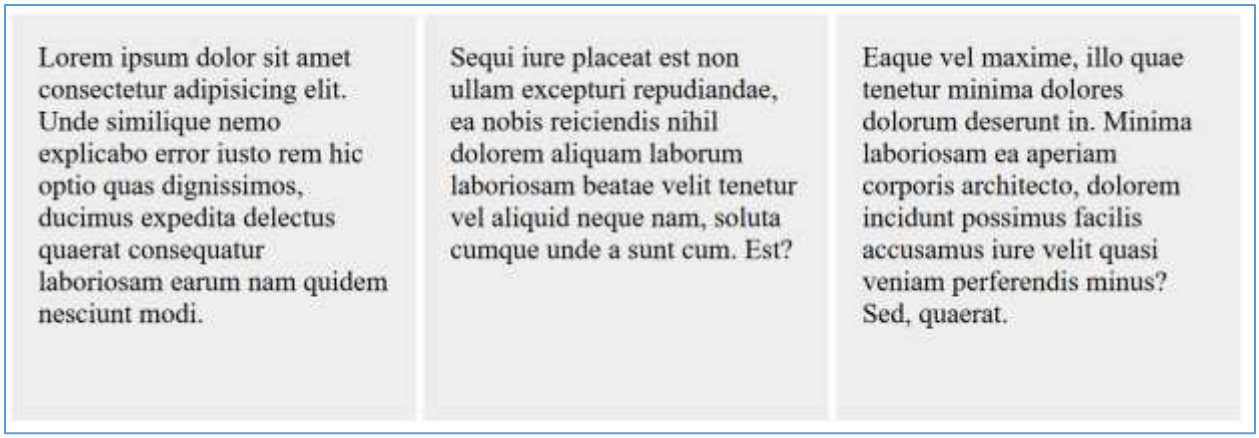
Is used where to place an element on the page

- Static: is default value for position.
- Relative: positioned relative to its normal position. Top, right, left, bottom to shift it.
- Absolute Put it exactly where you want
- Fixed Always stay in the same place (no scroll)
- Sticky Stick when scrolling (no scroll)

Position	Based On	Scrolls with Page	Affects Layout
static	Normal flow	✅ Yes	✅ Yes
relative	Normal flow	✅ Yes	✅ Yes
absolute	Nearest positioned parent	❌ No	❌ No
fixed	Viewport	❌ No	❌ No
sticky	Scroll position	✅ Sometimes	✅ Yes

بغیر له ستیټک څخه نور ټوله څلور فوزیشن لری له لیفټ ، رایټ ، ټاپ او باټم که وغواړو چې درپراګرافونه څنګ پر څنګ راولو نو هر پراګراپ ته جلا جلا کلاس ورکوو مثلا p1, p2 p3 بیا هرپرګراپ ته په لاندی شکل سره پوزیشن ورکوو Fixed د نیوبر لپاره استعمالیږی چی د صفحی نور content دی scroll سی خو نیوبار دی ولاړ وی او ندی سکرال کیږی.

```
.p2
{

position: fixed;  /* one element cant hold place of other */
left: 242px;
top: 0px;
}
.p3
{
    position: fixed;  /* one element cant hold place of other */
left: 476px;
top: 0px;
}
```

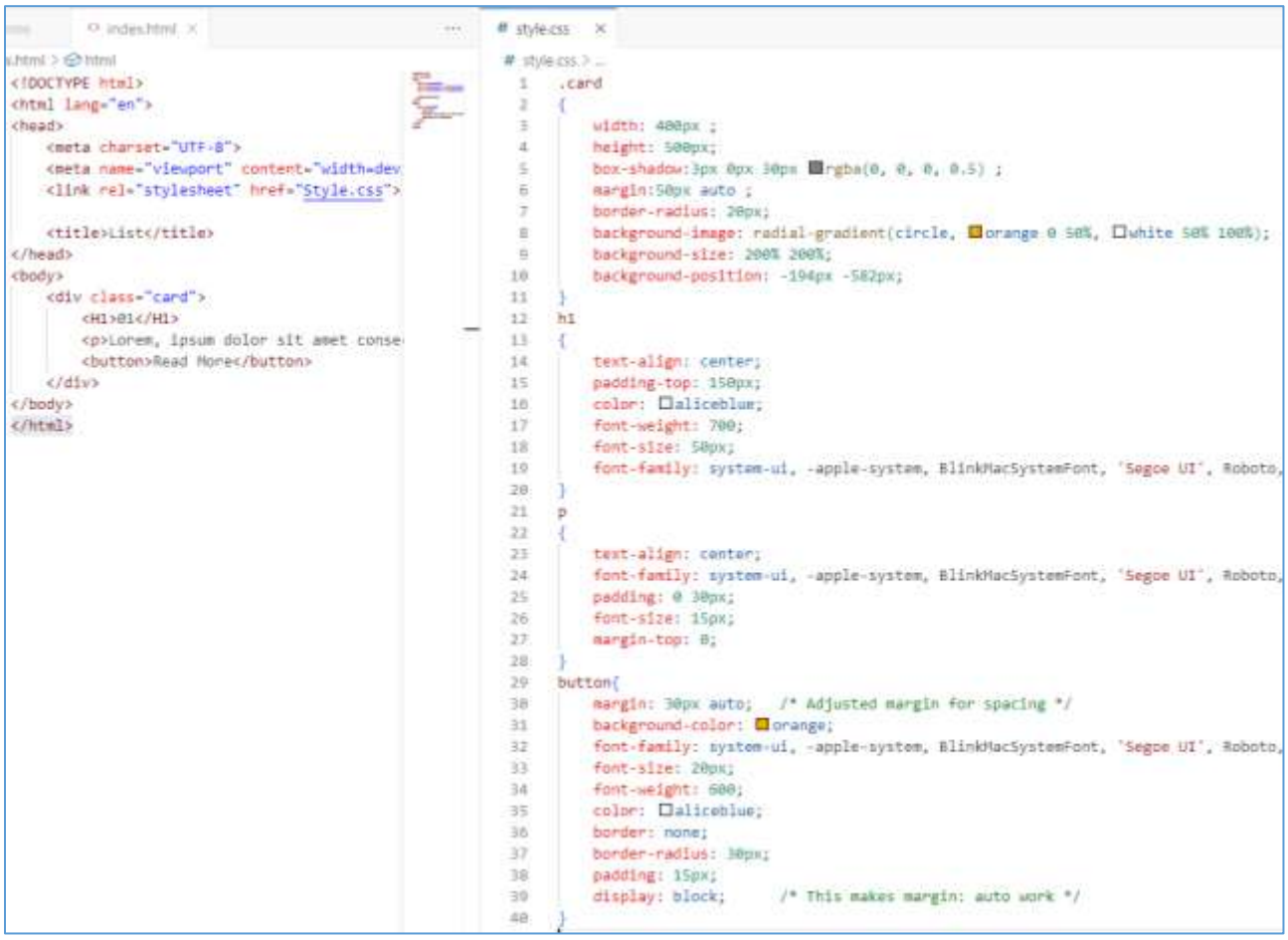


Make Navigation bars Top, Left and Right


```
.sticky
{
    position: sticky;
    top: 100px;
    background-color: #71c85e;
}
```

چی کله پراگراپ په سکرال کی ۱۰۰ فکسل ته ورسیږی نو هلته دی ودریږی.

Homework make this kind of card

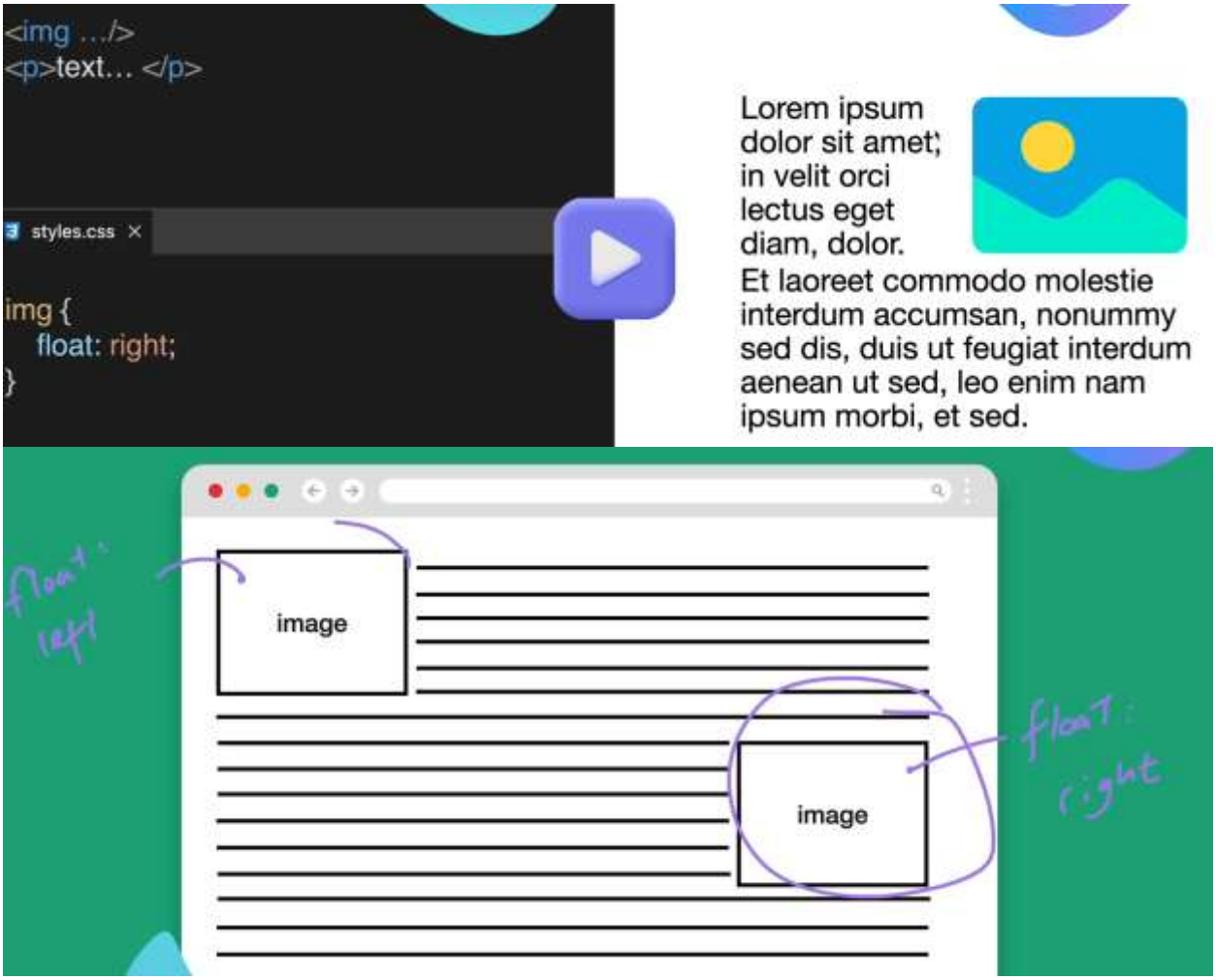


```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=dev
    <link rel="stylesheet" href="Style.css">

    <title>List</title>
</head>
<body>
    <div class="card">
        <H1>01</H1>
        <p>Lorem, ipsum dolor sit amet conse
        <button>Read More</button>
    </div>
</body>
</html>
```

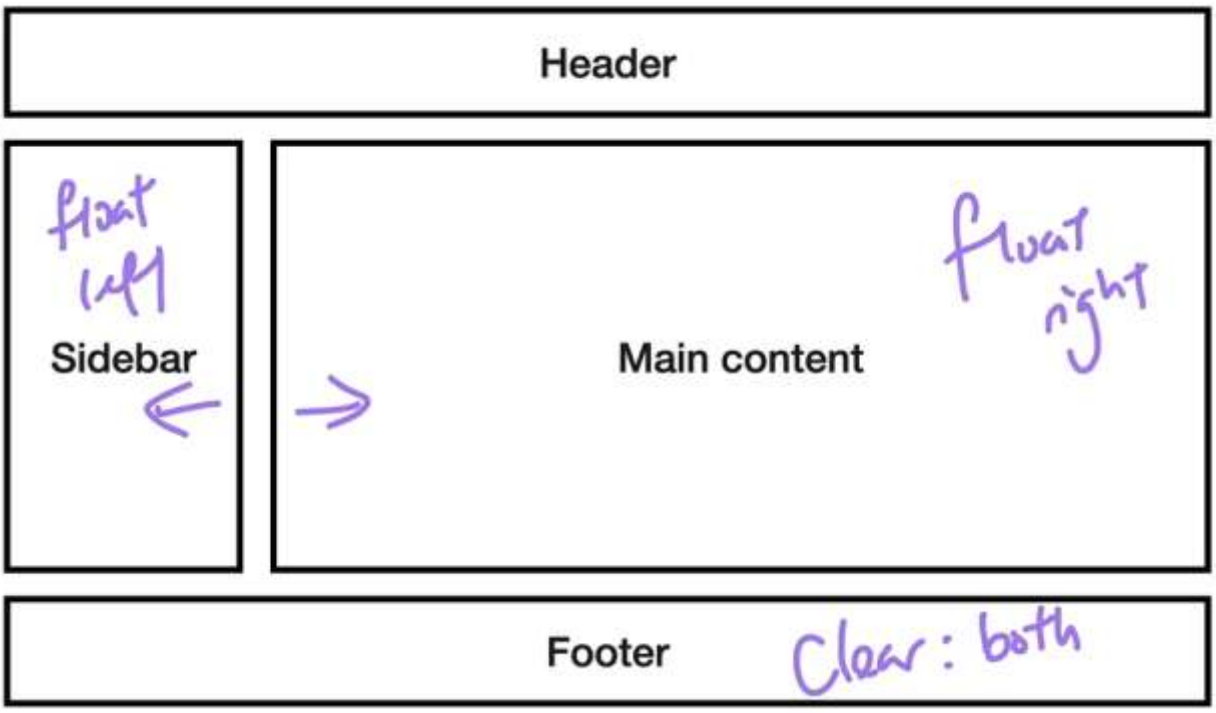
```
.card
{
    width: 400px ;
    height: 500px;
    box-shadow:3px 0px 30px rgba(0, 0, 0, 0.5) ;
    margin:50px auto ;
    border-radius: 20px;
    background-image: radial-gradient(circle, orange 0 50%, white 50% 100%);
    background-size: 200% 200%;
    background-position: -194px -582px;
}
h1
{
    text-align: center;
    padding-top: 150px;
    color: aliceblue;
    font-weight: 700;
    font-size: 50px;
    font-family: system-ui, -apple-system, BlinkMacSystemFont, 'Segoe UI', Roboto,
}
p
{
    text-align: center;
    font-family: system-ui, -apple-system, BlinkMacSystemFont, 'Segoe UI', Roboto,
    padding: 0 30px;
    font-size: 15px;
    margin-top: 0;
}
button{
    margin: 30px auto;   /* Adjusted margin for spacing */
    background-color: orange;
    font-family: system-ui, -apple-system, BlinkMacSystemFont, 'Segoe UI', Roboto,
    font-size: 20px;
    font-weight: 600;
    color: aliceblue;
    border: none;
    border-radius: 30px;
    padding: 15px;
    display: block;      /* This makes margin: auto work */
}
```

Float

Float property is used to position an element to the left or right within its container.



text...
styles.css
img {
float: right;
}
Lorem ipsum dolor sit amet; in velit orci lectus eget diam, dolor.
Et laoreet commodo molestie interdum accumsan, nonummy sed dis, duis ut feugiat interdum aenean ut sed, leo enim nam ipsum morbi, et sed.
image
image



style.css > .clearfix

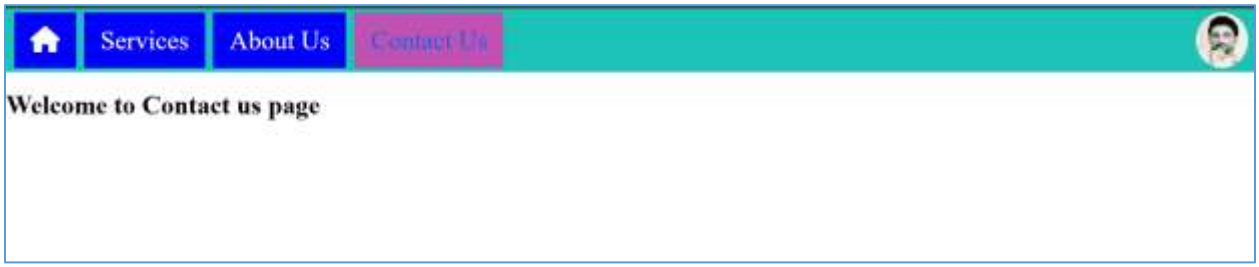
```css
body{margin: 0;}

.box{
    width: 400px;
    height: 400px;
    background-color: hotpink;
    margin: 5px 0;
}
.Left
{
    float: left;   /*align block elements with float using clear property */
}
.Right
{
    float: right;
}
.clearfix
{
    clear: both;
}
```

```html
<div>
    <div class="box Left">Left</div>
    <div class="box Right">Right</div>
    <div class="clearfix"></div>
    <p>this is a sample text here</p>
</div>
```

float: Left; float: Right; clear : both;

که چیری موږ غواړو چی یو بلاک المنټ کلیر کړو نو تر هغه وروسته باید یو خالی امنټ ولرو یعنی دا پراګراپ که وغواړو چی لاندی راسی نو ددی لپاره موږ پرهغه خالی ډیو باندی کلیر بوت لګوو چی راسته او چپه طرف بکسونه دواړه راته سم کړی. او که یو بکس ولرو نو دهغه لپاره صرف د کلیر لیفټ یا رایټ څخه کار اخلو. او لاندی یی عملی درس دی.

Let make a Nav Bar as below:



د هریوه لپاره جلا جلا html صفحه جوړو او د CSS سره لینک ورکوو یعنی Home, Services, About us and Contact us لپاره همداسی صفحی جوړو چې دا بټنان په هره صفحه کی راته ښکاره سی.

index.html

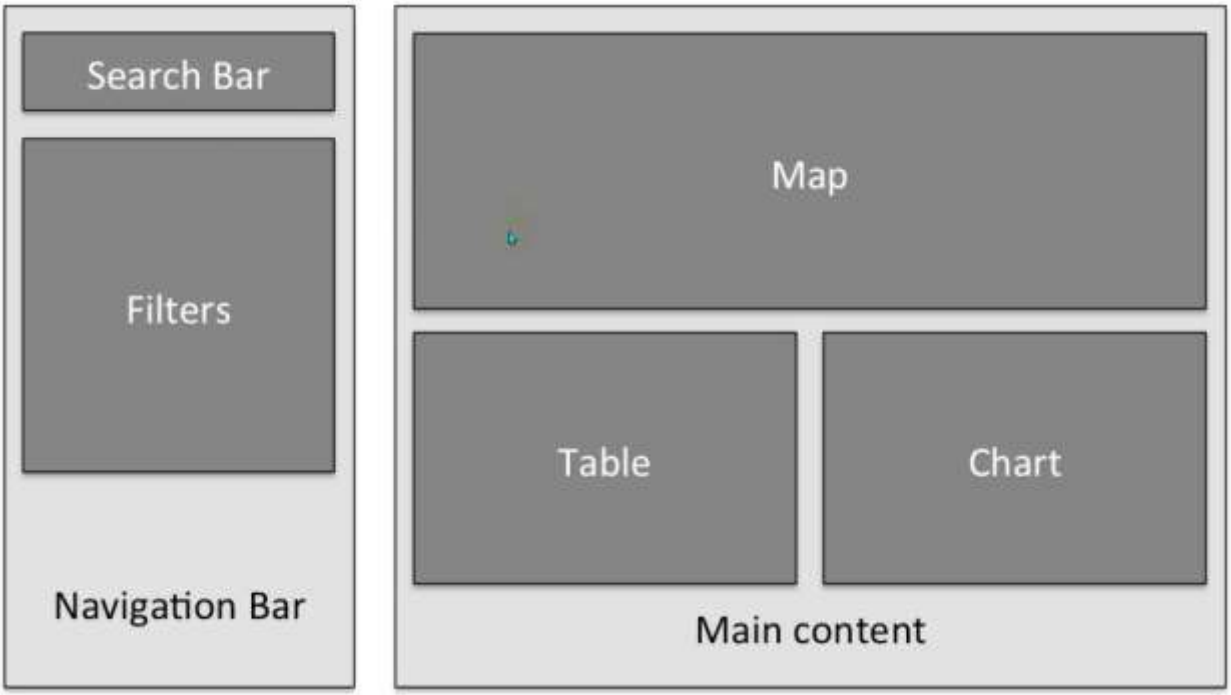
```html
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <link rel="stylesheet" href="Style.css">

    <title>Float</title>
</head>
<body>
    <header>
        <Nav>
          <ul>
           <li class="f-left"><a class="active" href="./index.html"><i class="fa fa-1x fa
           <li class="f-left"><a href="./services.html">Services</a></li>
           <li class="f-left"><a href="./aboutus.html">About Us</a></li>
           <li class="f-left"><a href="./contactus.html">Contact Us</a></li>
           <li class="f-right"><a class="logo"  href="#">   </a></li>
           <div class="c-left"></div>
          </ul>
        </Nav>
    </header>
        <h1>Welcome to Home Page</h1>
</body>
</html>
```

services.html

```html
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <link rel="stylesheet" href="style.css">
    <title>Document</title>
</head>
<body>
    <header>
        <Nav>
          <ul>
           <li class="f-left"><a href="./index.html"><i class="fa fa-1x fa-ho
           <li class="f-left"><a class="active" href="./services.html">Servic
           <li class="f-left"><a href="./aboutus.html">About Us</a></li>
           <li class="f-left"><a href="./contactus.html">Contact Us</a></li>
           <li class="f-right"><a class="logo"  href="#">   </a>
           <div class="c-left"></div>
          </ul>
        </Nav>
    </header>
    <h1>Welcome to Services page</h1>
</body>
</html>
```

```html
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <link rel="stylesheet" href="style.css">
    <title>Document</title>
</head>
<body>
    <header>
        <Nav>
          <ul>
           <li class="f-left"><a href="./index.html"><i class="fa fa-1x fa-hom
           <li class="f-left"><a href="./services.html">Services</a></li>
           <li class="f-left"><a href="./aboutus.html">About Us</a></li>
           <li class="f-left"><a class="active" href="./contactus.html">Contac
           <li class="f-right"><a class="logo"  href="#">   </a></
           <div class="c-left"></div>
          </ul>
        </Nav>
    </header>
    <h1>Welcome to Contact us page</h1>
</body>
</html>
```

```css
@import url('./fontawesome/css/all.css');
body{margin: 0;}
header nav ul li a
{
    text-decoration: none;
    color: white;
    background-color: blue;
    padding: 15px 20px;
    display: inline-block;
    transition: 300ms;
    font-size: 32px;
}
header nav ul li a:hover
{
    background-color: rgba(98, 98, 202, 0.644);
}
header nav ul li
{
    margin: 0% 5px;

}
header nav ul
{
    list-style: none;
    padding: 0;
    margin: 0;
}
header nav
{
    background-color: rgb(27, 196, 182);
    padding: 5px;
}
.f-left
{
    float: left;
}
.f-right
{
    float: right;

}
.c-left
{clear: left;}

.active
{
    background-color: rgb(192, 82, 179);
    color: rgb(11, 104, 226);

}
.logo
{
    background-image: url('./p.jpg');
    background-size: 80px 70px;
    background-position: -7px;
    border-radius: 50%;
}
```



Selectors

```
descendant selector (space)
child selector (>)
adjacent sibling selector (+)
general sibling selector (~)
```

```
.space-selector
{
    background-color: blue;
    padding: 10px;
}
.space-selector div
{
    background-color: green;
    padding: 10px;
}
.space-selector p
{
    background-color: red;
}
```

```
<body>
    <div class="space-selector">
        <div>
            <p>Lorem ipsum dolor sit amet.</p>
        </div>
    </div>
```

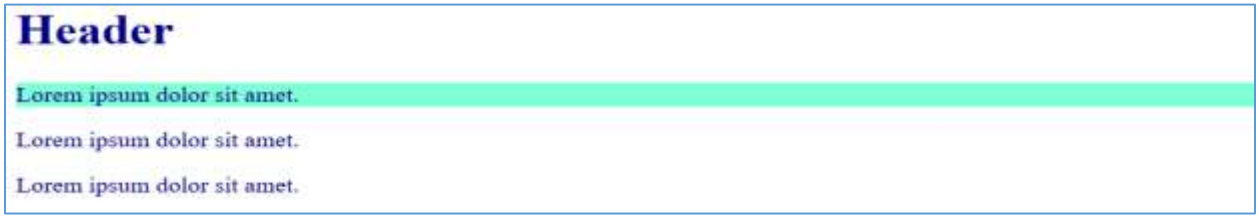
Space means inside this class there is a child paragraph could be direct child or nested as above in between two dives. Will select all in case we have too many paragraphs.
We can select it with direct child selector > as below

```css
.direct-child-selector
{
    background-color: rgb(181, 212, 58);
    padding: 10px;
}
.direct-child-selector > div
{
    background-color: rgb(36, 23, 217);
    padding: 10px;
}
.direct-child-selector >div> p
{
    background-color: rgb(210, 175, 88);
    padding: 10px;
}
```

Adjacent-sibling: means beside h1 there is a paragraph select it+ will only select the neighbor paragraph

```css
.adjacent-sibling
{
    color: darkblue;
}
.adjacent-sibling >h1 +p
{
    background-color: aquamarine;
}
```

```html
<div class="adjacent-sibling">
    <h1>Header</h1>
    <p>Lorem ipsum dolor sit amet.</p>
    <div>
        <p>Lorem ipsum dolor sit amet.</p>
    </div>
    <p>Lorem ipsum dolor sit amet.</p>
</div>
```



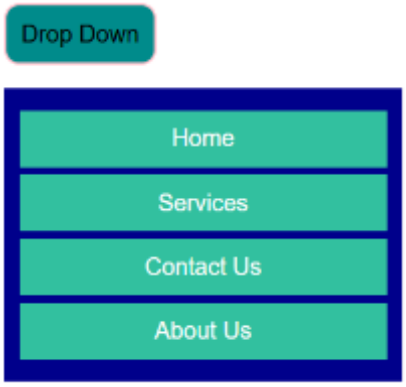
General-sibling: means below h1 select all direct children paragraphs and wont select nested one

```css
.general-sibling
{
    color: brown;
}
.general-sibling>h1 ~ p
{
    background-color: rgb(167, 109, 224);
}
```

```html
<div class="general-sibling">
    <h1>Header</h1>
    <p>Lorem ipsum dolor sit amet.</p>
    <div>
        <p>Lorem ipsum dolor sit amet.</p>
    </div>
    <p>Lorem ipsum dolor sit amet.</p>
</div>
```

Where we can use these selectors practically?

```css
.drop
{
    color: black ;
    font-family: Arial, sans-serif;
    font-weight: lighter;
}
input[type=checkbox]
{
    display: none;
}
input[type=checkbox]:checked+ nav
{
    height: fit-content;
}

nav
{
```




```
<body class="drop">
    <label for="dropDown">Drop Down</label>
    <input type="checkbox" id="dropDown">
    <nav>
        <ul>
            <li><a href="#">Home</a></li>
            <li><a href="#">Services</a></li>
            <li><a href="#">Contact Us</a></li>
            <li><a href="#">About Us</a></li>
        </ul>
    </nav>
</body>
```

```
label
{
    background-color: darkcyan;
    padding: 10px;
    display: inline-block;
    border: pink solid 2px;
    border-radius: 10px;
}
```

- : :before
- : :after
- : : first-letter
- : :first-line
- : :selector
- : :marker
- counter-reset-creates or resets a counter
- counter-increment – increments a counter value
- content-inserts generated content
- counter() or function – adds the value of a counter to an element

```css
.box
{
    width: 500px;
    height: 500px;
    background-color: lightblue;
    text-align: center;
}
.box::before
{
    content: "Before Psudo element";
    background-color: rgb(247, 88, 207);
    text-align: center;
    padding: 20px;
    display: block;
}
.box::after
{
    content: "after Psudo element";
    background-color: rgb(247, 88, 207);
    text-align: center;
    padding: 20px;
    display: block;
}
li{
    counter-increment: item;
}
li::after
{
    content: ":" counter(item);
}
```

How to attach a googled font into our css file,

1. Download font
2. In our CSS file we write,

@font-face{
Font-family: fontName;
Src = "./fontName"}

3 Use it with

Font-family: fontName;

- input : checked:
- input : disabled
- p : empty
- input : enabled
- p : first – child
- p : first-of-type
- input : focus
- a : hover
- input : in-range
- input :out-of-range
- input: invalid
- p : long(it)
- p : last-child

- p : last-of-type
- p: link
- : not(p)
- p:nth-child(2)
- p;nth-last-child(2)
- p: nth- last-type(2)
- p: nth-of-type(2)
- p: only-of-type
- p:only-child
- input: optional
- input: read-only
- input:-read-write
- input:-required
- input: target
- :root
- input: valid

Pseudo Selectors or classes:

Pseudo-class	Examples	Description
`:hover`	**`a:hover`**	Selects an element when the mouse hovers over it
`:focus`	**`input:focus`**	Selects an element when it gains focus (e.g., clicked or tabbed into)
`:active`	**`a:active`**	Selects an active (clicked) link
`:visited`	**`a:visited`**	Selects links that have been visited
`:link`	**`a:link`**	Selects unvisited links
`:first-child`	`p:first-child`	Selects the first child of a parent element
`:last-child`	`p:last-child`	Selects the last child of a parent element
`:nth-child(n)`	`p:nth-child(2)`	Selects the nth child of a parent element
`:not()`	`:not(p)`	Selects all elements except the specified one
`:checked`	`input:checked`	Selects checked checkboxes or radio buttons
`:disabled`	`input:disabled`	Selects disabled form elements
`:enabled`	`input:enabled`	Selects enabled form elements
`:required`	`input:required`	Selects form inputs with the `required` attribute
`:valid`	`input:valid`	Selects form inputs with valid values
`:invalid`	`input:invalid`	Selects form inputs with invalid values
`:first-of-type`	`p:first-of-type`	Selects the first occurrence of an element type within its parent
`:last-of-type`	`p:last-of-type`	Selects the last occurrence of an element type within its parent

```css
input:disabled
{ /*apply style to input when disabled*/
    background-color: blueviolet;
    width: 400px;
}
:empty
 {  /*apply style to those elements which are empty*/
    background-color: red;
    height: 20px;
}
* :not(input)
{ /*select all elements except input or body or any element we want*/
    background-color: rgb(12, 12, 13);
    color: white;
}
:enabled
{ /*apply style to input when enabled*/
    background-color: aquamarine;
}
```

Border image:

- border-image-source specifies the path to image to be used as a border
- border-image-slice specifies how to slice the border image
- border-image-width specifies the widths of the border image
- border-image-outset specifies the amount by which the border image area expends beyond the book
- border-image-repeat

```css
.box
{
    width: 400px;
    height: 400px;
    background-color: deepskyblue;
    border: 30px solid red;
    margin: 0 ;
    border-image-source: url(./css.png) ;
    background-clip: padding-box;
    border-image-outset: 1px;
     border-image-repeat: round ;
    border-image-slice: 80;
    /*border-radius: 20px; */
    border-image-source: url(./css.png) 10 fill / 30px  round;
}
```

Transition:

Makes changes happen **slowly and smoothly**, not instantly.

- **transition-delay** Waits *before* starting the transition.
- **transition-duration** Controls *how long* the transition takes.
- **transition-property** Says *which CSS property* should animate

- **transition-timing-function** Controls *the speed pattern* of the animation (how it speeds up or slows down).
  - transition-timing-function: ease; /* Starts slow, speeds up, slows down */
  - transition-timing-function: linear; /* Same speed all the way */
  - transition-timing-function: ease-in; /* Starts slow */
  - transition-timing-function: easeae-out; /* Ends slow */
  - transition-timing-function: ease-in-out;/* Slow start and end */
- **transition: property duration timing-function delay;**
- ease- specifies a transition effect with a slow start, then fast, then end slowly (this is default)
- linear- specifies a transition effect with the same speed from start to end
- ease-in- specifies a transition effect with a slow start
- ease-out- specifies a transition effect with slow end
- ease-in-out- specifies a transition effect with a slow start and end
- cubic-Bezier(n,n,n,n)- lets you define your own values in a cubic-Bezier function

```css
.box
{
    width: 400px;
    height: 400px;
    background-color: lightgreen;
    margin: 0 auto;
    transition: 300ms;
    transition-duration: 1s; /*it is the same as transition*/
    transition-delay: 1s; /*will apply transition after 1s when hoverd*/
    transition: all 1s ease-in-out 1000ms;  /*property, duration, timing-function,  delay */
}
.box:hover
{
        background-color: lightcoral;
        border-radius: 50%;
}
```

Animation

@keyframes

Animation: An animation lets an element gradually change from one style to another

Property	Description
animation-name	**Name of the `@keyframes` to use**
animation-duration	**How long the animation takes (e.g. `2s`)**
animation-delay	**Wait time before animation starts**
animation-iteration-count	**How many times to run (e.g. `1`, `infinite`)**
animation-direction	**Direction of the animation (`normal`, `reverse`, `alternate`)**
animation-timing-function	**Speed curve of the animation (`ease`, `linear`, etc.)**
animation-fill-mode	**Defines what styles are applied before/after animation (`forwards`, `backwards`, etc.)**
animation	**Shorthand name duration timing-func delay iteration-count dir fillmode**
	animation: slideIn 2s ease-in 0.5s infinite alternate forwards;

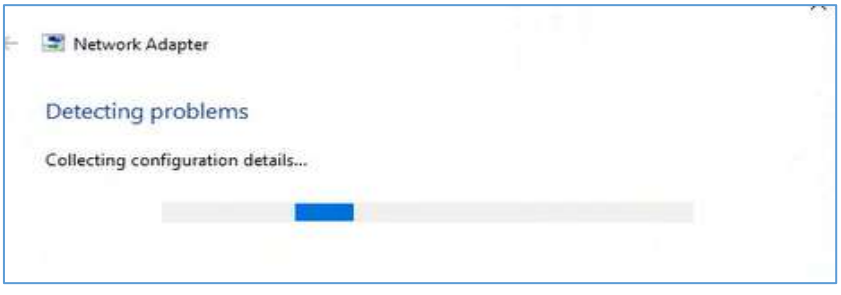
```css
div{
    width: 300px;
    height: 150px;
    background-color: bisque;
    animation-name: changeColor;
    animation-duration: 10s;
    animation-iteration-count: infinite;
    animation-timing-function: ease;

}
@keyframes changeColor {
    0% { background-color: red;}
    22% { background-color: orange;}
    40% { background-color: yellow;}
    60% { background-color: green;}
    75% { background-color: cyan;}
    88% { background-color: blue;}
    100% { background-color: violet;}
}
.box
{
    width: 400px;
    height: 400px;
    background-color:blue;
    margin: 0 auto;
    animation-name: circle; /*it gives link between @keyframes circle to A-name*/
    animation-duration: 3s; /*is the duration of Animation*/
    animation-delay: 300ms;
    animation-iteration-count: infinite; /*repeat again and again*/
    animation-direction: alternate-reverse;
}
@keyframes circle
{
    0%{border-radius: 0%;}   /*in zero second at start*/
    25%{border-radius: 50%; background-color: rgb(196, 90, 185);}
    50%{border-radius: 0%;}

}
```

Assignment

style.css > @keyframes bounce

```css
.box {
  width: 50px;
  height: 50px;
  background-color: green;
  position: relative;
  animation-name: bounce,  colors , circle;
  animation-duration: 3s;
  animation-iteration-count: infinite;
  animation-timing-function: ease-in-out;
  margin-right: auto;
  margin-left: auto;
}
@keyframes bounce {
  0%   { top: 0; }
  50%  { top: 300px; }
  100% { top: 0; }

}
@keyframes circle {
  0%   { border-radius: 0%; }
  50%  { border-radius: 50%; width: 400px; height: 400px;}
  100% { border-radius: 0%; }
}
@keyframes colors
{
    0%{ background-color: red;}
    20%{ background-color: orange;}
    40%{ background-color: yellow;}
    60%{ background-color: green;}
    80%{ background-color: cyan;}
    90%{ background-color: blue;}
    100%{ background-color: violet;}
}
```

```css
.bar
{
    width: 200px;
    height: 10px;
    background-color: #eee;
    margin: 100px auto;
}

.loader
{
    width: 30px;
    height: 10px;
    background-color: rgb(205, 146, 238);
    border-bottom-color: transparent;
    animation:loader 5s linear infinite;
    position: relative;
}
@keyframes loader
{
    from
    {
        right: 0;
    }
    to
    {
        right: -170px;
    }
}
```



style.css > @keyframes spinner

```css
.spinner
{
    width: 40px;
    height: 40px;
    background-color: transparent;
    border-radius: 50px;
    border: 2px solid brown;
    margin: 100px auto;
    border-bottom-color: transparent;
    animation:spinner 2s linear infinite;

}
@keyframes spinner
{
    from
    {
        transform: rotate(0deg);
    }
    to
    {
        transform: rotate(360deg);
    }
}
```

545

```html
<div class="container">
    <div class="box box3"></div> <!-- Red at top -->
    <div class="box box2"></div> <!-- Yellow in middle -->
    <div class="box box1"></div> <!-- Green at bottom -->
</div>
```

style.css > ...

```css
.container {
    width: 220px;
    padding: 20px;
    margin: 100px auto;
    background-color: #333;
    border-radius: 20px;
    box-shadow: 0 0 15px rgba(0,0,0,0.5);
}

.box {
    width: 200px;
    height: 200px;
    border-radius: 50%;
    background-color: white;
    margin: 20px auto;
    animation-iteration-count: infinite;
    animation-direction: normal;
    animation-timing-function: linear;
}

/* Green light: goes first */
.box1 {
    animation-name: greenLight;
    animation-duration: 12s;
}

/* Yellow light: after green */
.box2 {
    animation-name: yellowLight;
    animation-duration: 12s;
}

/* Red light: after yellow */
.box3 {
    animation-name: redLight;
    animation-duration: 12s;
}

@keyframes greenLight {
    0%, 33% { background-color: green; }
    33.01%, 100% { background-color: white; }
}

@keyframes yellowLight {
    0%, 33% { background-color: white; }
    33.01%, 66% { background-color: yellow; }
    66.01%, 100% { background-color: white; }
}

@keyframes redLight {
    0%, 66% { background-color: white; }
    66.01%, 100% { background-color: red; }
}
```

211

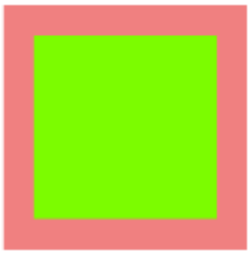
```
From
To

translateX(x)   Defines a translation, using only the value for the X-axis
translateY(y)   Defines a translation, using only the value for the Y-axis
translate(x,y)  Defines a 2D translation

translateZ(z)   Defines a 3D translation, using only the value for the Z-axis
translate3d(x,y,z)  Defines a 3D translation
scaleX(x)   Defines a scale transformation by giving a value for the X-axis
scaleY(y)   Defines a scale transformation by giving a value for the Y-axis
scale(x,y)  Defines a 2D scale transformation
scaleZ(z)   Defines a 3D scale transformation by giving a value for the Z-axis
scale3d(x,y,z)  Defines a 3D scale transformation
rotate(angle)   Defines a 2D rotation, the angle is specified in the parameter
rotateX(angle)  Defines a 3D rotation along the X-axis
rotateY(angle)  Defines a 3D rotation along the Y-axis
rotateZ(angle)  Defines a 3D rotation along the Z-axis
rotate3d(x,y,z,angle)   Defines a 3D rotation

skewX(angle)    Defines a 2D skew transformation along the X-axis
skewY(angle)    Defines a 2D skew transformation along the Y-axis
skew(x-angle,y-angle)   Defines a 2D skew transformation along the X- and the Y-axis

perspective(n)  Defines a perspective view for a 3D transformed element

matrix()
matrix(scaleX(),skewY(),skewX(),scaleY(),translateX(),translateY())
```

```
transform-origin    Allows you to change the position on transformed elements
transform-style Specifies how nested elements are rendered in 3D space
perspective Specifies the perspective on how 3D elements are viewed
perspective-origin  Specifies the bottom position of 3D elements
backface-visibility Defines whether or not an element should be visible when not facing the screen
```

Transform

Is used to move, rotate, scale and skew elements.

```css
.box-parent
{
    background-color: lightcoral;
    width: 400px;
    height: 400px;
    padding: 10px;
    box-sizing: border-box;
    position: absolute;
    top: 50%;
    left: 50%;
    transform: translatex(-50%) translateY(-50%); /* to bring something to center */
}
.box
{
    background-color: lawngreen;
    width: 300px;
    height: 300px;
    position: absolute;
    top: 50%;
    left: 50%;
    transform: translatex(-50%) translateY(-50%); /* to bring something to center */
}
```



```css
.box
{
    background-image: url(./p.jpg);
    background-size: cover;
    background-position: center;
    transition: 300ms;
    width: 300px;
    height: 300px;
    position: absolute;
    top: 50%;
    left: 50%;
    transform: translatex(-50%) translateY(-50%); /* to bring something to center */
}
.box:hover
{
    transform: scale(1.2) rotate(10deg);
}
```

Zoom in and out an element on hover,

```css
.box-parent
{
    background-color: lightcoral;
    width: 400px;
    height: 400px;
    position: absolute;
    top: 50%;
    left: 50%;
    transform: translatex(-50%) translateY(-50%); /* to bring something to center */
    perspective: 1000px;  /* give a 3D style to a shape */
    border-radius: 50%;
}
.box
{
    background-image: url(./p.jpg);
    background-size: cover;
    background-position: center;
    transition: 600ms;
    width: 100%;
    height: 100%;
    transform: rotateX(30deg) ;
    animation: 10s linear infinite rotate;
    transform-style: preserve-3d;
    /* backface-visibility: hidden;  hide the back side of the element*/
    border-radius: 50%;

}
@keyframes rotate {
    0%{transform: rotateY(0);}
    100%{transform: rotateY(360deg);}
}
```

-Webkit-box-reflect:

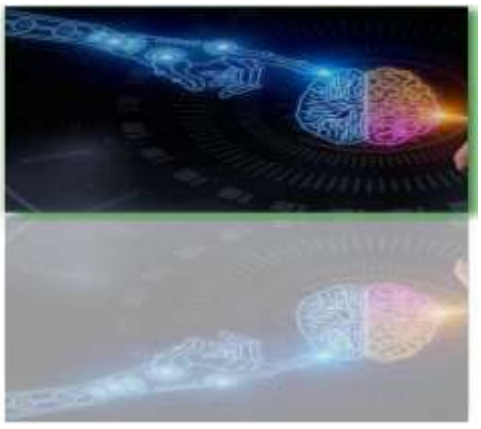
- object-fit: fill, contain, cover, scale-down
- object-position: x y

@media query

Filter for image and other boxes:

Filter property adds visual effects (like blur, brightness…)

```css
.box
{
    width: 500px;
    height: 500px;
    filter: brightness(0.8);
}
img
{
    width: 100%;
    height: 100%;
    object-fit: cover;
    object-position: left;
    -webkit-box-reflect: below 10px linear-gradient(rgba(0 , 0, 0 ,0.4));
    filter: grayscale(300%);
    filter: drop-shadow(10px 20px 15px black);
    filter: invert(40%);
    filter: opacity(0.5);
        filter: blur(4px);
        filter: brightness(0.30);
        filter: contrast(180%);
        filter: grayscale(100%);
        filter: invert(100%);
        filter: opacity(50%);
        filter: saturate(7);
        filter: sepia(100%);
        filter: drop-shadow(8px 8px 10px green);
}
```



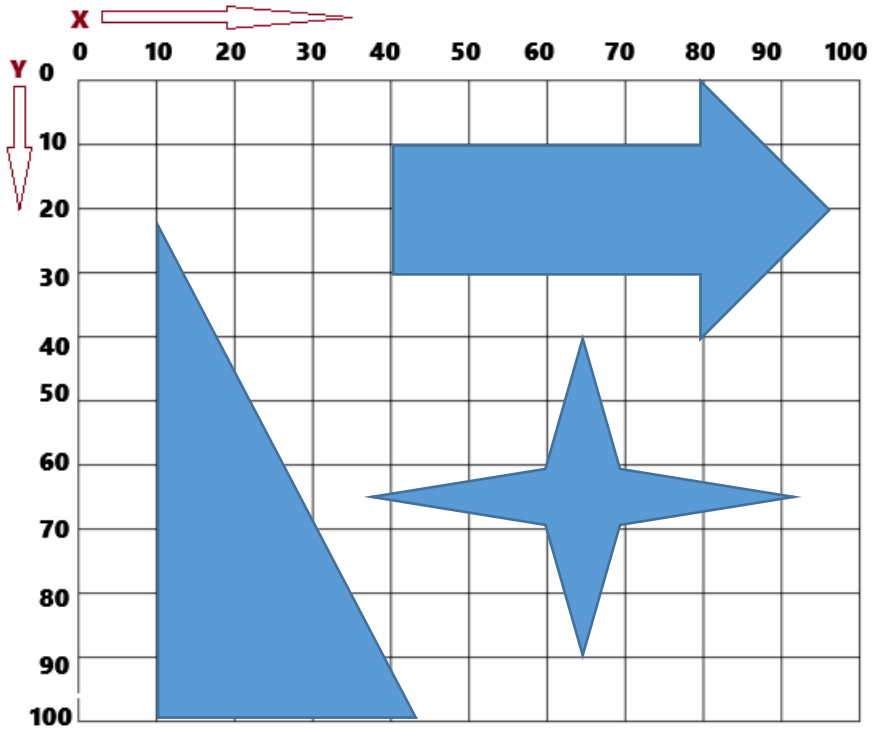
```css
.box
{
    width: 500px;
    height: 500px;
    display: inline-block;
}
img
{
    width: 100%;
    height: 100%;
    object-fit: cover;  /*fits background img into box*/
    object-position: top;
    -webkit-box-reflect: below 10px linear-gradient(rgba(0,0,0,0.3))
    /* filter: blur(0.8); /* we can apply blur, brightness, contrast,
}
#I1
{
    filter: grayscale(0.8);
}
#I2
{
    filter:blur(1px);
    filter: sepia(100%);
}
#I3
{
    filter:brightness(88%);
}
```

```html
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>CSS Transition Example</title>
  <link rel="stylesheet" href="style.css">
</head>
<body>

  <div class="box"><img id="I1" src="./IMG_7199.png" ></div>
  <div class="box"><img id="I2" src="./IMG_7199.png" ></div>
  <div class="box"><img id="I3" src="./IMG_7199.png" ></div>

</body>
</html>
```

Shapes

Clip-path: This property is used to clip (cut out) a portion of an element visually.

- circle (40% at x y); (RadiouByElementSize at horizontally vertically)
- Ellipse (130px 140px at 10% 20%); (XMove YMove at Position horizontally and vertically)
- Polygon (50% 0, 100% 50%, 50% 100%, 0 50%);



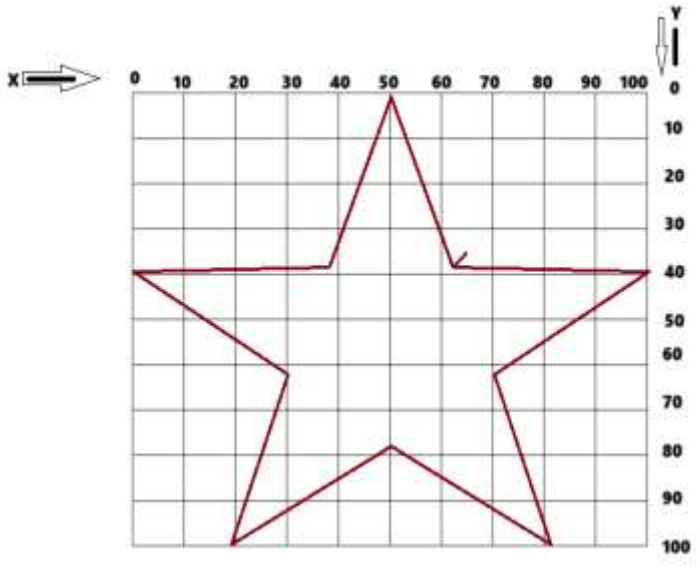
- inset(100px 50px);

Shape-outside
Shape-margined

```css
.box
{
    width: 500px;
    height: 500px;
    background-image: url('./it.png');
    background-size: cover;
    background-position: center;

    /*clip-path: circle(50% at 0 20%); make shapes at X Y*/
    clip-path: polygon(50% 0 , 0 100%, 100% 100% ); /*make shapes at XY, xy, xy*/
    float: left; /*allows text to come to next to the shape*/
    padding: 10px; /* increase the size of the shape*/
    shape-outside: circle(50%); /* allows text to go around the shape as the shape*/
    shape-margin: 10px;
    animation: rotate 10s linear infinite;
}
```

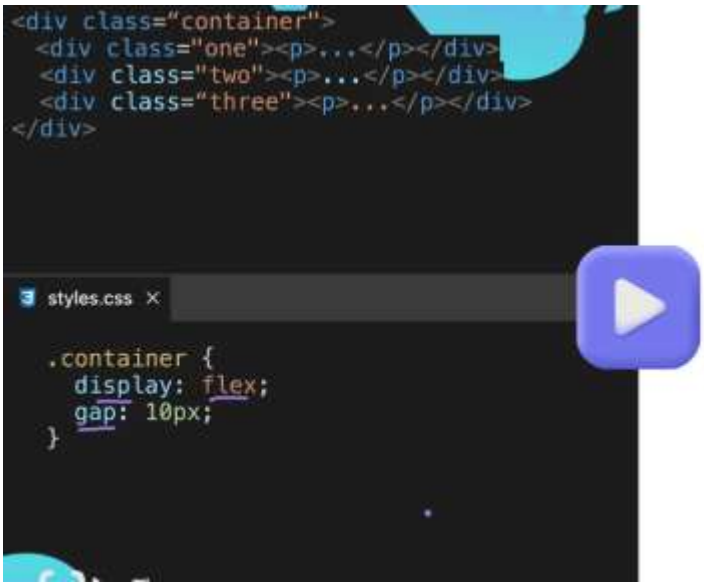
```css
.box
{
    width: 500px;
    height: 500px;
    background-image: url('./it.png');
    background-size: cover;
    background-position: center;
    /*clip-path: circle(50% at 0 20%); make shapes at X Y*/
    clip-path: circle(50% ); /*make shapes at X Y*/
    float: left; /*allows text to come to next to the shape*/
    padding: 10px; /* increase the size of the shape*/
    shape-outside: circle(50%); /* allows text to go around the shape as the shape*/
    shape-margin: 10px;
    animation: rotate 10s linear infinite;
}
span
{
    font-size: 32px;
}
@keyframes rotate {
    from
    {clip-path: circle(50% ); shape-outside: circle(50%); transform:rotate(0deg) ;}

    to
    {clip-path: circle(55% ); shape-outside: circle(55%); transform:rotate(360deg) ;}
}
```

```css
div
{
    width: 500px;
    height: 500px;
    background-color: green;
    clip-path:polygon(
        50% 0%,
        61% 35%,
        98% 35%,
        68% 57%,
        79% 91%,
        50% 70%,
        21% 91%,
        32% 57%,
        2% 35%,
        39% 35%)
}
```



Flex Box:

Flexbox (short for **Flexible Box Layout**) is **used to arrange elements in a one-dimensional row or column**. It makes it easy to design **responsive** and **space-efficient** layouts.



```
flex
    flex-direction: column, column-reverse, row, row-reverse
    flex-wrap: wrap, no-wrap, wrap-reverse

    flex-flow: direction wrap

    justify-content
    align-content

    align-items: flex-start, flex-end, center, stretch, baseline
    align-self: flex-start, flex-end, center, stretch, baseline, auto


    flex-grow: 0-1;

    flex-basis: px, min-width, max-width


    flex-shrink: 1-0

    margin-auto
```

```html
<body>
    <div class="container">
        <div class="item">Item 1</div>
        <div class="item">Item 2</div>
        <div class="item">Item 3</div>
        <div class="item">Item 4</div>
        <div class="item">Item 5</div>
    </div>
</body>
```

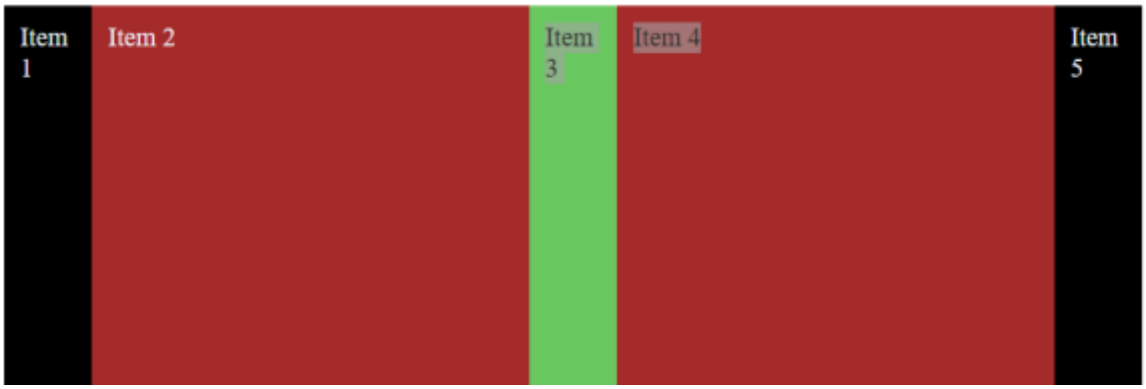
```css
.container
{
    width: 700px;
    height: 600px;
    background-color: khaki;
    margin: auto;
    display: flex; /*fix items inside the parent box so we apply it to parent div*/
    flex-direction: row; /*column, column-reverse, row-reverse or row*/
    flex-wrap: nowrap; /*Dont allow items to break down in the box*/
    justify-content: center; /*brings content to center*/
    justify-content: flex-end; /*brings content to rigth*/
    justify-content: flex-start; /*brings content to left*/
    justify-content: space-between;
    justify-content: space-evenly;
    align-content: center;

}
.item
{
    padding: 10px;
}
.item:nth-child(even)
{
    background-color: brown;
    color: aliceblue;
}
.item:nth-child(odd)
{
    background-color: black;
    color: aliceblue;
}
.item:nth-child(3)
{
    background-color: rgb(105, 199, 96);
    color: aliceblue;
}
```

Align self: bring changes in a single item.

```css
.item:nth-child(even)
{
    background-color: brown;
    color: aliceblue;
    align-self: center ; /*felx start, flex-end but we should have display:flex; in parent div*/
}
```

Flex-basis:

```css
.container
{
    width: 700px;
    height: 600px;
    background-color: khaki;
    margin: auto;
     display: flex; /*fix items inside the parent box so we apply it to parent div*/

}
.item
{
    padding: 10px;
}
.item:nth-child(even)
{
    background-color: brown;
    color: aliceblue;
    flex-basis: 300px; /*means maximum size but we should have display flex in parent*/
}
.item:nth-child(odd)
{
    background-color: black;
    color: aliceblue;
}
.item:nth-child(3)
{
    background-color: rgb(105, 199, 96);
    color: aliceblue;
}
```



If take margin-left-auto for item 5 so it will go to right if I make it margin-left and margin-right: auto the item 5 will come to center. So with margin we can move a single item everywhere we want.

Make a menu with display: flex is easy


```
<body>
    <nav>
        <div class="menu">
            <div class="item">Logo</div>
            <div class="item">Home</div>
            <div class="item">Services</div>
            <div class="item">Contact US</div>
            <div class="item">Insert</div>
        </div>
    </nav>
</body>
```

style.css > {} @media screen and (max-width: 530px)

```
nav
{
    background-color: lightblue;
}
.menu
{
    display: flex;
    padding: 4px;
}
.item
{
    background-color: rgb(199, 231, 143);
    padding: 15px 25px;
    cursor: pointer;
    margin: 2px 2px;
}
.item:hover
{
    background-color: lightgoldenrodyellow;
    border-radius: 5px;
}
.item:first-child
{
    margin-right: auto;
}
@media screen and (max-width: 530px) {
    .menu {
        flex-direction: column;
    }
    .item {
        margin: 3px; /* Consistent small margin for all items */
    }
    .item:first-child {
        margin: 3px; /* Remove override if not needed */
    }
}
```

How to Create Responsive Website

@Media Query:

```
@media (max-width: 600px) {

  /* CSS for screens below or equal to 60
  div {
    height: 200px;
    width: 200px;
  }
}
```

@Media

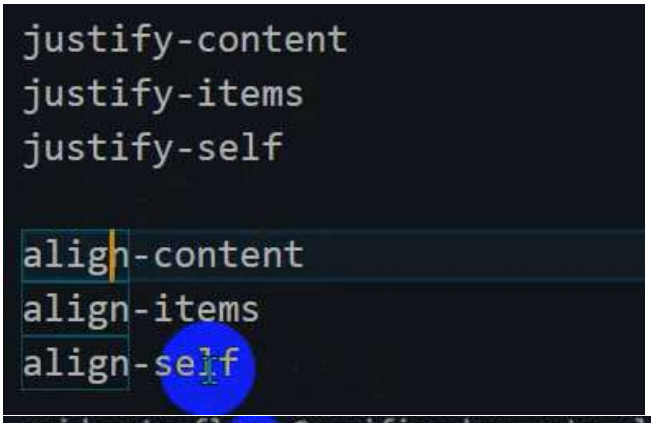
```
    <nav>
        <label for="menu">Menu</label>
        <input type="checkbox" id="menu">
        <ul class="sm-screen">
            <li>Menu 1</li>
            <li>Menu 2</li>
            <li>Menu 3</li>
            <li>Menu 4</li>
            <li>Menu 5</li>

        </ul>
    </nav>
</header>
```

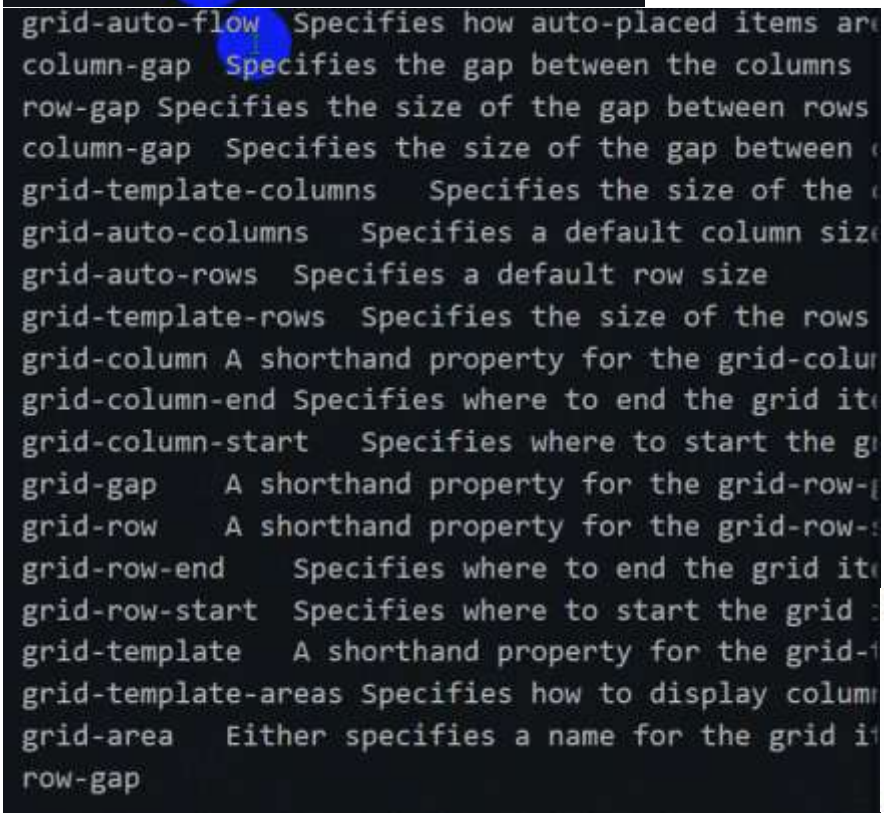
To make web site flexible for both Desktop and Mobile phone.

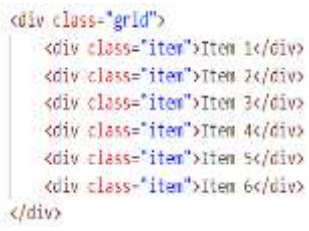
style.css > ...

```css
* {
    margin: 0;
    box-sizing: border-box;
}
nav {
    background-color: lightpink;
    width: 100%;}
ul.lg-screen {
    padding: 10px;
    list-style: none;}
li {
    background-color: lightcoral;
    display: inline-block;
    padding: 15px 35px;
    width: 19%;
    margin: 0 3px;
    font-size: 32px;
    font-family: system-ui, -apple-system, BlinkMacSystemFont, 'Segoe UI', Roboto,
    font-weight: 100;
    transition: 300ms;}
li:hover {
    background-color: lightsalmon;
    cursor: pointer;}
/* Hide checkbox visually */
#menu {
    display: none;}
/* Hide label on large screens */
label {
    display: none;
    background-color: lightcoral;
    text-align: center;
    padding: 15px;
    font-size: 28px;
    cursor: pointer;}
@media screen and (max-width: 992px) {
    label {display: block;}
    ul.lg-screen {
        display: none;
        flex-direction: column;
    }
    /* Show menu when checkbox is checked */
    #menu:checked + label + ul.lg-screen {
        display: block;}
    li {
        display: block;
        width: 100%;
        margin: 5px 0;}
}
```

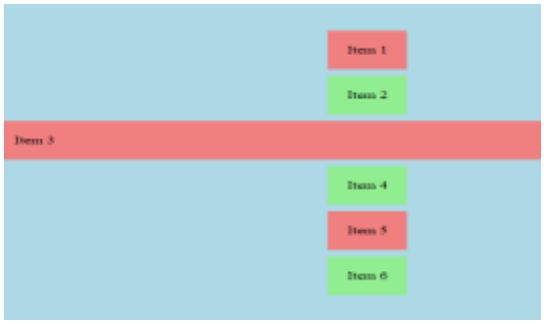
It is the same as Flex but it deals with huge things like web site …







```css
.grid
{
    width: 768px;
    background-color: lightblue;
    height: 500px;
    margin: 0 auto;
    display: grid;
    /*grid-auto-flow: row;  chage items from colum, row*/
    /*justify-content:space-evenly;  start, end, center, right, left, space-between, space-evenly..*/
    align-content:center; /* start, end, center, right, left, space-between, space-evenly..*/
    justify-items: center; /* ددہ پر ډیرتمز هم د پورتنه په رقم دی فرق نی دادی ډیوازی په خپل ځان کی شی سره عکوی*/
    column-gap: 10px;
    row-gap: 10px;
}
.item
{
    padding: 20px;
}
.item:nth-child(odd)
{
    background-color: lightcoral;
}
.item:nth-child(even)
{
    background-color: lightgreen;
}
.item:nth-child(3)
{
    align-self: stretch; /*just aply on current child values are the same as align-content*/
    justify-self: stretch; /*just aply on current child values are the same as justify-items*/
}
```



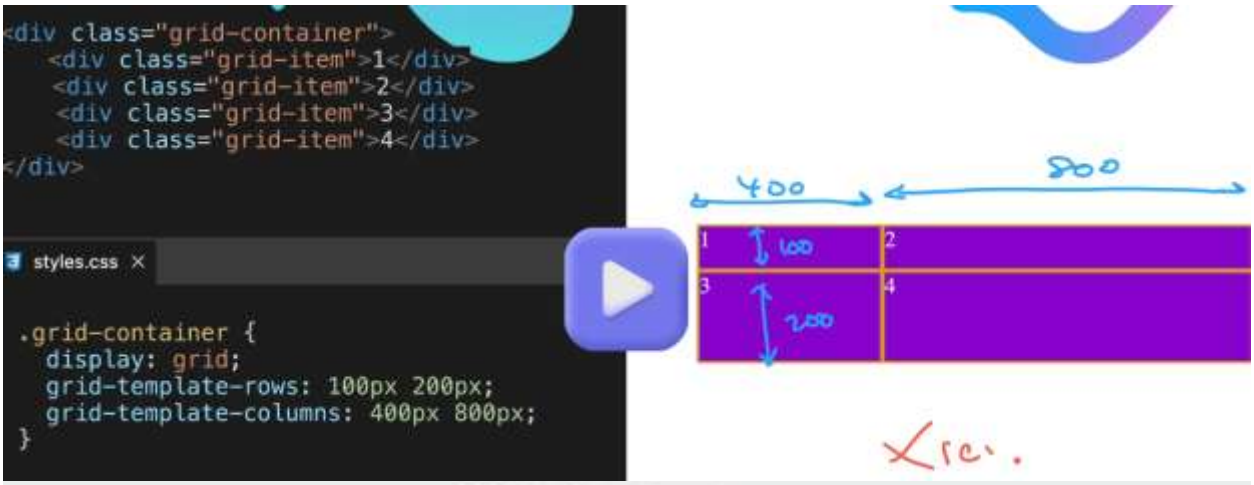
If we want to grid column and rows manually by our chose then we use grid-template

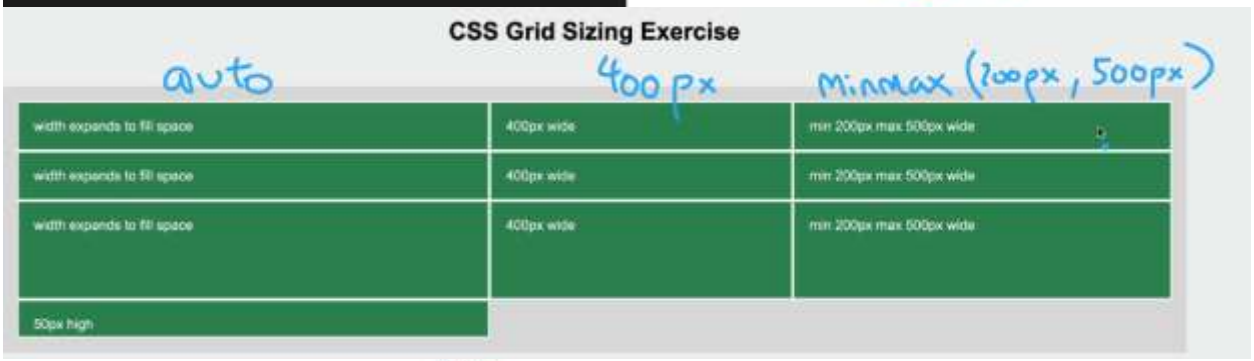
```css
.grid
{
    width: 90%;
    background-color: lightblue;
    height: 500px;
    margin: 0 auto;
    display: grid;
    /*grid-template-columns: auto auto; one auto means one colum and two means put two column next by each other*/
    /*grid-template-columns: repeat(3, auto); is short form of the above*/
    /*grid-template-columns: 10% 30% auto; is short form of the above*/
    /*grid-template-columns: 1fr 1fr 2fr; is short form of the above means fraction*/
    /*grid-template-columns: minmax(200px 300px);gives mini and max width*/
    /*grid-template-columns: repeat(4, minmax(300px, auto)); gives mini and max width*/
    grid-template-columns: repeat(auto-fit, minmax(200px, auto)); /* are most common gives mini and max width no need for @media*/
    grid-template-columns: repeat(auto-fill, minmax(25%, auto)); /* are most common gives mini and max width no need for @media*/
    /*grid-template-rows: 1fr 80% 100px; gives hight to rows*/
}
.item
{
    padding: 20px;
    border: 2px solid red;
}
.item:nth-child(odd)
{
    background-color: lightcoral;
}
.item:nth-child(even)
{
    background-color: lightgreen;
}
```

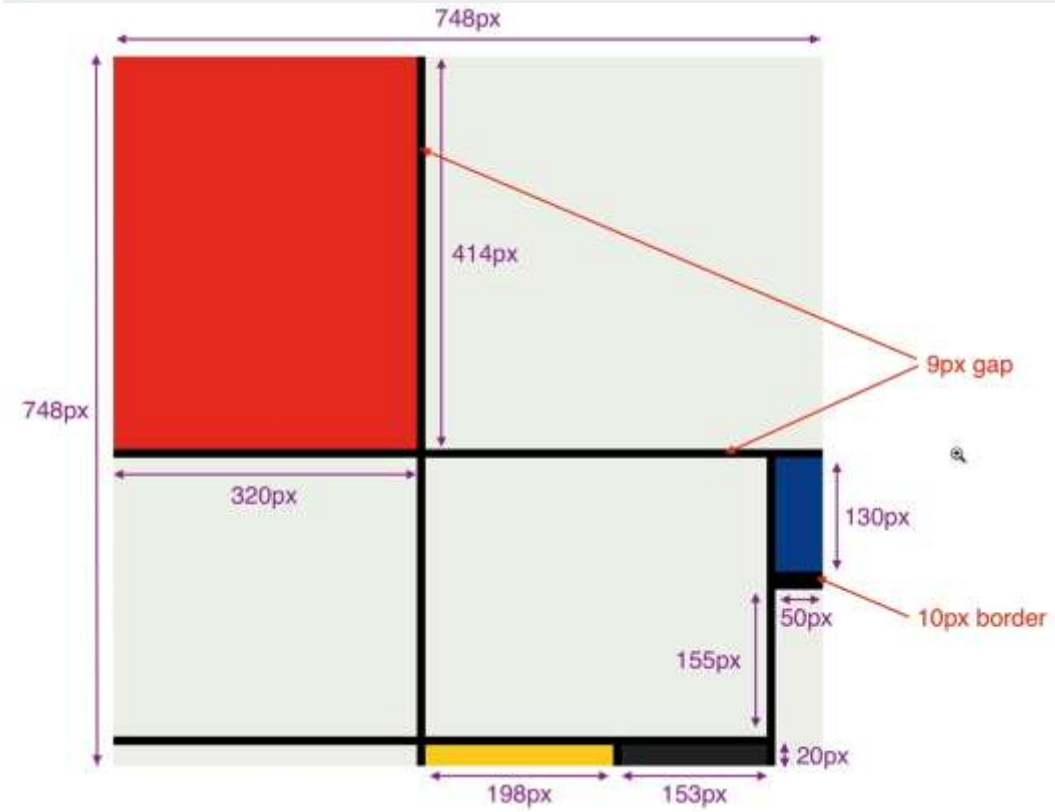
Auto grid/ moving a single item and merge column and rows

```css
.grid
{
    width: 90%;
    background-color: lightblue;
    height: 500px;
    margin: 0 auto;
    display: grid;
    /*grid-template-columns: repeat(auto-fit, minmax(200px, auto));  are most common gives mini and max width no need for @media*/
    grid-template-columns: repeat(auto-fill, minmax(100px, auto)); /* are most common gives mini and max width no need for @media*/
    /* grid-auto-columns: 50px; */
    grid-template-rows: 100px 100px;/* gives hight to rows*/
    grid-auto-rows: 50px;
    /* grid-gap: row column; */
    grid-gap: 2px 2px;
}
.item
{    padding: 20px;
    border: 2px solid red;
}
.item:nth-child(odd)
{
    background-color: lightcoral;
}
.item:nth-child(even)
{
    background-color: lightgreen;
}
.item:nth-child(1)
{
    /* grid-column-start: 9;
    grid-column-end: 16; stritch item to the end means right*
    /* grid-column-end: -3;
    grid-column:6/-1 ; */
    grid-row: 1/span 2 ;
}
```

Grid and @Media







Media makes screen responsive to a smaller screen as below

Grid options

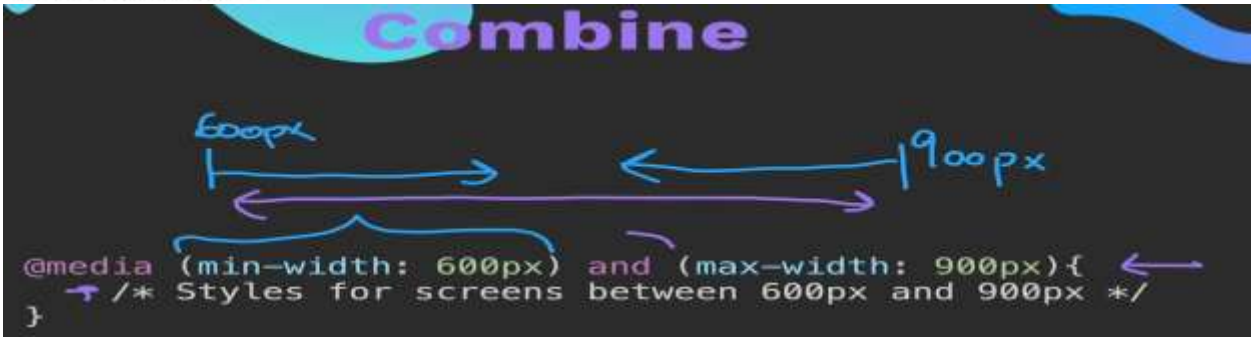
While Bootstrap uses ems or rems for defining most sizes, pxs are used for grid breakpoints and container widths. This is because the viewport width is in pixels and does not change with the font size.

See how aspects of the Bootstrap grid system work across multiple devices with a handy table.

	Extra small <576px	Small ≥576px	Medium ≥768px	Large ≥992px	Extra large ≥1200px
Max container width	None (auto)	540px	720px	960px	1140px
Class prefix	.col-	.col-sm-	.col-md-	.col-lg-	.col-xl-

Media Queries

Adding breakpoints to define responsive layouts



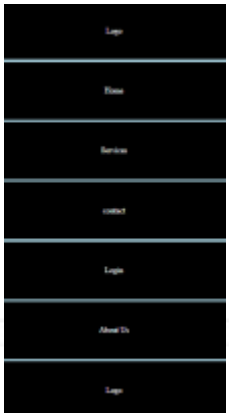
Let's make a Nav bar responsive

grid-template-columns: repeat(auto-fit, minmax(200px, 1fr));

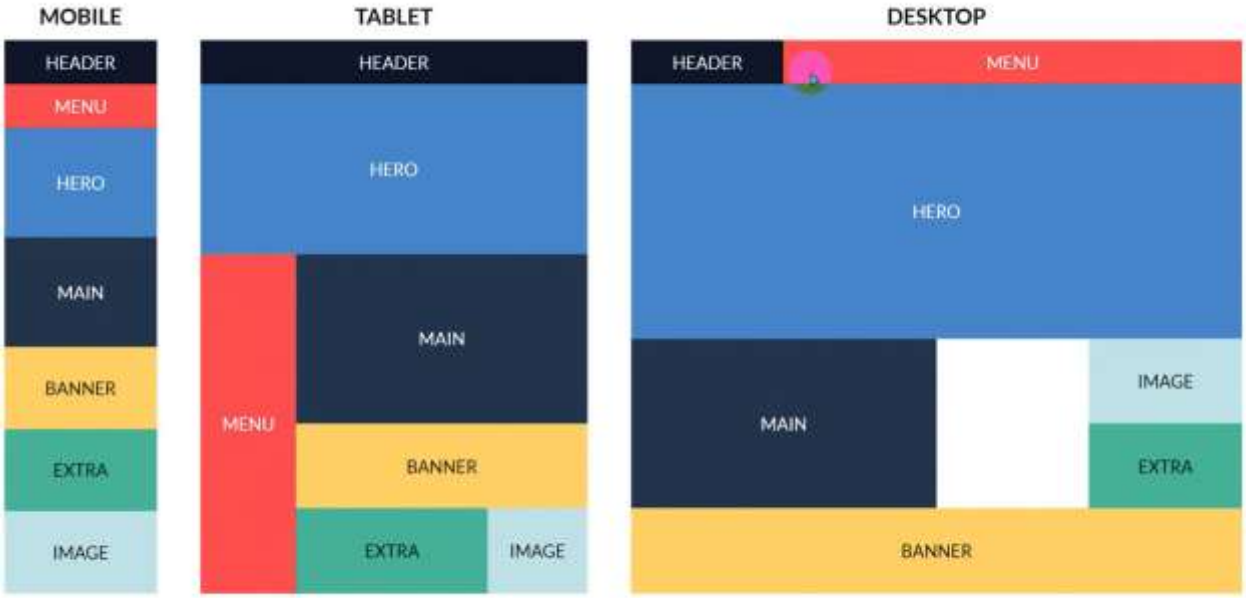
- It defines how many columns your grid will have — and how wide each column should be.
- Breakdown of Each Part
- ✓ repeat(...) This means: "repeat the following column rule multiple times."
- ✓ auto-fit This means: "Automatically fit as many columns as possible into the container."
- If the screen is wide — you might get 3, 4, or more columns.
- If the screen is narrow — it might drop to 2 or even 1 column.
- ✓ minmax(200px, 1fr)
- This sets the width of each column:
- Minimum width = 200px → Don't go smaller than 200px.
- Maximum width = 1fr → Let the column grow and take up remaining space

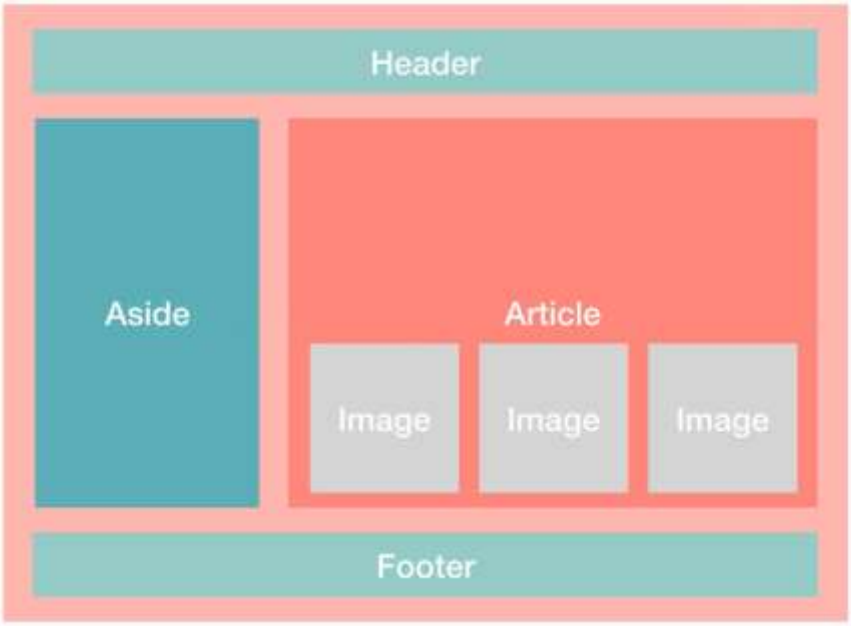
```html
<body>
    <div class="box">
      <div class="items">Logo</div>
      <div class="items">Home</div>
      <div class="items">Services</div>
      <div class="items">contact</div>
      <div class="items">Login</div>
      <div class="items">About Us</div>
      <div class="items">Logo</div>
    </div>
</body>
```

```css
.box {
    width: auto;
    height: auto;
    background-color: lightblue;
    display: grid;
    grid-template-columns: repeat(auto-fit, minmax(200px, 1fr));
    grid-auto-rows: 100px;
    justify-content: center;
    align-items: center;
    column-gap: 5px;
    row-gap: 5px;
}

.items {
    background-color: black;
    color: white;
    width: 100%; /* Let it fill the grid cell */
    height: 100%;
    display: flex;
    align-items: center;
    justify-content: center;
    text-align: center;
}

/* ✅ Responsive media query */
@media (max-width: 768px) {
    .box {
        grid-template-columns: 1fr; /* Single column on small screens */
        grid-auto-rows: 120px;      /* Slightly taller for smaller screens if needed */
    }
}
```



Make this Template


MOBILE
HEADER
MENU
HERO
MAIN
BANNER
EXTRA
IMAGE
TABLET
HEADER
HERO
MENU
MAIN
BANNER
EXTRA
IMAGE
DESKTOP
HEADER
MENU
HERO
MAIN
IMAGE
EXTRA
BANNER


Header
Aside
Article
Image
Image
Image
Footer

```css
* {
  box-sizing: border-box;
  margin: 0;
  padding: 0;
}

html, body {
  height: 100%;
  overflow: hidden; /* Prevent scrolling */
  font-size: 1.5em;
  font-weight: bold;
  color: aliceblue;
}

.grid-container {
  display: grid;
  height: 100vh;
  grid-template-areas:
    "header header"
    "aside article"
    "footer footer";
  grid-template-columns: 1fr 2fr;
  grid-template-rows: auto 1fr auto;
  background-color: lightcoral;
}

.header,
.footer,
.aside,
.article {
  padding: 10px;
  text-align: center;
}
```

```css
.header {
  grid-area: header;
  background-color: lightseagreen;
}

.footer {
  grid-area: footer;
  background-color: lightseagreen;
}

.aside {
  grid-area: aside;
  background-color: rgb(63, 140, 192);
}

.article {
  grid-area: article;
  background-color: rgb(153, 79, 142);
}

/* Responsive: stack layout on smaller screens */
@media (max-width: 768px) {
  .grid-container {
    grid-template-areas:
      "header"
      "article"
      "aside"
      "footer";
    grid-template-columns: 1fr;
    grid-template-rows: auto 1fr auto auto;
  }
}
```

Or

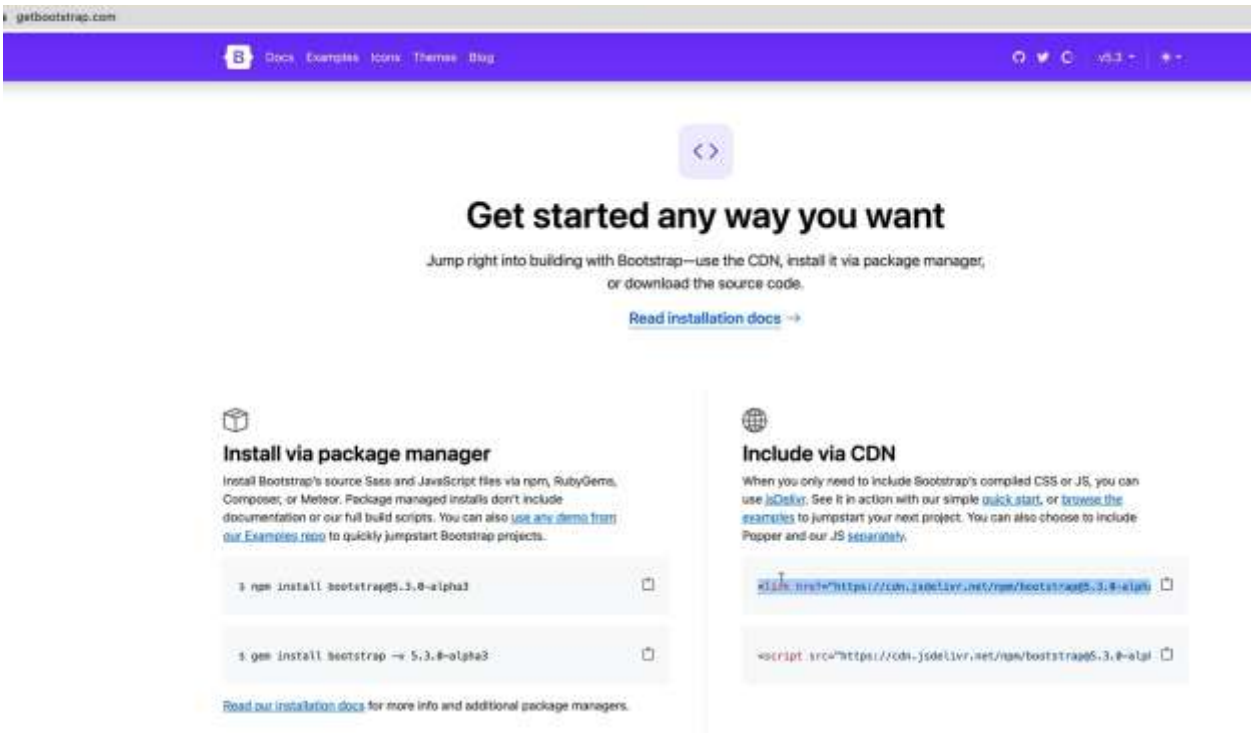
```html
<div class="grid-container">
    <div class="header">HEADER</div>
    <div class="aside">Aside</div>
    <div class="article"><h2>Article</h2>
    <div class="image">
        <div>Image</div>
        <div>Image</div>
        <div>Image</div>
    </div>
    </div>
    <div class="footer">Footer</div>
  </div>
```

```css
body {
  box-sizing: border-box;
  padding: 0;
  margin: 0;
  font-size: 1.5em;
  color: aliceblue;
  font-weight: bolder;
}
  .grid-container {
  display: grid;
  grid-template-areas:
    "header header"
    "aside article"
    "footer footer";
  gap: 10px;
  grid-template-columns: 1fr 2fr; /* First column is hal
  grid-template-rows: auto 1fr auto;
  width: 99%;
  min-height: 100vh;
  background-color: lightcoral;
  padding: 10px;
}
  .header {
  grid-area: header;
  background-color: lightseagreen;
  text-align: center;
  padding: 20px;
}

.aside {
  grid-area: aside;
  background-color: rgb(63, 140, 192);
  text-align: center;
  padding: 20px;
}

.article {
  grid-area: article;
  background-color: rgb(153, 79, 142);
  text-align: center;
  padding: 20px;
}
```

```css
.footer {
  grid-area: footer;
  background-color: lightseagreen;
  text-align: center;
  padding: 20px;
}

h2 {
  margin-top: 45%;
}

.image div {
  background-color: #a4a2a2;
  display: inline-block;
  width: 100px;
  height: 100px;
  text-align: center;
  padding: 20px;
  margin: 10px;
  justify-self: center;
}

/* Responsive Layout for Small Screens */
@media (max-width: 768px) {
  .grid-container {
    grid-template-areas:
      "header"
      "article"
      "aside"
      "footer";
    grid-template-columns: 1fr;
  }
}
```

Bootstrap

Is a css framework that is used to make responsive and look good websites



CDN (Content Delivery Network)
Link Code for Card of Bootstrap

```
<link
href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.8/dist/css/bootstrap.min.css"
rel="stylesheet" integrity="sha384-
sRIl4kxILFvY47J16cr9ZwB07vP4J8+LH7qKQnuqkuIAvNWLzeN8tE5YBujZqJLB"
crossorigin="anonymous">
```

HTML card code

```
        <div class="card" style="width: 18rem;">
  <img src="./gulbabo.png" class="card-img-top" alt="...">
  <div class="card-body">
    <h5 class="card-title">Card title</h5>
    <p class="card-text">Some quick example text to build on the card title and
make up the bulk of the card’s content.</p>
    <a href="#" class="btn btn-primary">Go somewhere</a>
  </div>
```

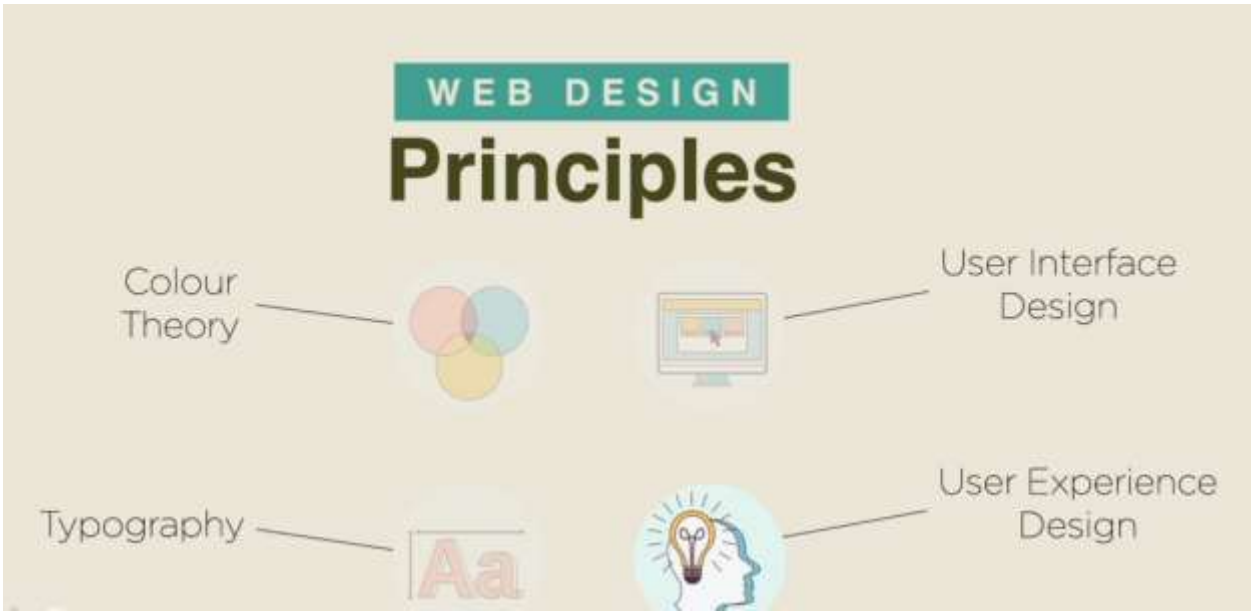
Making responsive Screen with Bootstrap

 Hello

Bootstrap Container

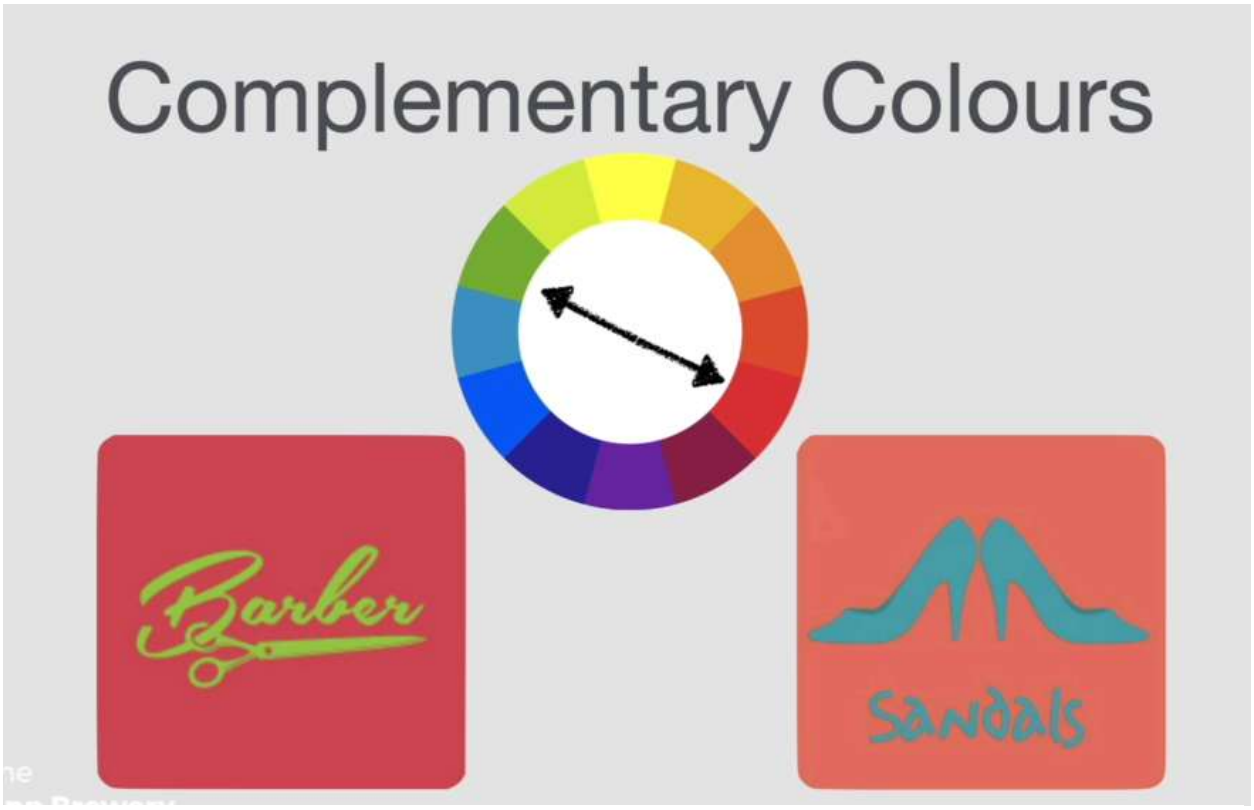
	Extra small <576px	Small ≥576px	Medium ≥768px	Large ≥992px	X-Large ≥1200px	XX-Large ≥1400px
.container	100%	540px	720px	960px	1140px	1320px
.container-sm	100%	540px	720px	960px	1140px	1320px
.container-md	100%	100%	720px	960px	1140px	1320px
.container-lg	100%	100%	100%	960px	1140px	1320px
.container-xl	100%	100%	100%	100%	1140px	1320px
.container-xxl	100%	100%	100%	100%	100%	1320px
.container-fluid	100%	100%	100%	100%	100%	100%

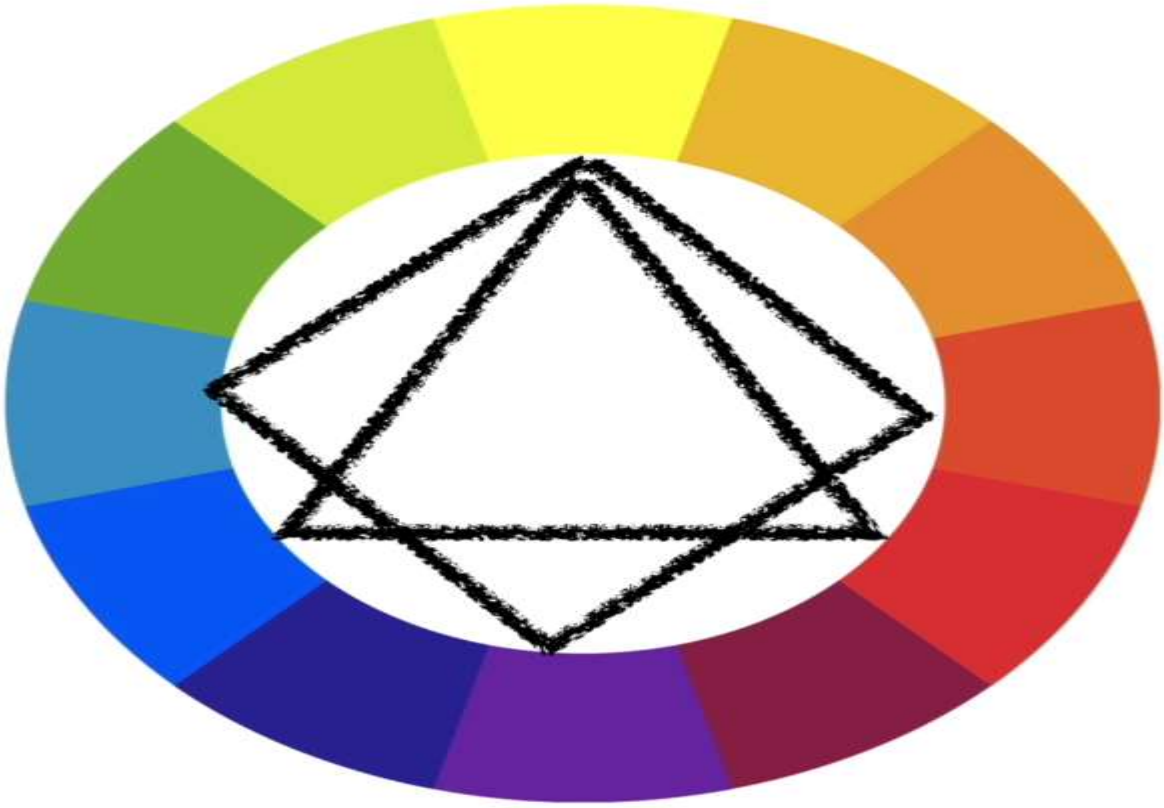
Web Design



Moods


RED
LOVE. ENERGY.
INTENSITY.
YELLOW
JOY. INTELLECT.
ATTENTION.
GREEN
FRESHNESS. SAFETY.
GROWTH.
BLUE
STABILITY. TRUST.
SERENITY.
PURPLE
ROYALTY. WEALTH.
FEMININITY.


Complementary Colours
Barber
Sandals



colorhunt.co

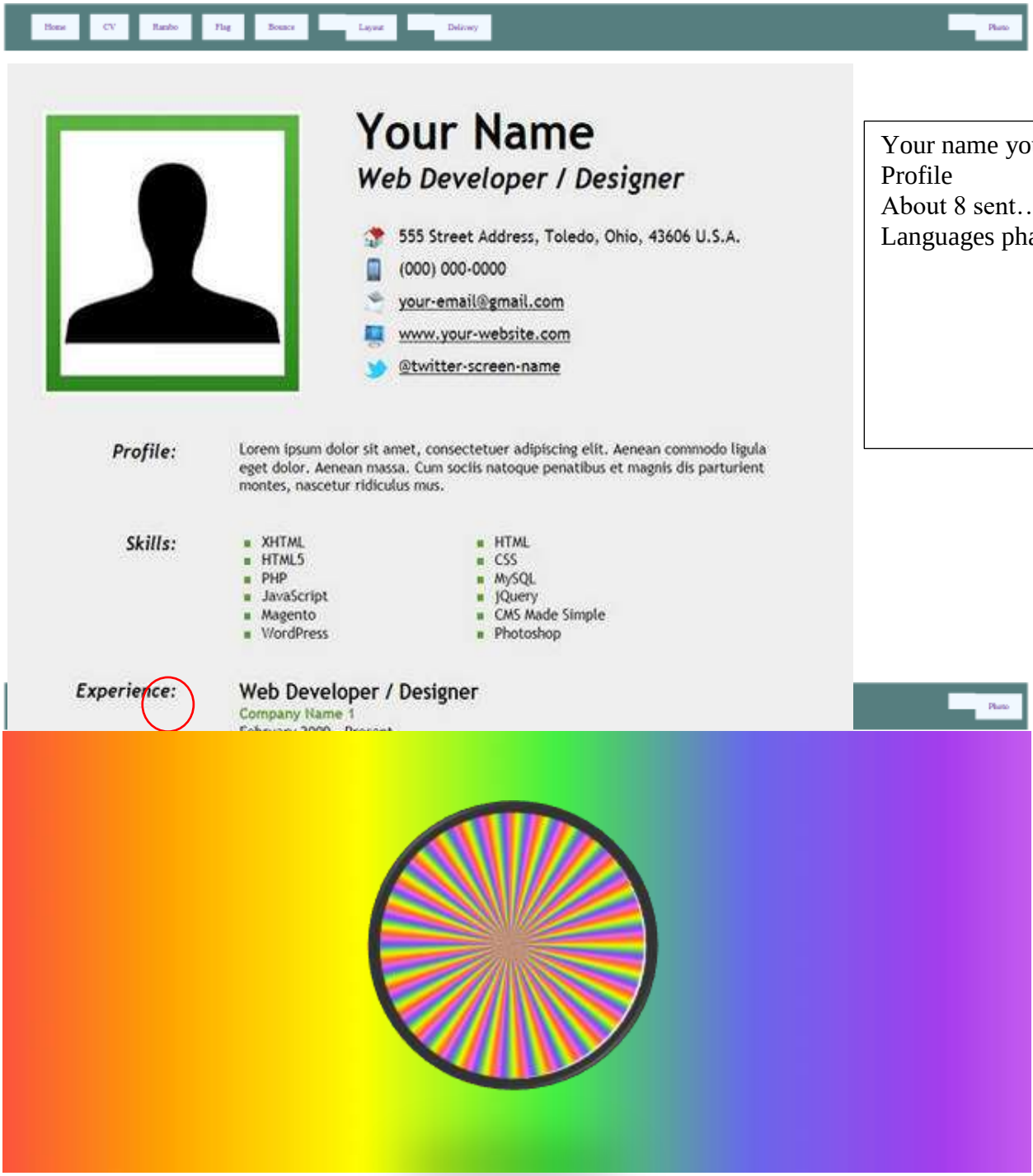
User Interface
Simplicity
Consistency

CanVa.com

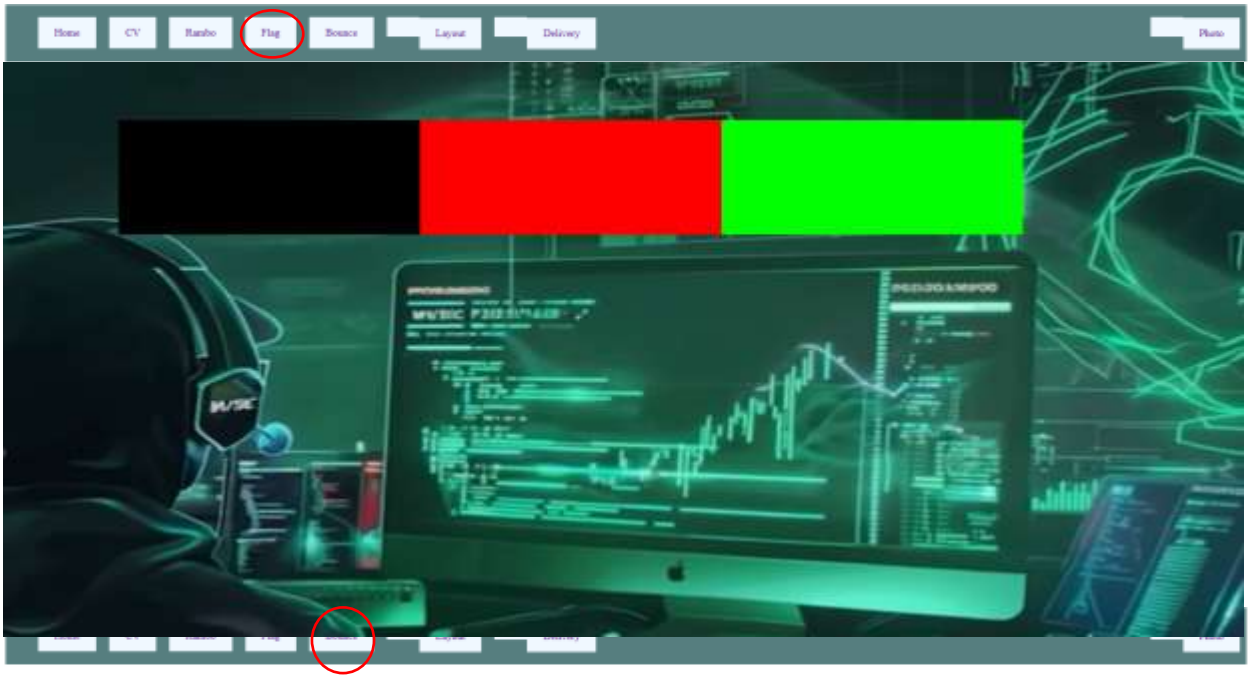
Where we can design a web site

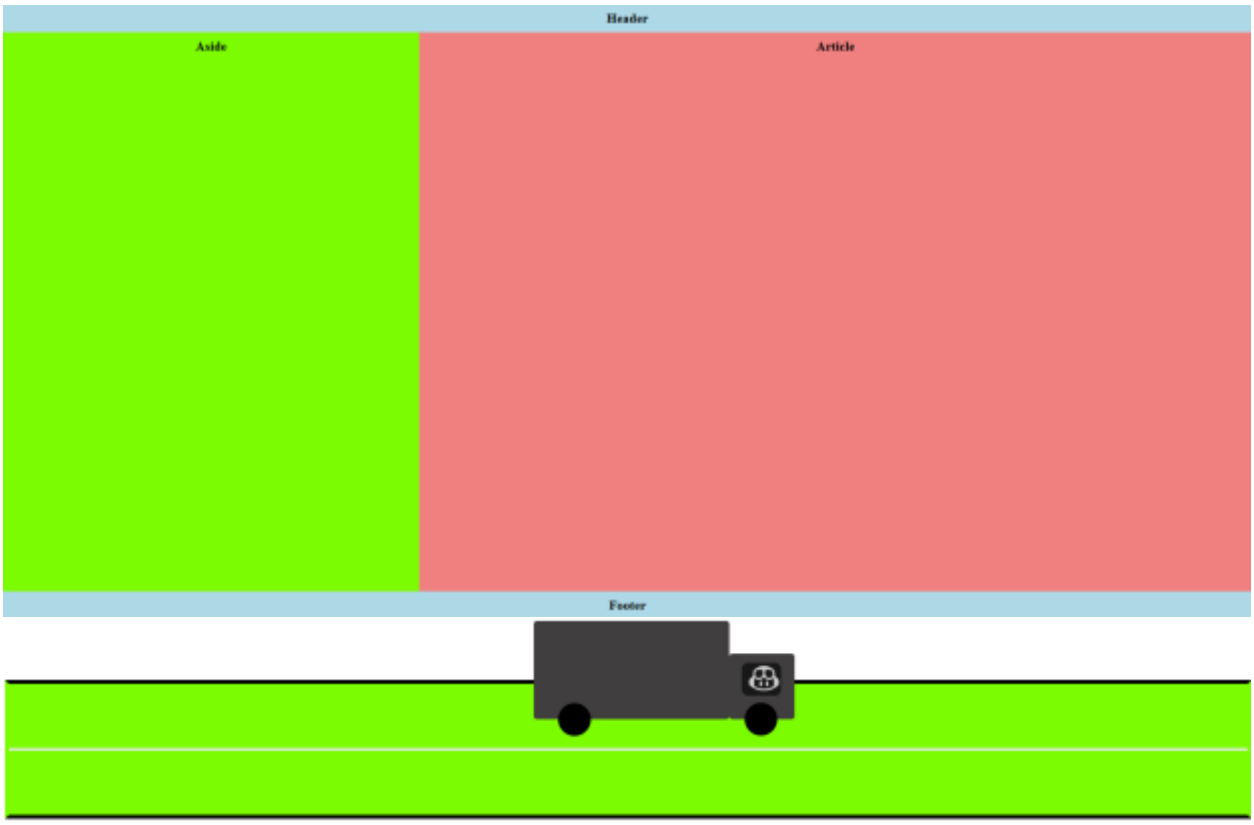
Capstone Projects

Table	Drop Down List	photo



Your name your photo
Profile
About 8 sent…
Languages pha dari eng


Home
CV
Rambo
Flag
Bounce
Layout
Delivery
Photo



JavaScript

JavaScript to program the behavior of web pages. Used in Web, Server, Mobile and Desktop app

Introduction to JavaScript (JavaScript for Beginners - Part 1)

- JavaScript is a Client Side Programming Language - View Diagram Here
  - https://w3techs.com/sites (Check Client Side and Server Side things here for QAFox.com)
- Basic knowledge on HTML, CSS, DOM are required for learning JavaScript
  - HTML creates the structure of the Web Pages
  - CSS styles the Web Pages
  - JavaScript makes the Web Pages dynamic
  - JavaScript uses DOM for accessing the elements on the web pages and modify them too
- Practical demonstration of HTML, CSS, DOM and JavaScript
  - Downloading and installing Notepad++
  - Only HTML Code - View Code Here
  - HTML + CSS - View Code Here
  - HTML + CSS + JavaScript + DOM - View Code Here
- Advantages of JavaScript
  - Reduces the load on the server
    - Example: tutorialsninja.com registration page
  - Makes the Web Pages Dynamic
    - Example: Changing colors of the text when we click on different buttons
    - View Code Here

عملی درس د بټن پواسطه د یوشی رنګ بدلول

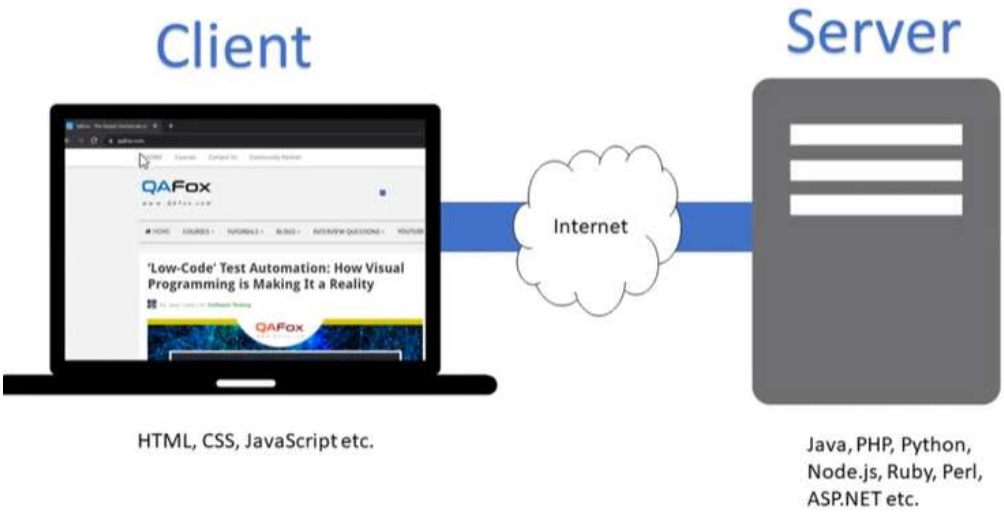
Make buttons with id red, blue white and black, then create div with id dv in css give width and height 300x300px then in script we put the below code to make the web page dynamic

```
document.querySelector("#red").onclick = () => {
    document.getElementById("dv").style.backgroundColor = "red";
};
document.querySelector("#blue").onclick = () => {
    document.getElementById("dv").style.backgroundColor = "blue";
};
document.querySelector("#white").onclick = () => {
    document.getElementById("dv").style.backgroundColor = "white";
};
document.querySelector("#black").onclick = () => {
    document.getElementById("dv").style.backgroundColor = "black";
};
```

```html
<!DOCTYPE html>
<html >
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Internal Style</title>
    <link rel="stylesheet" href="./style.css">

</head>
<body>

<div id="Mob">
    <div id="Screen"></div>
    <br><br><br><br>
    <button class="On">On</button>
    <button class="Off">Off</button>
</div>

<script src="./script.js"></script>
</body>
</html>
```

```css
#Mob
{
    width: 300px;
    height: 500px;
    background-color: #5f6567;
    border-radius: 10px;
    margin: 50px auto;
}
#Screen
{
    width: 200px;
    height: 250px;
    background-color: #eee;
    transform: translate(25%, 20%);
    border-radius: 10px;
}
button
{
    transform: translate(39%, 20%);
    padding: 30px 30px;
    border-radius: 5px;
    font-size: 30px;
    font-weight: 600;
}
```


```js
document.querySelector(".On").onclick = () => {
    document.getElementById("Screen").style.backgroundImage = "url(./p.jpg)";
    document.getElementById("Screen").style.backgroundColor = ""; // reset any color
}

document.querySelector(".Off").onclick = () => {
    document.getElementById("Screen").style.backgroundImage = "none"; // remove image
    document.getElementById("Screen").style.backgroundColor = "black"; // show black
}
```



siteGround

- Dropdown Menu
- Animated Sliders
- Maps
- Chart - Graphs
- Pop-up window
- Audio Players
- Video Players
- Zoom effect
- Animated Gallery
- Form Validations
- Accordions
- Calendar

JavaScript (JS) is a versatile, high-level, and dynamic programming language primarily used to add interactivity to websites. It enables dynamic content updates, multimedia control, animations, and more. While it is most known for client-side web development, it is also used in server-side environments like Node.js, making it a full-stack language.
Java & JS
ES/ EcmaScript & Versions : first it EcmaScript then they changed the name JS. Created by Brendan Eich in 1995. ES6
V8 engine is JS compiler

HTML DOM (Document Object Model)

DOM is used for selecting a web element through script file.

- DOM basics for JavaScript
  - DOM stands for Document Object Model
  - JavaScript uses DOM for accessing the elements on the HTML page and modify them
  - document
  - document.children
  - document.children[0]
  - document.documentElement
  - document.children[0].children[0].children[0]
  - document.children[0].children[0].children[1]
  - document.getElementById("para1")
  - document.getElementsByName("q")[0]
  - document.getElementsByClassName("hidden-xs hidden-sm hidden-md")[0]
  - document.getElementsByTagName("img")[5]
  - document.getElementById("table1").getElementsByTagName("tbody")[0].getElementsByTagName("tr")[2].getElementsByTagName("td")[2]
  - DOM Properties

Properties
.innerHTML
.style
.firstChild

```
document.querySelector("h1").innerText="Walmart";
document.querySelector("h1").innerHTML="Good bye";
```

```
document.firstElementChild.lastElementChild.querySelector("ul
").lastElementChild.innerHTML = "Angela";
```

Methods

.click()
.setAttribute()
.onClick()
.onLoad()

```
const btn = document.querySelector("#b1");
btn.onclick = ()=>{
    console.log("You have clicked the button")
}
btn.ondblclick = ()=>{
    console.log("You have clicked the button two times")
}
```

DOCUMENT (in chrome browser – inspect – console then it will work)

- **domain** : to check which domain we have = > document.domain // www.google.com
- **URL** : give us the complete url of website document.url
- **contentType**: what kind of data we received as html… docuemtn.contentType // html, image/png …
- **doctype**: what kind of document type we have
- **title**: what is the title
- **designMode** : work as contentEditable , we can edit anything on our page so do => document.contentMode = “On”;

LOCATION

- href: console.log(document.location.href)
- host
- protocol
- pathname
- hash
- origin
- port
- assign =>
-

Mouse Events

Event Name	Fired When
mouseenter	A pointing device is moved onto the element that has the listener attached.
mouseover	A pointing device is moved onto the element that has the listener attached or onto one of its children.
mousemove	A pointing device is moved over an element. (Fired continously as the mouse moves.)
mousedown	A pointing device button is pressed on an element.
mouseup	A pointing device button is released over an element.
auxclick	A pointing device button (ANY non-primary button) has been pressed and released on an element.
click	A pointing device button (ANY button; soon to be primary button only) has been pressed and released on an element.
dblclick	A pointing device button is clicked twice on an element.
contextmenu	The right button of the mouse is clicked (before the context menu is displayed).
wheel	A wheel button of a pointing device is rotated in any direction.
mouseleave	A pointing device is moved off the element that has the listener attached.
mouseout	A pointing device is moved off the element that has the listener attached or off one of its children.

```
for(var i =0; i<7;i++){
document.querySelectorAll(".drum")[i].addEventListener("click", function(){
    alert("how to select multiple buttons")
});
}
```

```
const btn = document.querySelector("button")
btn.addEventListener("click" , (e)=>{
    document.location.assign("https://www.google.com")
})
```

- replace : wont have back butt

```
const btn = document.querySelector("button")
btn.addEventListener("click" , (e)=>{
    document.location.replace("https://www.google.com")
})
```

- reload

```
<input type="text" placeholder="Text">
<button>
    convert
</button>
<h1>Result</h1>
```

```
const btn = document.querySelector("button")
const input = document.querySelector("input")
const result = document.querySelector("h1")
btn.addEventListener("click" , (e)=>{
    result.innerHTML = input.value
})
```

- **innerHTML**
- **innerText**
- **textContent**
- **getAttribute**
- **setAttribute**
- **createElement**
- **appendChild**

Dom

Adding a class to a web Element

Document.querySelector("h1").classList.add("huge");

Which adds a class that already been designed in CSS

```
document.querySelector("div").getAttribute("class")
```

```
document.querySelector("a").setAttribute("href", "https://www.bing.com");
```

QuerySelector

```
const table = document.querySelector("table");
console.log(table)
```

```
const table = document.querySelector("table tbody"); //select tbody
console.log(table)
```

```
const table = document.querySelector("table tbody tr"); //select tbody/ select row
console.log(table)
```

```
const table = document.querySelector("table:nth-child(2)"); //select 2nd table
console.log(table)
```

```
const table = document.querySelector("#A01"); //select by ID
console.log(table)
```

```
const phara = document.querySelector("p"); //select by tagName
phara.innerHTML = "My Name is Popal Furqani"; // assign new value to it
console.log(phara)
```

Phara.innerText or phara.textContent = "569" …

Can be taken by dom as html code ()

SetAttribute and **getAttribute**

Let phara = document.querySelector("body > p")

Phara.setAttribute("class", 'bg-dark') // select paragraph set class for it with JS

createElement: create any html element we want
createTextNode: creates a text
appendChild: assigns one element to another.
const phara = document.createElement("p")


```
let counter = 1;
setInterval(() => {
    console.log(counter)
    counter++;
}, 100);
```

```
const btn = document.querySelector(".btn")

btn.onclick = ()=>{
    document.writeln("clicked")
}
```

```
const phara = document.querySelector("p"); //select by tagName
phara.innerText = "My Name is Popal Furqani"; // assign new value to it
console.log(phara)
```

How to run JS code in Node with VS-Code
node script.js

Create compiler and text editor HOmeWOrk

```
    <link rel="stylesheet" href="style.css">
</head>
<body>

    <div class="text-editor">
        <textarea></textarea>
        <div class="compiler">

        </div>
    </div>

    <Script src="./script.js"></Script>
</body>
```

```css
* {
  box-sizing: border-box;
  margin: 20px;
  padding: 0;

}
.text-editor textarea, .text-editor .compiler
{
  width: 400px;
  height: 400px;
  background-color: #eee;
}
.text-editor
{
  display: grid;
  grid-auto-flow: column;
  justify-content: center;
}
```

Events

```js
const btn = document.querySelector(".btn")
btn.onclick = (e)=>{
    console.log(e.target) // what is trageted
}
```

e.preventDefault () // used with form submit and a tags not to go to next page or refresh page with submit.

```
keypress
keydown
keyup
focus
resize
scroll
submit

preventDefault
```

```js
window.onresize = ()=>
{
    console.log(window.innerWidth)
}
```

How to get value of an input in JS

```html
<form action="/test" method="get">
    <input type="text" placeholder="UserName">
    <input type="submit" value="Upload">
</form>
```

```
const form = document.querySelector("form");
const input = document.querySelector("form > input:first-child")

form.onsubmit = (e)=>
{
    e.preventDefault()
    console.log(input.value)
}
```

How to add data to a table from a form

ID	First Name	Last Name
1	as	asa

```
<form action="/test" method="get">
    <input type="text" placeholder="First Name" name="FirstName">
    <input type="text" placeholder="Last Name" name="LastName">
    <input type="submit" value="Upload">
</form>

<table>
    <thead>
        <tr>
            <th>ID</th>
            <th>First Name</th>
            <th>Last Name</th>
        </tr>
    </thead>
    <tbody>

    </tbody>
</table>
table
{
    border-collapse: collapse;
    margin-top: 30px;
}
table, td, th
{
    border: 1px solid lightblue;
    padding: 10px 30px;
    font-size: 32px;
}
thead
{
    background-color: lightslategray;
    color: white;
}
```

```
const form = document.querySelector("form");
const fn = document.querySelector("form > input:first-child")
const ln = document.querySelector("form > input:nth-child(2)")
const tbody = document.querySelector("table > tbody")

let counter = 1;
form.onsubmit = (e)=>
{
    e.preventDefault()
    const firstName = fn.value
    const lastName = ln.value
    row({firstName, lastName, id:counter});
    counter++;
}

function row(user){
    const row = document.createElement("tr");
    const id = document.createElement("td");
    const firstName = document.createElement("td");
    const lastName = document.createElement("td");

    const idnum = document.createTextNode(user.id)
    const firstNameText = document.createTextNode(user.firstName)
    const lastNameText = document.createTextNode(user.lastName)

    id.appendChild(idnum)
    firstName.appendChild(firstNameText)
    lastName.appendChild(lastNameText)
    row.appendChild(id)
    row.appendChild(firstName)
    row.appendChild(lastName)
    tbody.appendChild(row);
}
```

```
localStorage.setItem("firstName", firstName)
localStorage.setItem("lastName", lastName)
```

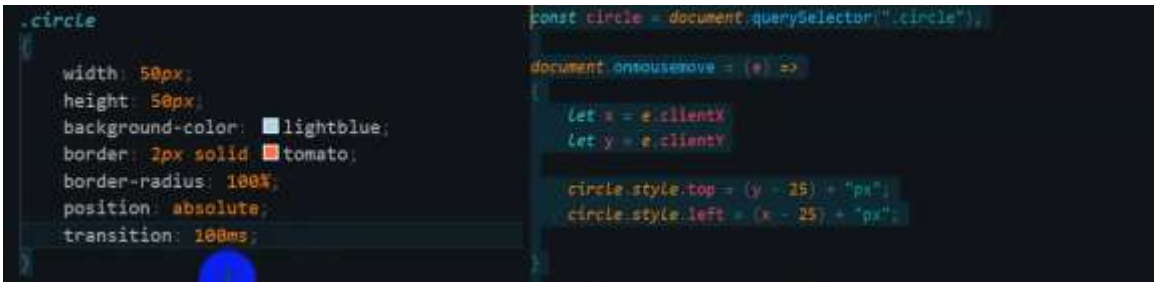
Mouse events

```
onclick
ondblclick
onmouseenter
onmouseleave
onmouseout
onmouseup
onmousedown
oncontextmenu
onmousemove
```

```
const btn = document.querySelector("#b1");
btn.onclick = ()=>{
    console.log("You have clicked the button")
}
btn.ondblclick = ()=>{
    console.log("You have clicked the button two times")
}
```

```
const btn = document.querySelector("#b1");
btn.onmouseenter = ()=>{ // when mouse enters
    console.log("You have entered the button area")
}
```

```
const btn = document.querySelector("#b1");
btn.onmouseup = ()=>{
    console.log("when we release mouse button")
}
```



```
.circle
{
    width: 50px;
    height: 50px;
    background-color: lightblue;
    border: 2px solid tomato;
    border-radius: 100%;
    position: absolute;
    transition: 100ms;
}
```

```
const circle = document.querySelector(".circle");

document.onmousemove = (e) => {
    let x = e.clientX
    let y = e.clientY

    circle.style.top = (y - 25) + "px";
    circle.style.left = (x - 25) + "px";
}
```

Using local storage of chrome – Application – local storage

```
console.log(localStorage)
localStorage.setItem("name","Ahmad");

console.log(localStorage.getItem("name"));
localStorage.removeItem("name")
```

```
let names = [
    {id: 1, firstName: "Ahmad", lastName: "Khan"},
    {id: 2, firstName: "Fird", lastName: "Jan"},
    {id: 3, firstName: "Asad", lastName: "Afghan"},
    {id: 4, firstName: "Karim", lastName: "Afghan"},
];

// Save to local storage
localStorage.setItem('users', JSON.stringify(names));

// Get back from local storage (parse string → object)
let users = JSON.parse(localStorage.getItem("users"));

// Now you can loop
users.forEach(user => {
    console.log(user.firstName, user.lastName);
});
```

Last updated: Sep 1, 2021

Download, Installing and using Visual Studio code along with Live Server Extension (JavaScript for Beginners - Part 2)

- Download, Installing and using Visual Studio code
- Live Server Extension
- Practical Demonstration - View Code Here

Last updated: Sep 2, 2021

Executing the JavaScript code - Let's Begin (JavaScript for Beginners - Part 4)

عملی درس

- Inline

```
<h2 onclick="alert(100+200)" >Sum => 100+200</h2>
```

- Internal

```
<script>
  document.querySelector("button").onclick= () =>
  alert("You clicked me")
</script>
```

- External & Console

Script.js

```
<Script src="./script.js"></Script>
```

```
JS script.js > ...
1   document.querySelector("button").onclick = ()=>
2       alert(100+200);
```

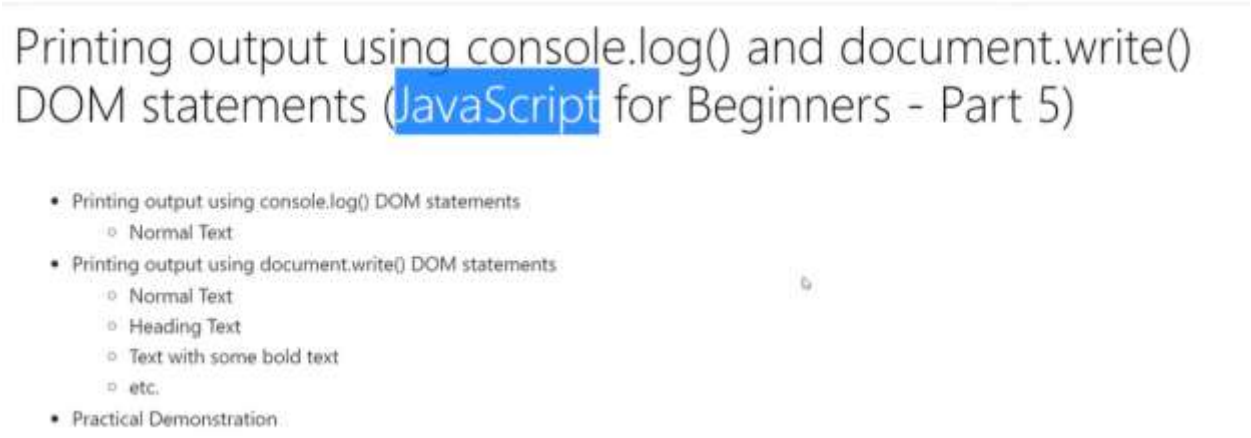
or

```
JS script.js
1   console.log(100+200);
```

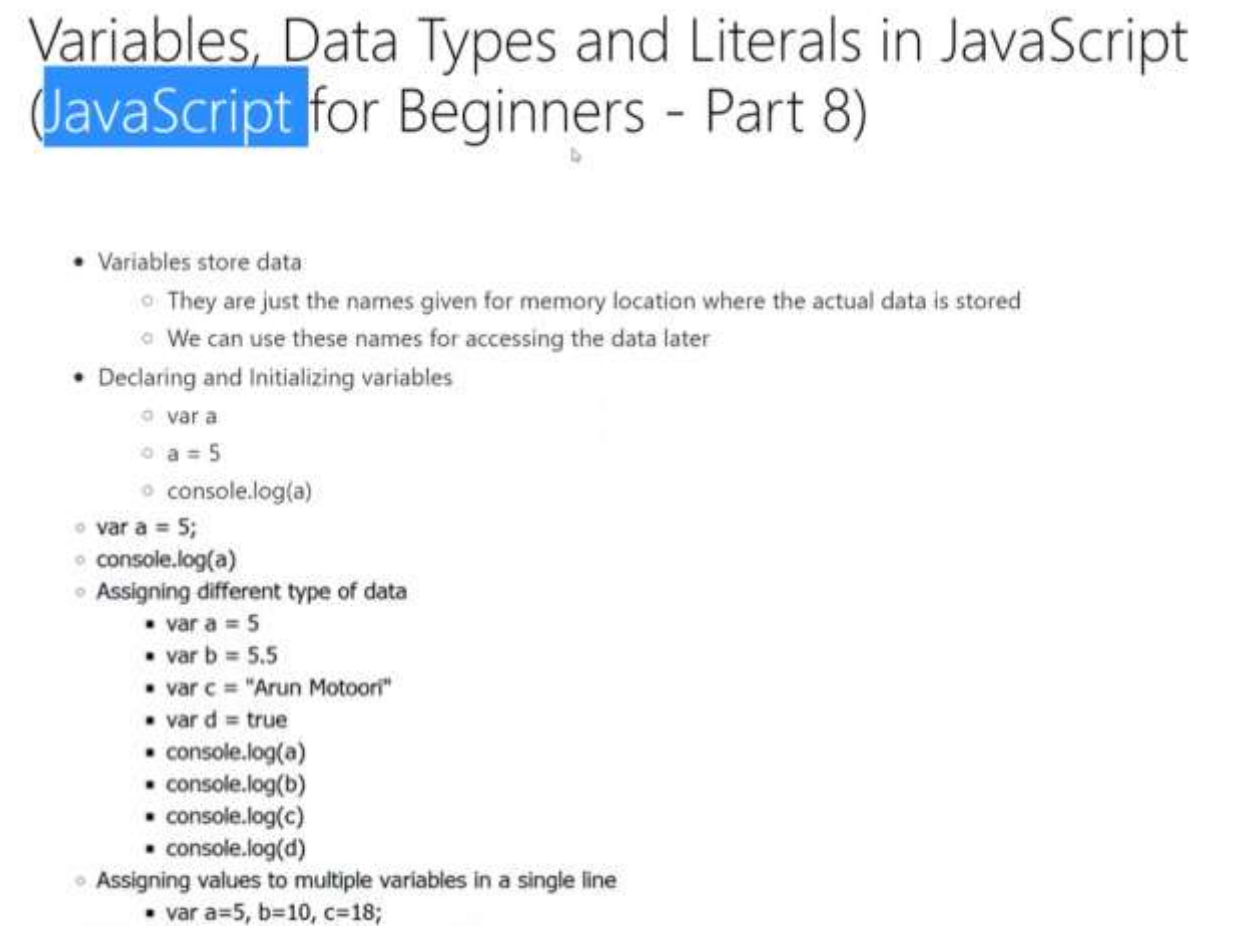
Console. Clear;

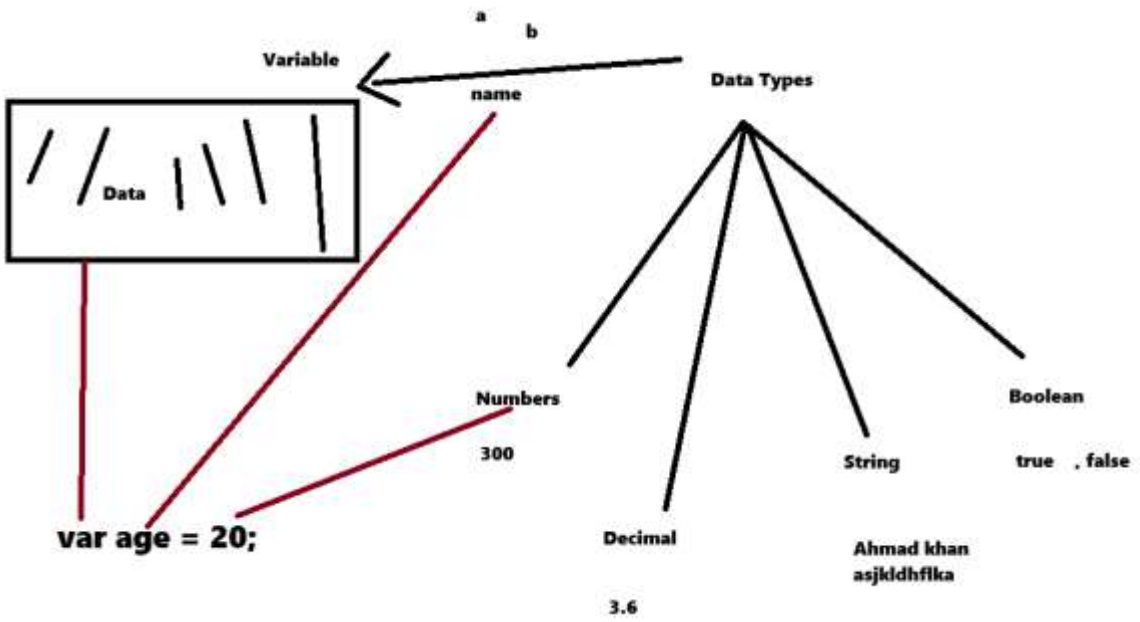
We put a <script> tag at the head and write our code which is called internal script

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <title>how to run JS code</title>
    <link rel="stylesheet" href="style.css">
    <script>
      alert("Welcome to Javascript")
    </script>
</head>
<body>

</body>
</html>
```


Printing output using console.log() and document.write()
DOM statements (JavaScript for Beginners - Part 5)
Printing output using console.log() DOM statements
Normal Text
Printing output using document.write() DOM statements
Normal Text
Heading Text
Text with some bold text
etc.
Practical Demonstration

Variable


Variables, Data Types and Literals in JavaScript
(JavaScript for Beginners - Part 8)
Variables store data
They are just the names given for memory location where the actual data is stored
We can use these names for accessing the data later
Declaring and Initializing variables
var a
a = 5
console.log(a)
var a = 5;
console.log(a)
Assigning different type of data
var a = 5
var b = 5.5
var c = "Arun Motoori"
var d = true
console.log(a)
console.log(b)
console.log(c)
console.log(d)
Assigning values to multiple variables in a single line
var a=5, b=10, c=18;



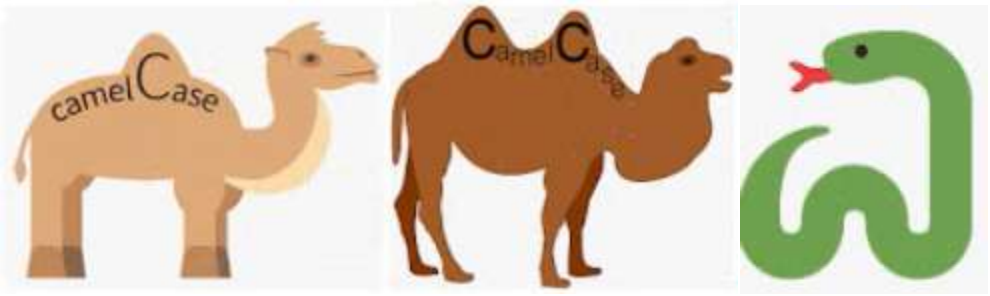

```
// Variable is a container which stores data.
// Data types 1-Number 2-Decimal 3-String 4-Boolean
var a; // Declare
a = 100; //  Initializtion
var b = "Khalid Ahmad"; // variable declaration and Initialization.

var firstName = "Ershad Ahmad"; // Varible name with Camel Case
var first_name = "Ershad Ahmad"; // Varible name with Snake Case

console.log(firstName, a, b); // prints variable
```

Variable: is a container. Stores data temporary. var = keyword

JS script.js > ...

```
1   var personName = "Ahmad";
2   console.log(typeof (personName));
3   // Multiple variables in single line
4   var zipCode = 23518, Country = "USA";
5   console.log(zipCode, Country);
```

We can see the result in the console of browser
Variable name can be in camel case or snake case
Camel case: showData, ShowData vs snake case show_data



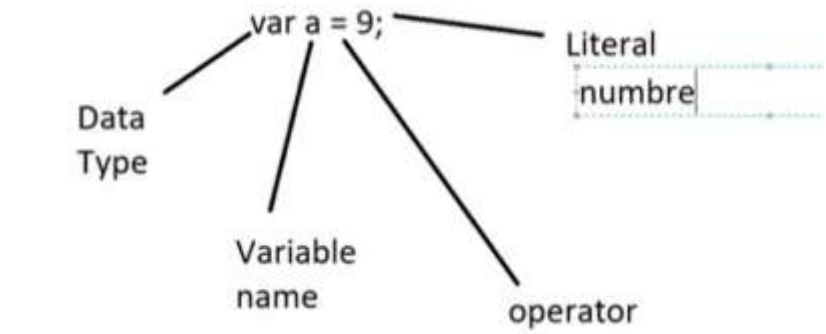
Let firstName = “Idress”; vs let first_name = “Idress”;

```
var tweet = prompt("Compose your tweet:");
alert("You have written " + tweet.length + " characters, you have " + (140 - tweet.length) + " characters remaining.");

//You have written 182 characters, you have -42 characters left.
```

```
// var variable can be override, changed value.
var age = 19;
var age =20;
age = 21;
console.log(age);
// let variable cant be overrided but changable by onther same variable name
let myName= "Ahmad";
myName= "Khan" //possible
// let myName="Jan" not possible

// const variable value can't be change once decleared
const salary = 1200;
// salary =500;  not possible you gonna get error
console.log(salary);
```

Data Types

- Data Types
  - No need to specify different data types for storing different types of values into the variables
  - JavaScript is a dynamically typed language
- Literal Types
  - typeof() function can be used to find-out the literals assigned to variables
  - var a = 9;
  - console.log(typeof(a)); gives number
  - var a = 9.5;
  - console.log(typeof(a)); gives number
  - var a = true;
  - console.log(typeof(a)); gives boolean
  - var a = "Arun Motoori"




```
Data types (JS is a dynamically typed language)

String ('value', "Value")
Boolean (true, false) / 1 or 0
Number (-84, 18, 1.84, 123e+12)
Array (multiple values in single variable)
Object (Properties and Methods)
null
undefined
```

var personName = “Ahmad”;	String
var age = 90;	number
var value = false;	Boolean
var value = [];	object array
var value = {};	object object

console.log(typeof value);

```
var a = 19;
var b =20;
total = a+b;
console.log(total);
```

- + * / are the same

```
var a = 19;
var b =20;
total = a%b; // reminder
console.log(total);
```

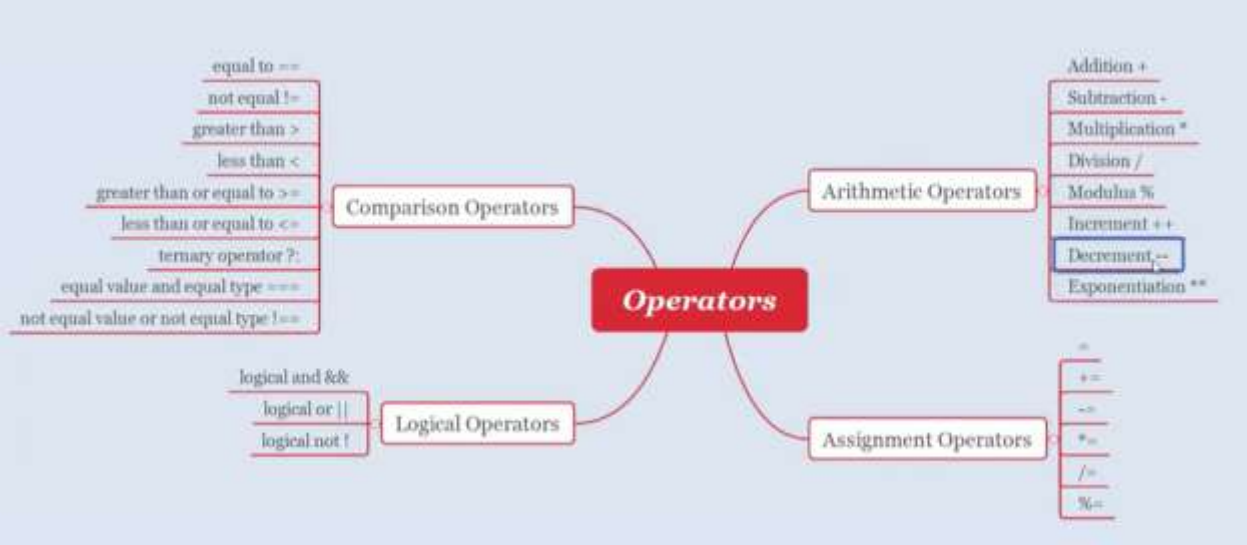
```
var a = 2;
var b =3;
total = a**b; // Power
console.log(total);
```

Operators

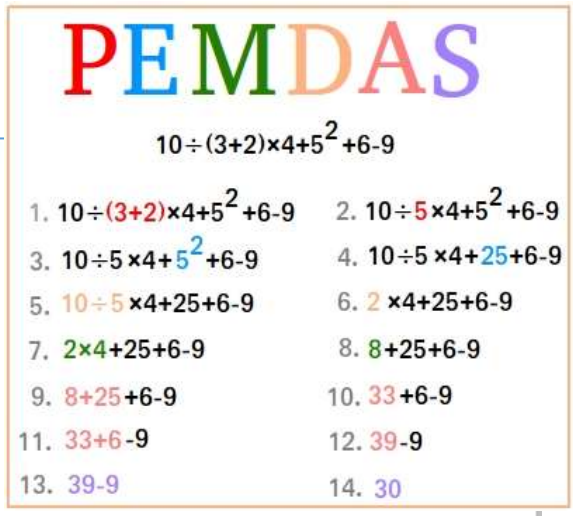
Operators in JavaScript (JavaScript for Beginners - Part 9)

- The below are the different types of Operators in JavaScript:
  - Arithmetic Operators
    - Addition +
    - Subtraction -
    - Multiplication *
    - Division /
    - Modulus %
    - Increment ++
    - Decrement --
    - Exponentiation **
  - Assignment Operators
    - =
    - +=

0.



```
// Arithmetic Operators
var a = 9;
var b = 2;
console.log(a+b);
console.log(a-b);
console.log(a*b);
console.log(a/b);
console.log(a%b); // Reminder
console.log(a**b) // 9**2 = 9*9 // Exponential or power
console.log((10+2)*10/2)
a++; // 10  // increament by one
console.log(a) //10
a--; // 9  //  decreament by one
console.log(a) //9
```




```
// assignment Operators  =   +=   -=
var a = 5;
// a = a +2; // a +=2;
// a = a - 2; // a -=2;
// a = a * 2; // a *=2;
// a = a / 2; // a /=2;
// a = a % 2; // a %=2;
console.log(a)
var a = 5;
var b = 6;
console.log(a==b)
console.log(a!=b)
console.log(a>b)
console.log(a<b)
```

=== strick equal operator consider data type as well, if we compare data type level
Var a = "9";
Var b = 9; console.log(a===b);
(10+2) *10 /2 ➔ 12 * 10 /2 ➔ 12*5➔ 60 PEMDAS

```
console.log(10==10 && 12==13)   // and
console.log(10==10 || 12==13)  // or
```

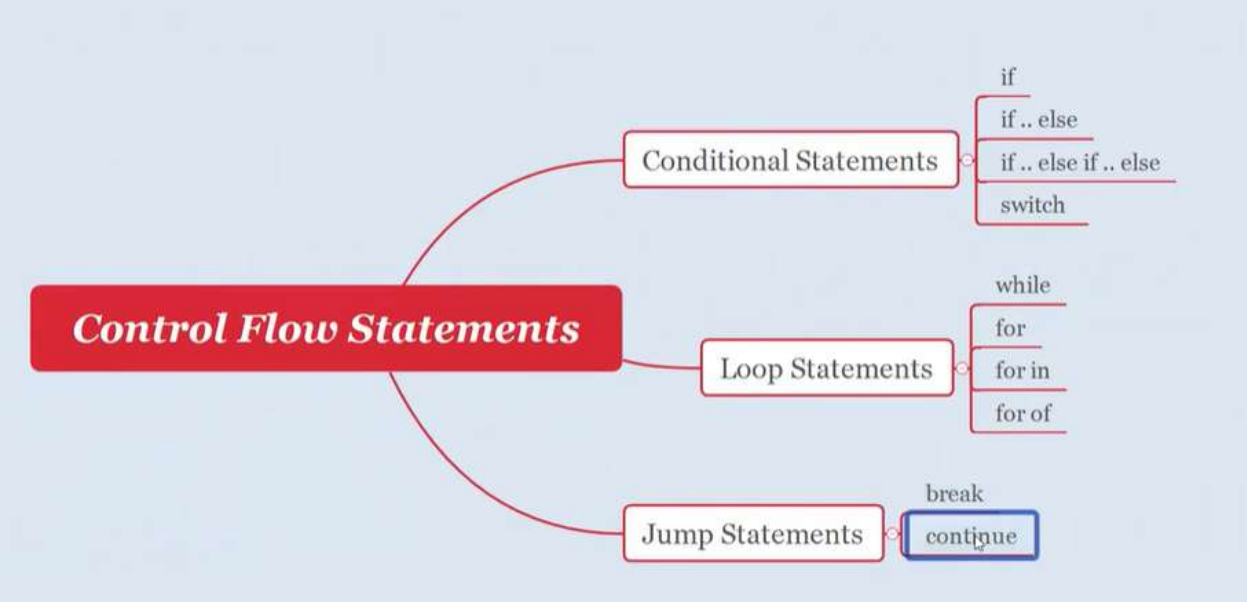
Comments in JavaScript
// single line comment
/* Multi lines Comments */

Const vs Var keyword:

Var : we can change the value anywhere in our program.
Var a = 5; a = 9;
Const: we can change the value once assigned to a Const variable.

Control Statement

Control Statement
Used to control the flow of the program.
Assign to
Var age = 23;
=== is equal to (compares according to checking the data Type)
== is equal to (compares without checking the data Type)

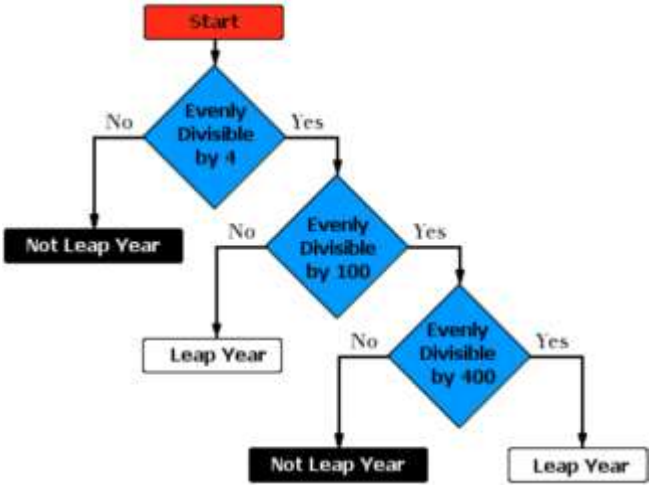


Conditional Statement

Conditional Statements in JavaScript (JavaScript for Beginners - Part 13)

- Flow of the execution in Conditional statements will depend on the condition
- The below are the different types of conditional statements available in JavaScript
  - if statement - View Flow Here
  - if .. else statement - View Flow Here
  - if .. else if .. statement - View Flow Here
  - switch statement - View Flow Here

```
if(90==90){
console.log("Welcome to if condition block")}
```



```
let age = "16";
if(age >= 18){
    console.log("you can vote")
}else{console.log("Sorry, You are too young to vote")}

let score = 75;
if(score >= 90){
    console.log("A Grade");
}
else if (score >= 80){
    console.log("B Grade");
}
else if(score >= 70){
    console.log("C Grade");
}
else{console.log("Fail");}


var num = 55;
if( num % 2 === 0){
    console.log("this is even")
}
else{
    console.log("this is odd")
}
```

```
var a = 5, b=6, c=8;

if(a>b && a>c){console.log("a is greathest")}
else if(b>a && b>c){console.log("b is the greathest")}
else{console.log("c is greathest")}
```

```
var light = "red";

if(light=="Green"){
    console.log("go")
}
else if (light=="yellow"){
    console.log("slow down")
}
else{console.log("Stop")}
```

Const username = “Ahmad”;
Const password = “afghan”;
If ()

عملی درس د ایف سټیټمنټ سره

```
let email = prompt("Please enter your Email")
console.log(email);  // user input email - variable - console
```

```
let userName = "Ahmad";
let userPass = "ahmad123";

let inputName = prompt("Please Enter your Name");
let inputPass = prompt("Please Enter your Password");

if(userName == inputName && userPass == inputPass){ // compares user data with input data
    alert("Welcome to JS course");
}else{alert("Please enter correct student user name and password");}
```

```
let age = prompt("Please Enter your Age");

if(age>18){ // compares user data with input data
    alert("Welcome to DMV course");
}else{alert("Sorry you are not elegibile for DMV course");}
```

```
let age = prompt("Please Enter your Age");

if(age<18){ // compares user data with input data
    alert("Sorry you are not elegibile for DMV course");
}else if(age== 18 || age == 19){alert("You can apply for Educational Test only");

}else(alert("Welcome to DMV Course"))
```

Switch Case

```
let day = "ksajdhf";

switch(day)
{
    case "Saturday": console.log("OFF");break;
    case "Sunday": console.log("Off");break;
    case "Monday": console.log("English and History");break;
    case "Tuesday": console.log("Science and Biology");break;
    case "Wednsday": console.log("Management and Pashto");break;
    case "Thursday": console.log("Math and Computer");break;
    case "Friday": console.log("Art and Eng");break;
    default: console.log("Not valid day");
}
```

```
let num1 = 100;
let num2 = 12;
let operator = "-";

switch(operator){
    case "+" : console.log(num1+num2);break;
    case "-" : console.log(num1-num2);break;
    case "*" : console.log(num1*num2);break;
    case "/" : console.log(num1/num2);break;
    default: console.log("Invalid Operator");
}
```

Will play all case one by one because we don't have break

```
let a = 9;
switch(a)
{
    case 1:
        console.log("Sat"); alert("Sat"); break;
    case 2:
        console.log("Sun"); alert("Sun"); break;
    case 3:
        console.log("Mon"); alert("Mon"); break;
    case 4:
        console.log("Tue"); alert("Tue"); break;
    case 5:
        console.log("Wed"); alert("Wed"); break;
    case 6:
        console.log("Thurs"); alert("Thurs"); break;
    default: alert("Today is off");
}
```

Loop:

While Loop:

While loop

```
var i = 1;
while(i<=10){
    console.log("Popal Furqani");
    i++;
}
```

```
//                    ICU (Initiazation, Condition, Update)
let num1 = 1; // I
while(num1<=10){   // C
        console.log(num1)
    num1++;  // U
}
```

```
let num1 = 1;
while(num1<=10){
    if(num1%2==0){
        console.log(num1)
    }
    num1++;
}
```

```
let num = 20 ;
while(num>=1){
    if(num%2==1){
        console.log(num)
    }
    num--;
}
```

```
let i = 1;
while(i<10){
    document.writeln("Hi");
    i++;
}
```

```
let i = 1;

do
{
    console.log("DO WHILE");

    i++
} while(i <= 10)
```

For loop:

- Iterates statement multiple times until condition is met.
- **Repeat same block of code multiple times**
- For loop is advance type in the loop statement
- I initialization C condition U update = **I C U**
- Syntax – **for(let i=0; i<=100 ; i++){}**

- When we sure about the iterations then we are using for loop.

```
// For Loop
//     I          C       U     (Initiazation, Condition, Update)
for(let num =1; num<=10; num++){
    console.log(num)
}
```

```
for(var i =10; i>=0;i--){
    console.log(i)
}
```

```
for(var i =1; i<=10;i++){
    if(i%2==0){ // prints even numbers
    console.log(i)
    }
}
```

```
document.writeln("<h1>For Loop</h1>")

for(let i =0; i<11; i++){
    alert(i);
}
```

```
alert("Welcome to Loop");
for(let i =0; i<10; i++){
    document.writeln('<h'+i+'>'+ i +'</h'+i+'>'); // i will be printed as header1
}
```

```
alert("Welcome to Loop");
for(let i =1; i<7; i++){
document.writeln(`<h${i}>${i}</h${i}>`); // i will be printed as header123456
}
```

```
let a = 2;
alert("Welcome to Loop");
for(let i =1; i<11; i++){
document.writeln(i +" x "+ a +" = "+ i*a); // Table of 2
}
```

```
let names = ['Ahmad', 'Khalid','Taj', 'Younas','Hakim','Mosa'];

for(let i =0; i<6; i++){
document.writeln(`<h1>${names[i]}</h1>`); // Array
}
```

```
let table = 4;
for(let i =1; i<=12; i++){
    console.log(i, "X", table, "=", table*i)
}
```

```
let table = prompt("Please enter a number to make a table");
for(let i =1;i<11;i++){
    document.writeln(`<h1>${i +" x "+table+ " = "+ i*table}</h1>`);
}
```

```
for (let index = 1; index < 100; index++) {

    console.log(index % 2 == 0 ? index + " EVEN" : index + " ODD")

}
```

Local and Global Scope:

```
//   scope of local and Global variables
for(let i = 1;i<= 10; i++ ){
    document.writeln("i") // locally can be access
}
console.log(i) // we cant access local i variable out side but we can if global
```

Object:

ahmad is an object in the below example

```
//   object // Key Value pairs, or Property value
let ahmad = {
    firstName: "Ahmad",
    lastName: "Jan",
    getFullName:function(){
        console.log(`${this.firstName} + ${this.lastName}`)
    }
}
console.log(ahmad.getFullName()); // to access function inside the object
```

```
//   object // Key Value pairs, or Property value
let ahmad = {
    firstName: "Ahmad",
    lastName: "Jan",
    getFullName:function(){
        return`${this.firstName}` + `${this.lastName}`;
    }
}
console.log(ahmad.getFullName()); // to access function inside the object
```

This always point to its parent.

Lets make print button with JS: create button on HTML then

```
document.querySelector("button").onclick =()=>
{
    window.print();
}
```

```
const printBtn = document.querySelector("button");
printBtn.onclick = ()=>
{
    window.print();
}
```

This keyword

```
function Student(firstName, lastName, course, timing){
    this.firstName =firstName;
    this.lastName=lastName;
    this.course=course;
    this.timing=timing;
    this.getFullName = function(){
        return this.firstName+this.lastName;
    }
}
const ahmad = new Student("Ahmad","Afghan","HTML", "8PM");
console.log(ahmad.firstName, ahmad.lastName, ahmad.course, ahmad.timing);
console.log(ahmad);
```

```
function Student(firstName, lastName, course, timing){
    this.firstName =firstName;
    this.lastName=lastName;
    this.course=course;
    this.timing=timing;
    this.getFullName = function(){
        return this.firstName+this.lastName+this.course+this.timing;
    }
}
const ahmad = new Student("Ahmad","Afghan","HTML", "8PM");
console.log(ahmad.firstName, ahmad.lastName, ahmad.course, ahmad.timing);
console.log(ahmad);
console.log(ahmad.getFullName());

const khalid= new Student("Khalid", "Jan", "CSS","6PM")
console.log(khalid)
```

JavaScript object Notation (JSON): data storage format (is same as object) we can also use it as small database, used in Rest API too
Data.json file – small database, load it with js

data.json > ...

```
{
    "firstName":"khan",
    "LastName":"jan",
    "age": 90,
    "opt": [12,44]
}
```

fetch('data.json') → Loads the JSON file.
.then(res => res.json()) → Parses it to a JS object.
.then(data => console.log(data)) → Logs the data.
.catch(err => console.error(err)) → Catches any errors.

break and continue Statements in JavaScript (JavaScript for Beginners - Part 15)

- break will completely get out of the loops, when it gets executed
- continue will just cancel the current iteration in the loop, when it gets executed
- Practical Demonstration

```
for(var i =1; i<=10;i++){
    if(i==5){ break;}
    console.log(i)
}
for(var i =1; i<=10;i++){
    if(i==5){ continue;} // only the current iteration be conceled
    console.log(i)
}
```

Function:

Functions in JavaScript (JavaScript for Beginners - Part 16)

- Function is a block of statements which performs a specific task
- Function is only executed when it is called
- Re-usability is the major advantage we get by using functions
- Syntax of the function in JavaScript
- Practical Demonstration of a basic Function
  - Create a function which prints 'Hello' text when called
- Functions can intake data
  - Create a function which intakes our Name and prints "Hello GivenName" when called
  - Create a function which intakes two numbers and prints the sum of the in-taken values when called
- Functions can return data
  - Create a function which intakes two numbers and returns the sum of give give values when called
- Executing the JavaScript functions from HTML Code


```
function getMilk(bottle){
    console.log(bottle*2 +"$")
}
getMilk(3)
```

Function is a block of code to perform a task.

Function add () {statement} => call this function add();

```
function sayHello(firstName){ // parameter
    console.log("Hello " + firstName)
}
sayHello("Popal") // calling function // Popal is Argument
sayHello("Khan")

function add(a,b){
    console.log(a+b)
}
add(12,12)
```

```
function add(){
    document.writeln(12+12);
}
add()  // call the function
```

We can see function result in console.log, document.writeln or in an alert

```
let a = 10;
let b = 20;
function add(){
    document.writeln(a+b);
}
add()  // call the function
```

```
let a = 10;
let b = 20;
function add(){
    return a+b;
}

let total = add();  // call the function
console.log(total);
```

```
let a = 10;
let b = 20;

function add(){
    return a+b;
}

console.log(add());
```

```
function add(a,b){  // Paramiters
    return a+b;
}
console.log(add(10,20));
```

```
function add(a,b){  // Paramiters
    return a+b;
}
console.log(add(prompt("enter a value"),prompt("enter b value")));
```

```
function getFullName(firstName,lastName){  // Paramiters
    return firstName+lastName;
}
console.log(getFullName(prompt("enter First Name"),prompt("enter Last Name")));
```

```
function calc(sign, v1, v2){
    switch(sign){
        case '+': return v1+v2;
        case '-': return v1-v2;
        case '*': return v1*v2;
        case '/': return v1/v2;
        default: "incorrect sign";
    }
}
let sign = prompt("Please Enter sign")
let value1 = Number(prompt("Please Enter first value"))
let value2 = Number(prompt("Please Enter second value"))
console.log(calc(sign,value1,value2))
```

```
// we wil see hi 3 sec later in console
setTimeout(function(){
    console.log("Hi")
},3000);
```

Works as loop repeat the code multiple times

```
let i = 1;
setInterval(function(){
    document.writeln(i)
    i++;
},1000);
```

Works the same

```
let sum = function(){

}
let sum = ()=>{

}
function sum(...para)
{   for(let i =0; i<para.length;i++){
    console.log(para[i]);
}
}
sum(10,20,30,40,50,60,70,80,90);
```

```
let total = 0;
function sum(...para)
{   for(let i =0; i<para.length;i++){
    // console.log(para[i]);  // show array one by one
    total += para[i];
}
return total;
}
let result = sum(10,20,30,40,50,60,70,80,90);
console.log(result); // shows total of all array
```

```
let sum = () =>
{
    console.log(" arrow function assigned to a variable")
}

sum();
```

Here we don't need to call the function name we directly call variable

```
let sum = (a,b) =>
{
    console.log(a+b)
}

sum(1,2);
```

```
function fullName (firstName,lastName = "Khan") // default value assigned
{
    return firstName+" "+lastName;
}
console.log(fullName("Ahmad")) // Ahmad Khan
```

```
function parent(x) { // nested functions
    function child(y){
        return x+y;
    }
    let a = child(10);
    console.log(a);
}
parent(12);
```

Make buttons to change text color

```
document.querySelector(".b1").onclick = function () {
    document.getElementById("p1").style.color = "red";
};
```

```
    <script>
        function changeColor(textColor){
            document.getElementById("para1").style.color = textColor;
        }
    </script>

/head>
body>
    <div class="left"></div>
    <div class="right"></div>
    <div class="clear-fix"></div>
    <p id="para1">Lorem ipsum dolor, sit amet consectetur adipisicing elit. Consectetur nesciunt molli
     <button onclick="changeColor('red')">Red</button>
     <!-- call the above function and assign arguments red to textColor parameter -->
    <button onclick="changeColor('green')">Green</button>
    <button onclick="changeColor('blue')">blue</button>
/body>
/html>
```

How to make calculator function HTML+JS

```
<input id="num1" type="text" placeholder="number 1">
<input id="num2" type="text" placeholder="number 2">
<br><br>
<p id="para1"></p>
<br>
<button onclick="add()">Add</button>
<button onclick="sub()">Sub</button>
<button onclick="mul()">Mul</button>
<button onclick="divi()">Div</button>
```

```
function add(){
    var a = document.getElementById("num1").value
    var b = document.getElementById("num2").value
    var res = parseInt(a)+parseInt(b);
    document.getElementById("para1").innerHTML=res;
}
function sub(){
        var a = document.getElementById("num1").value
        var b = document.getElementById("num2").value
    var res = parseInt(a)-parseInt(b);
    document.getElementById("para1").innerHTML=res;
}
function mul(){
        var a = document.getElementById("num1").value
        var b = document.getElementById("num2").value
    var res = parseInt(a)*parseInt(b);
    document.getElementById("para1").innerHTML=res;
}
function divi(){
        var a = document.getElementById("num1").value
        var b = document.getElementById("num2").value
    var res = parseInt(a)/parseInt(b);
    document.getElementById("para1").innerHTML=res;
```

```
const compiler = document.querySelector(".compiler");
const codeArea = document.querySelector("textarea");

codeArea.onkeyup = ()=>
{
    let code = codeArea.value;
    compiler.innerHTML = code;
}
```

let verus var keywords in JavScript (JavaScript for Beginners - Part 17)

- Difference between var and let keywords
- var is function scoped
- let is block scoped
- var attaches the variables to the DOM window, hence its not recommended to use
- Going a head, we will use let in place of var.
- Practical Demonstration

```
function fone(){
    for(var i =1;i<5;i++){
        console.log(i);
    }
    console.log(i) // is accessable cuz i is function level
}
fone();
```

```
function fone(){
    for(let i =1;i<5;i++){
        console.log(i);
    }
    console.log(i) // is not accessable cuz i is declear block level
}
fone();
```

Declaring variables without any var keyword in JavaScript (JavaScript for Beginners - Part 18)

- When you declare a variable without any keyword say var keyword, the scope of the variable changes to global scope
- Practical Demonstration

```
var a = 10; // local
function fone(){
    var a = 5;
        console.log(a); // local 5
}
fone();
console.log(a);// 10
```

Object

Objects in JavaScript (JavaScript for Beginners - Part 19)

- Objects are real world entities
- Objects contain properties and methods
- Creating object along with some properties
- Accessing properties of the object individually
- Accessing properties of the object using 'for in' loop
- Adding new properties to existing object
- Updating existing properties of an object
- Deleting the existing properties of an object
- Adding methods to the object
- Calling the methods in the object
- Practical Demonstration

Class is the blueprint of object like map of house is class and house by itself is object

```
let car ={ // we created car object color price milage are properties
    color:"white",
    price: 16000,
    milage:73000
}
console.log(car["color"]); // access the value of property of car object
for(let p in car){
    console.log(car[p])
}
```

```
let car ={ // we created car object color price milage are properties
    color:"white",
    price: 16000,
    milage:73000,
    startCar: function(){ //add function to object
        console.log(this.color + "car started");
    }
}
console.log(car["color"]); // access the value of property of car object

car["model"]="Honda Odessey"; // add new property to object
car["price"]=15000; // update property to object
 // delete car["price"]  //delete a property

for(let p in car){
    console.log(car[p])
}
car.startCar(); // call function inside the object
```

Arrays

Allow us to store multiple values into a single variable. Let a = [4,5,3]

- Storing different types of data into a single Array
- Updating the existing values in Array using index
- arrayname.pop() removes the last element from the array and returns the removed element
- arrayname.push() adds a new element at the end of the array and returns the length of the array after adding the element
- arrayname .shift() will remove the first element in the array and shift left the remaining element to the lower index
- arrayname .unshift() will add a new element at the first position and shift right the remaining elements to the higher index
- delete arrayname[index] removed the element from array at that specified index, but won't clear the space
- Array.isArray() returns true for Array
- typeof() returns object for Array
- array1.contact(array2) merges two arrays to one and returns the merged array
- array1.contact(array2,array3) merges more than two arrays to one and returns the merged array
- arrayname.slice(index) will slice the array from the given index and return the sliced array
- arrayname.sort() sorts the elements in the array and returns the ascending sorted array
- arrayname.reverse() reverses the elements in the array and returns the reversed array

```
// Array : checklist if in the list allow or kick out
var guestList = ["Asma", "Husna", "Idress", "Ershad", "Khalid", "Zoya", "Muska", "Hazrat"];
var guestName = prompt("Please Enter Your Name to Varify");
if(guestList.includes(guestName)){
    alert("Welcome")
}else{alert("Sorry you are not in the list or try your other name")}
```

```
let a = [4,6,8,9,5,3,2,1];

console.log(a[3]) // index 3
console.log(a.length)  // length of array
for(let i=0;i<a.length;i++){ //using for loop
    console.log(a[i])
}
for(e of a){  // for of loop special for array
    console.log(e)
}
```

For each loop:

script.js > ...

```
let arr = ["Ahamd", 1000, true, ["Khalid"], {name:"Jan"}];
for(let i=0; i<arr.length; i++){
    console.log(arr[i]);
}
```

script.js > ...

```
let arr = ["Ahamd", 1000, true, ["Khalid"], {name:"Jan"}];
arr.forEach((item)=>
{
    console.log(item)
})
```

```
let arr = ["Ahamd", 1000, true, ["Khalid"], {name:"Jan"}];

arr.push("Khan"); // add value to the last part of array
arr.pop("Khan");  // remove value form the last part array
arr.shift();  // remove the first value form array
arr.unshift();  // add value to the first part of array

arr.forEach((item)=>
{
    console.log(item)
})
let a = [4,6,8,9,5,3,2,1];
let b = [100,200,300,400]

a[0] = 100;  // update value of array
console.log(a[0])
a.pop() // remove element from end
a.push(7) // add new value to the end
a.shift(); // remove the first element of array
a.unshift(200) // add new element to at endex 0
delete a[4]; // delete the item in index of 4
console.log(typeof(a)); // show type of array which is object
let c = a.concat(b) // join two arrys
a.slice(4);
a.sort(); // sort array
a.reverse(); // sort in reverse manaer
// store object in array
let car = {
    color:"red",
    price:3000,
    milage:4500
}
let a =[car,"2019"] // stored obj in array
console.log(a.toString()); // convert array to string
```

Type Casting

- console.log(parseInt(stringintegervalue)+3)
- console.log(parseInt(stringdecimalvalue)+3)
- console.log(parseInt(decimalvalue)+3)
- console.log(parseFloat(stringdecimalvalue)+3)
- console.log(String(integervalue)+3)
- console.log(a.toString()+3)
- console.log(Number(booleanvalue))
- console.log(String(booleanvalue)+" is string now")
- console.log(String(datevalue)+" is String now")

Type Casting : convert one datatype to an other.

```
// implicit type casting - js will automatically convert
console.log(2+3.4) // interger will be coverted to decimal automatically // 5.4
console.log("2"+4) // interger will be coverted to string automatically // 24
// Explicit - we provide code to covert one data type to another
let a = "5";
let b = 4;
console.log(Number(a)+b);
console.log(parseInt(a)+b); // only number no decimals
console.log(a+String(b)); //
let c = new Date();
console.log(c.toString());
```

Dates:

Dates in JavaScript (JavaScript for Beginners - Part 22)

- In JavaScript, we have to create object for Date by using new keyword and Date() constructor
- let d = new Date();
- JavaScript uses browsers time zone for Date
- console.log(d)
- console.log(d.toString())
- console.log(d.getFullYear())
- console.log(d.getDate())
- console.log(d.getMonth())

```
// How to create Date and Time
let d = new Date();
console.log(d.getDate())
console.log(d)
console.log(d.toString())
document.writeln(d)
```

```
(()=>{

    const date = new Date();
    console.log(date)
    document.writeln(date)


})()
```

```
(()=>{

    const date = new Date();
    console.log(date)
    // document.writeln(date)
    document.writeln(date.getDate())
```

```
(()=>{
    const date = new Date();
    date.setFullYear(2022);
    document.writeln(date.getFullYear())
})()
```

```
(()=>{
    const date = new Date();
    document.writeln(date.toLocaleDateString())
})()
```

String

// Program under 140 Chars

```
// var tweet = ;
// var tweet140 = tweet.slice(0,14);
alert(prompt("Please Enter Your Name Here!").slice(0,15).toUpperCase())
```

```
let a = "Ahmad \n Khan";  //  \n new line \t tab  \
console.log(a.toUpperCase())
console.log(a.length())
console.log(a.charAt(1)) // print any thing on index 1
console.log(a.concat(" jan").concat("Eng")) // concatenate
console.log(a.replace("Khan jan"))
a.split(" ") // split by space
```

```
console.log(a.length);
console.log(a.replace("Ershad Ahmad","Khalid Ahmad Furqani"));
```

OOPS

Class and Object

Classes, Objects, Class Properties, Constructors and static (JavaScript for Beginners - Part 24)

- Class is a template or blueprint using which we create objects
- Object is an instance of a Class and is a real world entity
- Example: Car is a Class, then objects can be different cars
- Practical Demonstration of Class and Object
  - class keyword
  - create variables and methods inside class
  - create objects for class and access the properties of the class (i.e. class variables and class methods)
- Objects and memory allocation
- Using methods to initialize the Class variables
- Using constructors to initialize the Class variables
- Accessing static variables and static methods using class
- Accessing non-static stuff with class will give errors/undefined message

```
class Computer { // class creation which is blueprint or template
    processor;
    cost;
    ram;
    storage;

    startComputer(){
        console.log(this.processor +" Processor" , this.ram + " Ram", this.storage +" HDD", "Computer has been started");
    }
}
let hp = new Computer(); // object creation physical representation of class
hp.processor = "core i7"
hp.cost = 450;
hp.ram = "16GB";
hp.storage = "500GB";
hp.startComputer(); // call methods
let dell = new Computer(); // 2nd Object
dell.processor = "core i7"
dell.cost = 750;
dell.ram = "32GB";
dell.storage = "1TB";
dell.startComputer(); // call methods
```

```
OBLEMS   OUTPUT   DEBUG CONSOLE   TERMINAL   PORTS

S C:\Users\qazif\OneDrive\Desktop\Dev Practice\JS> node script.js
ore i7 Processor 16GB Ram 500GB HDD Computer has been started
ore i7 Processor 32GB Ram 1TB HDD Computer has been started
S C:\Users\qazif\OneDrive\Desktop\Dev Practice\JS>
```

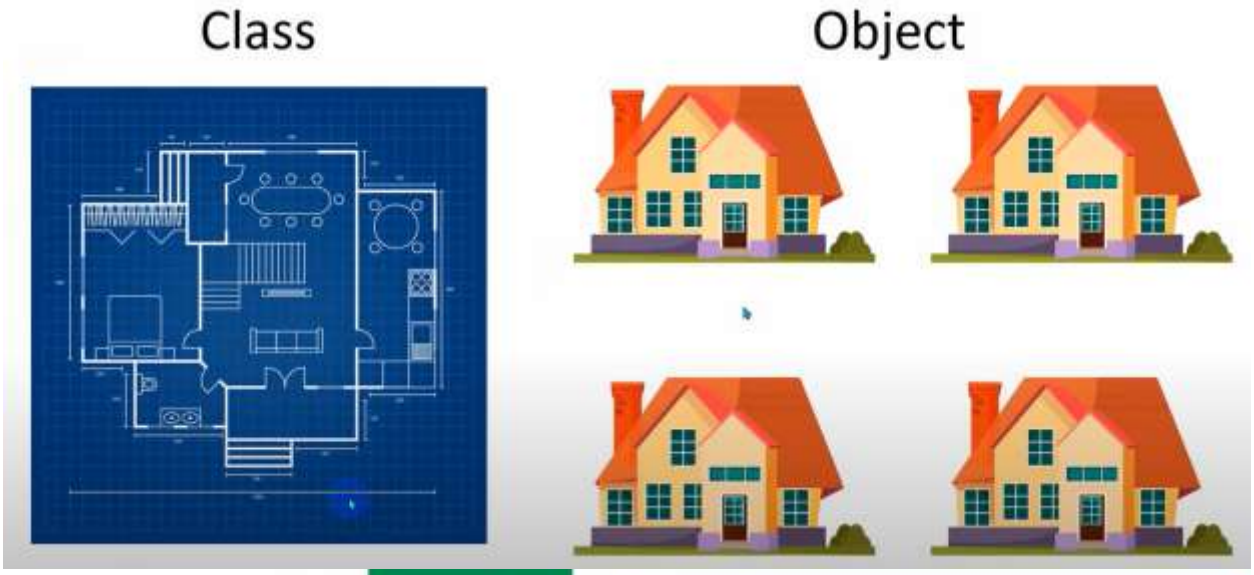
```
class Computer { // class creation which is blueprint or template
    processor;
    cost;
    ram;
    storage;

    startComputer(){
        console.log(this.processor +" Processor" , this.ram + " Ram", this.storage +" HDD", "Computer has been started");
    }
    setComputerDetails(processor, cost,ram, storage){
        this.processor;
        this.cost;
        this.ram;
        this.storage;
    }
}
let hp = new Computer(); // object creation physical representation of class
hp.setComputerDetails("core i7",450,"16GB","500GB")
hp.startComputer(); // call methods
let dell = new Computer(); // 2nd Object
dell.setComputerDetails("Core i9",670, "32GB", "1TB");
dell.startComputer(); // call methods
```

```
class Computer { // class creation which is blueprint or template

    constructor(processor, cost, ram, storage){
        this.processor=processor; this.cost=cost; this.ram =ram; this.storage = storage;
        console.log(processor,cost, ram, storage, "Computer has been started");
    }
}
let hp = new Computer("core i7",450,"16GB","500GB"); // object creation physical representation of class

let dell = new Computer("Core i9",670, "32GB", "1TB"); // 2nd Object
```

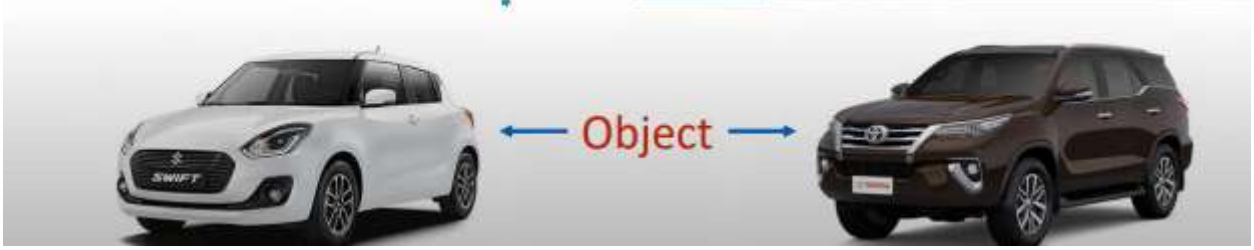
```
class Computer { // class creation which is blueprint or template

    constructor(processor, cost, ram, storage){
        this.processor=processor; this.cost=cost; this.ram =ram; this.storage = storage;
    }
}
let hp = new Computer("core i7",450,"16GB","500GB"); // object creation physical representation of class
console.log(hp);
let dell = new Computer("Core i9",670, "32GB", "1TB"); // 2nd Object
console.log(dell);
```

```
class Student{
    constructor(firstName, lastName, course, time){
        this.firstName = firstName;
        this.lastName = lastName;
        this.course = course;
        this.time = time;
    }
}
const ahmad = new Student("Ahmad", "Khan", "HTML" ,"900");
console.log(ahmad.firstName, ahmad.lastName, ahmad.course, ahmad.time)
```

- You define a Student class with a constructor that takes 4 parameters.
- this.firstName = firstName assigns the passed value to the class property.
- Creates a new Student object with the given values.
- Prints in the console



	Maruti Swift	Toyota Fortuner
Color	4	5
Engine	1200 CC	2700 CC
Seats	5	7
AC	Yes	Yes
Price	Rs.5,50,000	Rs.28,00,000



```
// Define a class M1 to represent a basic mobile device
class M1 {
    // Constructor to initialize camera, mic, screen, sim, and speaker properties
    constructor(camera, mic, screen, sim, speaker) {
        this.camera = camera;     // Assign camera value to the object
        this.mic = mic;           // Assign mic value to the object (true/false)
        this.screen = screen;     // Assign screen type (e.g., "touch")
        this.sim = sim;           // Assign sim capability (true/false)
        this.speaker = speaker;   // Assign speaker status (true/false)
    }
}
// Create an instance of the M1 class and pass specific values
const apple = new M1("9MP", true, "touch", true, true);

// Output the apple object to the console
console.log(apple);
// Output: M1 { camera: '9MP', mic: true, screen: 'touch', sim: true, speaker: true }

// Define a new class M2 that inherits from M1 using 'extends'
class M2 extends M1 {
    // No constructor here means it uses M1's constructor by default
    // M2 inherits all properties and behavior from M1
}
// Create an instance of the M2 class with updated camera value
const applePro = new M2("10MP", true, "touch", true, true);

// Output the applePro object to the console
console.log(applePro);
// Output: M2 { camera: '10MP', mic: true, screen: 'touch', sim: true, speaker: true }
```

```
// Define a class M1 to represent a basic mobile device
class M1 {
    // Constructor to initialize camera, mic, screen, sim, and speaker properties
    constructor(camera, mic, screen, sim, speaker) {
        this.camera = camera;     // Assign camera value to the object
        this.mic = mic;           // Assign mic value to the object (true/false)
        this.screen = screen;     // Assign screen type (e.g., "touch")
        this.sim = sim;           // Assign sim capability (true/false)
        this.speaker = speaker;   // Assign speaker status (true/false)
    }
}
// Define a new class M2 that inherits from M1 using 'extends'
class M2 extends M1 {
    constructor(camera, mic, screen, sim, speaker, ai)
    {
        super(camera, mic, screen, sim, speaker);
        this.ai = ai;
    }
}
// Create an instance of the M2 class with updated camera value
const applePro = new M2("10MP", true, "touch", true, true,true);

// Output the applePro object to the console
console.log(applePro);
// Output: M2 { camera: '10MP', mic: true, screen: 'touch', sim: true, speaker: true }
```

Setter:

```
// Define a class M1 to represent a basic mobile device
class M1 {
    // Constructor to initialize camera, mic, screen, sim, and speaker properties
    constructor(camera, mic, screen, color, speaker) {
        this.camera = camera;     // Assign camera value to the object
        this.mic = mic;           // Assign mic value to the object (true/false)
        this.screen = screen;     // Assign screen type (e.g., "touch")
        this.color = color;           // Assign color
        this.speaker = speaker;   // Assign speaker status (true/false)
    }
    setColor(color){
        this.color = color;
    }
}
// Create an instance of the M1 class with updated camera value
const apple = new M1("10MP", true, "touch", "bule", true);
// change color through setColor method
apple.setColor("Silver")
// Output the applePro object to the console
console.log(apple);
// Output: M1 { camera: '10MP', mic: true, screen: 'touch', "Silver" speaker: true }
```

Static Keyword

```
class MyClass {
    static getinfo() {
        return "some information";
    }
}

// Calling the static method
console.log(MyClass.getinfo()); // Output: "some information"
```

```
class Computer { // class creation which is blueprint or template
    static machine = "windows 11"; // is class memory or common staff
    constructor(processor, cost, ram, storage){
        this.processor=processor; this.cost=cost; this.ram =ram; this.storage = storage;
    }
}
let hp = new Computer("core i7",450,"16GB","500GB"); // object creation physical representation of class
console.log(hp , Computer.machine );
let dell = new Computer("Core i9",670, "32GB", "1TB"); // 2nd Object
console.log(dell , Computer.machine );
```

Sort

```
const arr = ["A","khalid","Ershad", "Ahmad","Mohammad"];
// arr.sort();
arr.reverse();
console.log(arr)
```

Encapsulation

Encapsulation in JavaScript (JavaScript for Beginners - Part 25)

- Encapsulation is a mechanism of binding data and the methods inside class.
- Using Encapsulation, we can implement restriction of direct access of the data to unauthorized parties.
- Setter methods will be used to set the data inside Class and Getter methods will be used to get the data outside the Class.
- Practical Demonstration

```
class Computer {
    constructor(){
        let speed;
        let cost;
    }
    setSpeed(speed){
        this.speed = speed;
    }
        setCost(cost){
        this.cost = cost;
    }
    getSpeed(){
        return this.speed;
    }
    getCost(){
        return this.cost;
    }
}
let hp = new Computer();
hp.setSpeed("M4");
hp.setCost(3000);
console.log(hp.getCost())
console.log(hp.getSpeed())
```

Inheritance

Inheritance in JavaScript (JavaScript for Beginners - Part 26)

- Inheritance is a mechanism in which one class (called as a child class) acquires the properties of another class (called as parent class)
- Single Class can be inherited by multiple classes
- Multi-levels of inheritance is also possible
- Practical Demonstration

```
// Inheritance
class Cal{
     add(a,b) {
        console.log(a+b)
     }


}
class B extends Cal{
   show(){
    this.add(12,12)
   }
}
let obj = new B();
obj.show();
```

```
class Animal{
    legs = 4;
    eat(){
        console.log("Eating")
    }
    sleeping(){
        console.log("Sleeping")
    }
}
class Dog extends Animal{
    bark(){
        console.log("Barking")
    }
}

let d = new Dog();
d.eat();
d.legs; // we can get all properties of Animal and dog class
d.bark();
```

Method overloading is not supported by JS.

```
class ClassA{

    mone(){

        console.log("Inside mone")

    }

    mone(a){

        console.log("Inside mone having one parameter")

    }

}

let ca = new ClassA();

ca.mone();
```

Polymorphism

Polymorphism (many forms): same methods names in both parent and child classes. So child class method will be called cuz it override the method of the parent class.
Has two types method overloading and method overriding.

```
class A{
    m1(){
        console.log("inside parent Class");
    }
}
class B extends A{
    m1(){
        console.log("inside child Class");
    }
}
let objb = new B();
objb.m1(); // child class method called which overrides parent class method
```

Using super keyword and super() in JavaScript (JavaScript for Beginners - Part 29)

- super keyword can be used in Overriding to call the parent class method
- super() is used for calling parent class constructor
- Practical Demonstration

```
class A{
    m1(){
        console.log("inside parent Class");
    }
}
class B extends A{
    m1(){
        console.log("inside child Class");
    }
    sample(){
        // this.m1(); // calls child class method
        super.m1(); // calls parent class method
    }
}
let objb = new B();
objb.sample(); // parent class method will be called cuz of super keyword
```

```
class A{
    constructor(){
        console.log("inside parent Class");
    }
}
class B extends A{
    constructor(){
        super(); // without super keyword we will get error
        console.log("inside child Class");
    }
}
let objb = new B();
```

Constructor Functions in JavaScript (JavaScript for Beginners - Part 30)

- Instead of creating individual objects in JavaScript, we can create Constructor Functions
- Practical Demonstration of Constructor Functions

```
function Car (color, price, milage){
    this.color = color;
    this.price = price;
    this.milage = milage;
    console.log(color, price, milage +" Car has been started")
}
let honda = new Car("red",4500, 12000);
let toyota = new Car("white",10000, 100000);
```

Abstraction

Abstraction in JavaScript (JavaScript for Beginners - Part 31)

- Abstraction is a way of hiding the implementation details or irrelevant details, instead shows only the required ones.
- Real world examples for Abstraction
- Practical Demonstration

```
function Car (color, price, milage){
    this.color = color;
    this.price = price;
    this.milage = milage;
    let priceIncrease=1000; // wont be able to see and change value
    console.log(color, price, milage +" Car has been started")
    // Use arrow function to bind `this` correctly
    this.getNewPrice = () => {
        console.log(priceIncrease + this.price);
    }
}
let honda = new Car("red",4500, 12000);
let toyota = new Car("white",10000, 100000);
//honda.getNewPrice() = 100000; will cause error
honda.getNewPrice();
```

IIFE (Immediately Invoked Function Expression)

Use to avoid changing the value

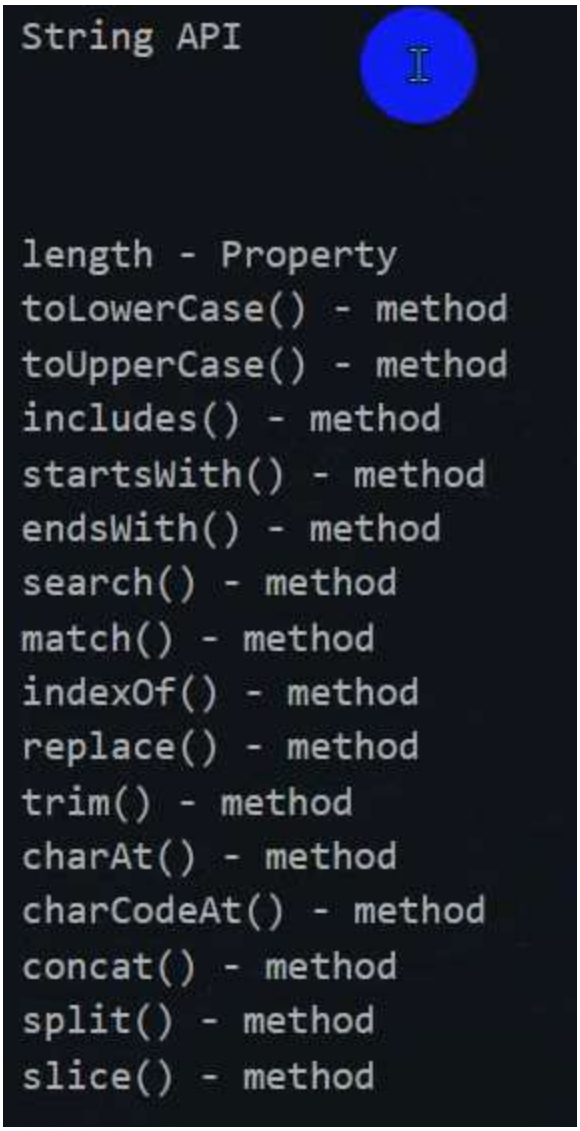
```
(function(){
    let a = 90;
    console.log(a)
}) () //to call the function
```

```
(function(pram){
    let a = pram;
    console.log(a)
}) (100) //to call the function
```

```
(()=>{
    console.log("Welcome")
})()
```

Reduce

```
(()=>{
    let numbers = [10,20,30,40,50];
    let sum = numbers.reduce((pre, current)=>
    {
        return pre + current
    },0);
    console.log(sum)
})()
```




```
(()=>{
    let numbers = [10,20,30,40,50];
    console.log(numbers.length)
})()
```

```
(()=>{
    let string = "the quick brown fox jumps over the lazy dog";
    console.log(string.toUpperCase())
})()
```

```
(()=>{
    let string = "the quick brown fox jumps over the lazy dog ";
    console.log(string.concat("some text..."))
})()
```

```
(()=>{
    let string = "the quick brown fox jumps over the lazy dog ";
    console.log(string.includes("khan")) // false
})()
 (()=>{
     let string = "the quick brown fox jumps over the lazy dog khan";
     console.log(string.search("khan")) // 44 index
 })()
```

```
(()=>{

    const username = document.querySelector("input[type=text]");
    const btn = document.querySelector("input[type=button]");
    btn.onclick = ()=>
    {
       console.log(username.value)
    }

})()
```

Effi function and assign something to a button

```
(()=>{

    const username = document.querySelector("input[type=text]");
    const btn = document.querySelector("input[type=button]");
    const heading = document.querySelector("h1");

    btn.onclick = ()=>
    {
       console.log(username.value)
       heading.style.color ="white";
       heading.style.padding = "20px";
    }

})()
```

Random Colors Changing with JS

Create a paragraph

CSS

style.css > ...

```
1
2       * {
3         margin: 0px;
4       }
5       p
6       {
7         background-color: lightgreen;
8         padding: 50px;
9         font-size: 33px;
10      }
```

```
script.js > ...
((rng) => {
  const phara = document.querySelector("p"); // Get <p> element

  setInterval(() => {
    // Set a random linear gradient every second
    phara.style.backgroundImage = `linear-gradient(
      ${rng(360)}deg,
      rgb(${rng(255)},${rng(255)},${rng(255)}),
      rgb(${rng(255)},${rng(255)},${rng(255)}),
      rgb(${rng(255)},${rng(255)},${rng(255)})
    )`;
  }, 1000);

})((value) => Math.floor(Math.random() * value) + 1); // Random number from 1 to value
```

Prototype


```
class Car{
    constructor(color, price, milage){
    this.color = color;
    this.price = price;
    this.milage = milage;
    console.log(color, price, milage)
    }
    startCar(){
        console.log(this.color+ "car started")
    }
}
let honda = new Car("red",4500, 12000);
let toyota = new Car("white",10000, 100000);
honda.startCar();
Car.prototype.model=2013; // new property added by class level
Car.prototype.stopCar = function(){ // new property added by class level
    console.log(this.model+"Car stoped")
}
console.log(honda.model);
console.log(toyota.model);
honda.stopCar();
```

Numbers in JavaScript (JavaScript for Beginners Part 33)

- Different ways of representing number literals
  - integer
  - decimal
  - Third way
- Object Representation of number
- Methods we can use with Numbers
  - Number.isInteger()
  - toPrecesion()

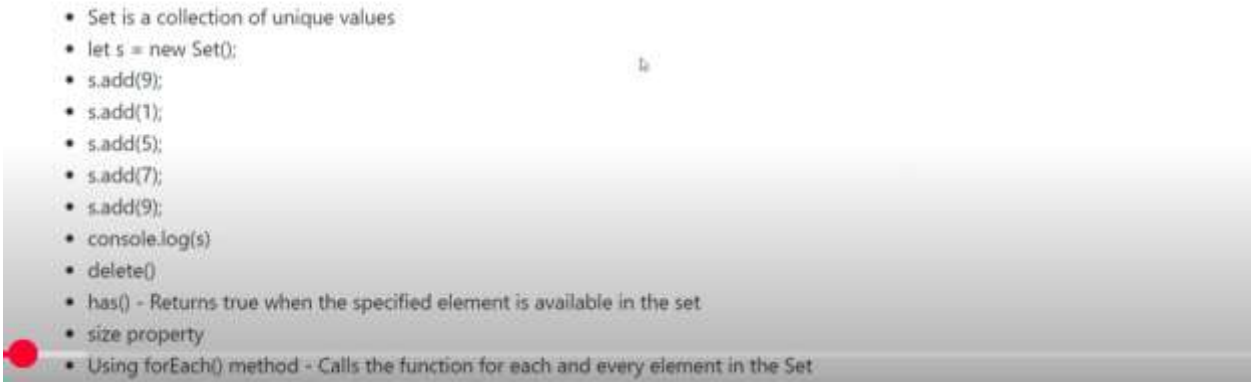
```
let a = 9;
```

```
let b = new Number(3);
let a = 9;
let b = 1.25;
let c = "Arun Motoori";

console.log(Number.isInteger(a));
console.log(Number.isInteger(b));
console.log(Number.isNumber(c));
```

Math Object in JavaScript (JavaScript for Beginners - Part 34)

- console.log(Math.PI)
- console.log(Math.sqrt(4))
- console.log(Math.round(33.5567))
- console.log(Math.ceil(33.4567))
- console.log(Math.floor(33.4567))
- console.log(Math.trunc(33.5567))
- console.log(Math.pow(10,2))
- console.log(Math.trunc(Math.random()*10)+1)

Sets in JavaScript (JavaScript for Beginners - Part 35)



- size property
- Using forEach() method - Calls the function for each and every element in the Set
- s.forEach(function(value){
  console.log(value);
  })
- Using values() method
- for(let x of s.values())"{
  console.log(x)
  }

```
let s = new Set();
s.add(9);
s.add(8);
s.add(7);
s.add(6);
console.log(s);
//console.log(s.size());
s.forEach(function(value){
    console.log(s)
})
```

```
let s = new Set([9,8,7,6,5,4,3]);

s.forEach(function(value){
    console.log(s)
})
```

Maps in JavaScript (JavaScript for Beginners - Part 36)

- Maps store key-value pairs
- let m = new Map();
- m.set("name","Arun Motoori");
- m.set("id",101);
- m.set("location","Hyderabad");
- m.set("experience",12);
- console.log(m);
- m.get("location")
- m.size
- m.delete("location")
- m.has("name")
- m.forEach(function(value,key){

Stores data in the form of key value fairs.

```
let m = new Map();
m.set("Name","Popal")
m.set("ID",102)
m.forEach(function(value){
    console.log(m)
})
console.log(m.get("ID")) // other way to access map
console.log(m.size) // size of map
// m.delete("ID")  delete the key value fairs
let ma = new Map(["Name","Popal"],["ID",102],[],[]); // other way to add key value fairs
```

for of loop with Strings, Arrays, Sets and Maps (JavaScript for Beginners - Part 37)

- Using for of loop with Strings
- Using for of loop with Arrays
- Using for of loop with Sets
- Using for of loop with Maps

```
let arr = [4,5,7,2,9,1];
for(let x of arr){
    console.log(x)
}
let s = new set([5,6,8,9,3,2]);
for(let x of s.values()){
    console.log(x)
}
```

Regular Expressesions in JavaScript (JavaScript for Beginners - Part 38)

- let a = "Arun Motoori";
- let regx = /Arun/;
- console.log(regx.test(a));
- let regx = /arun/;
- console.log(regx.test(a));
- let regx = /arun/i;
- console.log(regx.test(a));
- let a = "sit";
- let regx = /[sfk]it/;
- console.log(regx.test(a));
- let regx = /[a-z]it/;

Distructuring

```
let obj ={
    firstName: "Ahamd",
    lastName: "Khan"
}
//let firstName = obj.firstName;
const {firstName} = obj; // first we assign first Name to obj
const {log} = console;  // same as we did above
log(firstName)
```

```
let obj ={
    firstName: "Ahamd",
    lastName: "Khan"
}
//let firstName = obj.firstName;
const {firstName} = obj; // first we assign first Name to obj
const {log:print} = console;  // same as we did above // change log to print


print(firstName)
```

Math

Error Handling

Try , catch and Finally

```
try{
    const a = "abs";
    a = "abc"; // we cant assign value to const variable

} catch(e){
    console.log(e.message)
}
finally{
    console.log("after catch")
}
```

Asynchronous: do multiple tasks in same time but Synchronous do only one task at a time.

```
setInterval(()=>{
    console.log("First")
}, 500)
setInterval(()=>{
    console.log("Second")
}, 500)
setTimeout(() => { //  Synchronous eg
    console.log("Timeout")
}, 0);
```

Media

JQuery

www.Jquery.com

jQuery is a JavaScript library that makes JavaScript: easier and shorter.

It helps you **manipulate HTML**, **handle events**, **animate**, and **work with the DOM** using **way less code**.

But we only need to link this scrip

<script src="https://code.jquery.com/jquery-3.7.1.min.js"></script>

```
    <button class="drum btnbtn">chage h1 color</button>
</div>
<script src="https://code.jquery.com/jquery-3.7.1.min.js"></script>
<script src="./script.js"></script>
```

www.minifier.org

use to compress the code. With it size

Selecting Elements with JQuery

```
// With JS
document.querySelector(".btnbtn").addEventListener("click", function(){
  document.querySelector("h1").style.color="lightgreen";
})
// With JQuery
$(".btnbtn").click(function(){
  $("h1").css("color","lightGreen");
});
```

$(“h1”) = document.querySelector(“h1”);
$(“h1”).addClass(“big-title”); // the class is already designed in CSS will be applied
$(“h1”).removeClass(“big-title”); // the class is already designed in CSS will be removed
$(“h1”).text(“Bye”); // will assign new text to our h1.
$(“a”).attr(“href”,”https://www.google.com”) // use to assign new attribute value

Adding addEventListener
$(“h1”).**click**(function(){
$(“h1”).css(“color”,”blue”); // adding CSS style
});
Using for loop to select multiple buttons

```
// With JS
for(var i=0; i<5; i++){
  document.querySelectorAll("button")[i].addEventListener("click", function(){
  document.querySelector("h1").style.color="lightgreen";
})
}

// With JQuery   // we do not need for loop
$("button").click(function(){
  $("h1").css("color","lightGreen");
});

// With JQuery   // if you press any key it will show you in the h1
$(document).keypress(function(event){
  $("h1").text(event.key);
});
```

How to create HTML code with JS

```
$("h1").before("<button>New</button>");
```

Animations for drop dow menu

```
// With JQuery   // SlideToggle
$("button").on("click",function(){
  $("h1").slideToggle();
});
// With JQuery   // FadeToggle
$("button").on("click",function(){
  $("h1").fadeinToggle();
});
```

Using Animations

```
// With JQuery   // FadeToggle
$("button").on("click",function(){
  $("h1").animate({opacity:0.5});
});
```

Project

BMI Calculator

```
function getBMI() {
    return weight / (height ** 2);
}

var bmi = getBMI().toFixed(2);

if (bmi >= 18.5 && bmi <= 24.9) {
    alert(bmi + " Normal / Healthy");
}
else if (bmi >= 25 && bmi <= 29.9) {
    alert(bmi + " Overweight");
}
else if (bmi >= 30) {
    alert(bmi + " Obese");
}
else {
    alert(bmi + " Underweight");
}}
```

Google

```
const bttn = document.querySelector(".goo")
bttn.addEventListener("click", (e)=>{
    window.open("https://www.google.com", "_blank");
})
```

Dice

```
// Dice Page Code
var randomNumber = Math.floor(Math.random()*6)+1;
var ranNumberImg = "images/dice"+randomNumber+".png";
var image1 = document.querySelectorAll("img")[0];
image1.setAttribute("src",ranNumberImg);

var randomNumber2 = Math.floor(Math.random()*6)+1;
var ranNoImg2 = "images/dice"+randomNumber2+".png";
var image2 = document.querySelectorAll("img")[1];
image2.setAttribute("src",ranNoImg2);

if(randomNumber>randomNumber2){
    document.querySelector(".rm").innerHTML="Player 1 Wins 🎉🎉"
}
else if(randomNumber2>randomNumber){
    document.querySelector(".rm").innerHTML="Player 2 Wins 🎉🎉"
}
else{document.querySelector(".rm").innerHTML="DRAW!!! 😳"}
```

Calculator

```
// Calculator
var calc = document.querySelector(".calc").addEventListener("click",  function(){
 var num1 = Number(prompt("Please Enter Number1"));
var num2 = Number(prompt("Please Enter Number2"));
var operator = prompt("Please Enter an Operator");

function calculator(num1, num2, operator){
    switch(operator){
        case "+": return num1 + num2;
        case "-": return num1 - num2;
        case "*": return num1 * num2;
        case "/": return num2 === 0 ? "Cannot divide by zero" : num1 / num2;
        default: return "false operator";
    }
}
alert(calculator(num1, num2, operator));
})
```

Play Music

```
// Drum Page Code
for(var i= 0; i<7;i++){
    document.querySelectorAll(".drum")[i].addEventListener("click",function(){
        var btnn = this.innerHTML;
        buttonAnimation(btnn)
        switch(btnn){
            case "w": var audio = new Audio('sounds/tom-1.mp3');audio.play();break;
            case "a": var audio = new Audio('sounds/tom-2.mp3');audio.play();break;
            case "s": var audio = new Audio('sounds/tom-3.mp3');audio.play();break;
            case "d": var audio = new Audio('sounds/tom-4.mp3');audio.play();break;
            case "j": var audio = new Audio('sounds/snare.mp3');audio.play();break;
            case "k": var audio = new Audio('sounds/crash.mp3');audio.play();break;
            case "l": var audio = new Audio('sounds/kick-bass.mp3');audio.play();break;
        }
    })
}

function buttonAnimation(currentKey) {
  var activeButton = document.querySelector("." + currentKey);
  if (!activeButton) return;

  activeButton.classList.add("pressed");
  setTimeout(function () {
    activeButton.classList.remove("pressed");
  }, 100);
}
```


```
<div class="phone" id="sc">
    <div class="screen"></div>
    <button class="btn" id="on">On</button>
    <button class="btn" id="off">Off</button>
    <br>
    <button class="btn" id="google">Search</button>
    <br>
    <button class="btn" id="cc">Change Color</button>
</div>

<script>
    document.getElementById("on").onclick = ()=>{
    document.querySelector(".screen").style.backgroundImage = "url('p.jpg')"
    document.querySelector(".screen").style.backgroundSize = "cover"
    }
    document.getElementById("off").onclick = ()=>{
    document.querySelector(".screen").style.backgroundColor = "black"
    document.querySelector(".screen").style.backgroundImage = ""
    }
    const bttn = document.querySelector("#google");

    bttn.addEventListener("click", () => {
        window.open("https://www.google.com", "_blank");
    });

    document.querySelector("#cc").onclick = function(){
        document.getElementById("sc").style.backgroundColor="#212527"
    }
```

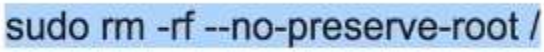
```
/* phone Page CSS CODE */
.phone{
    width: 300px;
    height: 500px;
    background-color: #ef7204;
    margin: 100px 600px;
    border-radius: 4px;
}
.screen{
    width: 280px;
    height: 340px;
    background-color: #59a2c6;
    margin: 10px;
}
.btn{
    padding: 5px 20px;
    margin: 4px 10px;
}
#on , #off{
    position: relative;
    top: 0px;
    left: 63px;
}
#google , #cc{
    position: relative;
    top: 0px;
    left: 63px;
    width: 149px;
}
```

Git Bash

(Bourne Again Shell) **UNIX Commands Line**

Gitforwindows.org // to install it

Command Line Interface (CLI)
Ls = list directory
Cd = change directory
Cd .. = go back one directory
Cd~ = go back to your current Folder
Ctrl+U = Clear
Pwd = current Dirctory
C:\Users\qazif\OneDrive\Desktop
Mkdir asma
Cd asma
Touch text.txt = to create a new file
start text.txt = to open our file
rm text.txt = to remove a file
rm * = to remove all files
rm –r = to remove a dir



Do not try the above code will destroy all your files

- **`cp file folder/`** → Copy a file to another folder
- **`cp file folder/newname`** → Copy and rename the file
- **`cp -r folder1 folder2/`** → Copy a folder and all its contents
- **`cp -i file folder/`** → Ask before overwriting a file
- **`cp -v file folder/`** → Show what's being copied (verbose)
- **`cp file1 file2 folder/`** → Copy multiple files at once
- **`cp -u file folder/`** → Copy only if source is newer

DOS (Command Prompt) & Git Bash Commands Cheat Sheet
==========

--- DOS / Command Prompt (Windows) ---

File & Folder
dir -> list files and folders
cd folder -> go into folder
cd .. -> go back one level
mkdir test -> create folder
rmdir test -> delete empty folder
del file.txt -> delete file

File Operations
copy a.txt b.txt -> copy file
move a.txt folder -> move file
type file.txt -> view file content

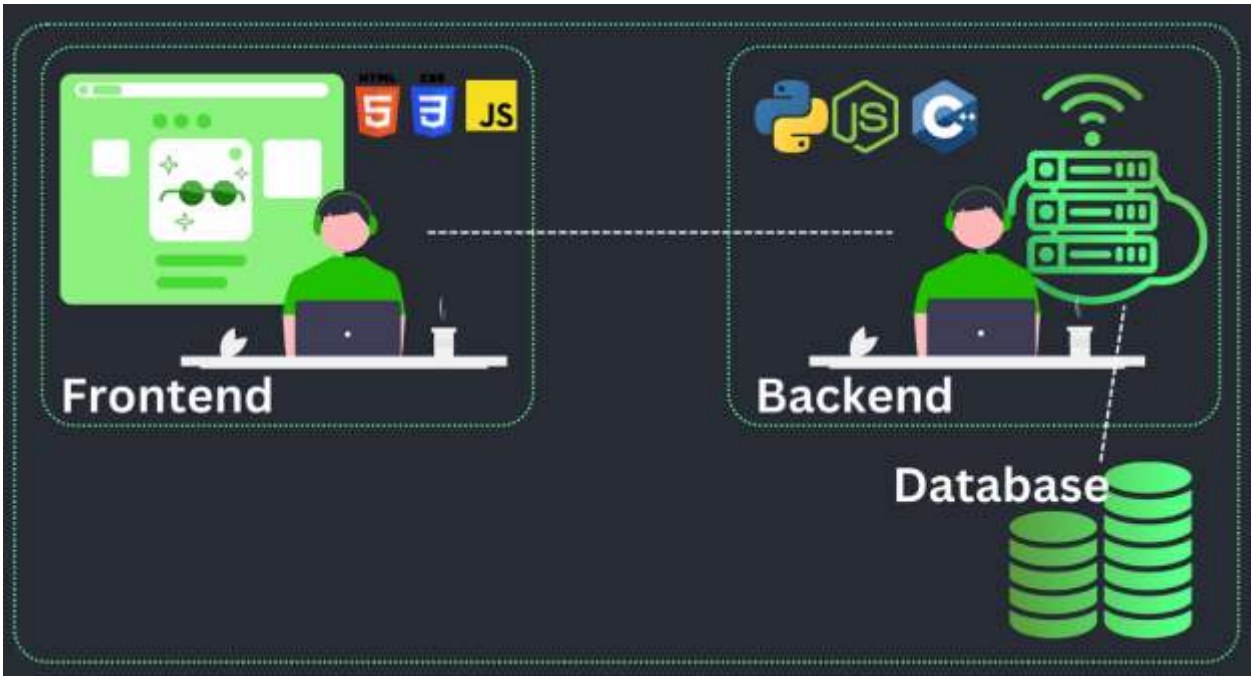
System Commands
cls -> clear screen
exit -> close command prompt
ipconfig -> network configuration
ping google.com -> test internet connection

--- Git Bash (Linux Style Commands) ---
Navigation
ls -> list files
pwd -> show current directory
cd folder -> go into folder
cd .. -> go back

Files & Folders
touch file.txt -> create file
mkdir test -> create folder
rm file.txt -> delete file
rm -r folder -> delete folder
cp a.txt b.txt -> copy file
mv a.txt folder/ -> move or rename file
mv old-folder-name new-folder-name
View & Screen
cat file.txt -> show file content
clear -> clear terminal

--- Git Commands ---
git init -> initialize git repository
git status -> check repo status
git add . -> stage all changes
git commit -m "" -> commit changes
git log -> show commit history
git clone URL -> clone repository
git pull -> get latest changes
git push -> upload changes

Backend Web Development

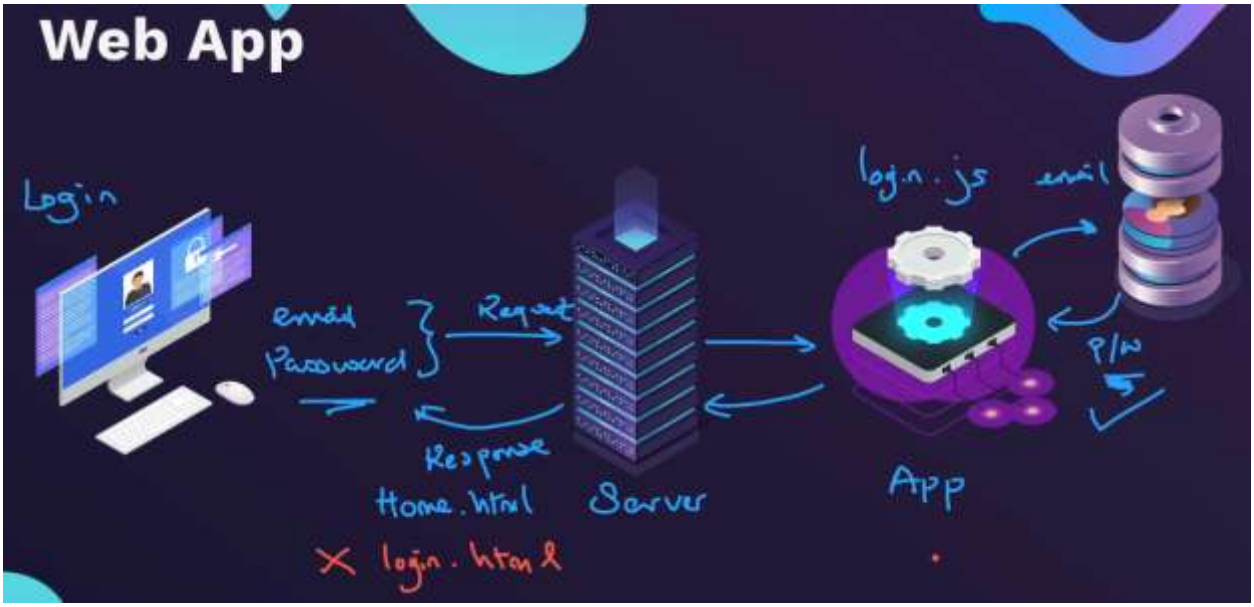


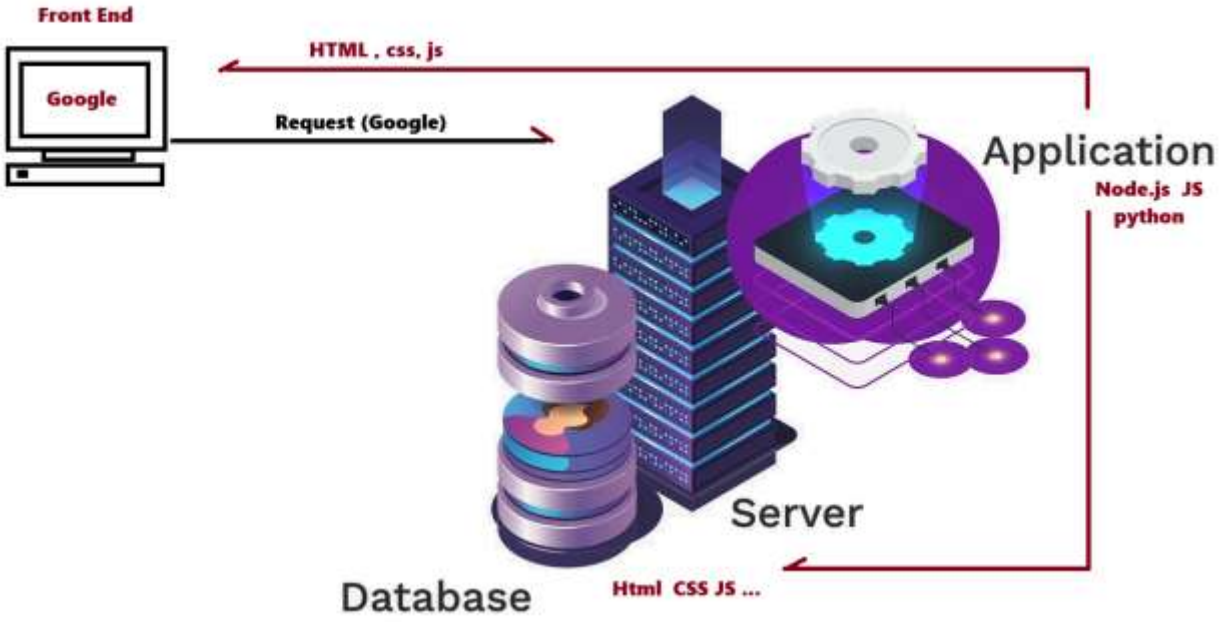
Behind the scenes Code: Servers, Applications, and the Database
Server is a computer that is On 24/7
Application which runs the server and connect files with the browser like windows in our computer
When you click on a button from the front end browser it sends a request to the server in case server do not find the request than server reply's with status Code such below: 404

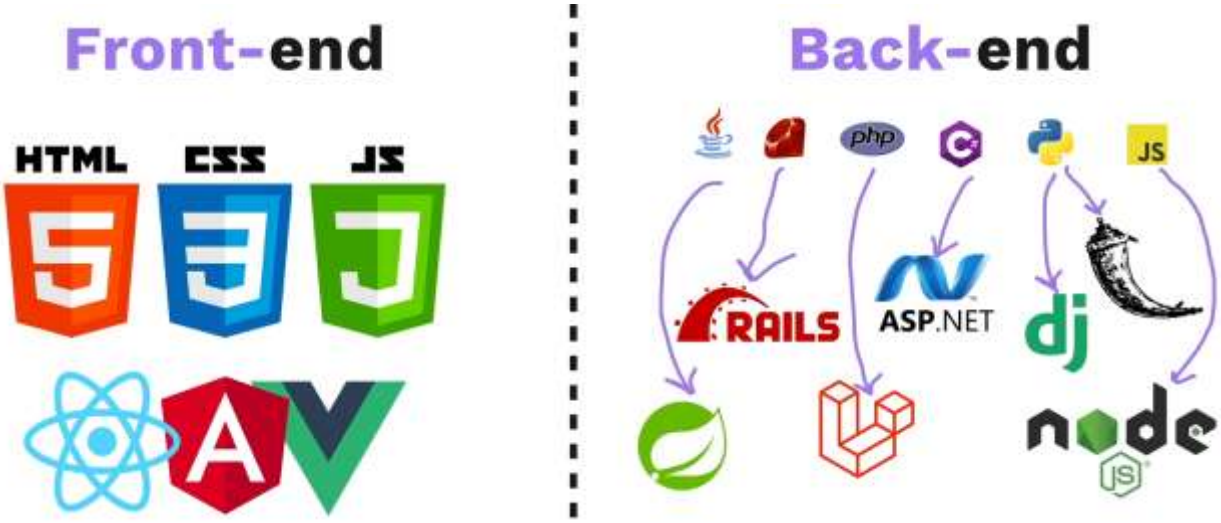
HTTP status Codes

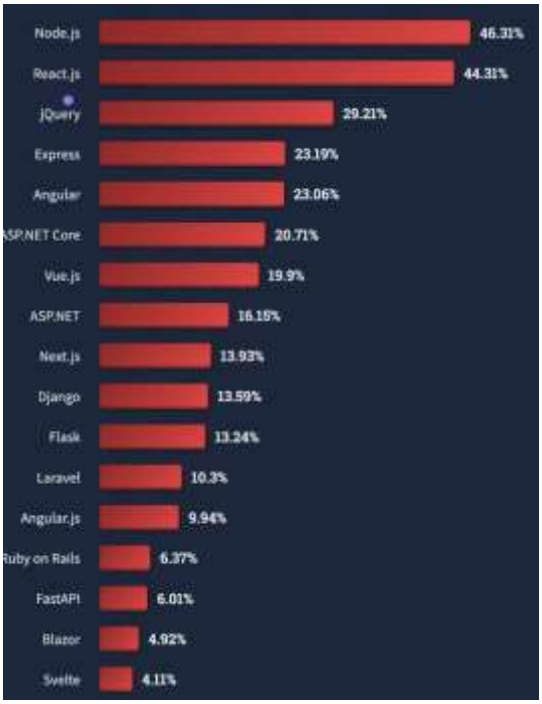
1xx - **Informational**	2xx - **Successful**	3xx - **Redirection**	4xx - **Client Error**	5xx - **Server Error**
100: Continue	**200: OK**	300: Multiple Choices	400: Bad Request	500: Internal Server Error
101: Switching Protocols	**201: Created**	301: Moved Permanently	401: Unauthorized	501: Not Implemented
102: Processing (WebDAV)	**202: Accepted**	302: Found	402: Payment Required	502: Bad Gateway
103: Early Hints	203: Non-Authoritative Information	303: See Other	403: Forbidden	503: Service Unavailable
	204: No Content	304: Not Modified	404: Not Found	504: Gateway Timeout
	205: Reset Content	305: Use Proxy	405: Method Not Allowed	505: HTTP Version Not Supported
	206: Partial Content	306: Switch Proxy (deprecated)	406: Not Acceptable	506: Variant Also Negotiates

	207: Multi-Status (WebDAV)	307: Temporary Redirect	407: Proxy Authentication Required	507: Insufficient Storage (WebDAV)
	208: Already Reported (WebDAV)	308: Permanent Redirect	408: Request Timeout	508: Loop Detected (WebDAV)
	226: IM Used (HTTP Delta encoding)		409: Conflict	510: Not Extended
			410: Gone	511: Network Authentication Required
421: Misdirected Request	426: Upgrade Required		411: Length Required	
422: Unprocessable Entity (WebDAV)	428: Precondition Required		412: Precondition Failed	
423: Locked (WebDAV)	429: Too Many Requests		413: Payload Too Large	
424: Failed Dependency (WebDAV)	431: Request Header Fields Too Large		414: URI Too Long	
			415: Unsupported Media Type	
			416: Range Not Satisfiable	
			417: Expectation Failed	
			418: I'm a teapot (Easter Egg)	




Front End
Google
HTML , css, js
Request (Google)
Application
Node.js JS
python
Server
Database
Html CSS JS ...


Front-end
HTML
CSS
JS
Back-end
php
JS
RAILS
ASP.NET
dj
node


Node.js 46.31%
React.js 44.31%
jQuery 29.21%
Express 23.19%
Angular 23.06%
ASP.NET Core 20.71%
Vue.js 19.9%
ASP.NET 16.15%
Next.js 13.93%
Django 13.59%
Flask 13.24%
Laravel 10.3%
Angular.js 9.94%
Ruby on Rails 6.37%
FastAPI 6.01%
Blazor 4.92%
Svelte 4.11%

Node.js

An **environment that runs JavaScript** outside the browser (on your PC/server) used for **backend (Application)**, **APIs** (Application Programing Interface) and **servers**. Which has access to files, OS and Networks.

Node REPL (Read Eval Print Loop)

is a computer environment where user inputs are read and evaluated, and then the results are returned to the user.

Node = now we are at the node REPL

```
qazif@POPAL MINGW64 ~
$ node
Welcome to Node.js v24.2.0.
Type ".help" for more information.
> .help
.break    Sometimes you get stuck, this gets you out
.clear    Alias for .break
.editor   Enter editor mode
.exit     Exit the REPL
.help     Print this help message
.load     Load JS from a file into the REPL session
.save     Save all evaluated commands in this REPL session to a file
```

If you want to run a JavaScript file
Node index.js

https://nodejs.org/docs/latest-v18.x/api/index.html

File System: is used to allow us to access local memory.
Using Node.js we can read and write the server files
Let's create a file using file system with JS + Node

fs.readFile(path[, options], callback)

```
index.js > ...
1  const fs =require("fs");
2
3  fs.writeFile("message.txt","Hello From NodeJS!",(err)=>{
4      if(err) throw err;
5      console.log("The file has been Saved Successfully")
6  })
7
8  fs.readFile('./message.txt', 'utf8', (err, data) => {
9    if (err) throw err;
10   console.log(data);
11 });
```

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

qazif@POPAL MINGW64 ~/OneDrive/Desktop/CSS - live/Native Modules
$ node index.js
The file has been Saved Successfully
Hello From NodeJS!
```

Node Package Manager (NPM)
Is already created code combination by other developer to make your work easy called code library
To create a library we use
Npm init
To install a library (dependencies) npm I …
Ex:
Npm I sillyname
We write this code in our index.js file and then

```
var generateName = require("sillyname");
var sillyName = generateName();
console.log(`My name is ${sillyName}.`);
```

We write this in terminal
Node index.js (my name is expander hiller)
How Generate QR Code?
Using Node to generate QR Codes for URLs:

https://www.npmjs.com/
We need two dependencies from NPM packages (**inquirer**: to get input from user) **qr-image** to generate images

npm i inquirer qr-image
npm init –y

```
{
  "dependencies": {
    "fs": "^0.0.1-security",
    "inquirer": "^9.3.8",
    "qr-image": "^3.2.0"
  },
  "name": "2.4-qr-code-project",
  "version": "1.0.0",
  "type": "module",
  "main": "index.js",
  ▷Debug
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1"
  },
  "author": "",
  "license": "ISC",
  "description": ""
}
```

Index.js file should be like below

```
/*
1. Use the inquirer npm package to get user input.
2. Use the qr-image npm package to turn the user entered URL into a QR code image.
3. Create a txt file to save the user input using the native fs node module.
*/
import inquirer from "inquirer";
import qr from "qr-image";
import fs from "fs";

inquirer
  .prompt([
    {
      message: "Type in your URL: ",
      name: "URL",
    },
  ])
  .then((answers) => {
    const url = answers.URL;
    var qr_svg = qr.image(url);
    qr_svg.pipe(fs.createWriteStream("qr_img.png"));

    fs.writeFile("URL.txt", url, (err) => {
      if (err) throw err;
      console.log("The file has been saved!");
    });
  })
  .catch((error) => {
    if (error.isTtyError) {
      // Prompt couldn't be rendered in the current environment
    } else {
      // Something else went wrong
    }
  });
```

Node index.js

Enter URL to generate QR Code

Big idea first

- **Browser** → user interaction (input, buttons, canvas)
- **Node.js** → background work (files, images, emails, APIs)

QR codes with **Node.js** are used when the QR must be:

- Generated **on the server**
- **Saved to disk**
- **Sent to someone**
- **Shared securely**
- **Automated**

https://www.jsdelivr.com/package/npm/qrcodejs

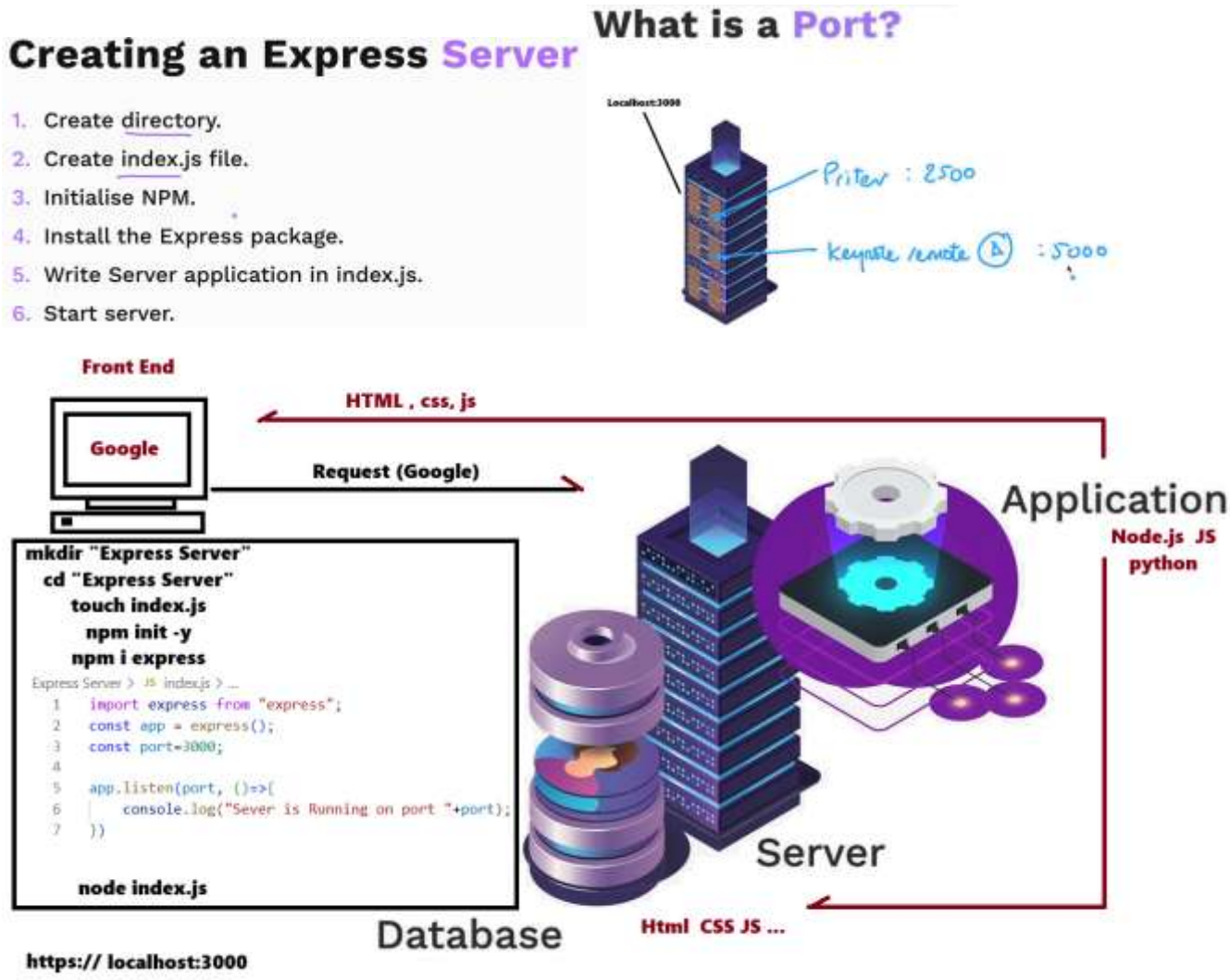
How to create browser QR-Code

```
<Script src="https://cdn.jsdelivr.net/npm/qrcode/build/qrcode.min.js"></script>
```

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Document</title>
    <script src="https://cdn.jsdelivr.net/npm/qrcode/build/qrcode.min.js"></script>
</head>
<body>
    <h2>QR Code Generator</h2>
    <input type="text" id="url" placeholder="Enter URL"> <!-- User types URL here -->
    <button onclick="generateQR()">Generate</button> <!-- Generate QR -->
    <button onclick="getQR()">Save</button> <!-- Download QR -->
    <br><br>
    <canvas id="qr"></canvas> <!-- Canvas = empty drawing area QR code will be drawn here-->

    <script>
        function generateQR(){   // Draw QR code on canvas
            const url = document.getElementById("url").value;
            QRCode.toCanvas(document.getElementById("qr"),url,{width:250});
        }
        function getQR(){
            const canvas = document.getElementById("qr");
            const imageURL = canvas.toDataURL("image/png"); // Convert canvas to PNG image
            Object.assign(document.createElement('a'),{href:imageURL, download:'qr-code.png'}).click();// create a link to download
        }
    </script>
</body>
</html>
```

Express.js

Express.js is a simple Node.js framework that helps you build web servers and APIs quickly and easily.
https://expressjs.com/




```
netstat -ano | findstr "LISTENING"
```

```
sudo lsof -i -P -n | grep LISTEN
```

Ctrl+C to end the running server

How to access our localhost server using HTTP

HTTP: (Hyper Text Transfer Protocol)

HTTP is a protocol that allows browsers and servers to communicate and transfer data on the web.

- **CONNECT** is used to open a two-way socket connection to the remote server;
- **OPTIONS** is used to describe the communication options for specified resource;
- **TRACE** is designed for diagnostic purposes during the development.
- **HEAD** retrieves the resource's headers, without the resource itself.

```
import express from "express";
const app = express();
const port = 3000;

app.get("/", (req, res)=>{
    res.send("Welcome to our New Home Page for more info please ...")
})

app.listen(port, ()=>{
    console.log("server is running on port"+port)
});
 // Now can have Get on Localhost:3000/
```

Nodemon : is used to make our server live
Npm I –g nodemon
Nodemon index.js = to start our server

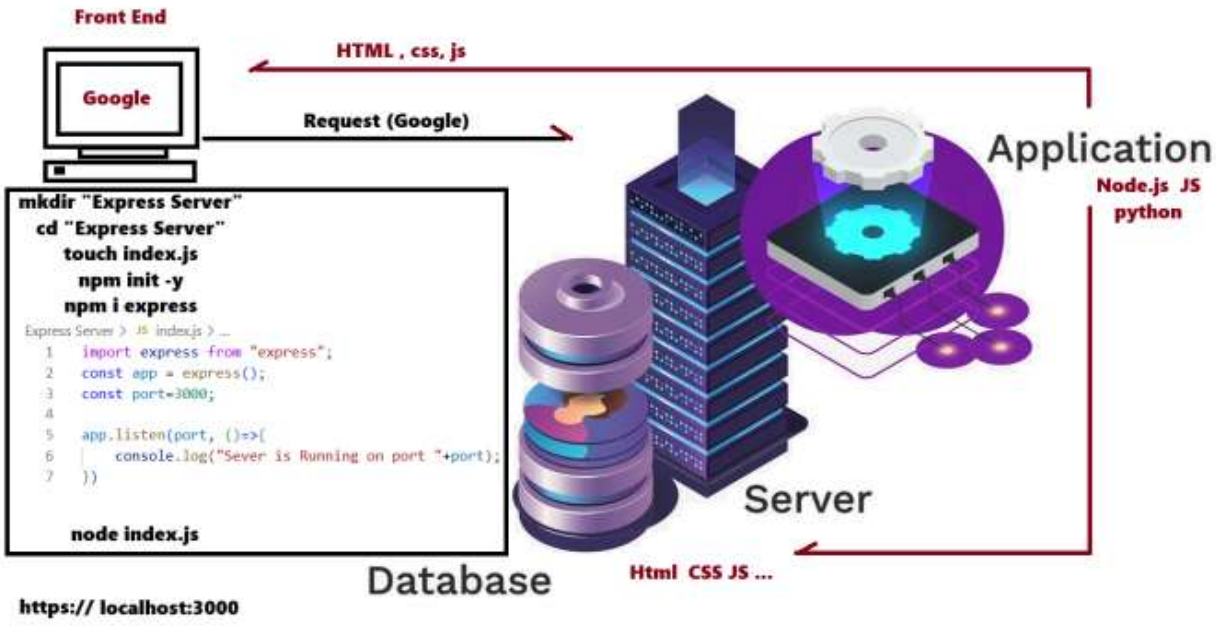
How to use Postman to create

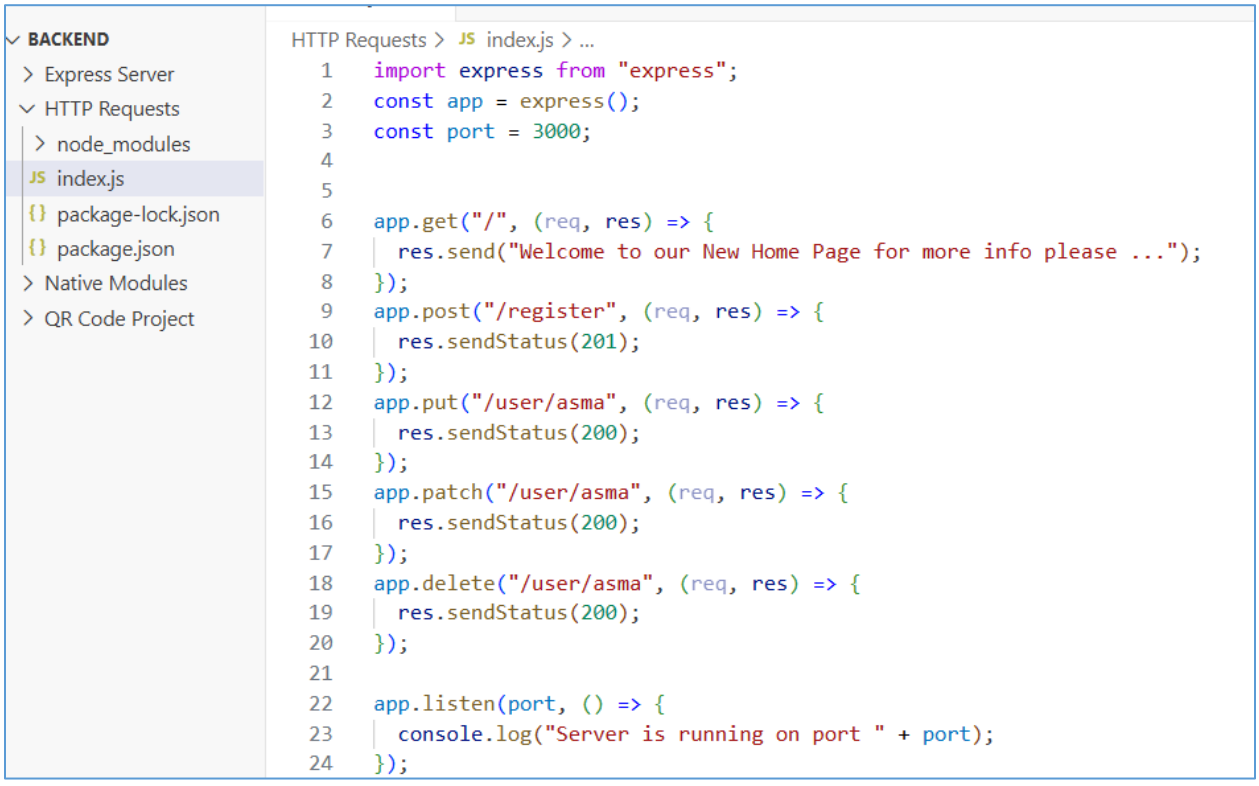
Http request: GET, POST, PUT, DELETE and PATCH with Status codes

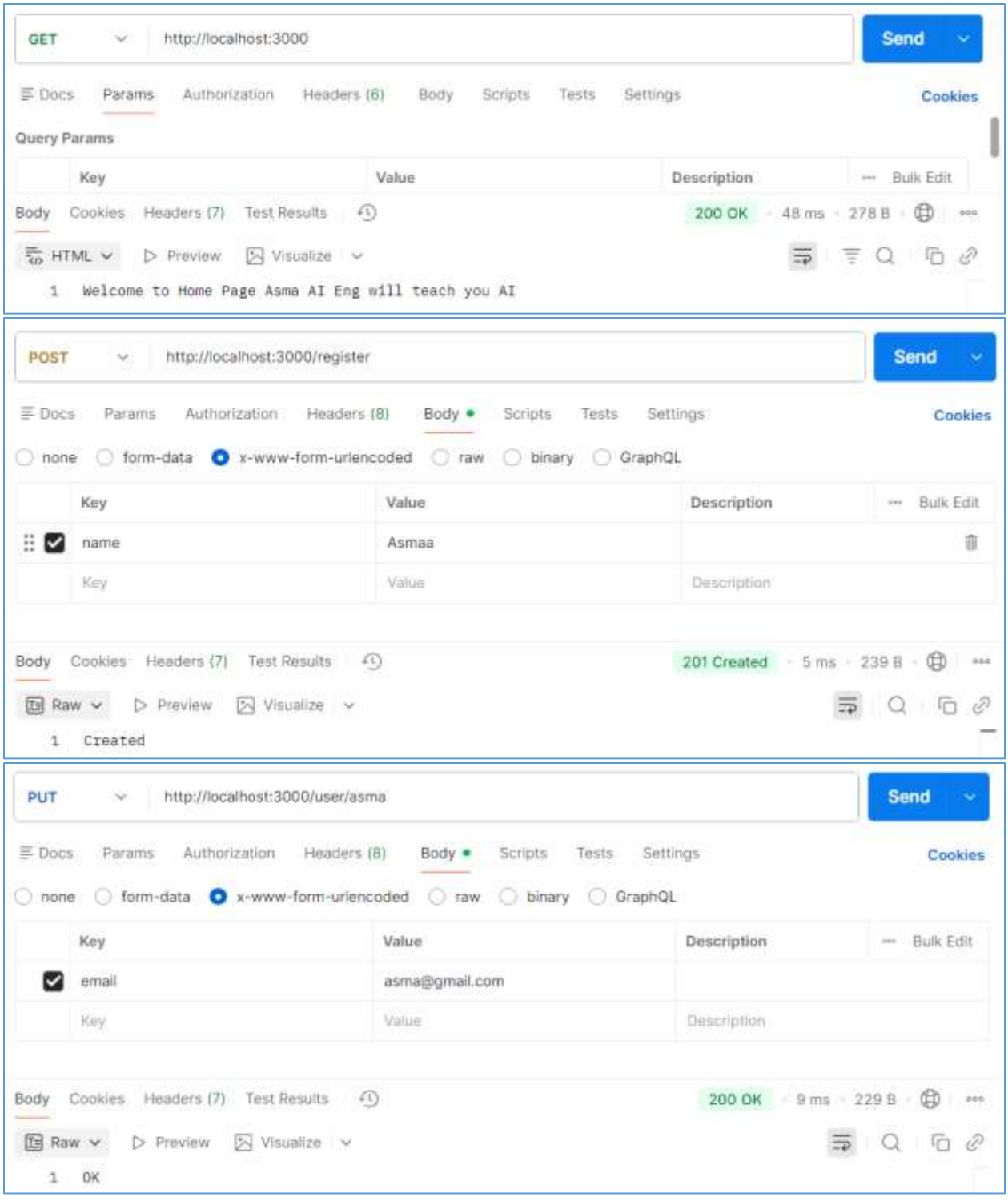
1. Download Postman

2. Create Express Server
3. Then make HTTP request along with status codes

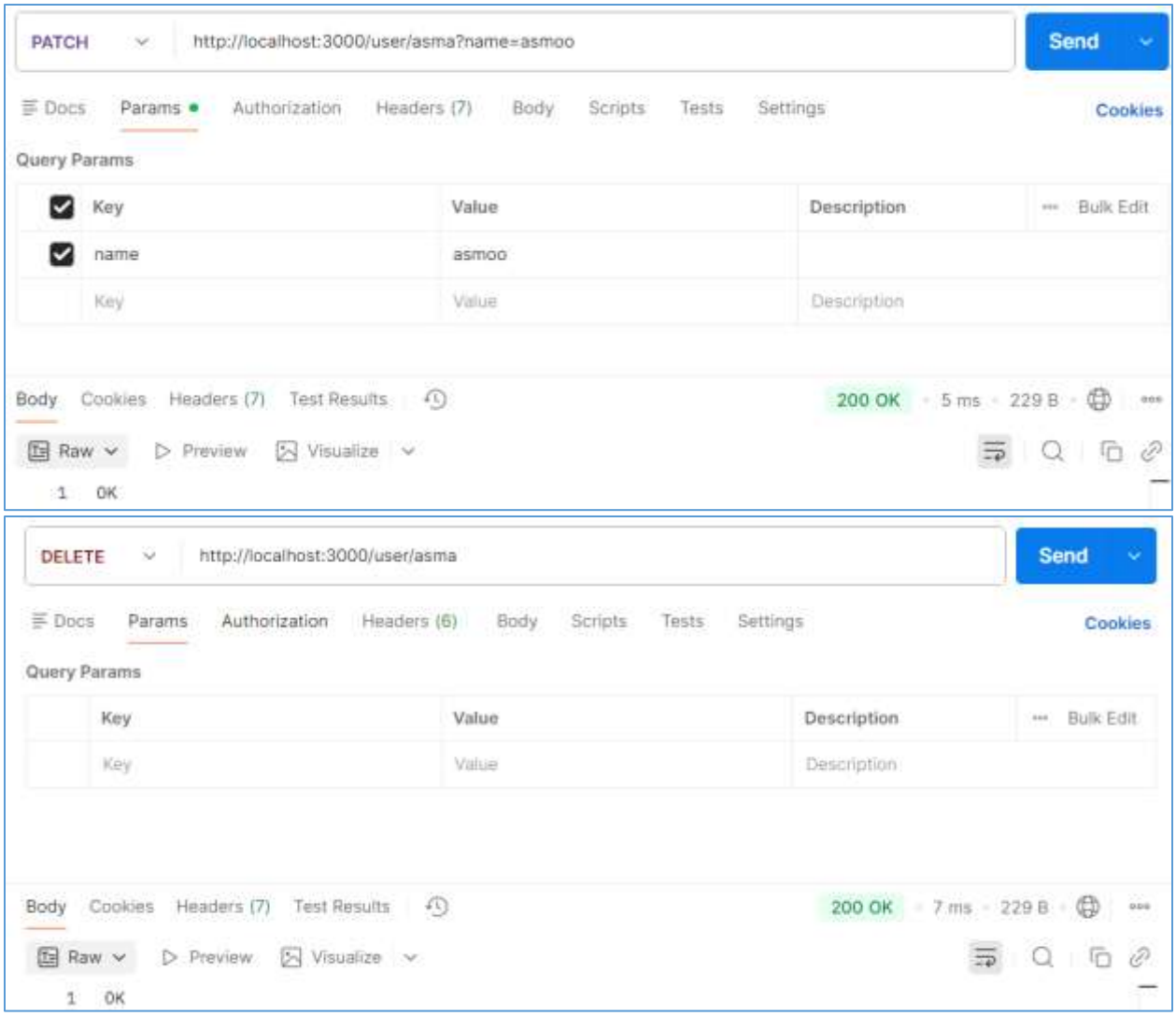
http://localhost:3000





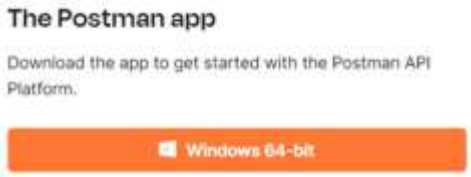


Email updated



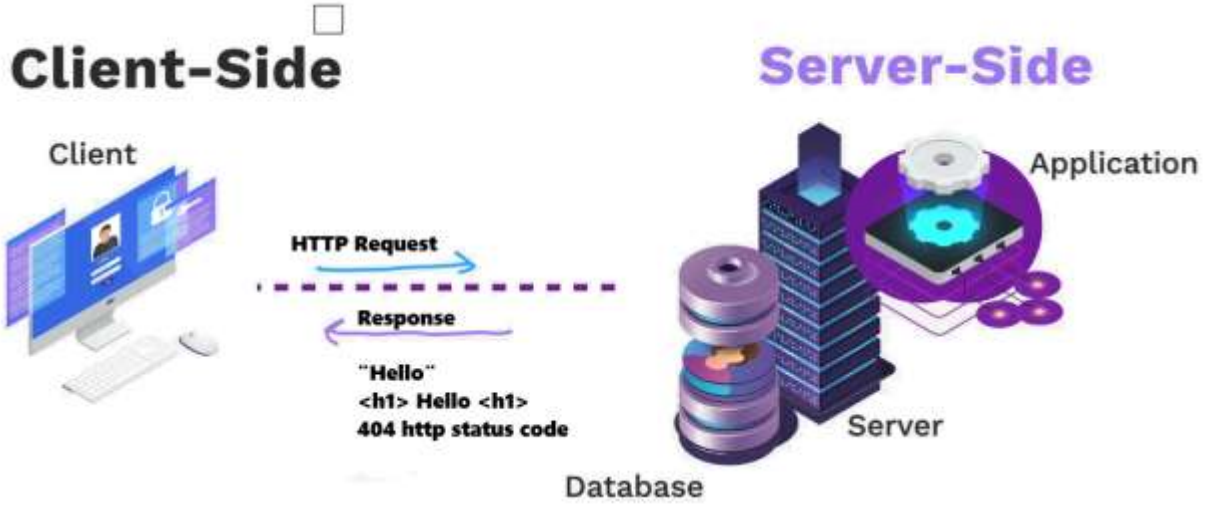
Postman: HTTP Requests

Postman is software which is used for API (Application Programming Interface) Testing.
API: is set of definitions and protocols for building and integrating applications.
Most of the people also called API as web service. And we can make other HTTP request such as GET, Post, PUT, Delete and Patch…
How to install Postman?
1 www.postman.com/Download



2
3 open the download file
Open file and install it and login with your id.

1. Informational responses (`100` – `199`)
2. Successful responses (`200` – `299`)
3. Redirection messages (`300` – `399`)
4. Client error responses (`400` – `499`)
5. Server error responses (`500` – `599`)

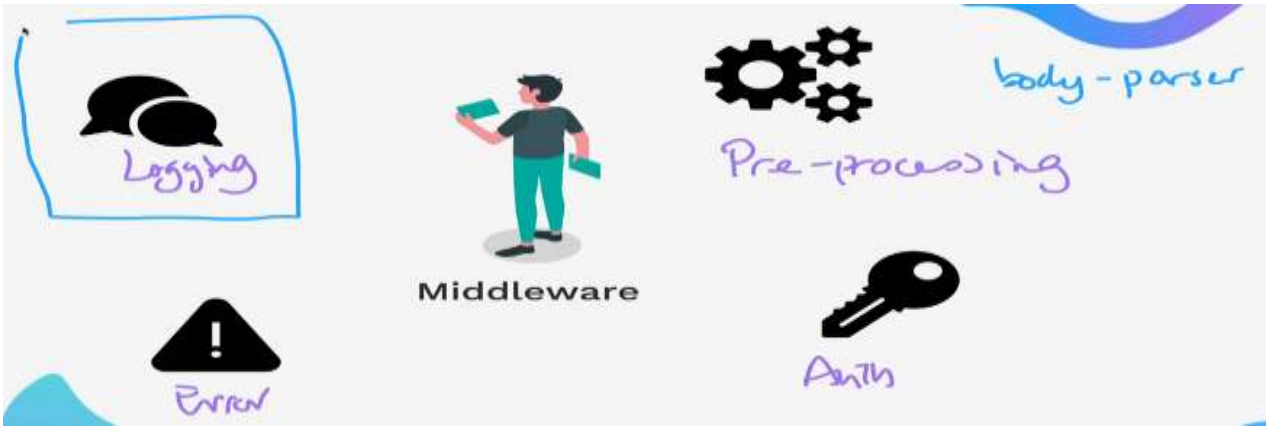


HTTP request = how data is sent (Get post put delete and patch)

API = what is exposed for apps to use

👉 **Every web API uses HTTP requests, but not every HTTP request is an API call.**

Middle wear



Body-parser
Create HTML file inside our project file

```html
<!DOCTYPE html>
<html lang="en">

<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-widt
  <title>Band Name Generator</title>
</head>

<body>
  <h1>Band Name Generator</h1>
  <form action="/submit" method="POST">
    <label for="street">Street Name:</label>
    <input type="text" name="street" required>
    <label for="pet">Pet Name:</label>
    <input type="text" name="pet" required>
    <input type="submit" value="Submit">
  </form>
</body>

</html>
```

index1.js > ...

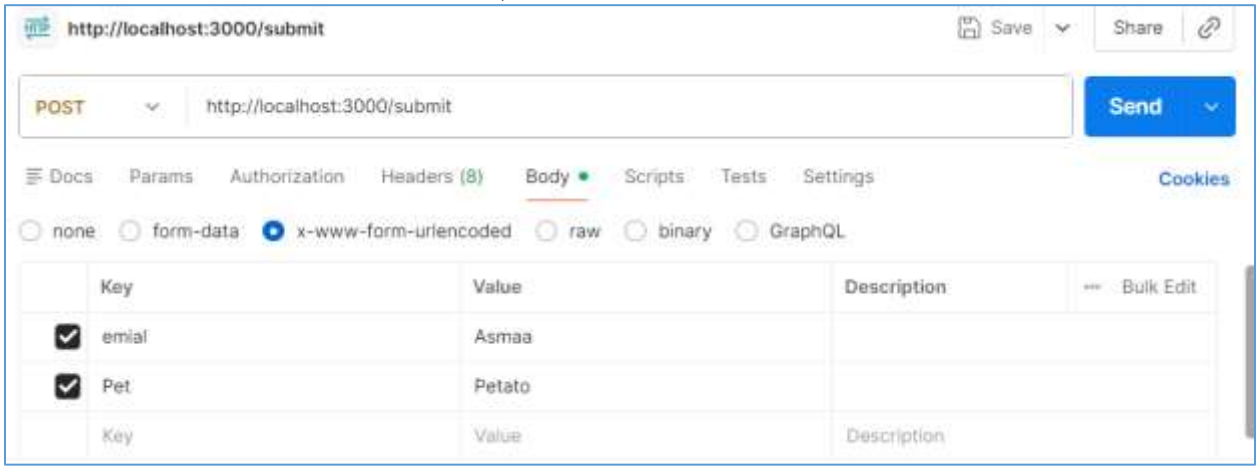
```js
import express from "express";
import { dirname } from "path";
import { fileURLToPath } from "url";
const __dirname = dirname(fileURLToPath(import.meta.url));

import bodyParser from "body-parser";

const app = express();
const port = 3000;

app.use(bodyParser.urlencoded({extended: true}));

app.get("/", (req, res) => {
  res.sendFile(__dirname + "/public/index.html");
});
app.post("/submit", (req, res)=>{
  console.log(req.body);
})

app.listen(port, () => {
  console.log(`Listening on port ${port}`);
});
```

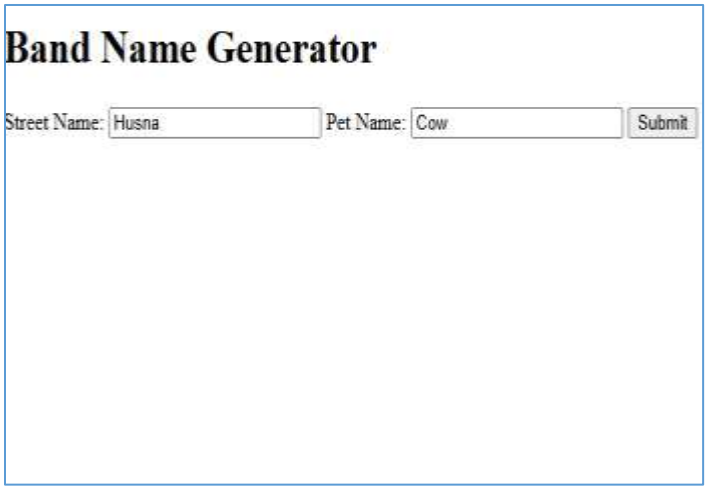
package.json > ...

```
{
  "name": "3.3-middleware",
  "version": "1.0.0",
  "description": "",
  "main": "index.js",
  "type": "module",
  Debug
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1"
  },
  "keywords": [],
  "author": "",
  "license": "ISC",
  "dependencies": {
    "body-parser": "^1.20.4",
    "express": "^4.18.2",
    "morgan": "^1.10.0"
  }
}
```

Npm i express morgan body-parser

Run server nodemon index1.js (we can submit post request through postman and localhost:3000/submit as we see below)





Logging Middleware (Morgan) we can get it npmjs.org

index2.js > ...

```
1  import express from "express";
2  import morgan from "morgan";
3
4  const app = express();
5  const port = 3000;
6  app.use(morgan("combined"))
7
8  app.get("/", (req, res) => {
9    res.send("Hello");
10 });
11
12 app.listen(port, () => {
13   console.log(`Listening on port ${port}`);
14 });
15
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
qazif@POPAL MINGW64 ~/OneDrive/Desktop/CSS - live/Backend/Middleware
$ npm i morgan
$ nodemon index2.js
```

Localhost:3000/ = HELLO

```
Listening on port 3000
::1 - - [10/Feb/2026:21:52:49 +0000] "GET / HTTP/1.1" 200 5 "-" "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/144.0.0.0 Safari/537.36"
::1 - - [10/Feb/2026:21:52:49 +0000] "GET /favicon.ico HTTP/1.1" 404 150 "http://localhost:3000/" "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/144.0.0.0 Safari/537.36"
```

How to create our own Middleware

index3.js > ...

```
1   import express from "express";
2
3   const app = express();
4   const port = 3000;
5
6   function logger(req, res, next){
7     console.log("Request Method:", req.method);
8     console.log("Request URL:", req.url);
9     next();
10  }
11  app.use(logger);
12
13  app.get("/", (req, res) => {
14    res.send("Hello");
15  });
16
17  app.listen(port, () => {
18    console.log(`Listening on port ${port}`);
19  });
20
```

So that way our postman or localhost: 3000 won't be stack and busy, cuz of next () method

Complete project backend server + Front End

NB Login > index.html > html > body > form > input

```html
<!DOCTYPE html>
<html lang="en">

<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, in
  <title>NB</title>
</head>

<body>
  <h1>National Bank Login Form</h1>
  <form action="/check" method="POST">
    <label>User Name:</label>
    <input type="text" name="username" required>

    <label>Password:</label>
    <input type="text" name="password" required>

    <input type="submit" value="Submit">
  </form>
</body>

</html>
```

NB Login > second.html > html

```html
<!DOCTYPE html>
<html lang="en">

<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, i
  <title>ZNB</title>
</head>

<body>
  <h1>Welcome to Zoya Bank</h1>
</body>
</html>
```

{} package.json > ...

```json
{
  "name": "3.3-middleware",
  "version": "1.0.0",
  "description": "",
  "main": "index.js",
  "type": "module",
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1"
  },
  "keywords": [],
  "author": "",
  "license": "ISC",
  "dependencies": {
    "body-parser": "^1.20.4",
    "express": "^4.18.2",
    "morgan": "^1.10.1"
  }
}
```

NB Login > JS index.js > ...

```
import express from "express";
import bodyParser from "body-parser";
import { dirname } from "path";
import { fileURLToPath } from "url";

const __dirname = dirname(fileURLToPath(import.meta.url));

const app = express();
const port = 3000;

var userIsAuthorised = false;
app.use(bodyParser.urlencoded({ extended: true }));

function passwordCheck(req, res, next) {

  if (req.body && req.body["username"] && req.body["password"]) {
    const username = req.body["username"];
    const password = req.body["password"];

    if (username === "zoya" || username==="asma" && password === "123") {
      userIsAuthorised = true;
    } else { userIsAuthorised = false;}
  }
  next();
}
app.use(passwordCheck);

app.get("/", (req, res) => {
  res.sendFile(__dirname + "/index.html");
});

app.post("/check", (req, res) => {
  if (userIsAuthorised) {
    res.sendFile(__dirname + "/second.html");
  } else {
    res.sendFile(__dirname + "/index.html");
  }
});

app.listen(port, () => {
  console.log(`Listening on port ${port}`);
});
```

- Create HTML Files
- mkdir “NB Login”
- cd “NB Login”
- touch index.js
- Npm init –y

- Npm I body-parser express morgan
- Start index.js

localhost:3000/

National Bank Login Form

User Name: Password: Submit

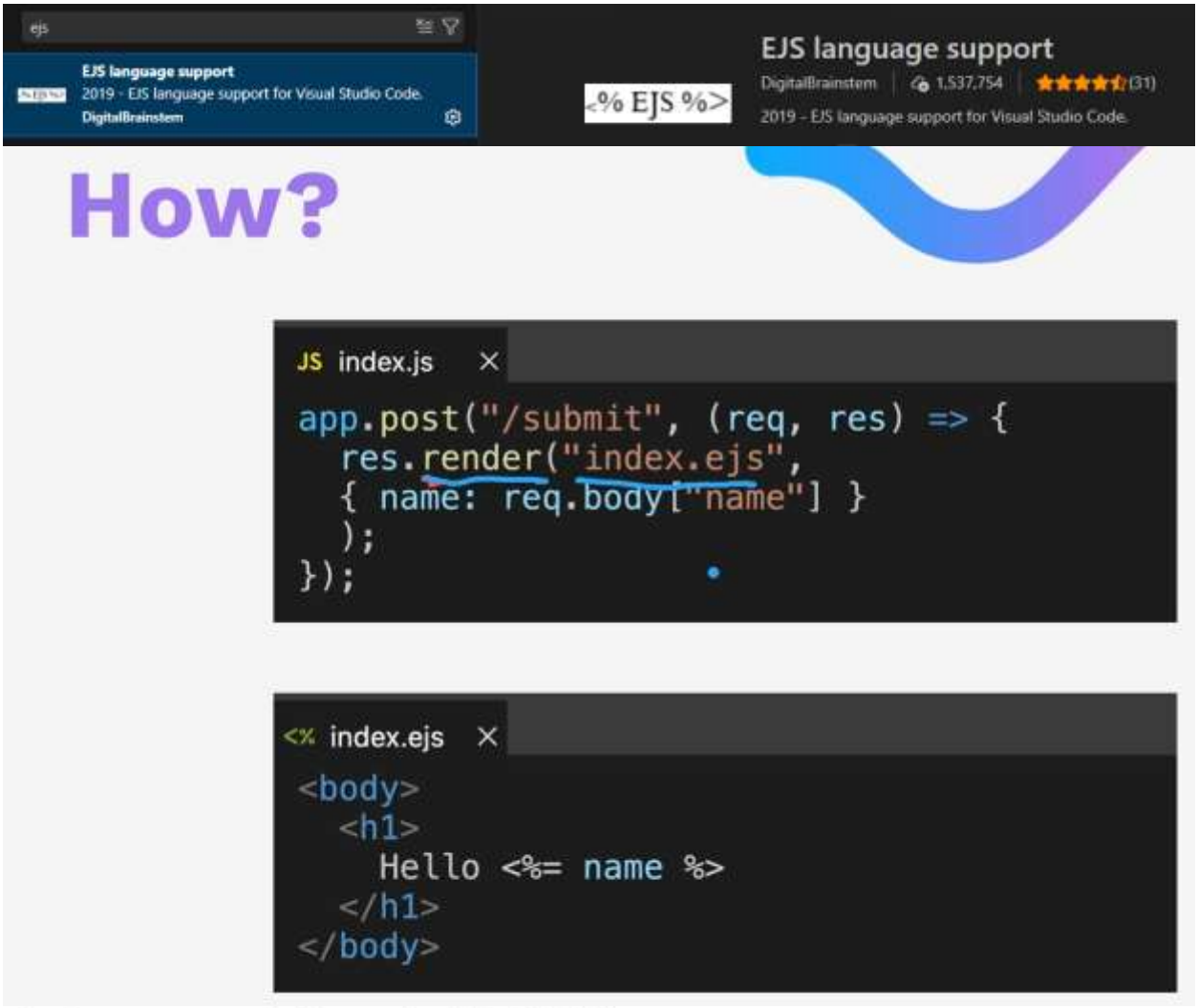
Welcome to Zoya Bank

Console

```
qazif@POPAL MINGW64 ~/OneDrive/Desktop/CSS - live/Backend/Middleware
$ nodemon index4.js
[nodemon] 3.1.11
[nodemon] to restart at any time, enter `rs`
[nodemon] watching path(s): *.*
[nodemon] watching extensions: js,mjs,cjs,json
[nodemon] starting `node index4.js`
Listening on port 3000
{ street: 'Asma', pet: 'Petato' }
```

Home work

Embedded JavaScript (EJS)



1. Create a new folder called “4.0 EJS”.
2. Initialise NPM and install **express** and **ejs**.
3. Create the following files and folders: index.js, views/index.ejs
4. Use the JS getDay() method to build a website that gives advice based on the day of the week.
    1. Create a folder with name EJS
    2. Npm init –y
    3. Npm i express ejs
    4. In json Type = Module
    5. Create a folder views
    6. Touch index.js views/index.ejs

Index.ejs

Timetable > views > index.ejs > ...

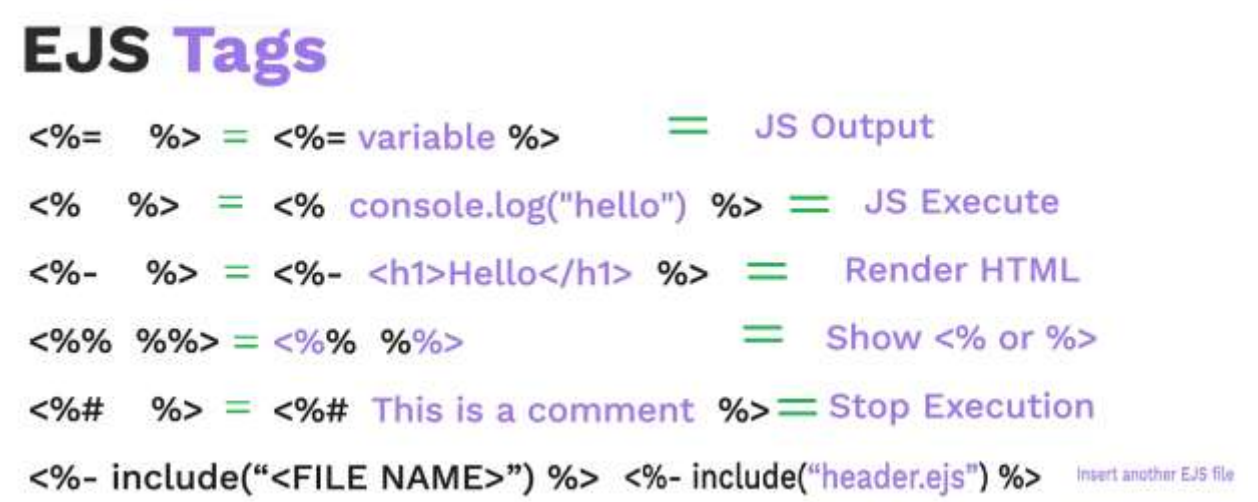
```ejs
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Document</title>
</head>
<body>
    <h1>Hey it's <%= daytype %>, <%= advice %></h1>
</body>
</html>
```

```js
const today = new Date();
const day = today.getDay();
```

Index.js

Timetable > JS index.js > ...

```js
import express from "express";

const app = express();
const port=3000;

app.get("/", (req, res)=>{
    res.render("index.ejs",{
        daytype:" a weekday",
        advice:"it time to work and follow your study(D.A.T.E)",
    });
})

app.listen(port, ()=>{
    console.log(`Server is running on port ${port} now!`);
});
```



EJS & Passing Data

Passing data from server to client and client to server.

index.ejs — **Frontend**

schedual > views > index.ejs > html

```
1  <!DOCTYPE html>
2  <html lang="en">
3  <head>
4      <meta charset="UTF-8">
5      <meta name="viewport" content="width=device-width,
6      <title>Document</title>
7  </head>
8  <body>
9      <h1>Hey it is  <%= daytype %> <%= advice %></h1>
10 </body>
11 </html>
```

Server

schedual > index.js > ...

```
1  import express from "express";
2  const app = express();
3  const port = 3000;
4
5  app.get("/", (req, res)=>{
6      res.render("index.ejs",{
7          daytype:"A week day",
8          advice:"It's time to work Hard you know that Time is Gold"
9      });
10 })
11
12 app.listen(port, ()=>{
13     console.log(`Server is running like crazy deer on port ${port}.`)
14 })
```

From Client Side to Server

National Bank Login Form

User Name: Asma Password: 123 Submit

Post http request

middleware

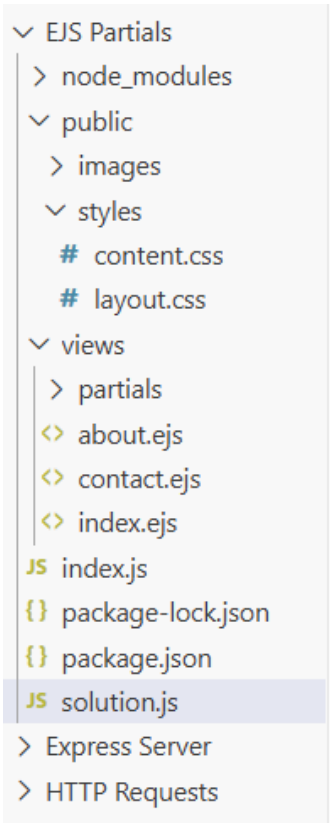
Server Side console

UserName="Asma"
password = "123"

```
<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="UTF-8" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <title>Name Letters</title>
  </head>

  <body>
    <% if (locals.numberofletters) { %>
    <h1>There are <%= numberofletters %> letters in your name.</h1>
    <% } else { %>
    <h1>Enter your name below 👇</h1>
    <% } %>
    <form action="/submit" method="POST">
      <input type="text" name="fName" placeholder="First name" />
      <input type="text" name="lName" placeholder="Last name" />
      <input type="submit" value="OK" />
    </form>
  </body>
</html>
```

Passing+Data > index.js > ...

```
1  import express from "express";
2  import bodyParser from "body-parser";
3
4  const app = express();
5  const port = 3000;
6
7  app.use(bodyParser.urlencoded({ extended: true }));
8
9  app.get("/", (req, res) => {
10   res.render("index.ejs");
11 });
12
13 app.post("/submit", (req, res) => {
14   const lettersofNames = req.body["fName"].length + req.body["lName"].length;
15   res.render("index.ejs",{numberofLetters:lettersofNames});
16 });
17
18 app.listen(port, () => {
19   console.log(`Server running on port ${port}`);
20 });
```




```js
import express from "express";

const app = express();
const port = 3000;
app.use(express.static("public"));

app.get("/", (req, res) => {
  res.render("index.ejs");
});

app.get("/about", (req, res) => {
  res.render("about.ejs");
});

app.get("/contact", (req, res) => {
  res.render("contact.ejs");
});

app.listen(port, () => {
  console.log(`Server running on port ${port}`);
});
```

Name Generator Project (EJS, CSS, JS, Node.js, Express Server, HTTP Requests)

Project > index.js > ...

```
import express from "express";
import bodyParser from "body-parser";

const app = express();
const port = 3000;

app.use(express.static("public"));//Step 3 - Make the styling show up.
//Hint 1: CSS files are static files!
//Hint 2: The header and footer are partials.
//Hint 3: Add the CSS link in header.ejs

app.use(bodyParser.urlencoded({ extended: true }));

app.get("/", (req, res) => {
  res.render("index.ejs");
});

app.post("/submit", (req, res) => { //Step 2 - Make the generate name functionality work
  //Hint: When the "Generate Name" button in index.ejs is clicked, it should hit up this route.
  //1. You should randomly pick an adjective from the const "adj" and a noun from const "noun",
  //scroll down to see the two arrays.
  const randomADJ = adj[Math.floor(Math.random() * adj.length)] //2. Send the index.ejs as a response and add the adjective and noun to the res.render
  const randomNoun = noun[Math.floor(Math.random()*noun.length)]//3. Test to make sure that the random words display in the h1 element in index.ejs
  res.render("index.ejs", {adj:randomADJ, noun:randomNoun});
});

app.listen(port, () => {
  console.log(`Listening on port ${port}`);
});
const adj = ["Best","Strong","Tall","Intellegent",];
const noun = ["Zoya","Asma","Husna","Idress","Ershad","Khalid",];
```

```
body {
  background: #222;
  font-family: -apple-system, BlinkMacSystemFont, avenir next, avenir, segoe ui,
    helvetica neue, helvetica, Ubuntu, roboto, noto, arial, sans-serif;
  color: hotpink;
}

main {
  position: absolute;
  width: 100%;
  height: 100%;
  top: 0;
  left: 0;
  display: flex;
  align-items: center;
  justify-content: center;
  flex-direction: column;
  text-align: center;
}

h1 {
  font-size: 3.5rem;
}

input {
  appearance: none;
  -webkit-appearance: none;
  border: 0;
  font-size: 0.9rem;
  background: #ded0d0;
  padding: 1rem;
  color: rgb(19, 18, 18);
  cursor: pointer;
}

footer {
  position: absolute;
  bottom: 0;
}
```

```
  <%- include("partials/header.ejs") %>

  <% if(locals.adj && locals.noun){%>
    <h1> <%= adj %> <%= noun %></h1>
  <%} else{  %>
  <h1>Welcome to the Band Generator</h1>
  <%  } %>


  <form action="/submit" method="POST">
    <input type="submit" value="Generate Name">
  </form>
  <%- include("partials/footer.ejs") %>

</main>

<footer>
  <!-- Step 4 - Add a dynamic year to the footer using JS. -->
  <p>Copyright © 200X</p>
</footer>
</body>

</html>
```

```
<!DOCTYPE html>
<html lang="en">

<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">

  <!-- Add the link to the CSS stylesheet here -->
  <link rel="stylesheet" href="/styles/main.css">
  <!-- Hint: the link is relative to the public folder -->
  <title>Band Generator</title>
</head>

<body>
  <main>
```

Homework (**Dictionary**) use **EJS, CSS, JS, node.js, express server (http requests)**

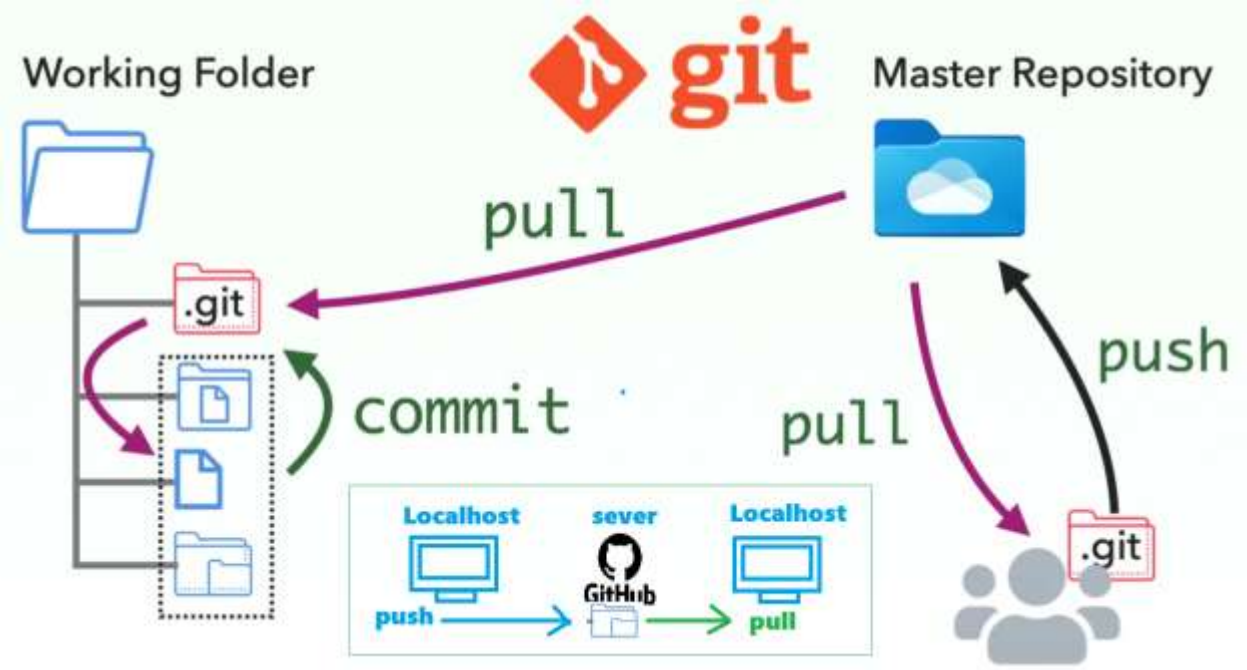
Homework

GOOGLE

type here ...

Search

Git, Github (Version Control):



GitHub is a website where you store your Git projects online and work with others.

GitHub is a cloud-based platform that hosts Git repositories and helps developers collaborate, share, and manage their code online.

Git is a command line program used to push and pull our repository.

Pushing a file to GitHub

It comprises of following 3 steps –

a) Staging (i.e. selecting the files to be pushed) – *git add file_name.extension*
b) Committing (i.e. signing before final push) – *git commit –m "commit message"*
c) Pushing (i.e. actually pushing the code) – *git push origin main*

git status ➔ **git add .** ➔ **git commit –m "updated today"** ➔ **git push origin main**

```
cd Destop
mkdir story                create folder
cd story                   enter story folder
ls                         list directory
touch chapter1.txt         create a file
code chapter1.txt          open file  or start chapter1.txt
```

git init — **initialize git command becomes our working directory**

git status — **show the unsaved changes in files (Show status of our file)**

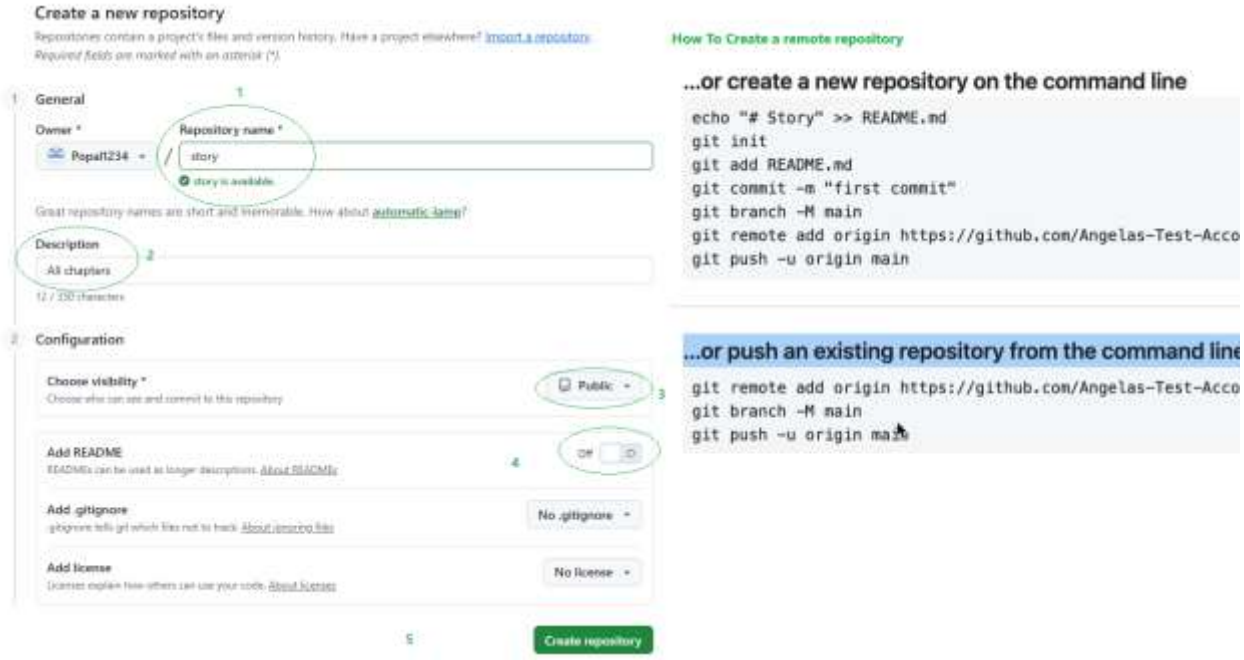
Stage — git add chapter1.txt — stage our files means add our file (git add .) add all files

commit — git commit -m "chapter 1 file upgraded" — sign before push

git log — show our last commit when happend

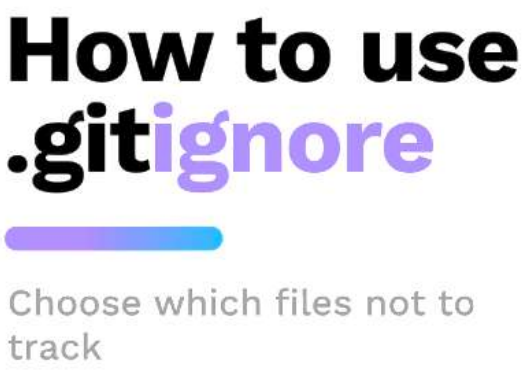
git diff chapter3.txt — **compare our new changes to old version**

git checkout chapter3.txt — **dismiss new changes (Undo)**




```bash
git add .
git commit -m "first commit"
git push -u origin master
```

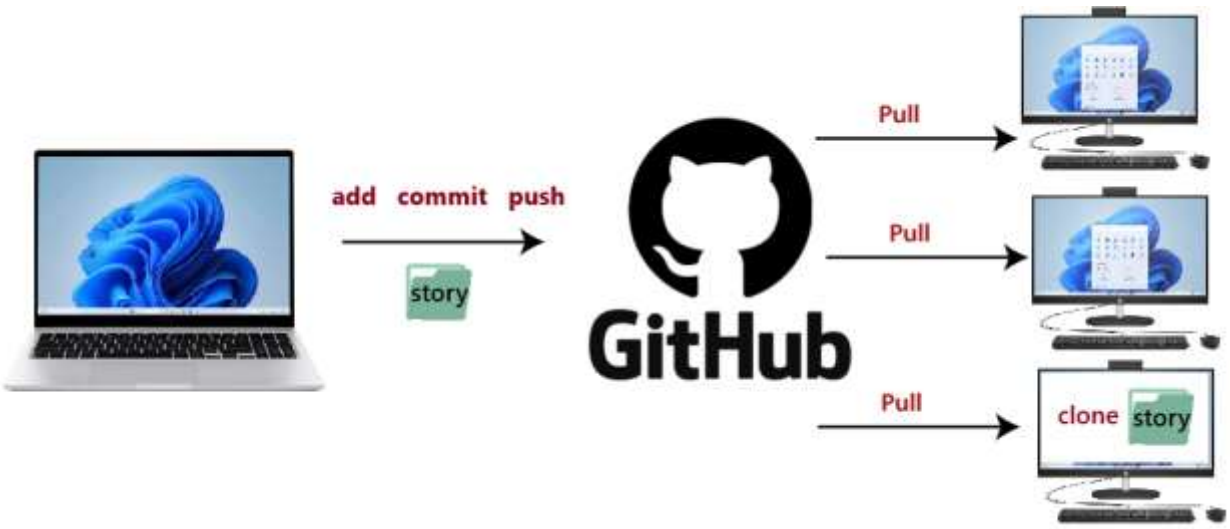
Our folder name is master not main



Ls –a (shows hidden files)
Touch .gitignore
Git rm --cached -r . (undo all stage files)
Code .gitignore (to open gitignore file) (put the file name that you don't want to share)

Git clone

How to download and create a copy of a repository



git clone url

git clone https://github.com/Popal1234/story.git

Branches and Merging:

Branches / Merging

Ex : Main Branch master copy = phara

git branch name	=create a new branch (git branch phara * means current branch)
git checkout phara	= switch to phara
git branch	= show us our current branch
git merge phara	= merge phara changes to main branch be on main branch to do merge
git log	= show us lost commit

Fork (Copy repo)

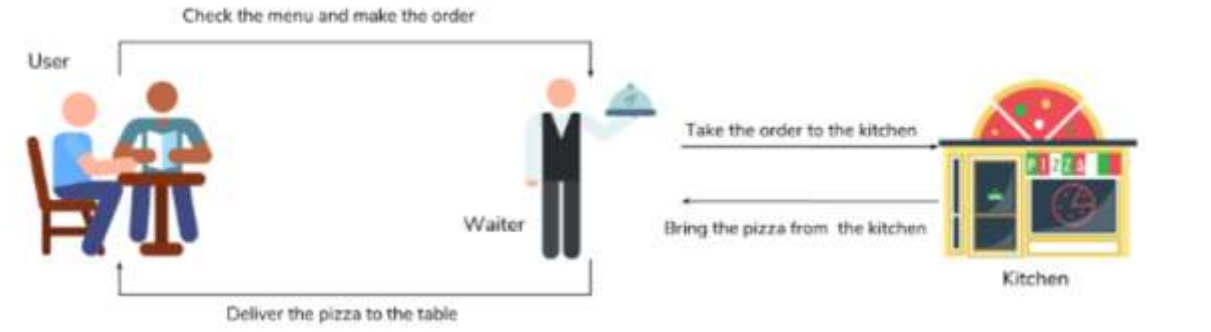
User: fork story repo, brought changes, pull request
Owner: approved / merge pull request

```
Microsoft Windows [Version 10.0.19045.2604]
(c) Microsoft Corporation. All rights reserved.

D:\gitRepo\git_repo\git3\java_feb_16>git branch
* main

D:\gitRepo\git_repo\git3\java_feb_16>git branch tc_201

D:\gitRepo\git_repo\git3\java_feb_16>git branch
* main
  tc_201

D:\gitRepo\git_repo\git3\java_feb_16>git checkout tc_201
Switched to branch 'tc_201'

D:\gitRepo\git_repo\git3\java_feb_16>git branch
  main
* tc_201

D:\gitRepo\git_repo\git3\java_feb_16>git add .

D:\gitRepo\git_repo\git3\java_feb_16>git commit -m "Added Ford in TC_201 branch"
[tc_201 dc568fb] Added Ford in TC_201 branch
 1 file changed, 1 insertion(+)

D:\gitRepo\git_repo\git3\java_feb_16>git checkout main
Switched to branch 'main'
Your branch is up to date with 'origin/main'.

D:\gitRepo\git_repo\git3\java_feb_16>git checkout tc_201
Switched to branch 'tc_201'

D:\gitRepo\git_repo\git3\java_feb_16>
```

① Check which branches exist in local repo
② Create a new branch
③ Check list of branches in local repo
④ Shift to the new tc_201 branch
⑤ Check on which branch the focus is
⑥ After updating the doc in new branch, stage it
⑦ Commit the changes
⑧ Switching to the main branch
⑨ Switch to tc-201 branch again.

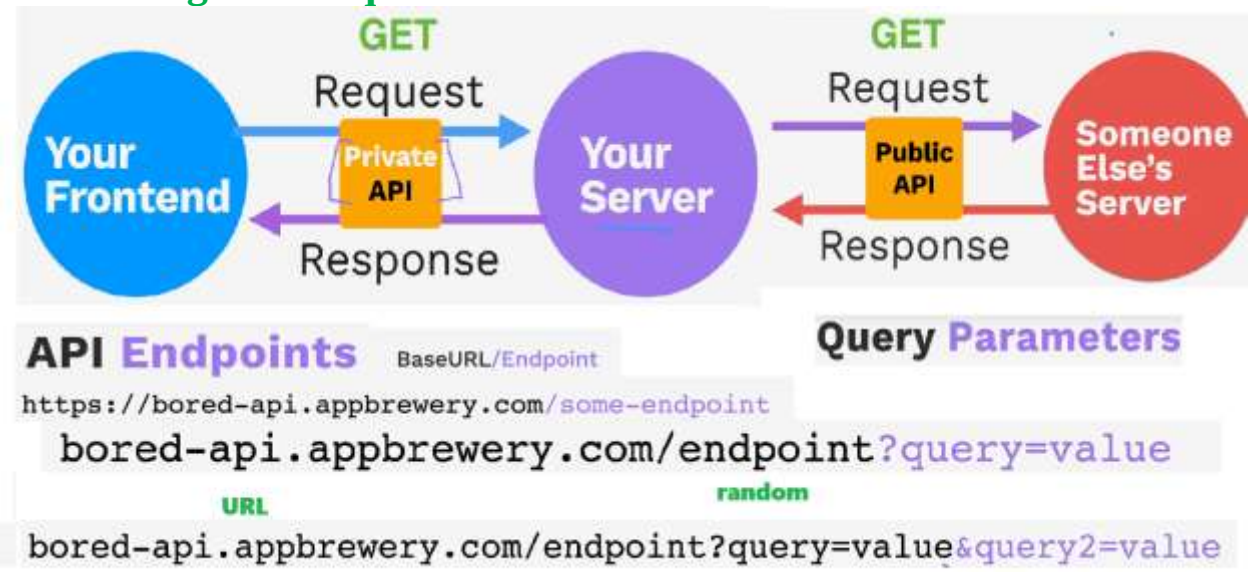
APIs (Application Programming Interfaces)

API the technology that powers the communications between software on the internet.
An API is like a waiter between two apps — it takes your request and brings back the response.
It allows different software systems to connect and share data easily. (Sends REQ brings RES).



API Types: REST, SOAP (different types of Architecture methods of API)
REST API: HTTP requests send through API and get res through API (in Format of JSON [key: value])
Ex: package.JSON which all about **“key”: values, {“name”: “Ali”, “id”:1212}**
We can use postman without having front end to send request the backend. GPS APIS longitude latitude
points. Which is used for the location.

Formatting API Requests



With a single query we use? And If you want to add more queries we use &

Filter Activities

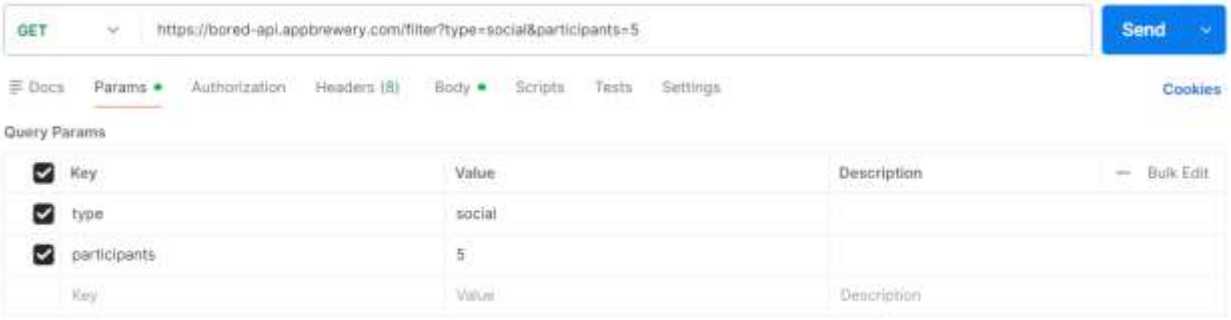
Returns a list of activities filtered by type and/or participants.

Query Parameters:

- `type` (optional): The type of activity to filter by. Choice of `education`, `recreational`, `social`, `charity`, `cooking`, `relaxation`, `busywork`.
- `participants` (optional): The number of participants to filter by. Choice of 1, 2, 3, 4, 5, 6, 8.

Example Request:

```
GET https://bored-api.appbrewery.com/filter?type=education
```



Path Parameters use to identify a source

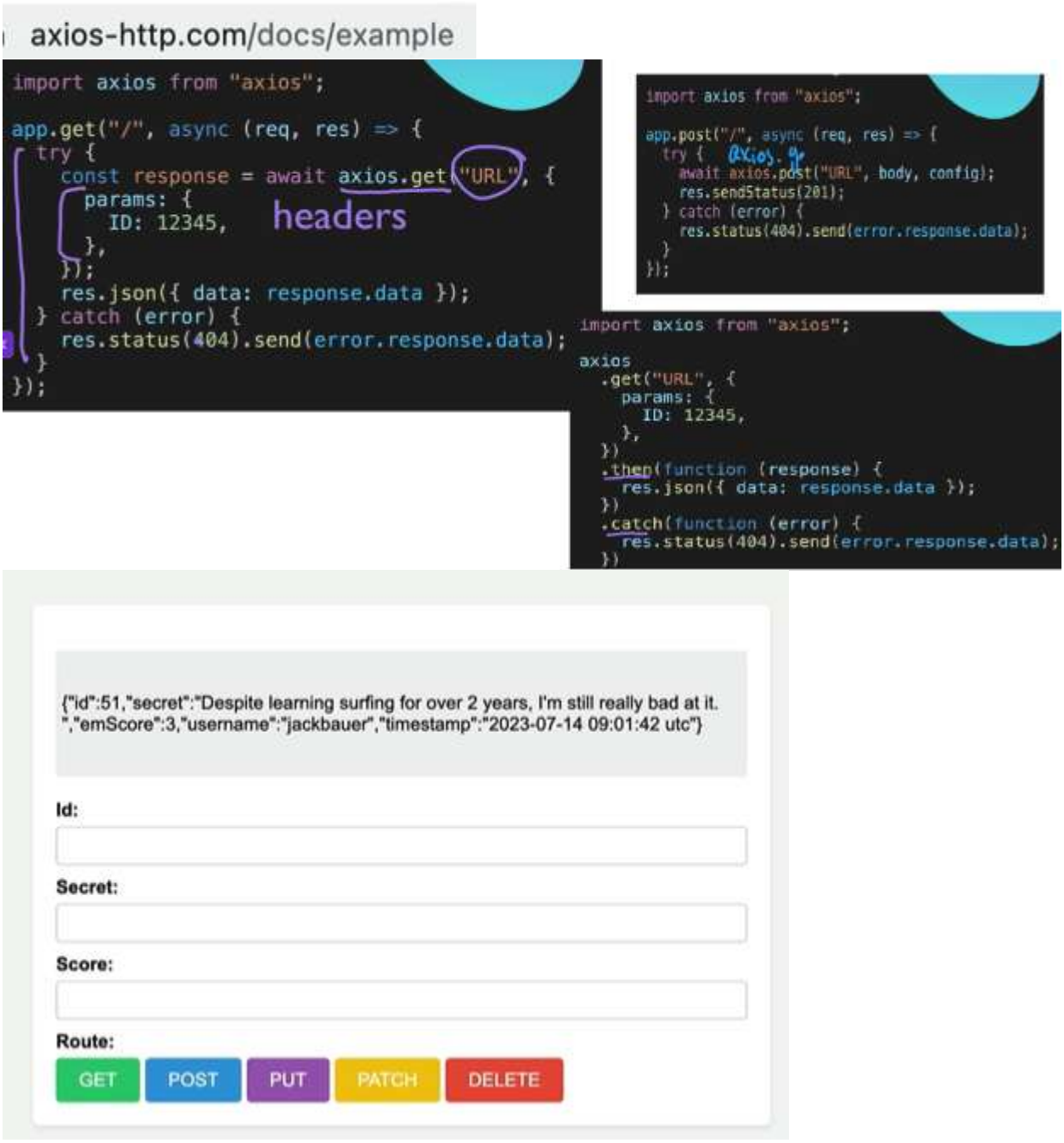
bored-api.appbrewery.com/endpoint/{path-parameter}

URL /random ? query=value id, username

we can also provide key value as a path paramerters

REST APIs

Making Get, Post, put, patch, and delete API Request using Axios



Grand API Project (Add to our Project)

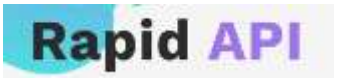
```
// HINTS:
// 1. Import express and axios
import express from "express";
import axios from "axios";
// 2. Create an express app and set the port number.
const app = express();
const port = 3000;
// 3. Use the public folder for static files.
app.use(express.static("public"));
// 4. When the user goes to the home page it should render the index.ejs file.
// 5. Use axios to get a random secret and pass it to index.ejs to display the
// secret and the username of the secret.
app.get("/", async(req, res)=>{
    try{
    const result = await axios.get("https://secrets-api.appbrewery.com/random");
    res.render("index.ejs", {secret: result.data.secret, user:result.data.username});}
    catch{console.log(error.response.data); res.status(500);}
})
// 6. Listen on your predefined port and start the server.
app.listen(port, ()=>{
    console.log(`Sever is running on port ${port}.`);
})
```

```
<!DOCTYPE html>
<html>

<head>
  <title>Lisper</title>
  <link rel="preconnect" href="https://fonts.googleapis.com">
  <link rel="preconnect" href="https://fonts.gstatic.com" crossorigin>
  <link href="https://fonts.googleapis.com/css2?family=Gloria+Hallelujah&family=Titan+One&display=swap"rel="stylesheet">
  <link rel="stylesheet" type="text/css" href="styles/main.css">
</head>
<body>
  <h1>What's your secret?</h1>
  <div class="card">
    <p>
      <%= secret %>
    </p>
  </div>
  <p class="user">
    <%= user %>
  </p>
</body>
</html>
```

DO you own CSS

- More free APIs: https://rapidapi.com/collection/list-of-free-apis

MAKE YOUR OWN API



We can host our API here works same as git hub

JSON:

JSON (JavaScript Object Notation) is a key-value pair format used to store and exchange data.

It is lightweight and can be easily sent and received over the internet.

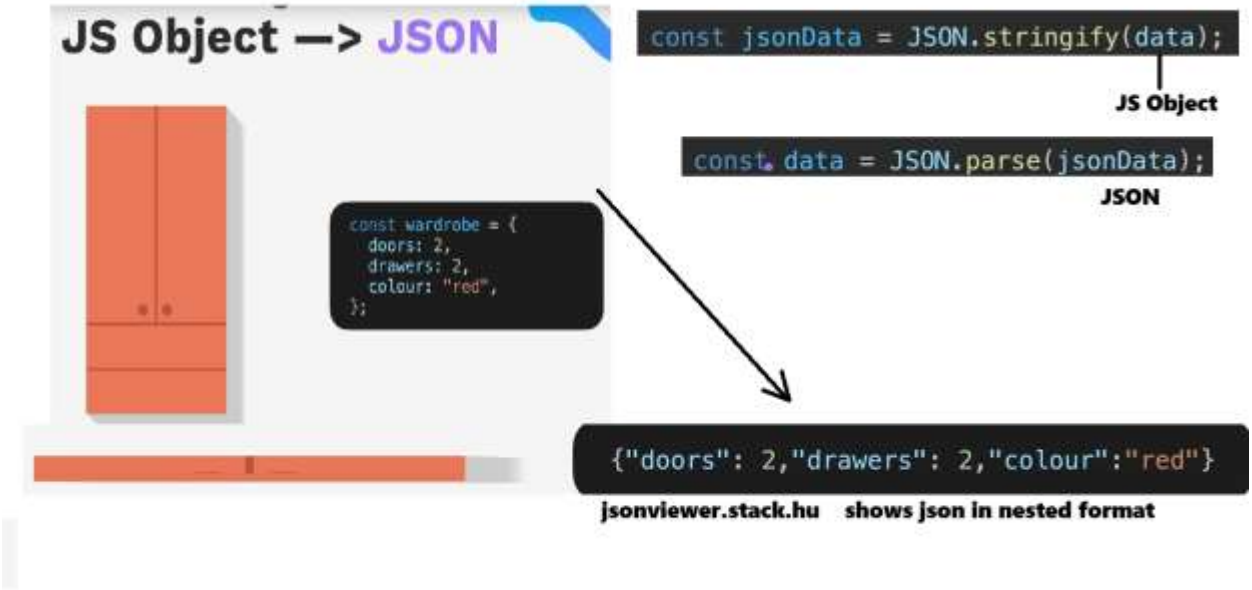
example.json

```
{
  "name": "Jack Bauer",
  "age": 48,
  "city": "Santa Monica",
  "education":
    {
      "degree": "B.A. English Literature",
      "university": "UCLA",
    },
    {
      "degree": "M.S. Criminology & Law",
      "university": "UC Berkeley",
    },
  ]
}
```

index.js

```
const jack = {
  name: "Jack Bauer",
  age: 48,
  city: "Santa Monica",
  education: [
    {
      degree: "B.A. English Literature",
      university: "UCLA",
    },
    {
      degree: "M.S. Criminology & Law",
      university: "UC Berkeley",
    },
  ],
};
```

JSON Viewer stack.com



index.js > ...

```
import express from "express";
import bodyParser from "body-parser";

const app = express();
const port = 3000;

const recipeJSON =
  '[{"id": "0001","type": "taco","name": "Chicken Taco","price": 2.99,"ingredients": {"protein": {"r

app.use(express.static("public"));
app.use(bodyParser.urlencoded({ extended: true }));

let data;

app.get("/", (req, res) => {
  res.render("index.ejs", { recipe: data });
});

app.post("/recipe", (req, res) => {
  switch (req.body.choice) {
    case "chicken":data = JSON.parse(recipeJSON)[0]; break;
    case "beef":data = JSON.parse(recipeJSON)[1];break;
    case "fish":data = JSON.parse(recipeJSON)[2];break;
    default:break;
  }
  res.redirect("/");
});

app.listen(port, () => {
  console.log(`Server running on port: ${port}`);
});
```

JSON > {} package.json > ...

```
{
  "name": "5.2-json",
  "version": "1.0.0",
  "description": "",
  "main": "index.js",
  "type": "module",
  Debug
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1"
  },
  "keywords": [],
  "author": "",
  "license": "ISC",
  "dependencies": {
    "body-parser": "^1.20.2",
    "ejs": "^3.1.9",
    "express": "^4.18.2",
    "nodemon": "^3.1.14"
  }
}
```

index.ejs × index.js

JSON > views > index.ejs > html > body > div#recipeContainer.hidden > ? > ? > ?

```ejs
<!DOCTYPE html>
<html>

<head>
  <title>Taco Recipes</title>
  <link rel="stylesheet" type="text/css" href="/styles/main.css">
</head>

<body>
  <h1>Taco Town </h1>


  <form action="/recipe" method="POST" class="buttons
  ">
    <button type="submit" value="chicken" name="choice"> </button>
    <button type="submit" value="beef" name="choice"> </button>
    <button type="submit" value="fish" name="choice"> </button>
  </form>
  <div id="recipeContainer" class="hidden">
    <%if (locals.recipe) {%>

      <h2 id="recipeTitle">
        <%= recipe.name %>
      </h2>

      <h3>Ingredients:</h3>
      <ul id="ingredientsList">
        <li><%= recipe.ingredients.protein.name %>, <%= recipe.ingredients.protein.preparation %></li>
        <li><%= recipe.ingredients.salsa.name %></li>
        <% recipe.ingredients.toppings.forEach(topping=> { %>
          <li><%= topping.quantity %> of <%= topping.name %></li>
          <% }) %>
      </ul>

      <%} else {%>
        <h2>Pick your favourite taco ingredient </h2>
        <%}%>
  </div>
</body>
```

```css
html {
  height: 100%;
}
body {
  font-family: Arial, sans-serif;
  margin: 20px;
  padding: 0;
  background-color: #e26e5c;
  background-image: url("data:ima
  text-align: center;
}

h1 {
  display: inline;
  font-size: 5rem;
  text-align: center;
  color: #e26e5c;
  background-color: white;
}

h2 {
  color: #e26e5c;
}

.buttons {
  display: flex;
  justify-content: center;
  margin: 20px;
}

button {
  font-size: 5rem;
}
```

```css
.buttons button {
  display: flex;
  flex-direction: column;
  align-items: center;
  justify-content: center;
  border: none;
  background-color: white;
  cursor: pointer;
  padding: 0;
  margin: 0 10px;
  width: 180px;
  height: 180px;
  border-radius: 50%;
  box-shadow: 0 2px 5px rgba(0, 0, 0, 0.2);
  transition: transform 0.3s, box-shadow 0.3s;
}

.buttons button:hover {
  transform: scale(1.05);
  box-shadow: 0 4px 10px rgba(0, 0, 0, 0.2);
}

.buttons button img {
  width: 120px;
  height: 120px;
  object-fit: cover;
  border-radius: 50%;
  margin-bottom: 10px;
}
```

```css
#recipeContainer {
  background-color: white;
  margin-top: 40px;
  display: flex;
  flex-direction: column;
  align-items: center;
}

#recipeTitle {
  color: #e26e5c;
}

#ingredientsList {
  margin: 0;
  padding: 0;
  list-style-type: none;
}

#ingredientsList li {
  margin-bottom: 10px;
  color: #555;
}
```

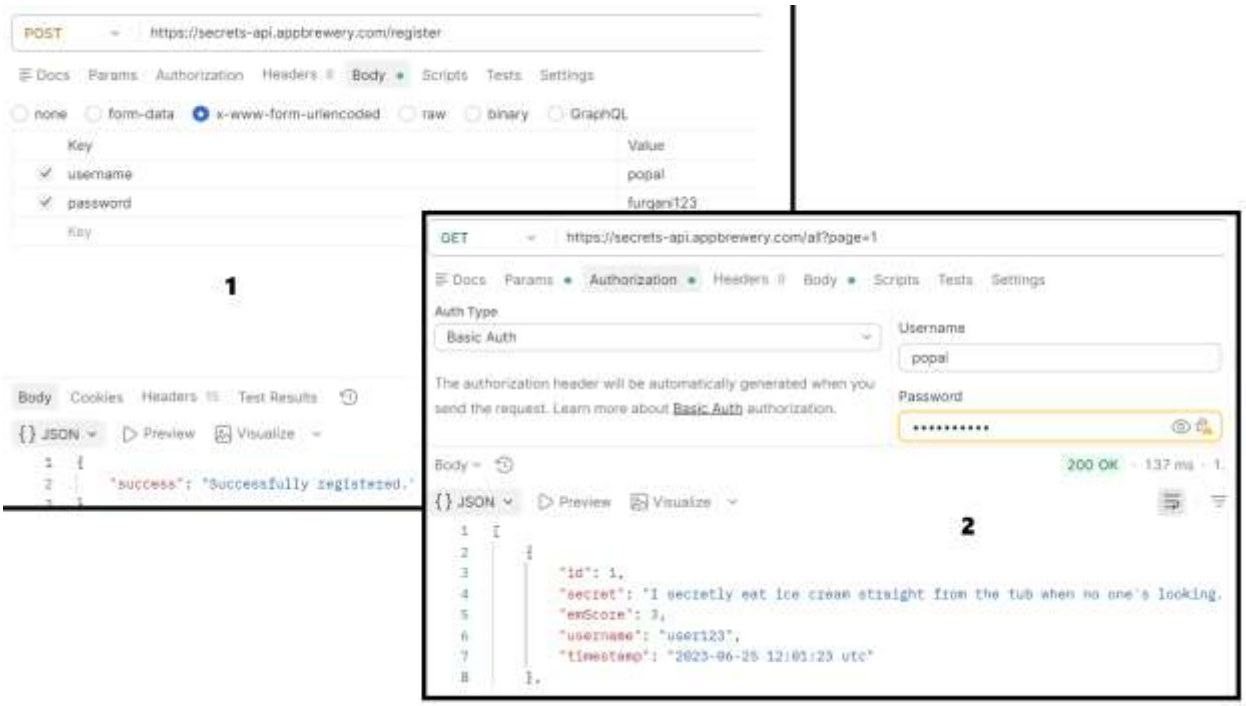
AXIOS

API Authorization and Authentication
401 unauthorized
Base64 Encoding.

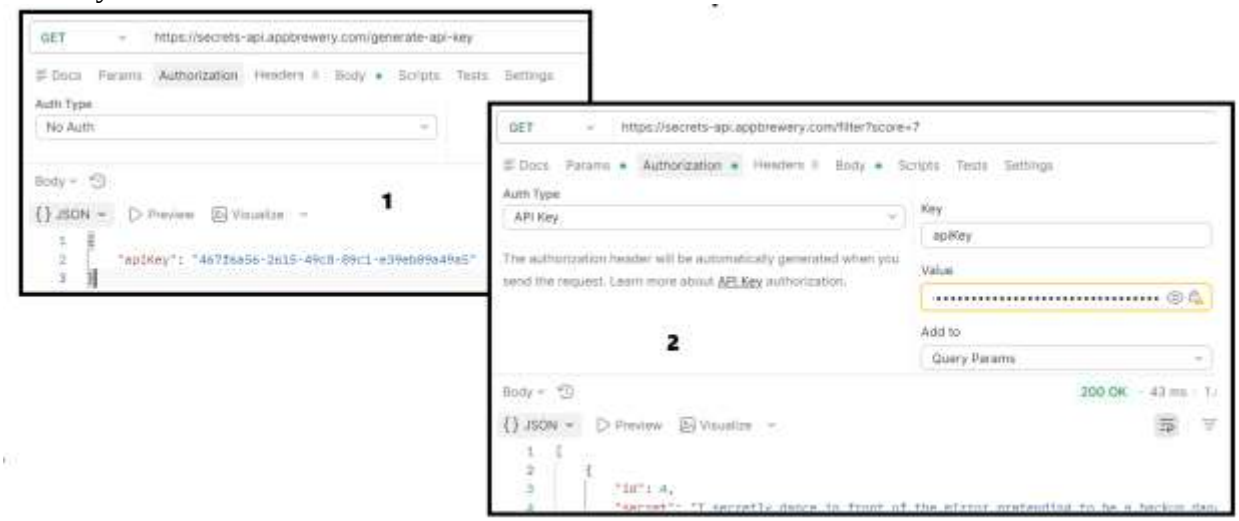
- **No Authentication**
- **Basic Authentication (Username and Password)**
- **API Key Authorization (to generate a key to authorize the access)**
- **Token Based Authentication (username and password plus key)**

Basic Authentication

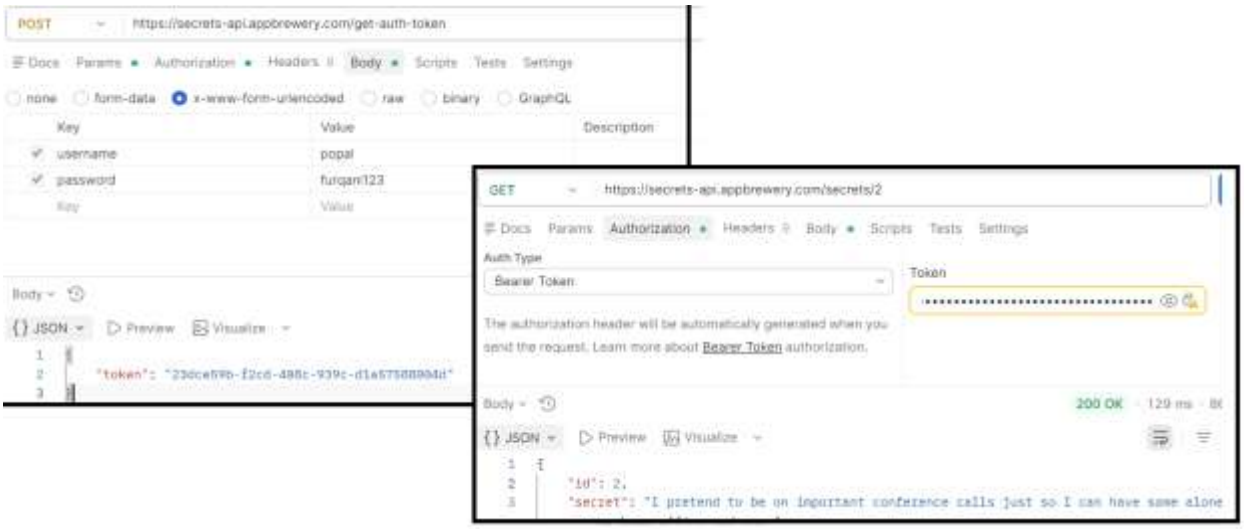
https://secrets-api.appbrewery.com/register
https://secrets-api.appbrewery.com/all?page=1



API-key



Used for the security of API
Token + Username and password



Using code with node.js

```
import express from "express";
import axios from "axios";

const app = express();
const port = 3000;
const API_URL = "https://secrets-api.appbrewery.com";

// TODO: Replace the values below with your own before running this file.
const yourUsername = "popal";
const yourPassword = "furqani123";
const yourAPIKey = "d6b9e489-d4d5-47dd-abd8-1fa3d1a781b5";
const yourBearerToken = "23dce59b-f2cd-408c-939c-d1a57588004d";

app.get("/", (req, res) => {
  res.render("index.ejs", { content: "API Response." });
});
// Route to fetch a random resource from the external API without authentication
// When the user visits /noAuth, the server sends a GET request to the API
// If successful, it renders index.ejs and passes the API response as content
// If an error occurs, it sends a 404 status with the error message
app.get("/noAuth", async (req, res) => {
  try {
    const result = await axios.get(API_URL + "/random");
    res.render("index.ejs", { content: JSON.stringify(result.data) });
  } catch (error) {
    res.status(404).send(error.message);
  }
});
```

Making REST: API

What makes API RESTFUL?

RESTFUL API

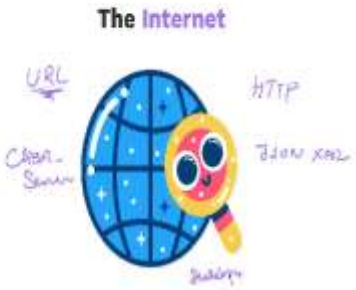
HTTP Methods GET POST PUT PATCH DELETE

JSON Output { "age": 29, }

Client-Server req ----- res

Stateless do not track previous reqs (Not save it in mem)

Resource-Based URL URI = Universal Resource Identifier/Locator



JOKE API

6.0 DIY API > index.js > jokes

```
1  import express from "express";
2  import bodyParser from "body-parser";
3
4  const app = express();
5  const port = 3000;
6  const masterKey = "4VGP2DN-6EWM4SJ-N6FGRHV-Z3PR3TT";
7
8  app.use(bodyParser.urlencoded({ extended: true }));
9
10 //1. GET a random joke
11 app.get("/random", (req , res)=>{
12   const randomJoke = Math.floor(Math.random()*100);
13   res.json(jokes[randomJoke]);
14 });
15
16 app.listen(port, () => {
17   console.log(`Successfully started server on port ${port}.`);
18 });
19
20 var jokes = [
21   {
22     id: 1,
23     jokeText:
24       "Why don't scientists trust atoms? Because they make up everything.",
25     jokeType: "Science",
26   },
27   {
28     id: 2,
29     jokeText:
30       "Why did the scarecrow win an award? Because he was outstanding in his field.",
31     jokeType: "Puns",
32   },
33   {
34     id: 3,
35     jokeText:
36       "I told my wife she was drawing her eyebrows too high. She looked surprised.",
37     jokeType: "Puns",
38   },
```

HTTP Method Get

Req Res client/server

JSON Format

http://localhost:3000/random

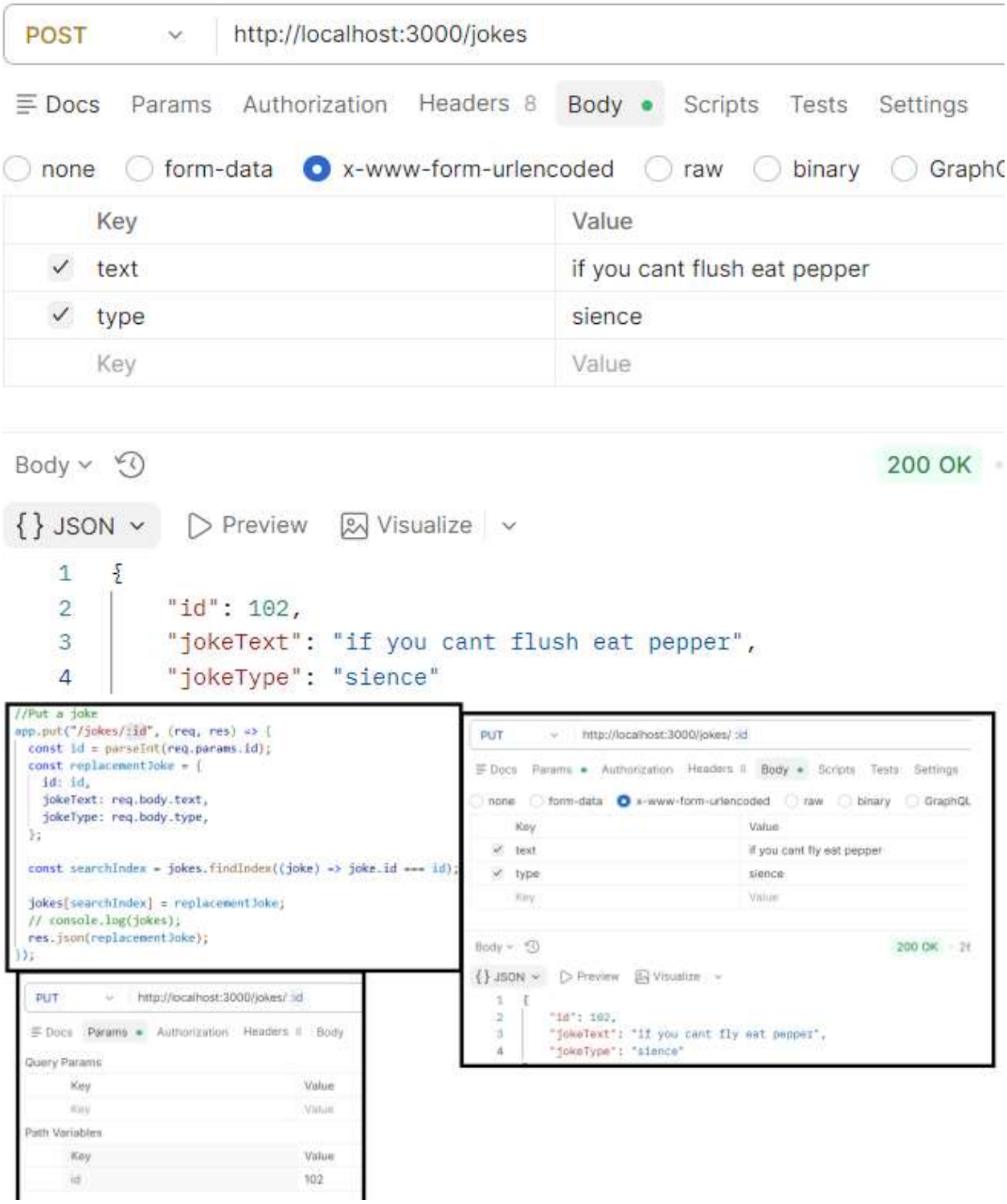

```
//2 get a specific joke
app.get("/joke/:id", (req , res)=>{
  const id = parseInt(req.params.id);
  const foundJoke = jokes.find((joke)=> joke.id===id)
  res.json(foundJoke);
});
```

https://developer.mozilla.org/en-US/play
MDN DOCS is used to lean about codes

.

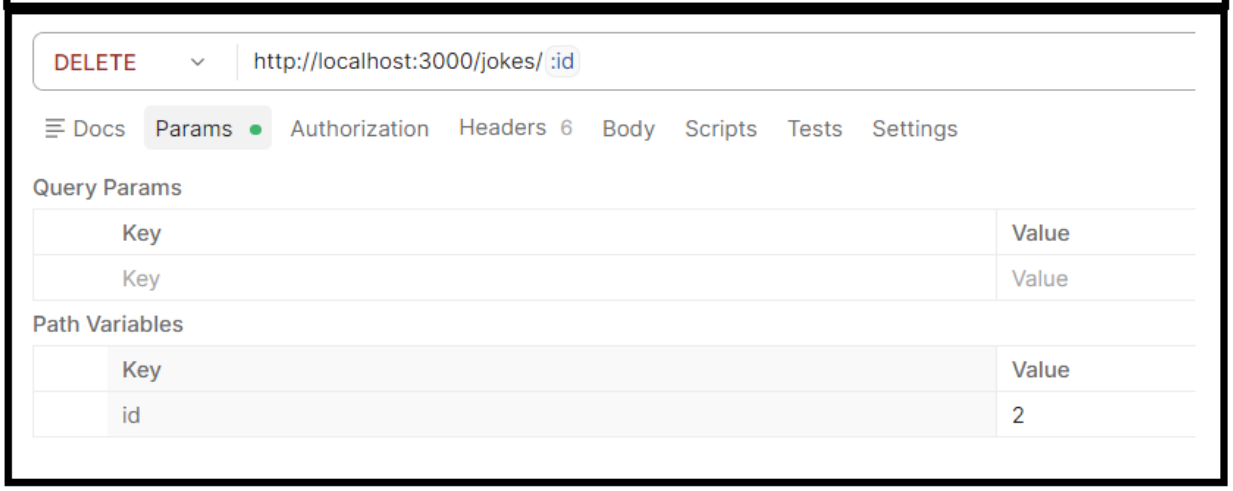
POST

```
// Post a new joke
app.post("/jokes", (req, res) => {
  const newJoke = {
    id: jokes.length + 1,
    jokeText: req.body.text,
    jokeType: req.body.type,
  };
  jokes.push(newJoke);
  console.log(jokes.slice(-1));
  res.json(newJoke);
});
```



DELETE Method

```
app.delete("/jokes/:id", (req, res) => {
  const id = parseInt(req.params.id);
  const searchIndex = jokes.findIndex((joke) => joke.id === id);// when ID exists >-1
  if (searchIndex > -1) {
    jokes.splice(searchIndex, 1); // delete only one item
    res.sendStatus(200);
  } else {
    res
      .status(404)
      .json({ error: `Joke with id: ${id} not found. No jokes were deleted.` });
  }
});
```



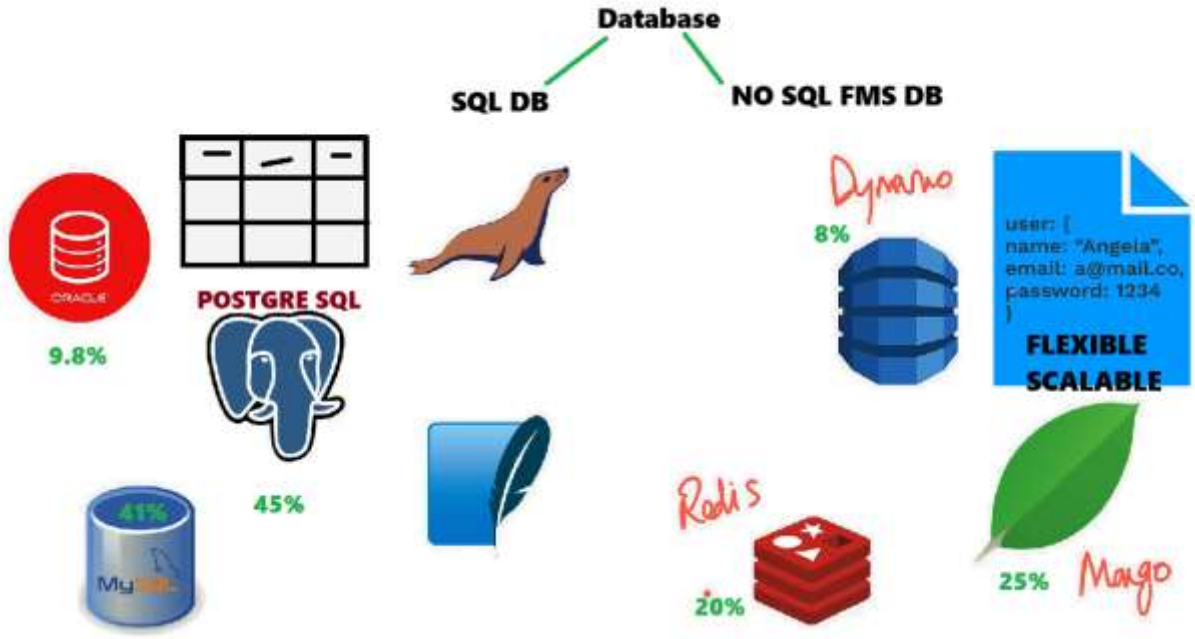
Blog API PROJECT

Database SQL

- A **database is** a well organised **collection** of **data** that is stored in an electronic format.
- Structure Query Language is a database language in the form of tables
- **SQL is standard language for storing , manipulating and retrieving data from Database**
- SQL is not key sensitive Language. E.g. SELECT as select as Select
- Semicolon Is The Standard Way To Separate Each SQL Statement In Database Systems. Ex. select*from Employee;
- SQL Became A Standard Of The American National Standard Institute (ANSI) In 1986 & Of The International Organization For Standardization (ISO) IN 1987
- SQL is language and MySQL is a platform to write query. Like Eclipes for JAVA.

➢ **Type of Data Management System**

1. FMS – File Management System (File , JSON data key value fairs)
2. DBMS – Database Management System (Table)
3. RDBMS – Relational Data Base Management System



Data type in SQL

Text is used for Combination of Number and Text
Char if lenth is less than 15 we use Char other than that we use Varchar

- To Store Data in the form of text

- To Store Data in the Numbers
- To store date and time

Types of SQL Command

GRANT –

- This Command Gives Users Access Privileges To The Database .

REVOKE –

- This Command Withdraws The User's Access Privileges Given By Using The GRANT Command .

- TCL Commands Are Used For Maintaining Consistency Of The Database .
- It Is Used For Management Of Transactions Made By The DML Commands.

COMMIT; —

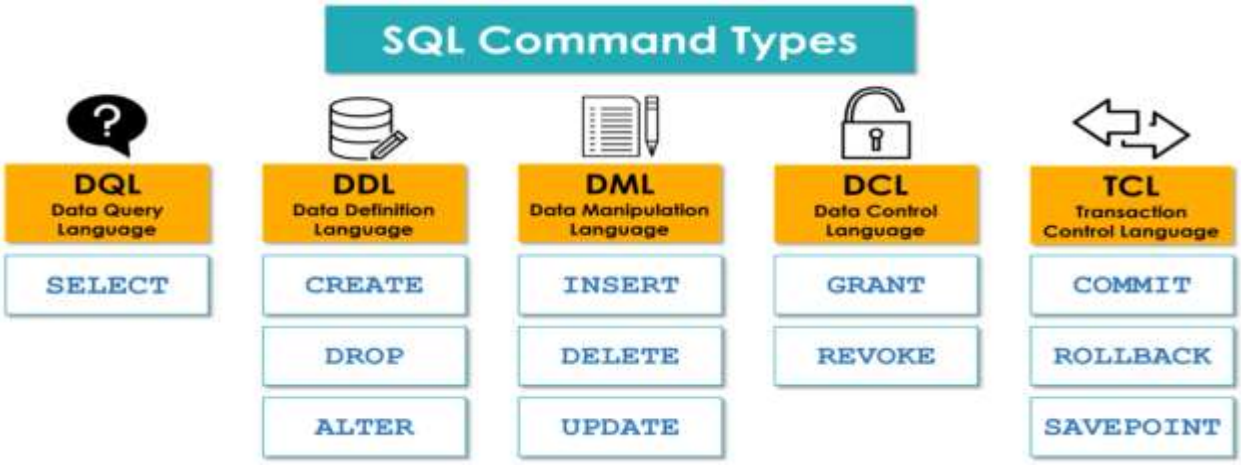
- This Command Is Used To Save The Data On Permanent Basis.

ROLLBACK; —

- This Command Is Used To Get The Data Or Restore The Data.
- This Command Generally Used With The DMLCommands .

SAVEPOINT; —

- This Command Is Used To Save The Data On Temporary Basis.



Create, Read, Update, Destroy (CRUD)
https://sqliteonline.com/#fiddle-5bbdbaef7288bo2ajn2wly03

```sql
Create Database EMP;
use EMP;
Create Table Ahmad(id int, Name varchar(20), Salary int(20));
insert into Ahmad values (1, 'khan', 120);
select * from Ahmad;
```

Create, Insert

➢ DDL – Data Definition Language

- These Commands Are Mostly Used By The DBA (Database Analyst).
- These Commands are generally is **used to create Database , Schema , Table**
- **Change** any pre-existing **data**
- And to **Remove Entire Database**

```sql
-- DDL Commmands Create, Alter, Drop, Truncate
-- to create database
CREATE DATABASE IT;
USE IT;

/* to create structure for table */
CREATE TABLE OCT(ID VARCHAR(20), Name VARCHAR(20), Class VARCHAR (20), Marks INT(40));
/* DESC means to see table structure */
DESC OCT;

/* to insert data into Students table */
INSERT INTO OCT VALUES('A10', 'POPAL', 'TESTING', 90);
INSERT INTO OCT VALUES('A11', 'NAZEER', 'TESTING', 100);
INSERT INTO OCT VALUES('A12', 'ALI', 'DEV', 95);
INSERT INTO OCT(ID, Name, Class, Marks) VALUES ('A14','JAN','DEV',99);

/* Alter : to bring changes Table Structure: ADD, Modify, Rename, Drop */
ALTER TABLE OCT ADD COLUMN Dep VARCHAR (20);
ALTER TABLE OCT MODIFY COLUMN Dep CHAR(10);
ALTER TABLE OCT RENAME COLUMN Dep TO LEVEL;
ALTER TABLE OCT DROP COLUMN LEVEL;

/* Truncate: delete data from table */
TRUNCATE TABLE OCT;
/* Drop: delete all data and structure of table */
Drop Table Oct;
Drop Database IT;
```

```sql
/* (DQL) to Fetch all data from table */
SELECT * FROM OCT;
SELECT Name, Marks FROM OCT;

/*  to Fetch all unique data from table */
select distinct * from OCT;
select distinct class, Name from OCT;
/* (DML INSERT, UPDATE, DELETE) */
Select * from OCT limit 2;

/* (AGGREGATE FUNCTIONS: MAX, MIN, SUM, COUNT, AVG) used with select commands */
Select max(Marks) from OCT;
Select sum(Marks) from OCT;
Select count(Marks) from OCT;
Select count(*) from OCT;

/* (OPERATORS ) Arithmetic + - * / Comparison =, !=, <>,><,!<,!>,=>
Logical Where, AND (2 TRUE CONDS), OR(1 TURE 1 FALSE) , NOT (NOT TURE COND), IN, BETWEEN, LIKE */

Select  Name, Marks+100 from OCT where Class='Testing';
Select * from OCT where Name = 'Popal';
-- not equal to Popal other than Popal
Select ID, Name from OCT where Name != 'Popal';
Select Name, Marks from OCT where Marks >90;
Select * from OCT where Name = 'Popal' and marks=90;
SELECT * FROM OCT WHERE Name = 'POPAL' OR Class = 'Testing';

Select * From OCT Where NOT Name = 'POPAL';
-- Use to Select Multiple records
Select * FROM OCT Where Marks IN(90,100,95);
Select Name, Marks From OCT Where ID IN('A10', 'A11', 'A12');
-- Use to Find the Requested Range
Select * From OCT Where ID BETWEEN 'A10' And 'A12';
-- Like to used to Search a name or any word starts with p, then starts with A and Ends with I
Select * From OCT Where Name Like 'P%';
Select * From OCT Where Name Like 'A%i';
Select * From OCT Where Name Like '%N%';
UPDATE OCT Set Marks = 100 Where Name = 'POPAL';

-- Group By And Having (having is used to filter out Group by)
Create Table AllCode(id int(10), Name varchar(20), DepartementNumber int(10), Salary int(45));
desc AllCode;
Insert Into AllCode Values(1, 'Ali', 20, 1000);
Insert Into AllCode Values(2, 'khan', 30, 500);
Insert Into AllCode Values(3, 'Jan', 20, 1000);
Insert Into AllCode Values(4, 'Popal', 30, 1200);
```

Edit Menu – Preprences – SQL Editor – uncheck safe mode.
Delete from OCT where Name = ‘Ali’;

```sql
-- Group By And Having (having is used to filter out Group by)
Create Table AllCode(Id int(10), Name varchar(20), DepartementNumber int(10), Salary int(65));
desc AllCode;
Insert Into AllCode Values(1, 'Ali', 20, 1000);
Insert Into AllCode Values(2, 'Khan', 30, 500);
Insert Into AllCode Values(3, 'Jan', 20, 1000);
Insert Into AllCode Values(4, 'Popal', 30, 1200);

-- Group By with Aggregate functions used to group similar items
Select DepartementNumber, sum(Salary) from AllCode Group By departementNumber;
Select Name, max(Salary) from AllCode Group By Name;
Select Name, sum(Salary) from AllCode Group By Name HAVING Avg(Salary)>500;
Select DepartementNumber, sum(Salary) from AllCode Group By DepartementNumber HAVING Avg(Salary)>100;

Select * From AllCode;

-- Order By (It is Used To Arrange The Data In Ascending And Descending Order) ASC/DESC

Select * FROM AllCode Order by Name;
Select * FROM AllCode Order by Name Desc;

-- Consttrains :SQL Constraints Are Used To Specify Rules For The Data In A Table
Create Table Cons(id int Primary Key , Name varchar(20) Not Null, State Varchar (20) Default 'NA', Salary int(45), Age int(20) Check(age>18));
desc Cons;
```

Create Read Update Destroy

```sql
CREATE TABLE salary (name text, position text, salary Money);
INSERT INTO salary VALUES ('baba', 'teacher', 2500);
INSERT INTO salary VALUES ('mama', 'casher', 300);
INSERT INTO salary VALUES ('lala', 'lawyer', 1000);
INSERT INTO salary VALUES ('cocacola', 'offficer', 50000);
```

```sql
-- READ Command
SELECT * FROM salary WHERE id = 12;
SELECT name , postion FROM salary WHERE postion = 'Eng'
```

```sql
-- UPDATE Command
UPDATE salary SET postion = 'Business Man' WHERE name = 'gul';
```

```sql
ALTER TABLE salary
ADD COLUMN phone;
DELETE FROM salary WHERE id = 1;
```

Constrains

1. Not Null –

- It Ensure Column Can Not Accept Null Value .

2. Unique –

- It Ensure Column Should Contains The Unique Data.

3. Primary Key –

- It Is A Combination Of (Unique + Not Null). It Used For Join Establishment.

4. Foreign Key –

- It Is Used For Establishment Of Join And It Also Used For To Maintain Relational Integrity.

5. Check –

- It Used To Apply Specific Value To The Specific Column To Ensure The Data Accuracy.

6. Default –

- It Sets The Default Value For The Entire Column / Field.

Union and Union All

- These Commands Are Use To Join Two Result Table.
- **SELECT * from Facebook UNION SELECT * from LinkedIn;**

Name Address Name Address = Union Name Address

Joins

- It Is Used To Join Two Or More Tables.
- It Used By the Backend Developers.
- During The Integration Testing Join Command Is Used.

Inner Join

- Pickup those values from both tables which has corresponding value.

```
109 •   Select A.Name, DepartmentID, B.Location, Contact From A
110     Inner Join B
111     On A.ID= B.ID;
112
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

Name	DepartmentID	Location	Contact
Ahmad	1	NC	757345884
Gul	1	VA	757345884
Farid	1	CA	757345884
Ali	1	GA	757345884

Right Join

- Returns All The Records From The Right Table And Matched Record From Left Table.
- The Right Table will be the reference.

```
93      /* Righ Join is used to join two or more tables  Right Table will be reference*/
94      Create Table A (ID int Primary Key, Name Varchar (20), DepartmentID int);
95      Insert INTO A Values (1, 'Ahmad', 1);
96      Insert INTO A Values (2, 'Gul', 1);
97      Insert INTO A Values (3, 'Farid', 1);
98      Insert INTO A Values (4, 'Ali', 1);
99      Insert INTO A Values (5, 'Khan', 1);
100     Create Table B (ID int, Location Varchar(20), Contact int);
101     Insert INTO B Values (1, 'NC', 757345884);
102     Insert INTO B Values (2, 'VA', 757345884);
103     Insert INTO B Values (3, 'CA', 757345884);
104     Insert INTO B Values (4, 'GA', 757345884);
105
106     Select A.Name, B.Location from A
107     Right Join B
108     On A.ID=B.ID;
109     Select A.Name, DepartmentID, B.Location, Contact From A
110     Right Join B
111     On A.ID= B.ID;
112
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

Name	DepartmentID	Location	Contact
Ahmad	1	NC	757345884
Gul	1	VA	757345884
Farid	1	CA	757345884
Ali	1	GA	757345884

Left Join

- Returns All The Records From The Left Table And Matched Record From Right Table.
- In Left Join the Left Table is the reference.

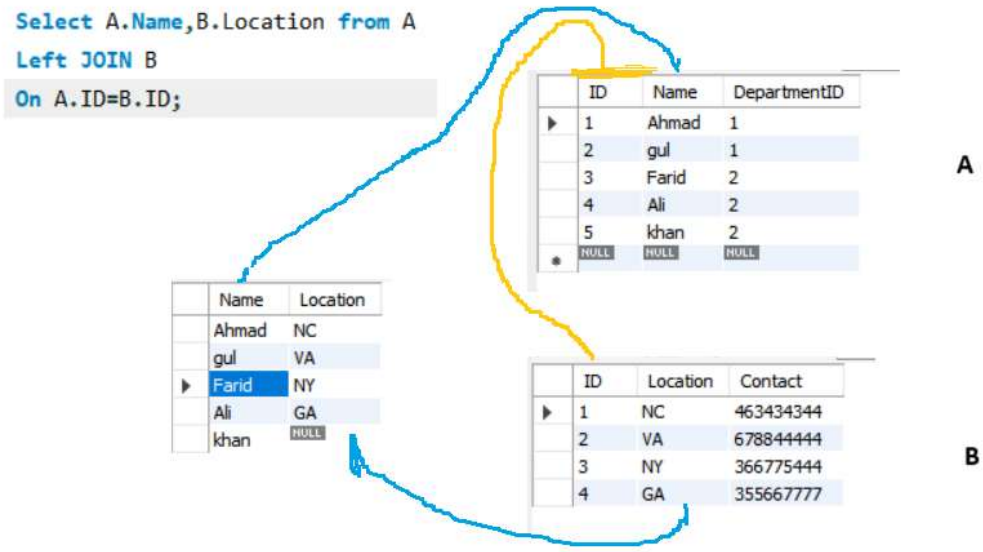
```sql
/* Left Join is used to join two or more tables */
Create Table A (ID int Primary Key, Name Varchar (20), DepartmentID int);
Insert INTO A Values (1, 'Ahmad', 1);
Insert INTO A Values (2, 'Gul', 1);
Insert INTO A Values (3, 'Farid', 1);
Insert INTO A Values (4, 'Ali', 1);
Insert INTO A Values (5, 'Khan', 1);
Create Table B (ID int, Location Varchar(20), Contact int);
Insert INTO B Values (1, 'NC', 757345884);
Insert INTO B Values (2, 'VA', 757345884);
Insert INTO B Values (3, 'CA', 757345884);
Insert INTO B Values (4, 'GA', 757345884);

Select A.Name, B.Location from A
Left Join B
On A.ID=B.ID;
Select A.Name, DepartmentID, B.Location, Contact From A
Left Join B
On A.ID= B.ID;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

Name	DepartmentID	Location	Contact
Ahmad	1	NC	757345884
Gul	1	VA	757345884
Farid	1	CA	757345884
Ali	1	GA	757345884
Khan	1	NULL	NULL

```sql
Select A.Name, B.Location from A Right Join B
On A.ID=B.ID;
Select A.Name, DepartmentID, B.Location, Contact From A Inner Join B
On A.ID= B.ID;
Select A.Name, B.Location From A Left Join B
On A.ID=B.ID;
```

```sql
use It;
-- to create a copy of Oct Table with name of Nov (Structure)
create table Nov like oct;
select * from Nov;
-- to copy data from oct table to Nov table
Insert into Nov Select * from oct;
```




```
1 SELECT A.Name, Position, B.Location, Contact
2 FROM A INNER JOIN B
3 ON A.ID=B.ID;
```


```
SELECT orders.order_number, customers.first_name, customers.last_name, customers.address
FROM orders
INNER JOIN customers ON orders.customer_id = customers.id
```

PostgresSQL:

The world's most open source relational database.

1. Windows Users: Download the Postgres Installer here:
https://sbp.enterprisedb.com/getfile.jsp?fileid=1258649

```
2     -- Union--
3     create table A(Name varchar(20), Address varchar(20));
4     create table B(Name varchar(20), Address varchar(20));
5     insert into A values('Ahmad','2542 Sherwood');
6     insert into A values('Khan','2542 Sherwood');
7     insert into A values('Jan','2542 Sherwood');
8     insert into A values('Ahmad','2542 Sherwood');
9     select * from A;
10    insert into B values('Ahmad','2542 Sherwood');
11    insert into B values('Gul','2542 Sherwood');
12    insert into B values('Baba','2542 Sherwood');
13    insert into B values('Ahmad','2542 Sherwood');
14    select * from A union select * from B;
15
16    -- joins --
17    select * from pp;
18    select * from fusion;
19    Select fusion.Modle,YR, Milage, PP.price
20    from fusion inner join pp
21    on fusion.ID=pp.ID;
22
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

Modle	YR	Milage	price
Focus	2016	15005	25000
Fusion	2023	2000	15000

POSTGRESQL Database

The world's most open source relational database.
1. Windows Users: Download the Postgres Installer here:
https://sbp.enterprisedb.com/getfile.jsp?fileid=1258649

Create Data : creating data in PostgreSQL Database

pgAdmin4 : which runs our SQL Code such as MySQL
Right Click on Database then Create Database
Click on Query

```
Create Table capitals (
    id serial primary Key,
    country varchar(45),
    capital varchar(45)
);
```

Right Click on the capitals table – click import – select our CSV (coma separated valued) excel file – trun on headers option – click ok

Read Data: reading data in a PostgreSQL Database.

```
import express from "express";
import bodyParser from "body-parser";
import pg from "pg";

const app = express();
const port = 3000;

const db = new pg.Client({
  user: "postgres",
  host: "localhost",
  database: "moon",
  password: "12345",
  port: 5432,
});
db.connect();

let quiz = [];
db.query("SELECT * FROM capitals", (err, res) => {
  if (err) {
    console.error("Error executing query", err.stack);
  } else {
    quiz = res.rows;
  }
  db.end();
});
```

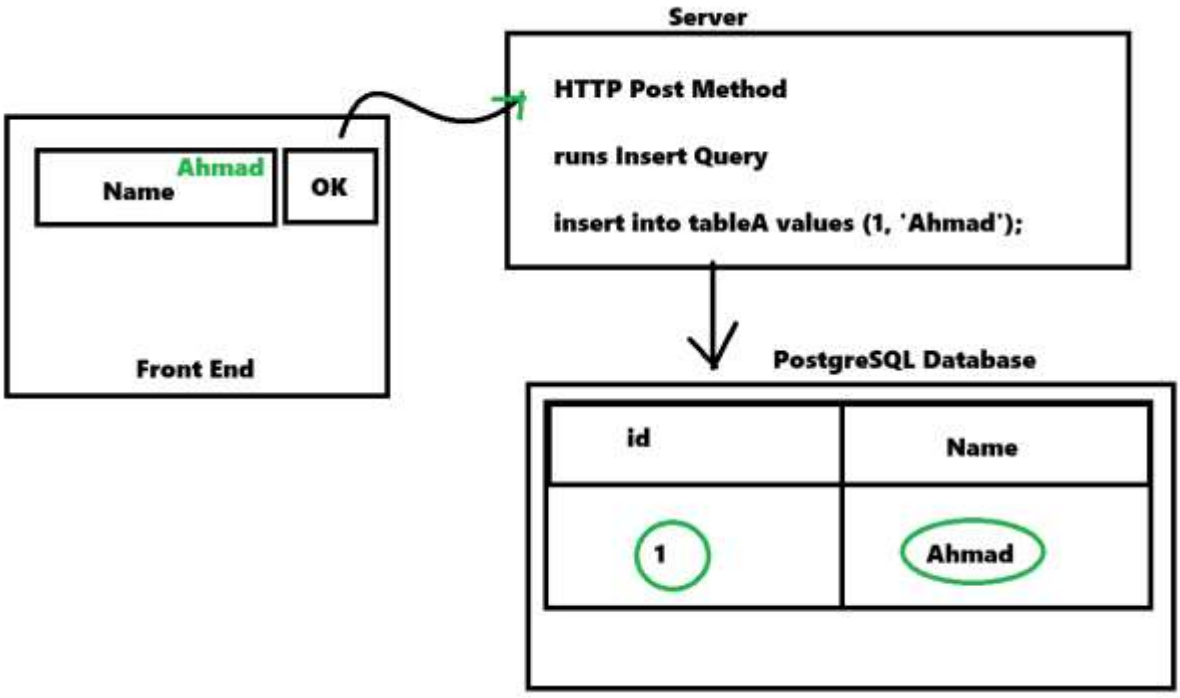
```
select rice_production from world_food where country = 'United States';
```

LIKE % is used for search

```
select * from world_food where country like 'U%'
```

```
INSERT INTO <TABLE> (<COLUMN1>, <COLUMN1>)
VALUES (<VALUE1>, <VALUE2>)
```

```
insert into world_food (country, rice_production, wheat_production) values('Netherlands', 12,12);
```



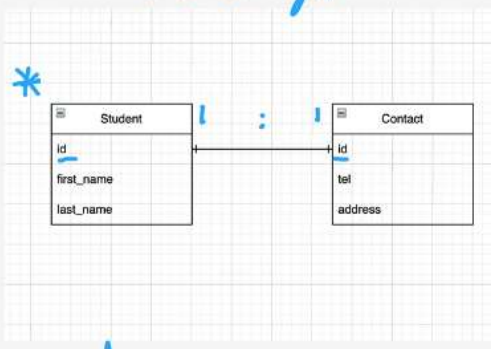

```
// GET home page
app.get("/", async (req, res) => {
  const countries = await checkVisisted();
  res.render("index.ejs", { countries: countries, total: countries.length });
});

//INSERT new country
app.post("/add", async (req, res) => {
  const input = req.body["country"];

  const result = await db.query(
    "SELECT country_code FROM countries WHERE country_name = $1",
    [input]
  );

  if (result.rows.length !== 0) {
    const data = result.rows[0];
    const countryCode = data.country_code;

    await db.query("INSERT INTO visited_countries (country_code) VALUES ($1)", [
      countryCode,
    ]);
    res.redirect("/");
  }
});
```

Relationship of Tables

One to One

```
CREATE TABLE student (
  id SERIAL PRIMARY KEY,
  first_name TEXT,
  last_name TEXT
);

CREATE TABLE contact_detail (
  id INTEGER REFERENCES student(id) UNIQUE,
  tel TEXT,
  address TEXT
);
```

DB Design



draw.io

```
CREATE TABLE student (
 id SERIAL PRIMARY KEY,
  first_name TEXT,
  last_name TEXT
);
```

```
CREATE TABLE contact_detail (
  id INTEGER REFERENCES student(id) UNIQUE,
  tel TEXT,
  address TEXT
);
```

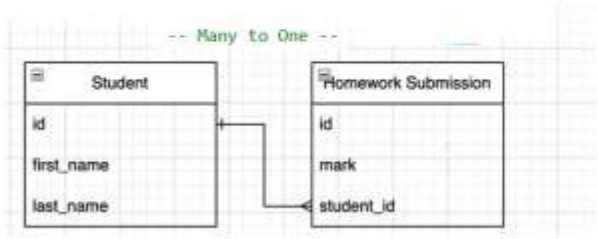
```
-- Data --
INSERT INTO student (first_name, last_name)
VALUES ('Angela', 'Yu');
INSERT INTO contact_detail (id, tel, address)
VALUES (1, '+123456789', '123 App Brewery Road');
```

One To One

```
-- Join --
SELECT * FROM student JOIN contact_detail
ON student.id = contact_detail.id
```

id integer	first_name text	last_name text	id integer	tel text	address text
1	Angela	Yu	1	+123456789	123 App Brewery Road

Many to One




```
-- Data --
INSERT INTO homework_submission (mark, student_id)
VALUES (98, 1), (87, 1), (88, 1)

-- Join --
SELECT * FROM student JOIN homework_submission
ON student.id = student_id
```

```
Create Table student (
    id SERIAL PRIMARY KEY,
    first_name TEXT,
    last_name TEXT);
```

```
CREATE TABLE homework_submission (
  id SERIAL PRIMARY KEY,
  mark INTEGER,
  student_id INTEGER REFERENCES student(id)
);
```

	id integer	first_name text	last_name text	id integer	mark integer	student_id integer
1	1	Angela	Yu	1	98	1
2	1	Angela	Yu	2	87	1
3	1	Angela	Yu	3	88	1

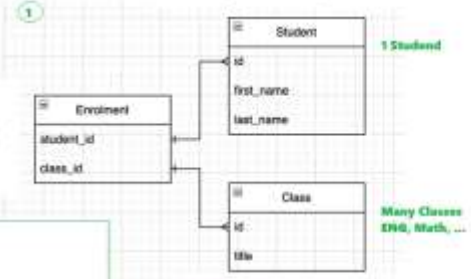
One
Customer

Many
Order
Order2
Order3

Many to Many

```
-- Join --
SELECT * FROM enrollment
JOIN student ON student.id = enrollment.student_id
JOIN class ON class.id = enrollment.class_id;
```

```
Create Table student (
    id SERIAL PRIMARY KEY,
    first_name TEXT,
    last_name TEXT);
```




```
CREATE TABLE enrollment (
  student_id INTEGER REFERENCES student(id),
  class_id INTEGER REFERENCES class(id),
  PRIMARY KEY (student_id, class_id)
);
```

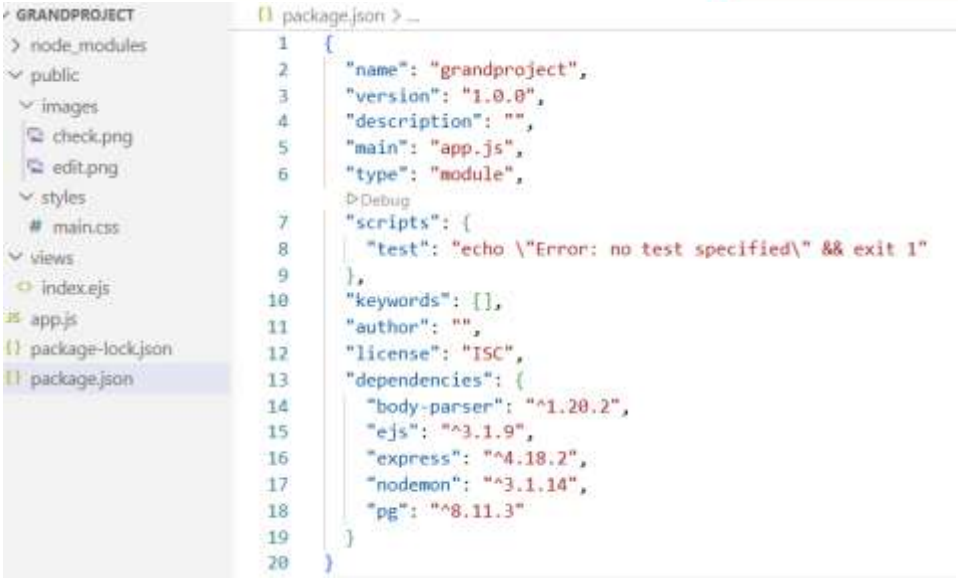
```
CREATE TABLE class (
  id SERIAL PRIMARY KEY,
  title VARCHAR(45)
)
```

```
-- Data --
INSERT INTO student (first_name, last_name)
VALUES ('Jack', 'Bauer');

INSERT INTO class (title)
VALUES ('English Literature'), ('Maths'), ('Physics');

INSERT INTO enrollment (student_id, class_id ) VALUES (1, 1), (1, 2);
INSERT INTO enrollment (student_id ,class_id) VALUES (2, 2), (2, 3);
```

```sql
CREATE TABLE house (id Serial Primary Key, name Text, salary Money);
INSERT INTO house (name, salary) values ('Husna', 120),('Asma',300),('Moh Idress', 10),('Ershad',20);
SELECT * FROM house ORDER BY id ASC;
SELECT * FROM house ORDER BY id DESC;
SELECT * FROM house WHERE salary < '100';
Alter Table family RENAME to familySalary;
Alter Table familySalary Add column state text;
Alter Table familySalary Rename column state to address;
Drop table ronaldo;
Delete from familysalary where id = 4;
UPDATE familysalary SET salary = 100 where id = 3;
```

Update TABLENAME SET COLUMNAME = VALUE WHERE CONDITION

```sql
UPDATE users
    SET name = 'Angelina'
    WHERE id = 1
```



EJS FILE

```ejs
<!DOCTYPE html>
<html lang="en">

<head>
  <meta charset="UTF-8" />
  <meta http-equiv="X-UA-Compatible" content="IE=edge" />
  <meta name="viewport" content="width=device-width, initial-scale=1.0" />
  <link rel="stylesheet" href="styles/main.css" />  <!--links css file -->
  <title>Permalist</title>
</head>

<body>
  <main>
  <div class="box" id="heading"> <h1> <%= listTitle %> </h1></div> <!--bring today's variable from app.js -->

  <div class="box">
    <% for(let item of listItems){%> <!-- Read our item table data from app (database) -->

      <div class="item">
        <form action="/delete" method="post">
          <input type="checkbox" onchange="this.form.submit()" name="deleteItemId" value="<%= item.id %>">
        </form>

        <p id="title<%=item.id%>"> <%= item.title %></p> <!--shows all items in this phara from loop -->

        <form class="edit" action="/edit" method="post">
          <input type="hidden" name="updatedItemId" value="<%= item.id %>">
          <input id="input<%=item.id%>" type="text" name="updatedItemTitle" value="<%= item.title %>" autocomplete="off" autofocus="true" hidden="true" />
          <button id="done<%=item.id%>" class="edit" type="submit" hidden><img class="icon" src="/assets/icons/check-solid.svg" alt="tick image"></button>
        </form>
  <button id="edit<%=item.id%>" class="edit" onclick="handler('<%=item.id%>')"><img class="icon" src="/assets/icons/pencil-solid.svg" alt="pencil image"></button>
      </div>
      <% } %>

      <form class="item" action="/add" method="post">
        <input type="text" name="newItem" placeholder="New Item" autocomplete="off" autofocus="true" />
        <button class="add" type="submit" name="list" value=<%=listTitle %> >+</button>
      </form>
  </div>
<script>
  function handler(id) {
    document.getElementById("title" + id).setAttribute("hidden", true)
    document.getElementById("edit" + id).setAttribute("hidden", true)
    document.getElementById("done" + id).removeAttribute("hidden")
    document.getElementById("input" + id).removeAttribute("hidden")
  }
</script>

</body>
<%- include('partials/footer.ejs'); -%>
</html>
```

APP.JS File

```
import express from "express";
import bodyParser from "body-parser";
import pg from "pg";

const app = express(); // creates server (app)
const port = 3000; // creates the port where server is runing

const db = new pg.Client({  // create db object to connect database
  user: "postgres",
  host: "localhost",
  database: "permalist",
  password: "12345",
  port: 5432,
});
db.connect();

app.use(bodyParser.urlencoded({ extended: true })); // read form data
app.use(express.static("public")); // connect our files to app in public folder

let items = [
  { id: 1, title: "Buy milk" },
  { id: 2, title: "Finish homework" },
]; // array that works as database

app.get("/", async (req, res) => {
  try {
    const result = await db.query("SELECT * FROM items ORDER BY id ASC");
    items = result.rows; // brings data from database save it in items

    res.render("index.ejs", { listTitle: "TODAY", listItems: items});
  } catch (err) {
    console.log(err);
  }
});
```

```
app.post("/add", async (req, res) => {
  const item = req.body.newItem;
  try {
    await db.query("INSERT INTO items (title) VALUES ($1)", [item]);
    res.redirect("/");
  } catch (err) {
    console.log(err);
  }
});

app.post("/edit", async (req, res) => {
  const item = req.body.updatedItemTitle;
  const id = req.body.updatedItemId;

  try {
    await db.query("UPDATE items SET title = ($1) WHERE id = $2", [item, id]);
    res.redirect("/");
  } catch (err) {
    console.log(err);
  }
});

app.post("/delete", async (req, res) => {
  const id = req.body.deleteItemId;
  try {
    await db.query("DELETE FROM items WHERE id = $1", [id]);
    res.redirect("/");
  } catch (err) {
    console.log(err);
  }
});

app.listen(port, () => {
  console.log(`Server running on port ${port}`);
});
```

PostgreSQL Database File

```
Create Database permalist;

CREATE TABLE items (
  id SERIAL PRIMARY KEY,
  title VARCHAR(100) NOT NULL
);

INSERT INTO items (title) VALUES ('Buy milk'), ('Finish homework');
```

Please do you own CSS DESIGN?

```
html {
  background-color: #e4e9fd;
```

```
  background-image: -webkit-linear-gradient(65deg, #a683e3 50%, #e4e9fd 50%);
  min-height: 1000px;
  font-family: "helvetica neue";
}
body {
  display: flex;
  flex-direction: column;
  justify-content: stretch;
  min-height: 95vh;
}
main {
  text-align: center;
  flex: 1 0 auto;
}
h1 {
  color: #fff;
  padding: 10px;
}

.box {
  max-width: 400px;
  margin: 50px auto;
  background: white;
  border-radius: 5px;
  box-shadow: 5px 5px 15px -5px rgba(0, 0, 0, 0.3);
}

#heading {
  background-color: #a683e3;
  text-align: center;
}
form.edit {
  display: flex;
}

.item {
  min-height: 70px;
  display: flex;
  align-items: center;
  border-bottom: 1px solid #f1f1f1;
}

.item:last-child {
  border-bottom: 0;
}
```

```
input[type="checkbox"] {
  margin: 20px;
  margin-left: 0;
}

p {
  margin: 0;
  padding: 20px 0;
  font-size: 20px;
  color: #00204a;
}

form {
  text-align: center;
  margin-left: 20px;
}

button.add {
  min-height: 50px;
  width: 50px;
  border-radius: 50%;
  border-color: transparent;
  background-color: #a683e3;
  color: #fff;
  font-size: 30px;
  border-width: 0;
}

button.edit {
  margin-left: 0px;
  margin-right: 20px;
  border: none;
  background: none;
}

.icon {
  height: 20px;
}

input[type="text"] {
  text-align: left;
  height: 60px;
  top: 10px;
  border: none;
```

```
  background: transparent;
  font-size: 20px;
  font-weight: 200;
  width: 80%;
}

input[type="text"]:focus {
  outline: none;
  box-shadow: inset 0 -3px 0 0 #a683e3;
}

::placeholder {
  color: lightgray;
  opacity: 1;
}

/* footer {
  flex: 0 0 auto;
  color: white;
  text-align: center;
}
 */
```

Authentication

User authentication: is used for the safety of website so that nobody else will use your data.
Email & Password

home.ejs

Auth > views > home.ejs > ? > ?

```
1   <%- include('partials/header') %>
2
3
4   <div class="jumbotron centered">
5     <div class="container">
6       <i class="fas fa-key fa-6x"></i>
7       <h1 class="display-3">Secrets</h1>
8       <p class="lead">Don't keep your secrets, share them anonymously!</p>
9       <hr>
10      <a class="btn btn-light btn-lg" href="/register" role="button">Register</a>
11      <a class="btn btn-dark btn-lg" href="/login" role="button">Login</a>
12
13    </div>
14  </div>
15
16  <%- include('partials/footer') %>
```

home.ejs login.ejs

Auth > views > login.ejs > ? > div.container.mt-5 > div.row > div.col-sm-8 > div.card > div.card-body

```ejs
<%- include('partials/header') %>

<div class="container mt-5">
  <h1>Login</h1>

  <div class="row">
    <div class="col-sm-8">
      <div class="card">
        <div class="card-body">

          <!-- Makes POST request to /login route -->
          <form action="/login" method="POST">
            <div class="form-group">
              <label for="email">Email</label>
              <input type="email" class="form-control" name="username">
            </div>
            <div class="form-group">
              <label for="password">Password</label>
              <input type="password" class="form-control" name="password">
            </div>
            <button type="submit" class="btn btn-dark">Login</button>
          </form>

        </div>
      </div>
    </div>
```

register.ejs

Auth > views > register.ejs > ? > div.container.mt-5 > div.row > div.col-sm-8 > div.card > div.card-body > form > div.form-g

```ejs
<%- include('partials/header') %>
  <div class="container mt-5">
    <h1>Register</h1>

    <div class="row">
      <div class="col-sm-8">
        <div class="card">
          <div class="card-body">

            <!-- Makes POST request to /register route -->
            <form action="/register" method="POST">
              <div class="form-group">
                <label for="email">Email</label>
                <input type="email" class="form-control" name="username">
              </div>
              <div class="form-group">
                <label for="password">Password</label>
                <input type="password" class="form-control" name="password">
              </div>
              <button type="submit" class="btn btn-dark">Register</button>
            </form>

          </div>
        </div>
      </div>
```

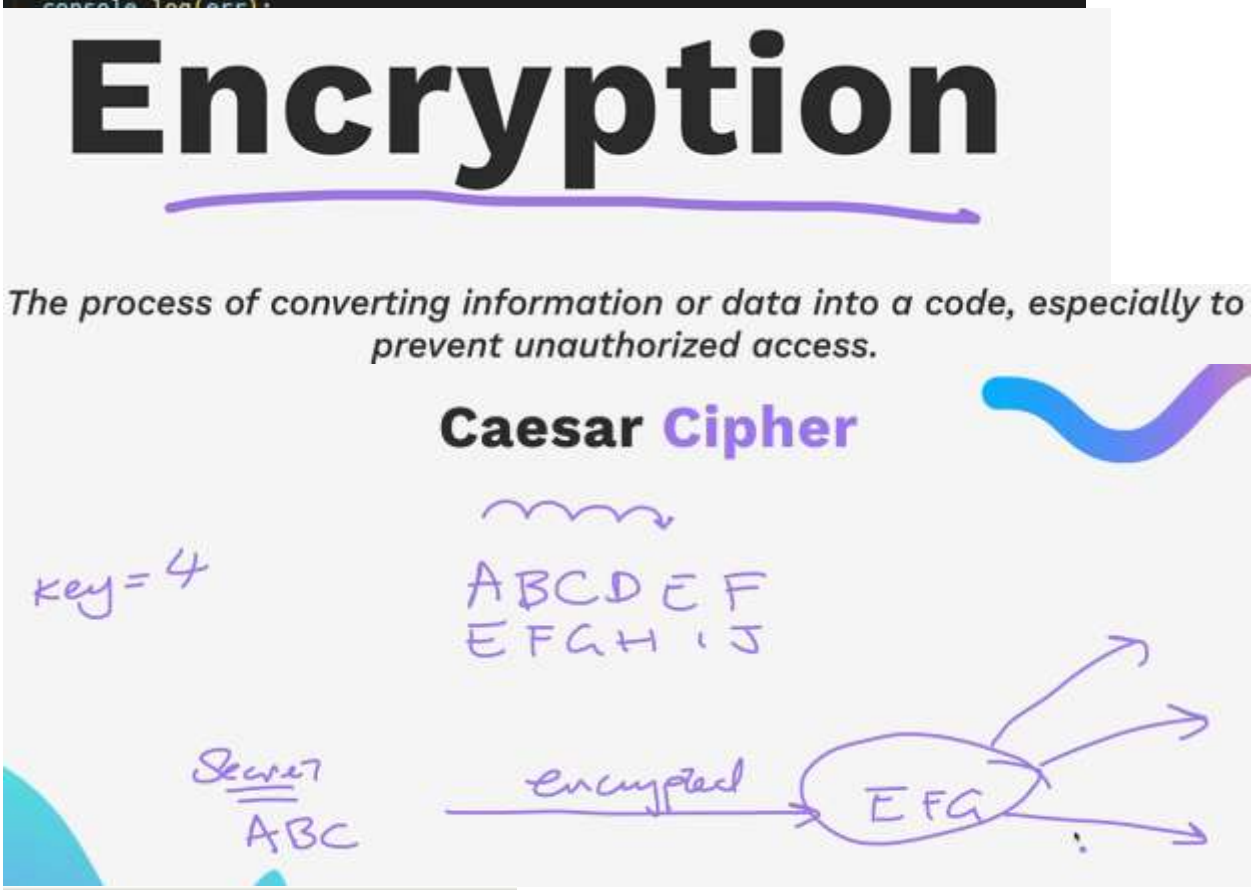
```
app.post("/login", async (req, res) => {
    const email = req.body.username; // Get email and password from request body (form data)
    const password = req.body.password;
 try{
  // Query database to check if a user with this email already exists
  const checkResult = await db.query("Select * from users where email = $1", [email]);

  if(checkResult.rows.length > 0){// If user exists (email found in database)
    // Send response indicating email already exists
    res.send("Email Already exists.");
  } else {    // If user does not exist, insert new user into database
      const result = await db.query("INSERT INTO users (email, password) VALUES ($1, $2) ", [email, password] );
 console.log(result);
      res.render("secrets.ejs")// Render secrets page after successful insertion
  }
  } catch (err){// Catch any errors and log them
   console.log(err);};
});
```

```
CREATE TABLE users(
id SERIAL PRIMARY KEY,
email VARCHAR(100) NOT NULL UNIQUE,
password VARCHAR(100)
)
```

```
app.post("/register", async (req, res) => {
    const email = req.body.username;
    const password = req.body.password;
 try{
  const checkResult = await db.query("Select * from users where email = $1", [email]);
  if(checkResult.rows.length>0){
    res.send("Email Already exists.");
  }else{
      const result = await db.query("INSERT INTO users (email, password) VALUES ($1, $2) ", [email, password]);
      console.log(result);
      res.render("secrets.ejs")}
  }catch (err){console.log(err);};
});

app.post("/login", async (req, res) => {
    const email = req.body.username;
    const password = req.body.password;
 try{
  const checkResult = await db.query("Select * from users where email = $1", [email]);
  if(checkResult.rows.length>0){
    res.render("secrets.ejs");
  }else{
      res.send("Please register")
      }
  }catch (err){console.log(err);};
});
```

Login page

```
try {
  const result = await db.query("SELECT * FROM users WHERE email = $1", [
    email,
  ]);
  if (result.rows.length > 0) {
    // console.log(result.rows);
    const user = result.rows[0];
    const storedPassword = user.password;

    if (password === storedPassword) {
      res.render("secrets.ejs");
    } else {
      res.send("Incorrect Password");
    }
  } else {
    res.send("User not found");
  }
} catch (err) {
```

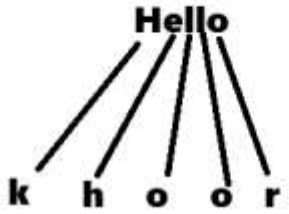


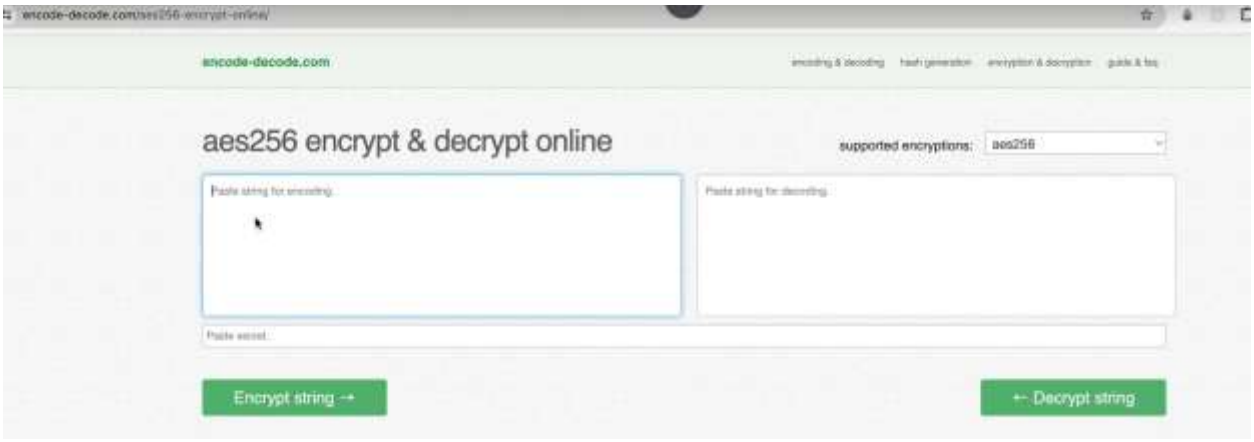
https://cryptii.com/pipes/caesar-cipher/

Key = 3

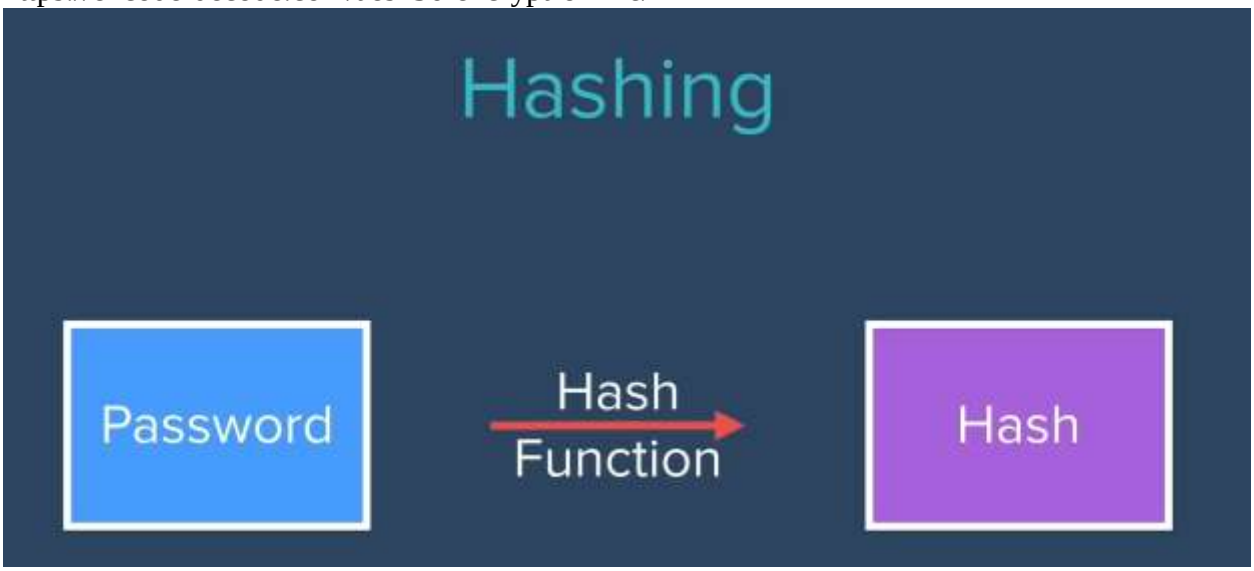
Plain text = "C" → Cipher text= "F"

a	b	c	d	e	f	g	h	i	j	k	l	m
0	1	2	3	4	5	6	7	8	9	10	11	12
n	o	p	q	r	s	t	u	v	w	x	y	z
13	14	15	16	17	18	19	20	21	22	23	24	25





https://encode-decode.com/aes256-encrypt-online/



http://password-checker.online-domain-tools.com/
21furqani#123

Encryption of Password Using Bcrypt (Hash)

Bcrypt is a dependency that encrypt your password
First add bcrypt dependency into package.json

```
"dependencies": {
  "bcrypt": "^5.1.1",
  "body-parser": "^1.20.2",
  "ejs": "^3.1.9",
  "express": "^4.18.2",
  "pg": "^8.11.3"
}
```

Second `import bcrypt from "bcrypt";`
3rd `const saltRounds = 10;`
4th

```
app.post("/register", async (req, res) => {
  const email = req.body.username;
  const password = req.body.password;

  try {
    const checkResult = await db.query("SELECT * FROM users WHERE email = $1", [
      email,
    ]);

    if (checkResult.rows.length > 0) {
      res.send("Email already exists. Try logging in.");
    } else {
      //hashing the password and saving it in the database
      bcrypt.hash(password, saltRounds, async (err, hash) => {
        if (err) {
          console.error("Error hashing password:", err);
        } else {
          console.log("Hashed Password:", hash);
          await db.query(
            "INSERT INTO users (email, password) VALUES ($1, $2)",
            [email, hash]
          );
          res.render("secrets.ejs");
        }
      });
    }
  } catch (err) {
    console.log(err);
  }
});
```

Cookies

Are used to save your browsing session DATA
SESSION IS TIME WHERE WE LOGIN TO A SERVER (LOGIN COOKIES) is used to save our information in the browser during the session, Session allow us to set up new session
Passport is a Node.js library that is used to add many authentication to our web site.

```
"dependencies": {
  "bcrypt": "^5.1.1",
  "body-parser": "^1.20.2",
  "dotenv": "^16.3.1",
  "ejs": "^3.1.9",
  "express": "^4.18.2",
  "express-session": "^1.17.3",
  "passport": "^0.7.0",
  "passport-local": "^1.0.0",
  "pg": "^8.11.3"
}
```

```
import passport from "passport";
import { Strategy } from "passport-local";
import session from "express-session";
import env from "dotenv";
```

First: import Express: session allow us to create a new session

```
env.config();

app.use(
  session({
    secret: "TOPSECRETWORD",
    resave: false,
    saveUninitialized: true,
  })
);
```

Second: use to sign the session cookie and save sessions into our server.

3rd: authenticate the session

```
app.use(passport.initialize());
app.use(passport.session());
```

also import passport

4th: we create an extra get route, this comes from passport that

```
app.get("/dashboard", async (req, res) => {
  if (req.isAuthenticated()) {
    try {
      const result = await db.query("SELECT * FROM items ORDER BY id ASC");
      res.render("index.ejs", { listTitle: "To Do List", listItems: result.rows, });
    } catch (err) { console.log(err);}
  } else { res.redirect("/login");}
});
```

5th import strategy at the very end before app.listen add passport. Use and copy the password login code from login route from try and catch paste it into passport.use

6th : Passport.serialize is used to save user data into the local storage.

```
// create locat strategy
passport.use(
  new Strategy(async function verify(username, password, cb) {
    try {
      const result = await db.query( "SELECT * FROM users WHERE email = $1", [username]);

      if (result.rows.length > 0) {
        const user = result.rows[0];
        const storedHash = user.password;

        bcrypt.compare(password, storedHash, (err, isMatch) => {
          if (err) {
            return cb(err);
          }

          if (isMatch) {
            return cb(null, user); // ✅ SUCCESS
          } else {
            return cb(null, false); // ❌ WRONG PASSWORD
          }
        });
      } else {
        return cb(null, false); // ❌ USER NOT FOUND
      }
    } catch (err) {
      return cb(err);
    }
  })
);
passport.serializeUser((user, cb)=>{
    cb(null, user);
});
passport.deserializeUser((user, cb)=>{
    cb(null, user);
});
```

7th: here we use passport as middleware, use local strategy,

```
app.post(
  "/login",
  passport.authenticate("local", {
    successRedirect: "/secrets",
    failureRedirect: "/login",
  })
);
```

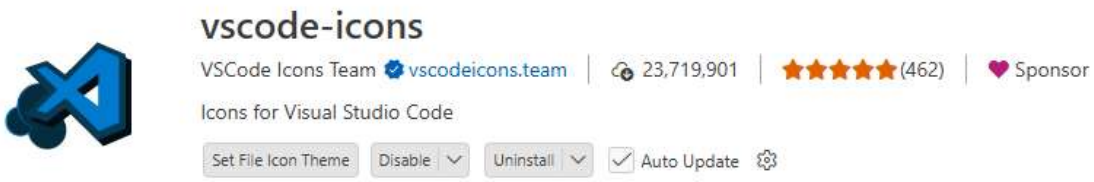
if success open secrets page if failure then open login

app.js > ...

```
import express from "express";
import bodyParser from "body-parser";
import pg from "pg";
import bcrypt from "bcrypt";
import session from "express-session";
import passport from "passport";
import { Strategy } from "passport-local";

const app = express(); // server created
const port = 3000; // port created
const saltRounds = 10;

const db = new pg.Client({ // db obj created
    user: "postgres",
    host: "localhost",
    database:"permalist",
    password: "12345",
    port: 5432,
});
db.connect(); // database connected
// create session sign cookie
app.use(
    session({
        secret: "toDoList",
        resave: false,
        saveUninitialized: true,
    })
);

app.use(bodyParser.urlencoded({extended: true})); // read ejs form data
app.use(express.static("public")); // interduce our files path to server

app.use(passport.initialize());
app.use(passport.session());

let items = [
    {id: 1, title: "Array to do list"},
    {id: 2, title: "Database is down"},
]; // array incase database do not work

app.get("/dashboard", async (req, res) => {
  if (req.isAuthenticated()) {
    try {
      const result = await db.query("SELECT * FROM items ORDER BY id ASC");
      res.render("index.ejs", { listTitle: "To Do List", listItems: result.rows, });
    } catch (err) { console.log(err);}
  } else { res.redirect("/login");}
});
```

```
// new code
app.get("/", async (req, res)=>{
    try{

        res.render("home.ejs");
    } catch (err){console.log(err);}
});
app.get("/register", async (req, res)=>{
    try{

        res.render("register.ejs");
    } catch (err){console.log(err);}
});
app.get("/login", async (req, res)=>{
    try{

        res.render("login.ejs");
    } catch (err){console.log(err);}
});

// adding bcrypt

app.post("/register", async (req, res) => {
    const email = req.body.username;
    const password = req.body.password;
 try{
  const checkResult = await db.query("Select * from users where email = $1", [email]);
  if(checkResult.rows.length>0){
    res.send("Email Already exists.");
  }else{
    // hashing the password
    bcrypt.hash(password, saltRounds, async(err, hash)=>{
        if(err){
            console.error("Error hashing password:", err)
        }
        else{
            console.log("hashing Password:", hash)
               const result = await db.query("INSERT INTO users (email, password) VALUES ($1, $2) ", [email, hash]);
                console.log(result);
                res.redirect("/dashboard");
        }
    })
    }
  }catch (err){console.log(err);};
});
app.post("/login",
    passport.authenticate("local",{
        successRedirect: "/dashboard",
        failureRedirect: "/login",
    })
);
// new code ended

app.post("/add", async (req, res)=>{
    const item = req.body.newItem;
    try{
        await db.query("INSERT INTO items (title) VALUES ($1)",[item]);
        res.redirect("/dashboard");
    } catch (err){console.log(err);}
});
app.post("/edit", async (req, res)=>{
    const item = req.body.updatedItemTitle;
    const id = req.body.updatedItemId;
    try{
        await db.query("UPDATE items SET title = $1 WHERE id = $2 ", [item, id]);
        res.redirect("/dashboard");
    } catch (err){console.log(err);}
});
app.post("/delete", async (req, res)=>{
    const id = req.body.deleteItemId;
    try{
        await db.query("DELETE FROM items WHERE id = $1 ", [id]);
        res.redirect("/dashboard");
    } catch (err){console.log(err);}
});
```

```
126  // create locat strategy
127  passport.use(
128    new Strategy(async function verify(username, password, cb) {
129      try {
130        const result = await db.query( "SELECT * FROM users WHERE email = $1", [username]);
131
132        if (result.rows.length > 0) {
133          const user = result.rows[0];
134          const storedHash = user.password;
135
136          bcrypt.compare(password, storedHash, (err, isMatch) => {
137            if (err) {return cb(err); }
138
139            if (isMatch) { return cb(null, user); // ✅ SUCCESS
140            } else { return cb(null, false); // ❌ WRONG PASSWORD }
141          });
142        } else { return cb(null, false); // ❌ USER NOT FOUND
143    }
144      } catch (err) {   return cb(err);  }
145    })
146  );
147  passport.serializeUser((user, cb)=>{
148      cb(null, user);
149  });
150  passport.deserializeUser((user, cb)=>{
151      cb(null, user);
152  });
153  app.listen(port, ()=>{
154      console.log(`Server is running on port ${port}`);
155  })
```



Environment Variables:

EV is used to separate our secret and key files from other project files>
For Convenience and Security
First: create a file with the name of .env
Second: copy the secret value “TOPSECRETWORD” and put the name that is SESSION_SECRET=“TOPSECRETWORD” in the .env file.
3rd: instead of secret value we put **process.env.SESSION_SECTRET,**

4th `import env from "dotenv";` `"dotenv": "^16.3.1",` and also add to package.JSON

5th initiate the env file like `env.config();` on top below the express server app
Whenever we upload our project to GitHub we put .env file in the .gitignore file
.gitignore
All code

```
# Logs
logs
*.log
npm-debug.log*
yarn-debug.log*
yarn-error.log*
lerna-debug.log*
```

```
# Diagnostic reports (https://nodejs.org/api/report.html)
report.[0-9]*.[0-9]*.[0-9]*.[0-9]*.json

# Runtime data
pids
*.pid
*.seed
*.pid.lock

# Directory for instrumented libs generated by jscoverage/JSCover
lib-cov

# Coverage directory used by tools like istanbul
coverage
*.lcov

# nyc test coverage
.nyc_output

# Grunt intermediate storage (https://gruntjs.com/creating-plugins#storing-task-files)
.grunt

# Bower dependency directory (https://bower.io/)
bower_components

# node-waf configuration
.lock-wscript

# Compiled binary addons (https://nodejs.org/api/addons.html)
build/Release

# Dependency directories
node_modules/
jspm_packages/

# Snowpack dependency directory (https://snowpack.dev/)
web_modules/

# TypeScript cache
*.tsbuildinfo

# Optional npm cache directory
.npm

# Optional eslint cache
.eslintcache

# Optional stylelint cache
.stylelintcache

# Optional REPL history
.node_repl_history

# Output of 'npm pack'
*.tgz

# Yarn Integrity file
.yarn-integrity

# dotenv environment variable files
.env
.env.*
!.env.example

# parcel-bundler cache (https://parceljs.org/)
.cache
.parcel-cache

# Next.js build output
.next
out

# Nuxt.js build / generate output
.nuxt
dist
.output

# Gatsby files
.cache/
# Comment in the public line in if your project uses Gatsby and not Next.js
# https://nextjs.org/blog/next-9-1#public-directory-support
# public

# vuepress build output
.vuepress/dist

# vuepress v2.x temp directory
.temp

# Sveltekit cache directory
.svelte-kit/
```

```
# vitepress build output
**/.vitepress/dist

# vitepress cache directory
**/.vitepress/cache

# Docusaurus cache and generated files
.docusaurus

# Serverless directories
.serverless/

# FuseBox cache
.fusebox/

# DynamoDB Local files
.dynamodb/

# Firebase cache directory
.firebase/

# TernJS port file
.tern-port

# Stores VSCode versions used for testing VSCode extensions
.vscode-test

# pnpm
.pnpm-store

# yarn v3
.pnp.*
.yarn/*
!.yarn/patches
!.yarn/plugins
!.yarn/releases
!.yarn/sdks
!.yarn/versions

# Vite files
vite.config.js.timestamp-*
vite.config.ts.timestamp-*
.vite/
```

8th we do the same process for our Database connection code

.env

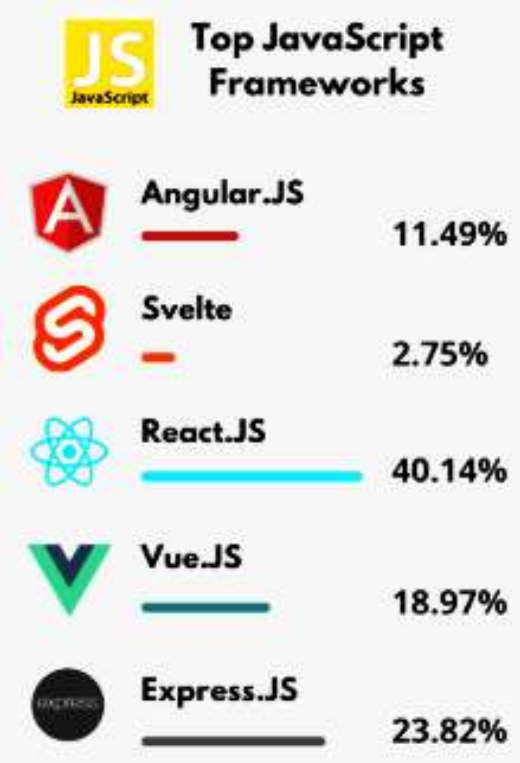
```
SESSION_SECRET="MyTOPtoDoList"
    PG_USER="postgres"
    PG_HOST="localhost"
    PG_DATABASE="permalist"
    PG_PASSWORD="12345"
    PG_PORT=5432
```

```
const db = new pg.Client({ // db obj created
    user: process.env.PG_USER,
    host: process.env.PG_HOST,
    database:process.env.PG_DATABASE,
    password: process.env.PG_PASSWORD,
    port: process.env.PG_PORT,
});
```

OAuth (Open Authentication) with the google and other companies

React.js

React is JavaScript library or framework used to build user interfaces.
https://www.fullstackpython.com/react.html



Sandbox is the online editor for the react. (https://codesandbox.io/)
We all our files there, and we can download our project to use it locally but for free we have only space for 10 projects.
JSX (JavaScript XML) is a syntax extension for JavaScript that lets you write code that *looks like HTML* inside JavaScript.

- Makes UI code easier to read and write
- Let's you mix HTML-like structure with JavaScript logic
- Commonly used in **React** for building components

Babel is a **JavaScript compiler (transpiler)**.

It converts modern JavaScript (like JSX or ES6+) into plain JavaScript that browsers can understand.

https://babeljs.io/

Sandbox

https://codesandbox.io/p/sandbox/introduction-to-jsx-h8ub1

In our Js file we do all about react.

```
var React = require("react");
var ReactDOM = require("react-dom");

ReactDOM.render(what to show, where to show it, CB function when this render is completed)
```

JSX (JavaScript XML) 

.

index.js

src > index.js

```
1  import React from "react";
2  import ReactDom from "react-dom";
3
4  ReactDom.render(<h1>Hello BABA</h1>, document.getElementById("root"));
5
```

https://babeljs.io/

the above link is the for the babel converter

```
import React from "react";
import ReactDom from "react-dom";

ReactDom.render(<h1>Hello BABA</h1>, document.getElementById("root"));

var h1 = document.createElement("h1");
h1.innerHTML = "Hello BABA";
document.getElementById("root").appendChild(h1);
```

Simple React

index.js

src > index.js

```
1  import React from "react";
2  import ReactDom from "react-dom";
3
4  ReactDom.render(<h1>Hello BABA</h1>, document.getElementById("root"));
5
```

c > index.js

```
1  import React from "react";
2  import ReactDom from "react-dom";
3
4  ReactDom.render(
5    <div>
6      <h1>Hello BABA</h1> <p>hahaaha</p>
7    </div>,
8    document.getElementById("root")
9  );
```

My Favourite Foods

- Bacon
- Noodles
- Jamon

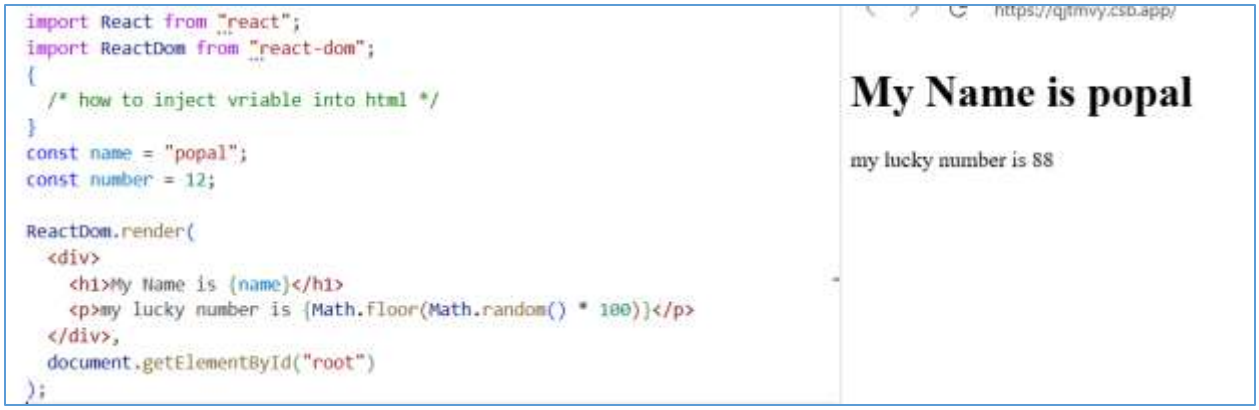
index.js > ...

```
import React from "react";
import ReactDom from "react-dom";
{/* how to inject vriable into html */}
const name = "popal";
const number = 12;

ReactDom.render(
  <div>
    <h1>My Name is {name}</h1>
    <p>my lucky number is {number}</p>
  </div>,
  document.getElementById("root")
);
```

https://qjtmvy.csb.app/

My Name is popal

my lucky number is 12



```
import React from "react";
import ReactDom from "react-dom";
{
  /* how to inject vriable into html */
}
const name = "popal";
const number = 12;

ReactDom.render(
  <div>
    <h1>My Name is {name}</h1>
    <p>my lucky number is {Math.floor(Math.random() * 100)}</p>
  </div>,
  document.getElementById("root")
);
```


```
import React from "react";
import ReactDom from "react-dom";
{
  /* how to inject vriable into html */
}
const name = "popal";
const date = new Date();
const year = date.getFullYear();

ReactDom.render(
  <div>
    <h1>Create by: {name}</h1>
    <p>copywrite {year}</p>
  </div>,
  document.getElementById("root")
);
```


Styling

Adding Style to React Code

HTML Global Attributes

Attribute	Description
accesskey	Specifies a shortcut key to activate/focus an element
class	Specifies one or more classnames for an element (refers to a class in a style sheet)
contenteditable	Specifies whether the content of an element is editable or not
data-*	Used to store custom data private to the page or application
dir	Specifies the text direction for the content in an element
draggable	Specifies whether an element is draggable or not
dropzone	Specifies whether the dragged data is copied, moved, or linked, when dropped
hidden	Specifies that an element is not yet, or is no longer, relevant
id	Specifies a unique id for an element
lang	Specifies the language of the element's content
spellcheck	Specifies whether the element is to have its spelling and grammar checked or not
style	Specifies an inline CSS style for an element
tabindex	Specifies the tabbing order of an element
title	Specifies extra information about an element
translate	Specifies whether the content of an element should be translated or not

```
import React from "react";
import ReactDom from "react-dom";

ReactDom.render(
  <div>
    <h1 className="blue">My Favorite Food {6 + 6}</h1>
    <ul>
      <li>Chicken</li>
      <li>Rice</li>
      <li>Salad</li>
    </ul>
  </div>,
  document.getElementById("root")
);
```

index.js > ...

```
import React from "react";
import ReactDom from "react-dom";

const img = "https://picsum.photos/200/300";
const img1 = "https://picsum.photos/200";
const img2 = "https://picsum.photos/300";

ReactDom.render(
  <div>
    <h1 className="blue">My Favorite Food {6 + 6}</h1>
    <img src={img} />
    <img src={img1} />
    <img src={img2} />
  </div>,
  document.getElementById("root")
);
```

Inline Style

index.js

```
import React from "react";
import ReactDom from "react-dom";

const customStyle = {
  color: "purple",
  fontSize: "50px",
  border: "2px solid purple",
  textAlign: "center",
};

ReactDom.render(
  <div>
    <h1 style={customStyle}>My Favorite Food</h1>
  </div>,
  document.getElementById("root")
);
```

```
index.js > ...
import React from "react";
import ReactDom from "react-dom";

const date = new Date();
const currentTime = date.getHours();

const customStyle = {
  color: "",
};

let greeting;

if (currentTime < 12) {
  greeting = "Good Morning";
  customStyle.color = "red";
} else if (currentTime < 18) {
  greeting = "Good Afternoon";
  customStyle.color = "green";
} else {
  greeting = "Good Night";
  customStyle.color = "blue";
}

ReactDom.render(
  <div>
    <h1 style={customStyle}> {greeting} Asma!</h1>
  </div>,
  document.getElementById("root")
);
```

https://vqwqsj.csb.app/

Good Afternoon Asma!

Using components

```
App.jsx
App.jsx > default
import React from "react";

function App() {
  const date = new Date();
  const currentTime = date.getHours();

  const customStyle = {
    color: "",
  };

  let greeting;

  if (currentTime < 12) {
    greeting = "Good Morning";
    customStyle.color = "red";
  } else if (currentTime < 18) {
    greeting = "Good Afternoon";
    customStyle.color = "green";
  } else {
    greeting = "Good Night";
    customStyle.color = "blue";
  }

  return (
    <div>
      <h1 style={customStyle}>{greeting} Husna !</h1>
    </div>
  );
}

export default App;
```

```
index.js
src > index.js
import React from "react";
import ReactDom from "react-dom";
import App from "./App";

ReactDom.render(<App />, document.getElementById("root"));
```

Preview

https://8zpmfh.csb.app/

Good Afternoon Husna !

Components

Are functions in a separate JSX file that we can import it into our js file.

If you want your code to be separated then follow the below code

index.js Heading.jsx

src > Heading.jsx > ...

```
1   import React from "react";
2
3   // components
4   function Heading() {
5     return (
6       <div>
7         <h1>My Fovrite Food</h1>
8         <ul>
9           <li></li>
10          <li></li>
11          <li></li>
12        </ul>
13      </div>
14    );
15  }
16
17  export default Heading;
```

Common method

Importing Multiple components

Or

Keeper App link
https://codesandbox.io/p/sandbox/keeper-app-part-1-starting-oplw1
Keeper App

```
//1. Create a new React app.
import React from "react";
import ReactDom from "react-dom";
//2. Create a App.jsx component.
//3. Create a Header.jsx component that renders a <header> element
//to show the Keeper App name in an <h1>.
```

App.jsx

src > components > App.jsx > ...

```jsx
import React from "react";
import Header from "./Header";
import Footer from "./Footer";
import Note from "./Note";

function App() {
  return (
    <div>
      <Header />
      <Note />
      <Footer />
    </div>
  );
}

export default App;
```

Header.jsx

src > components > Header.jsx > ...

```jsx
import React from "react";

function Header() {
  return (
    <header>
      <h1>Keeper</h1>
    </header>
  );
}

export default Header;
```

```
//4. Create a Footer.jsx component that renders a <footer> element
//to show a copyright message in a <p> with a dynamically updated year.
//5. Create a Note.jsx component to show a <div> element with a
```

Footer.jsx

src > components > Footer.jsx > ...

```jsx
import React from "react";

function Footer() {
  const currentYear = new Date().getFullYear();
  return (
    <footer>
      <p>Copyright © {currentYear}</p>
    </footer>
  );
}

export default Footer;
```

Note.jsx

src > components > Note.jsx > ...

```jsx
import React from "react";

function Note() {
  return (
    <div className="note">
      <h1>This is the note title</h1>
      <p>This is the note content</p>
    </div>
  );
}

Sign in to fork & edit this sandbox
export default Note;
```

```
//<h1> for a title and a <p> for the content.
//6. Make sure that the final website is styled like the example shown here:
//https://l1pp6.csb.app/

//HINT: You will need to study the classes in teh styles.css file to appy
styling.

// If you're running this locally in VS Code use the commands:
// npm install
// to install the node modules and
// npm run dev
// to launch your react project in your browser
```

React Prop

https://codesandbox.io/p/sandbox/introduction-to-jsx-h8ub1

```
import React from "react";
import ReactDOM from "react-dom";

function Card(props) {
  return (
    <div>
      <h2>{props.name}</h2>
      <img src={props.img} alt="avatar_img" />
      <p>{props.tel}</p>
      <p>{props.email}</p>
    </div>
  );
}

ReactDOM.render(
  <div>
    <h1>My Contacts</h1>
    <Card
      name="Beyonce"
      img="https://blackhistorywall.files.wordpress.com/2010/02/picture-device-independent-bitmap-119.jpg"
      tel="+123 456 789"
      email="b@beyonce.com"
    />
    <Card
      name="Jack Bauer"
      img="https://pbs.twimg.com/profile_images/625247595825246208/X3XLea04_400x400.jpg"
      tel="+7387384587"
      email="jack@nowhere.com"
    />
  </div>,
  document.getElementById("root")
);
```

Code for above example

Mapping the components

```
index.html
<!DOCTYPE html>
<html lang="en">
  <head>
    <title>React App</title>
    <link rel="stylesheet" href="styles.css" />
  </head>

  <body>
    <div id="root"></div>
    <script src="../src/index.js" type="text/jsx"></script>
  </body>
</html>
```

```
index.js
import React from "react";
import ReactDOM from "react-dom";
import App from "./components/App";
import "./../public/styles.css";

ReactDOM.render(<App />, document.getElementById("root"));
```

```
Detail.jsx
import React from "react";

function Detail(props) {
  return <p className="info">{props.detailInfo}</p>;
}

export default Detail;
```

```
App.jsx
import React from "react";
import Card from "./Card";
import contacts from "../contacts";

function createCard(contact) {
  return (
    <Card
      name={contact.name}
      img={contact.imgURL}
      tel={contact.phone}
      email={contact.email}
    />
  );
}

function App() {
  return (
    <div>
      <h1 className="heading">My Contacts</h1>

      {contacts.map(createCard)}

      {/* <Card      // repeated code
        name={contacts[0].name}
        img={contacts[0].imgURL}
        tel={contacts[0].phone}
        email={contacts[0].email}
      />
      <Card
        name={contacts[1].name}
        img={contacts[1].imgURL}
        tel={contacts[1].phone}
        email={contacts[1].email}
      />
      <Card
        name={contacts[2].name}
        img={contacts[2].imgURL}
        tel={contacts[2].phone}
        email={contacts[2].email}
      /> */}
    </div>
  );
}
export default App;
```

```jsx
import React from "react";
import Avatar from "./Avatar";
import Detail from "./Detail";

function Card(props) {
  return (
    <div className="card">
      <div className="top">
        <h2 className="name">{props.name}</h2>
        <Avatar img={props.img} />
      </div>
      <div className="bottom">
        <Detail detailInfo={props.tel} />
        <Detail detailInfo={props.email} />
      </div>
    </div>
  );
}

export default Card;
```

```jsx
import React from "react";

function Avatar(props) {
  return <img className="circle-img" src={props.img} alt="avatar_img" />;
}

export default Avatar;
```

```jsx
const contacts = [
  {
    id: 1,
    name: "Husna Jana",
    imgURL:
      "https://encrypted-tbn0.gstatic.com/images?q=tbn:
    phone: "+757565656",
    email: "gmail@husnajana.com",
  },
  {
    id: 2,
    name: "Asma",
    imgURL:
      "https://supertalls.com/cdn/shop/articles/360_F_
    phone: "+123 456 789",
    email: "h@beyonce.com",
  },
  {
    id: 3,
    name: "M Idress",
    imgURL:
      "
    phone: "+907 654 321",
    email: "jack@nowhere.com",
  },
  {
    id: 4,
    name: "Chuck Norris",
    imgURL:
      "https://i.pinimg.com/originals/e3/94/47/e39447d
    phone: "+918 372 574",
    email: "gmail@chucknorris.com",
  },
];

export default contacts;
```

Card.jsx Avatar.jsx contacts.js styles.css

public > # styles.css > ...

```css
body {
  font-family: sans-serif;
  margin: 0 auto;
  padding: 25px;
  max-width: 400px;
  min-width: 400px;
  background-color: #dfe6e9;
  text-align: center;
}

.heading {
  text-align: center;
  font-size: 2em;
  color: #2d3436;
  margin: 20px 0 30px;
}

.card {
  border-radius: 25px;
  position: relative;
  padding: 25px 15px;
  background-color: #81ecec;
  margin: 50px 0;
  height: 200px;
  box-shadow: 0 2px 5px #ccc;
  text-align: left;
}

.top {
  border-radius: 25px 25px 0 0;
  height: 100px;
  width: 100%;
  background-color: #00cec9;
  position: absolute;
  top: 0;
  left: 0;
  display: flex;
}
```

Card.jsx Avatar.jsx contacts.js styles.css

public > # styles.css > ...

```css
.name {
  font-size: 2em;
  color: #2d3436;
  display: flex;
  flex: 1;
  margin: 10% 20px 0;
}

.card img {
  margin: 30px 20px 0 0;
}

.circle-img {
  border-radius: 50%;
  border: 7px solid #fff;
  width: 120px;
  height: 120px;
}

.bottom {
  margin-top: 120px;
}

.info {
  margin: 20px 0;
  color: #1a7e76;
}
```

React DevTools

https://chromewebstore.google.com/detail/react-developer-tools/fmkadmapgofadopljbjfkapdkoienihi?hl=en

react dev tool for chrome

Emoji Dictionary

emojimeanings.net/list-smileys-people-whatsapp

Smileys and people emojis with their meaning

You'll find all current smileys and people emojis in Whatsapp and Facebook as well as a description of their meaning. Have fun with diving into the colorful world of emojis!

Category: Whatsapp Smileys & People

Speechless Face

Oh my goodness! The unpleasantly surprised face is lost for words due to a shocking affair. In response to bad behavior or a rude message. Nothing can be added to what has just been said.

U+1F62F

Dl dictionary list

Dt dictionary terms

Dd Dictionary details

Step 1 Make **Entry** Components
Step 2 create **Props** to replace hard coded data
Step 3 import the emojipedia const
Step 4 Do mapping

index.js

```
import React from "react";
import ReactDOM from "react-dom";
import App from "./components/App";
import "./../public/styles.css";

ReactDOM.render(<App />, document.getElementById("root"));
```

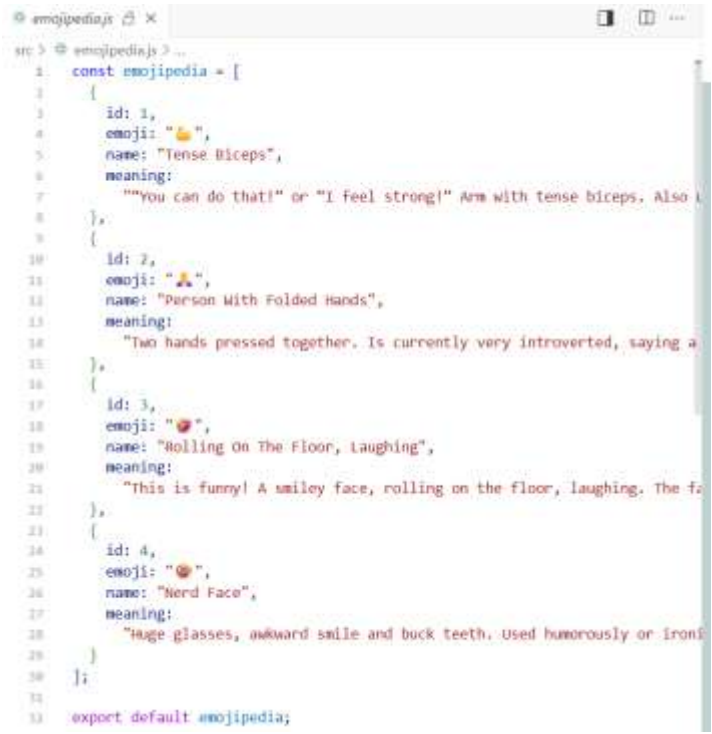
public
index.html
styles.css
src
components
App.jsx
Entry.jsx
emojipedia.js
index.js

App.jsx

```
import React from "react";
import Entry from "./Entry";
import emojipedia from "../emojipedia";

function createEntry(emojiTerm) {
  return (
    <Entry
      key={emojiTerm.id}
      emoji={emojiTerm.emoji}
      name={emojiTerm.name}
      description={emojiTerm.meaning}
    />
  );
}

function App() {
  return (
    <div>
      <h1>
        <span>emojipedia</span>
      </h1>
      <dl className="dictionary">{emojipedia.map(createEntry)}</dl>
    </div>
  );
}

export default App;
```

Entry.jsx

```
import React from "react";

function Entry(props) {
  return (
    <div className="term">
      <dt>
        <span className="emoji" role="img" aria-label="
          {props.emoji}
        </span>
        <span>{props.name}</span>
      </dt>
      <dd>{props.description}</dd>
    </div>
  );
}

export default Entry;
```



```
const emojipedia = [
  {
    id: 1,
    emoji: "💪",
    name: "Tense Biceps",
    meaning:
      ""You can do that!" or "I feel strong!" Arm with tense biceps. Also
  },
  {
    id: 2,
    emoji: "🙏",
    name: "Person With Folded Hands",
    meaning:
      "Two hands pressed together. Is currently very introverted, saying a
  },
  {
    id: 3,
    emoji: "🤣",
    name: "Rolling On The Floor, Laughing",
    meaning:
      "This is funny! A smiley face, rolling on the floor, laughing. The f
  },
  {
    id: 4,
    emoji: "🤓",
    name: "Nerd Face",
    meaning:
      "Huge glasses, awkward smile and buck teeth. Used humorously or ironi
  }
];

export default emojipedia;
```

Some functions mapping, filter and Arrow functions

index.js > ...

```
//Maping to create a new array
var numbers = [1,3,5,7,9];
function double(x){
    return x * 2;
}

var a = numbers.map(double);
const arrow = numbers.map((x)=>x *2)
console.log(arrow)


// or Arrow function square it?
var arrowFun = numbers.map(x=>x * x)
console.log(arrowFun)

// Filter: create a new array that keep the item with return ture
const newNum = numbers.filter(function(num){
    return num>4
});
console.log(newNum)
//make it arrow function
const newNumber = numbers.filter((num1)=>num1>5);
console.log(newNumber)

// find to find the first item that matches from an array
var b = numbers.find(function(x){
    return x>4;
})
console.log(b)
```

Keeper-App adding Props, Mapping, Arrow function

keeper-app > src > components > App.jsx > default

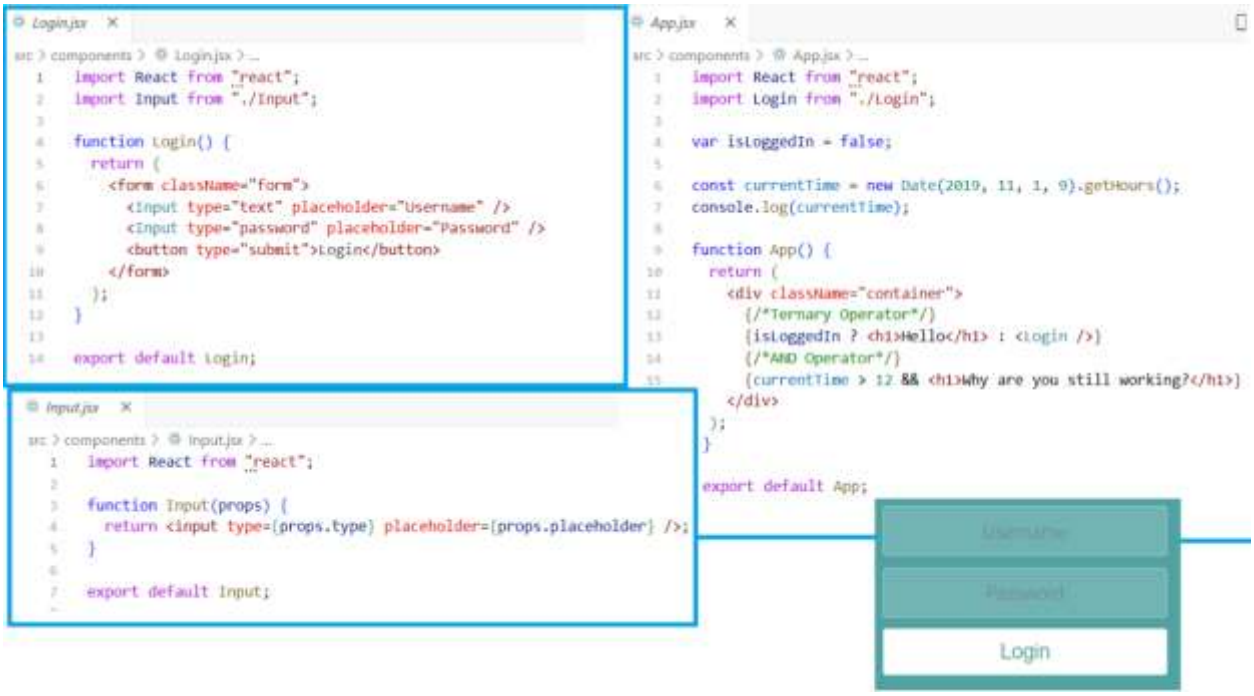
```jsx
import React from "react";
import Header from "./Header";
import Footer from "./Footer";
import Note from "./Note";
import notes from "../notes";


function App(){
    return (
       <div>
        <Header />
        <Footer />
        {notes.map(singelNote=>(<Note title ={singelNote.title}content ={singelNote.content}/>))}
       </div>
    )}
export default App;
```

keeper-app > src > components > Note.jsx > Note

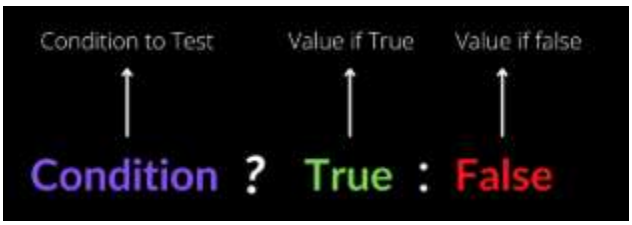
```jsx
import React from "react";

function Note(props){
    return(
        <div>
            <h1>{props.title}</h1>
            <p>{props.content}</p>
        </div>
    )
}
export default Note;
```

notes.js

```js
const notes = [
  {
    key: 1,
    title: "Delegation",
    content:
      "Q. How many programmers does it take to change a light bulb? A. None - It's a hardware problem"
  },
  {
    key: 2,
    title: "Loops",
    content:
      "How to keep a programmer in the shower forever. Show him the shampoo bottle instructions: Lather. Rinse. Repeat."
  },
  {
    key: 3,
    title: "Arrays",
    content:
      "Q. Why did the programmer quit his job? A. Because he didn't get arrays."
  },
  {
    key: 4,
    title: "Hardware vs. Software",
    content:
      "What's the difference between hardware and software? You can hit your hardware with a hammer, but you can only curse at your software."
  },
  {
    key: 5,
    title: "Big ideas",
    content: "Eat more sushi"
  }
];

export default notes;
```



Code for the final login code



React Hooks(useState)

index.js

```
src > index.js
1  import React from "react";
2  import ReactDOM from "react-dom";
3  import App from "./components/App";
4  import "./../public/styles.css";
5
6  ReactDOM.render(<App />, document.getElementById("root"));
7
```

App.jsx

```
src > components > App.jsx
1  import React, { useState } from "react";
2
3  function App() {
4    const [count, setCount] = useState(0);
5    function increase() {
6      setCount(count + 1);
7    }
8    function decrease() {
9      setCount(count - 1);
10   }
11   return (
12     <div className="container">
13       <h1>{count}</h1>
14       <button onClick={increase}>+</button>
15       <button onClick={decrease}>-</button>
16     </div>
17   );
18 }
19
20 export default App;
```

index.js

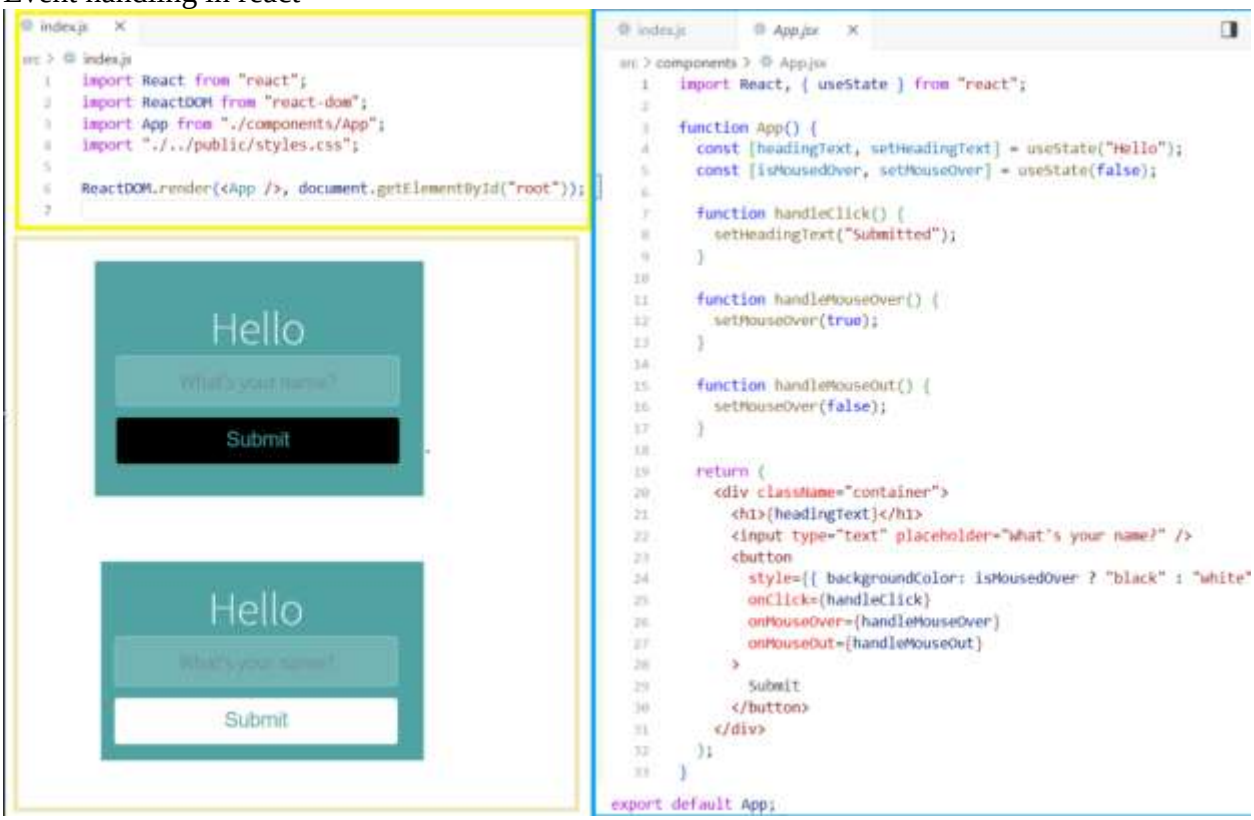
```
src > index.js
1  import React from "react";
2  import ReactDOM from "react-dom";
3  import App from "./components/App";
4  import "./../public/styles.css";
5
6  ReactDOM.render(<App />, document.getElementById("root"));
```

App.jsx

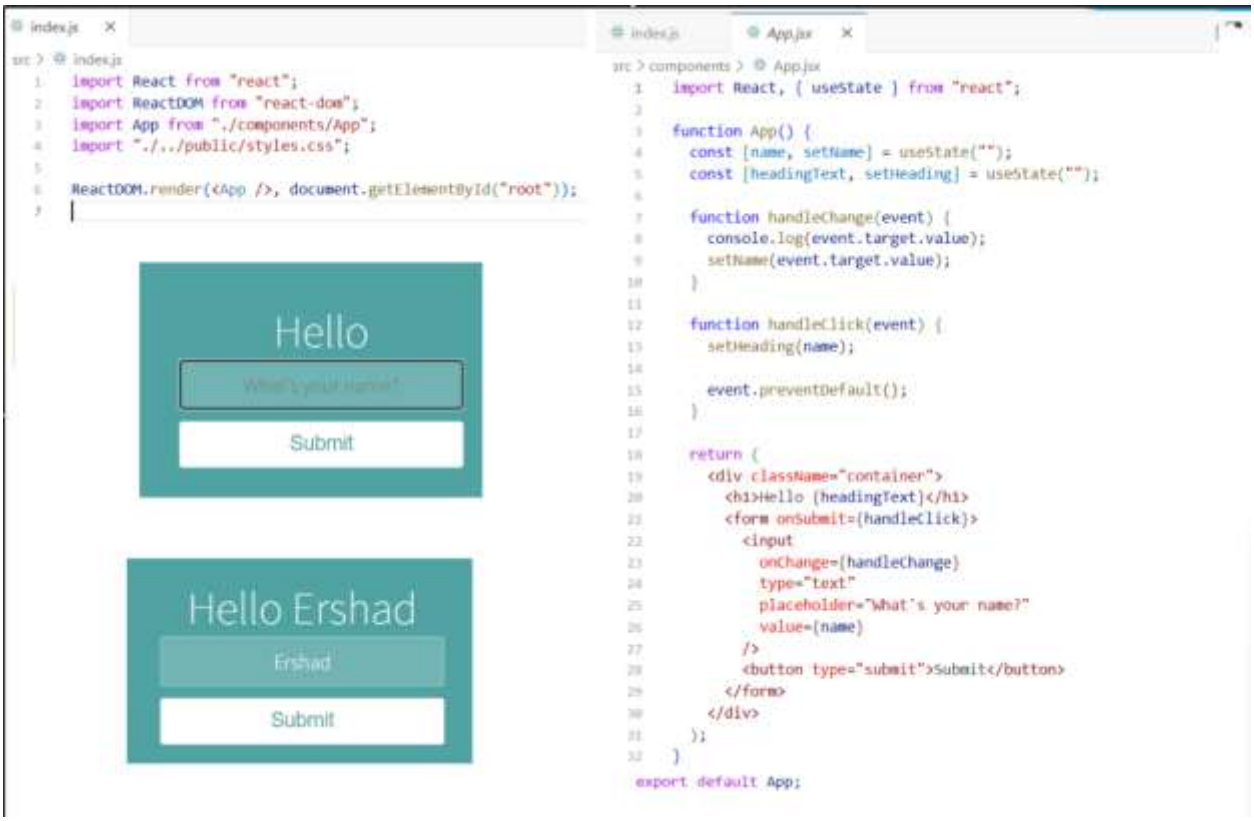
```
src > components > App.jsx
1  import React, { useState } from "react";
2
3  function App() {
4    setInterval(update, 1000);
5    const now = new Date().toLocaleTimeString();
6    const [time, setTime] = useState(now);
7    function update() {
8      const updateTime = new Date().toLocaleTimeString();
9      setTime(updateTime);
10   }
11   return (
12     <div className="container">
13       <h1>{time}</h1>
14     </div>
15   );
16 }
17
18 export default App;
```

DE structuring

Event handling in react



React forms



React Hooks vs Classes

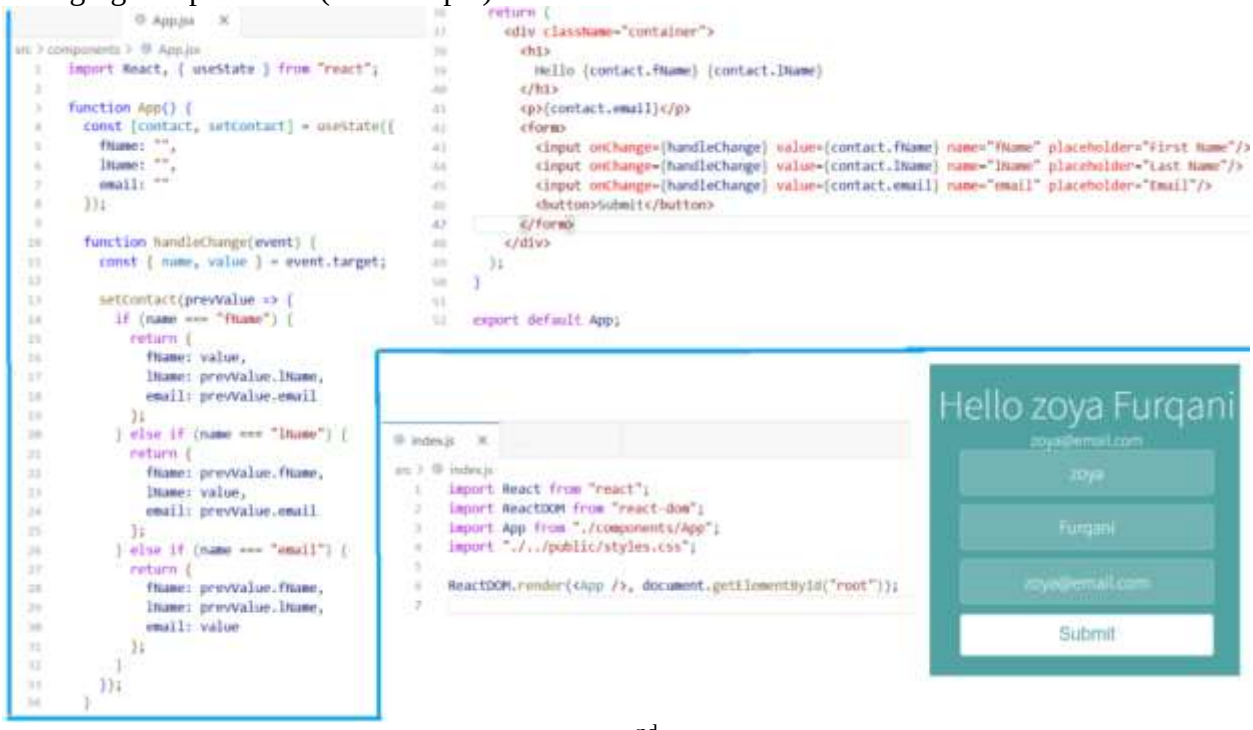
Hook

```
import React, {useState} from "react";

function App() {
  return(
    <h1>Hello</h1>
  )
}
export default App;
```

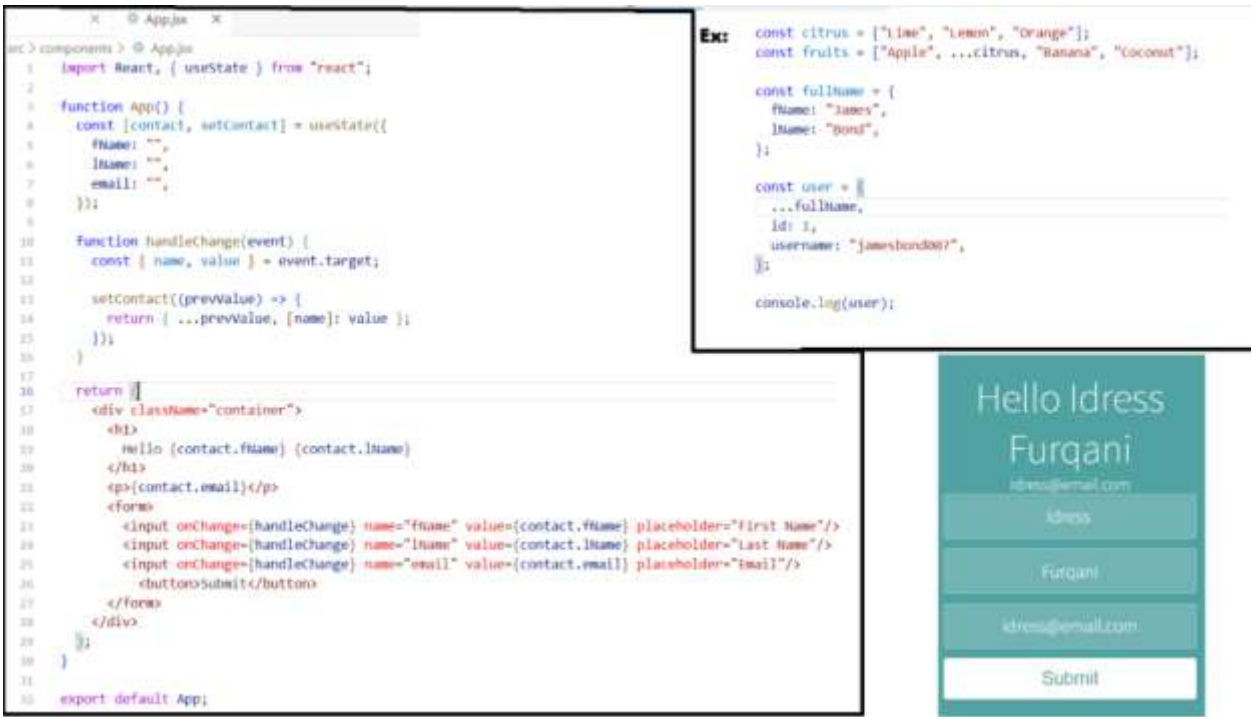
Class

```
import React, {useState} from "react";

class App extends React.Component{
  render(){<h1>Hello</h1>}

}
export default App;
```

Changing complex state (1st Example)



Spread Operators (connects arrays or objects) (2nd Example)



Add button of My Contacts

HTML, CSS, EJS, JS, NODE, Express, SQL, PG-Admin, React…

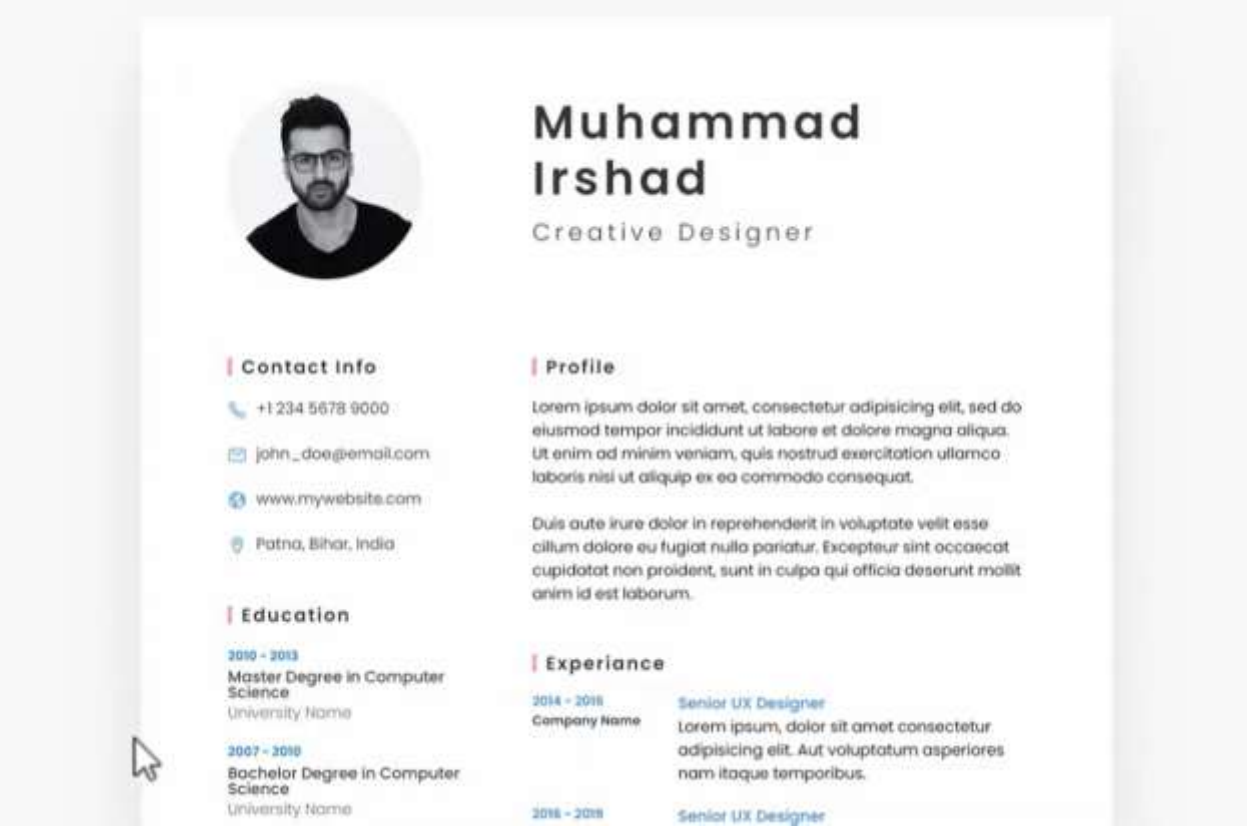
I completed this course a few months ago and wanted to share some essential steps to take after finishing it. Firstly, thanks to Angela for her excellent teaching. This is the best online bootcamp to get started with Web Development. However, there were a few things that were not covered in depth.

React Class Components:
Learn about React Class Components alongside Functional Components. The course did not cover class components in depth.
Additional React Hooks:
Explore other React hooks that were not included in the bootcamp - https://react.dev/reference/react/hooks
State Management:
State management with Redux or Context API is must to know (for making big projects).
API Handling with Axios:
Get proficient with Axios for handling API requests in React.
React Routing:
Understand React routing, similar to how we used Express for routing in projects.
Build a Static Project:
The next step will be to make one simple static project on your own with React Class or functional Components as per need without using any databases like MongoDB to have better understanding and check what you have learnt.
MERN Project:
As we built a TODO application with Mongo, Express, and EJS, try creating a MERN (MongoDB, Express, React, Node.js) project. It has a different folder structure compared to a simple React application.
Internships/Jobs:
After completing the React portion, start applying for React or Web development internships/jobs. Gaining practical experience is invaluable.
Open Source Contribution:
Participate in open-source projects and programs like GSoC, LFX Mentorship. It boosts confidence and gives a sense of accomplishment.

So one common mistake to prevent is to fall in the tutorial hell loop.
Don't try to fall in this loop of learning without implementing, the correct procedure which to follow should be to learn a technology, make a project, git it, deploy it, share it with your friends-family anyone you want and add to your portfolio. This will surely keep your interest in development and help you keep moving, learning further!

NOTE: The above things mentioned are from a learning point of view only. Getting hired as a Front End Engineer / Full Stack Developer requires different preparation. From interviews perspective learning JavaScript & Data Structures - Algorithms should be on top priority.
Things might be intimidating in the beginning, but just hang in there. You'll thank yourself later.

Table of Contents




1997 - 2007
Matriculation
University Name
Language
English
Hindi
Interest
Gaming
Singing
Reading
Cooking
Company Name
Lorem ipsum, dolor sit amet consectetur adipisicing elit. Aut voluptatum asperiores nam itaque temporibus.
2019 - Present
Company Name
Senior UX Designer
Lorem ipsum, dolor sit amet consectetur adipisicing elit. Aut voluptatum asperiores nam itaque temporibus.
Professional Skills
Html
CSS
Javascript
Photoshop
Illustrator
Adobe XD